



Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture 1

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

INTRODUCTION

- A **database-management system (DBMS)** is a collection of **interrelated data** and a set of programs to access those data.
- The collection of data, usually referred to as the **database**, contains information relevant to an enterprise.
- The primary goal of a DBMS is to provide a way to store and retrieve database information that is both *convenient* and *efficient*.
- Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information.

- Example:

When you access a bank Web site and retrieve your bank balance and transaction information, the information is retrieved from the bank's database system.

Applications

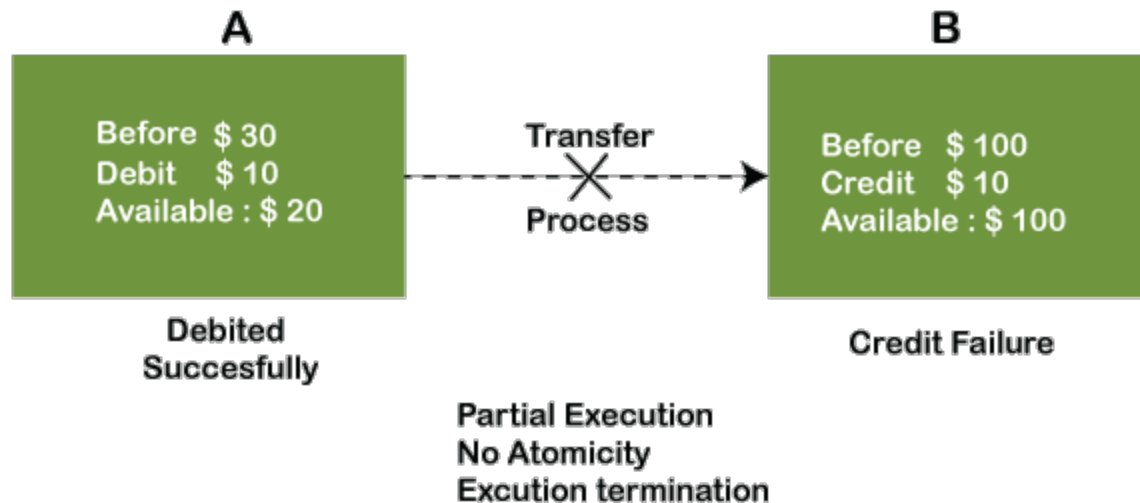
- Enterprise Information
 - Sales
 - Accounting
 - Human Resource
 - Retailing
- Banking and Finance
 - Banking
 - Credit Card transactions
 - Finance
- Universities
- Telecommunications

ATOMICITY PROBLEMS:

Consider a program to transfer 100 rupees from the account balance of **department A** to the account balance of **department B**.

If a system failure occurs during the execution of the program, it is possible that the 100 rupees was removed from the balance of **department A** but was not credited to the balance of **department B**, resulting in an inconsistent database state.

Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur. That is, the funds transfer must be atomic—it must happen in its entirety or not at all.



Concurrency

- In a multi-user system, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database.

1. Lost Update Problems (W - W Conflict)

The problem occurs when two different database transactions perform the read/write operations on the same database items in an interleaved manner (i.e., concurrent execution) that makes the values of the items incorrect hence making the database inconsistent.

Consider the below diagram where two transactions T_x and T_y , are performed on the same account A where the balance of account A is \$300.

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A - 50$	—
t_3	—	READ (A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	WRITE (A)	—
t_7	—	WRITE (A)

LOST UPDATE PROBLEM

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300 (only read).
- At time t_2 , transaction T_x deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
- Alternately, at time t_3 , transaction T_y reads the value of account A that will be \$300 only because T_x didn't update the value yet.
- At time t_4 , transaction T_y adds \$100 to account A that becomes \$400 (only added but not updated/write).
- At time t_6 , transaction T_x writes the value of account A that will be updated as \$250 only, as T_y didn't update the value yet.
- Similarly, at time t_7 , transaction T_y writes the values of account A, so it will write as done at time t_4 that will be \$400. It means the value written by T_x is lost, i.e., \$250 is lost.

2. Dirty Read Problems (W-R Conflict)

The dirty read problem occurs when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction. There comes the Read-Write Conflict between both transactions.

Consider two transactions T_x and T_y in the below diagram performing read/write operations on account A where the available balance in account A is \$300:

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A + 50$	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300.
- At time t_2 , transaction T_x adds \$50 to account A that becomes \$350.
- At time t_3 , transaction T_x writes the updated value in account A, i.e., \$350.
- Then at time t_4 , transaction T_y reads account A that will be read as \$350.
- Then at time t_5 , transaction T_x rolls back due to server problem, and the value changes back to \$300 (as initially).
- But the value for account A remains \$350 for transaction T_y as committed, which is the dirty read and therefore known as the Dirty Read Problem.

3. Unrepeatable Read Problem (W-R Conflict)

Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.

Consider two transactions, T_x and T_y , performing the read/write operations on account A, having an available balance = \$300. The diagram is shown below:

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	$A = A + 100$
t_4	—	WRITE (A)
t_5	READ (A)	—

UNREPEATABLE READ PROBLEM

- At time t_1 , transaction T_x reads the value from account A, i.e., \$300.
- At time t_2 , transaction T_y reads the value from account A, i.e., \$300.
- At time t_3 , transaction T_y updates the value of account A by adding \$100 to the available balance, and then it becomes \$400.
- At time t_4 , transaction T_y writes the updated value, i.e., \$400.
- After that, at time t_5 , transaction T_x reads the available value of account A, and that will be read as \$400.
- It means that within the same transaction T_x , it reads two different values of account A, i.e., \$ 300 initially, and after updation made by transaction T_y , it reads \$400. It is an unrepeatable read and is therefore known as the Unrepeatable read problem.

Concurrency Control Protocols

- The concurrency control protocols ensure the *atomicity, consistency, isolation, durability* and *serializability* of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:
 1. Lock Based Concurrency Control Protocol
 2. Time Stamp Concurrency Control Protocol
 3. Validation Based Concurrency Control Protocol

Database ACID Properties

Atomicity

means either all successful or none.

Consistency

ensures bringing the database from one consistent state to another consistent state.
ensures bringing the database from one consistent state to another consistent state.

Isolation

ensures that transaction is isolated from other transaction.

Durability

means once a transaction has been committed, it will remain so, even in the event of errors, power loss etc.

Multiple statements make up a transaction. These statements can be broken down into the parts of the transaction like the input, data processing, checking, storage and output.

- **Assume 3 Processes A, B and C. For a transaction t to complete, all processes must be validated and have completed.**

Therefore, it must follow an AND condition:

$$t = A \wedge B \wedge C = 1$$

The transaction cannot be an OR condition:

$$t = A \vee B \vee C = 0$$

- The reason a transaction must be validated is in order to keep the database in a consistent state. Any partial saving of data can lead to inconsistencies that can severely affect the accuracy of information.

ISOLATION

- In order to prevent conflicts in the database which could result in inconsistencies, there must be process isolation. This ensures that process t1 and t2 don't attempt to execute at the same time and write data to the database.

CONSISTENCY

- A transaction must be one completed operation in order for data to be consistent. Validation rules must be put in place to ensure data consistency.

Let $A + B = 1000$, where A and B are any integer that when added with each other results in 1000.

A or B cannot be > 1000 , but can be $=$ to 1000 or 0.

e.g.

$A = 0$, $B = 1000$

$A + B = 1000$ (Accept)

$A = 1000$, $B = 1000$

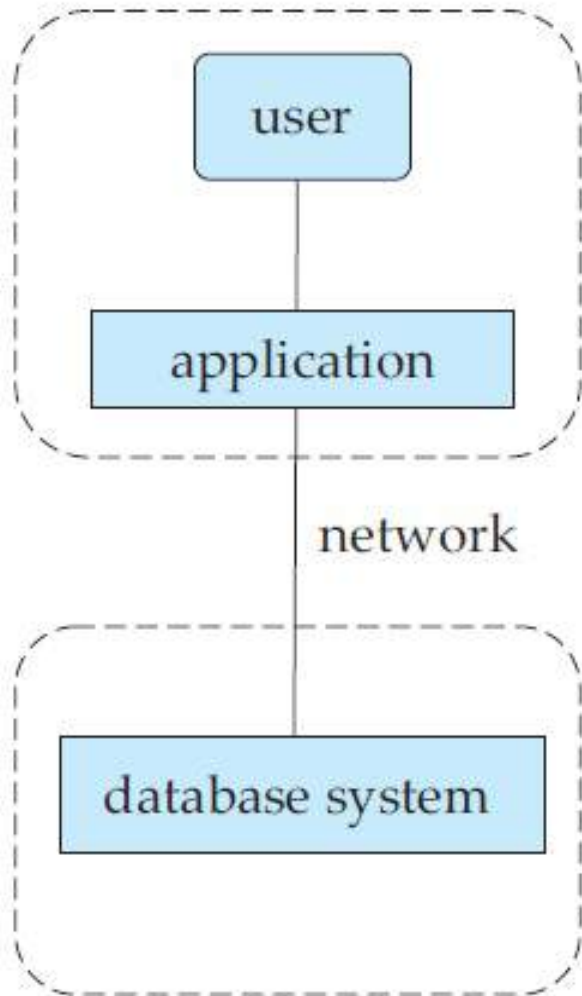
$A + B = 2000$ (Reject)

I can create a simple checker to make sure that the input data for A or B can only be 0 to 1000, and when added together it cannot be > 1000 .

DURABILITY

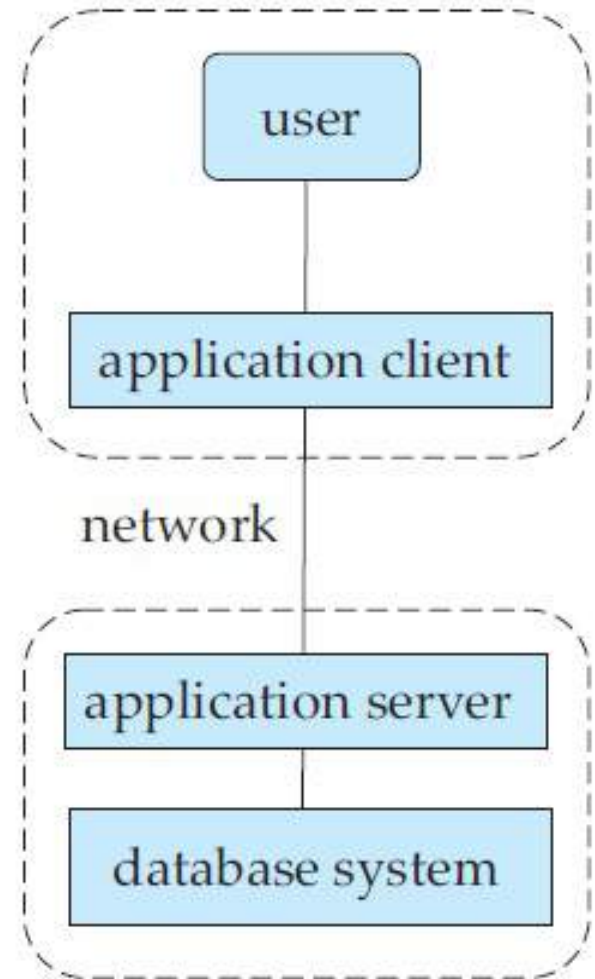
- Persistent data storage and backup tests a database system's effectivity.
- DB on the network, either in local data centers or on the cloud. Making sure that the data is properly secured and managed by an administrator is the most important part of database administration.

DATABASE ARCHITECTURE



(a) Two-tier architecture

client



(b) Three-tier architecture

server

- In a **two-tier architecture**, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements.
- Application program interface standards like ODBC and JDBC are used for interaction between the client and the server.
- **Three-tier applications** are more appropriate for large applications, and for applications that run on the Worldwide Web.

DATA MINING

- The term data mining refers loosely to the process of semi automatically analyzing large databases to find useful patterns.
- Like knowledge discovery in artificial intelligence (also called machine learning) or statistical analysis, data mining attempts to discover rules and patterns from data.
- However, data mining differs from machine learning and statistics in that it deals with large volumes of data, stored primarily on disk. That is, data mining deals with “knowledge discovery in databases.”

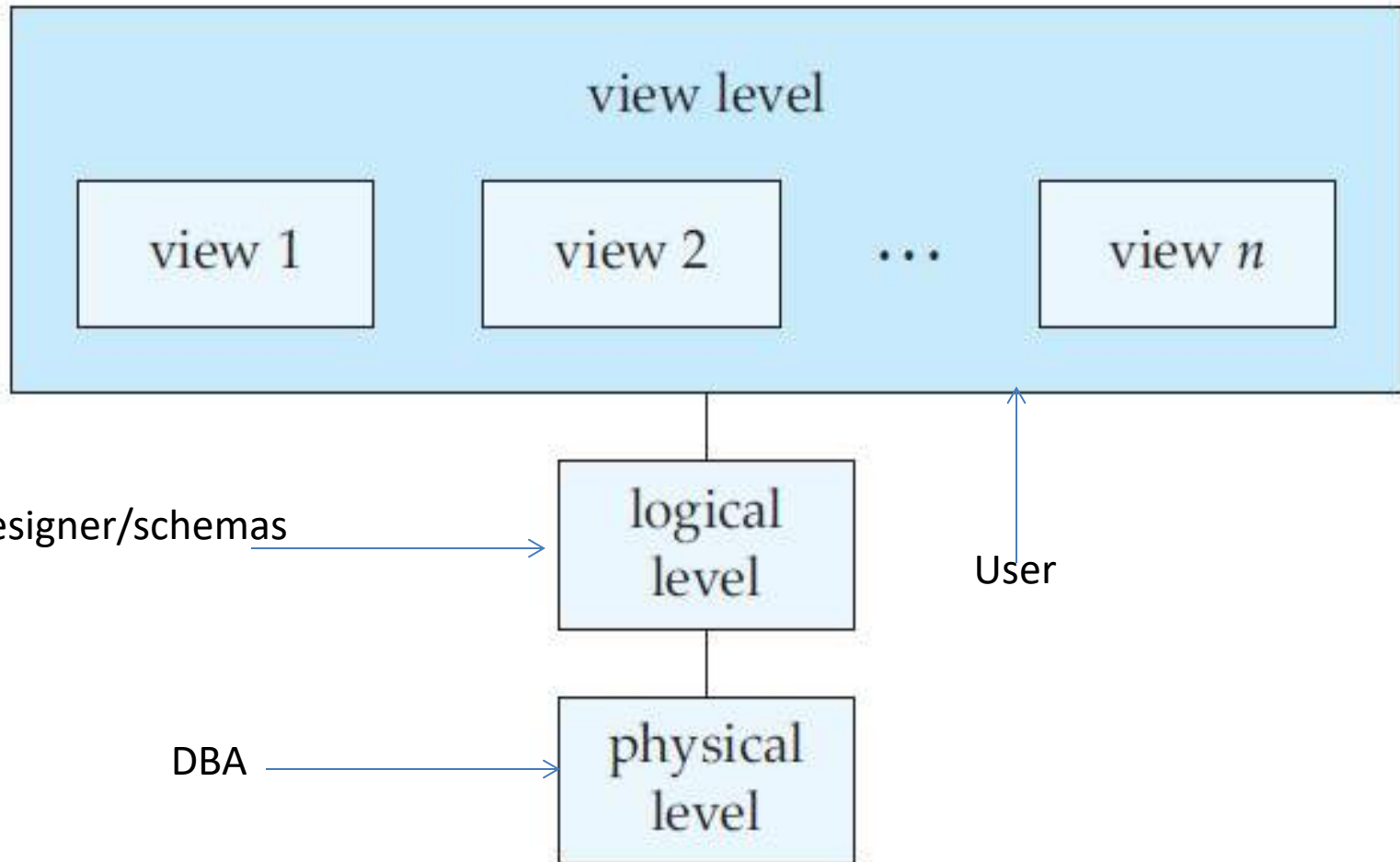
Information Retrieval

- Textual data, too, has grown explosively.
- Textual data is unstructured, unlike the rigidly structured data in relational databases. Querying of unstructured textual data is referred to as *information retrieval*.
- Information retrieval systems have much in common with database systems—in particular, the storage and retrieval of data on secondary storage.

Data Abstraction

- A major purpose of a database system is to provide users with an abstract view of the data.
- Since many database-system users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interaction with the system.

- **Physical level:** The lowest level of abstraction describes how the data are actually stored.
- **Logical level:** The next-higher level of abstraction describes *what* data are stored in the database, and what relationships exist among those data.
- **View Level:** Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system.





Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture-2

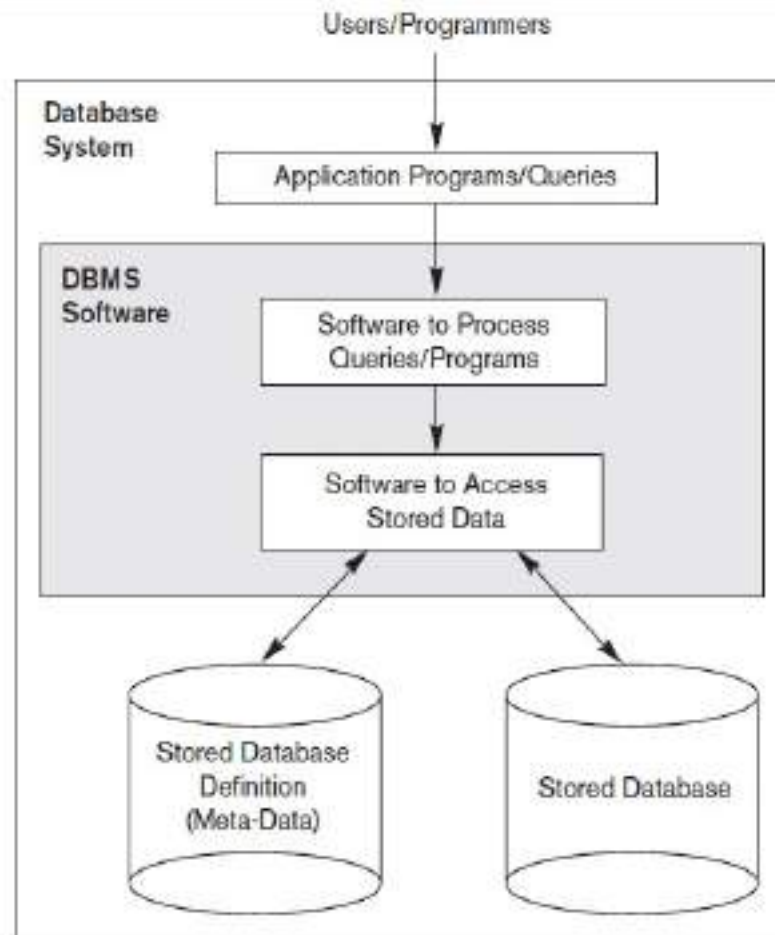
B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Objective/motivation

- A database is collection of interrelated data organized in such a way so that it is easy to access, retrieve, manage and understand data.
 - for example, a list of students - you will have a list of the names, along with their roll nos. You may also have a table somewhere about the marks of each student on your list. There will be separate files for mark lists for different subjects, and courses. These mark lists may be queried to find the aggregate marks, and overall percentages, which may be stored in some result files. All this collective information, derived from raw data, is called a database.
 - One way to keep information on a computer is to store it in permanent files. A company system has a number of application programs; each of them is designed to manipulate data files. These application programs have been written at the request of the users in the organization. New applications are added to the system as the need arises. The system just described is called the **file-based system**.

- These difficulties, among others, prompted the development of database systems. In what follows, we shall see the concepts and algorithms that enable database systems to solve the problems with file-processing systems.



Characteristics of the Database Approach

- A number of characteristics distinguish the database approach from the much older approach of programming with **files**.
 - For example, one user, the *grade reporting office*, may keep files on students and their grades.
 - A second user, *the accounting office*, may keep track of students' fees and their payments.
 - Although both users are interested in data about students, each user maintains separate files
 - Now, Data manipulation leads to data inconsistency because...each requires some data not available from the other user's files.

- This redundancy in defining and storing data results in wasted storage space and in redundant efforts to maintain common up-to-date data.
- In the database approach, a single repository maintains data that is defined once and then accessed by various users.
- In file systems, each application is free to name data elements independently. In contrast, in a database, the names or labels of data are defined once, and used repeatedly by queries, transactions, and applications.

- The main characteristics of the database approach versus the file-processing approach are the following:
 - Self-describing nature of a database system
 - Insulation between programs and data, and data abstraction
 - Support of multiple views of the data
 - Sharing of data and multiuser transaction processing

Self-describing nature of a database system

A database system is referred to as self-describing because it not only contains the database itself, but also **metadata** which defines and describes the data and relationships between tables in the database. This information is used by the DBMS software or database users if needed.

Insulation between program and data

- In the file-based system, the structure of the data files is defined in the application programs so if a user wants to change the structure of a file, all the programs that access that file might need to be changed as well.
- On the other hand, in the database approach, the data structure is stored in the system catalogue¹ and not in the programs. Therefore, one change is all that is needed to change the structure of a file. This insulation between the programs and data is also called **program-data independence**.

¹The system catalog consists of tables and views that describe the structure of the database

Support for multiple views of data

A database supports multiple views of data. A view is a subset of the database, which is defined and dedicated for particular users of the system. Multiple users in the system might have different views of the system. Each view might contain only the data of interest to a user or group of users.

Sharing of data and multiuser system

Current database systems are designed for multiple users. That is, they allow many users to access the same database at the same time. This access is achieved through features called **concurrency control strategies**. These strategies ensure that the data accessed are always correct and that data integrity is maintained.

The design of modern multiuser database systems is a great improvement from those in the past which restricted usage to one person at a time.

Evolution

- Information processing drives the growth of computers, as it has from the earliest days of commercial computers. In fact, automation of data processing tasks predates computers.
- Techniques for data storage and processing have evolved over the years and major stages in the evolution of database systems are as follows:
 - **File Systems:** In the early days of computing, data was primarily stored in flat file systems. Each application had its own set of files for data storage. File systems lacked data organization, and data redundancy was common. The data was managed at the file level, making it challenging to maintain and retrieve data efficiently.
 - **Hierarchical Database Model:** In the 1960s, the hierarchical database model emerged. It organized data in a tree-like structure, with parent-child relationships. While it provided some improvement over file systems, it was rigid and inflexible for handling complex data relationships

- **Network Database Model:** Following the hierarchical model, the network database model was developed in the late 1960s. It allowed more flexible data relationships by enabling each record to have multiple parent and child records. However, managing complex networks of data was still difficult.
- **Relational Database Model:** The breakthrough in database systems came with the advent of the relational database model in the 1970s. Edgar Codd introduced the concept, which organized data into tables with rows and columns. This model provided a more intuitive and flexible way to represent data and defined a standardized query language (SQL) for data manipulation. The relational model gained widespread adoption due to its simplicity and ability to handle complex data relationships effectively.
- **Object-Oriented Database Model:** In the 1980s and 1990s, object-oriented database management systems (OODBMS) were developed. These systems combined the principles of object-oriented programming with database management, allowing the storage of complex objects and their relationships. However, OODBMS adoption remained limited due to compatibility issues and the dominance of the relational model.

- **Object-Relational Database Model:** To bridge the gap between the relational and object-oriented worlds, object-relational database management systems (ORDBMS) emerged. ORDBMS extended the relational model to include object-oriented features, enabling the storage of complex data types and supporting inheritance and polymorphism. This approach was well-suited for certain application domains.
- **NoSQL Databases:** As internet applications and big data became more prevalent, the limitations of traditional relational databases in handling massive volumes of unstructured or semi-structured data became apparent. This led to the development of NoSQL (Not only SQL) databases. NoSQL databases use various data models, such as **key-value**, document, column-family, and graph, to offer flexible and scalable solutions for specific data needs.
- **NewSQL Databases:** NewSQL databases emerged as a response to the need for scalable, high-performance relational databases. These databases aim to combine the benefits of traditional SQL databases with horizontal scalability and distributed computing capabilities.
- **Cloud Databases:** With the rise of cloud computing, database systems have evolved to take advantage of cloud infrastructure. Cloud databases offer various advantages like scalability, automatic backups, and accessibility from anywhere, making them popular choices for modern applications.

Recent Advances in Database technology

- **In-Memory Databases:** In-memory databases store data in RAM instead of traditional disk storage, resulting in significantly faster data retrieval and processing. They have become popular for real-time analytics and high-performance applications that require low-latency data access.
- **Cloud Databases:** Cloud-based databases have become mainstream, allowing businesses to offload their database management tasks to cloud service providers. Cloud databases offer scalability, automatic backups, high availability, and seamless integration with cloud infrastructure.

Example: Amazon RDS (Relational Database Service), Google Cloud SQL, Microsoft Azure SQL Database, MongoDB Atlas, and Amazon DynamoDB.

- **Database-as-a-Service (DBaaS):** DBaaS solutions have emerged, providing managed database services that enable organizations to focus on application development rather than database administration. DBaaS offerings from various cloud providers have gained traction, making it easier to deploy, scale, and manage databases in the cloud.
- **Machine Learning in Databases:** Some databases have integrated machine learning capabilities to perform tasks like automatic query optimization, anomaly detection, and predictive analytics. These advancements aim to improve database performance and user experience.

Machine Learning in Databases

- **Anomaly Detection:** Machine learning can be employed within databases to detect anomalies or outliers in data. This is particularly useful in detecting fraudulent activities, monitoring sensor data for abnormal behavior, or identifying potential issues in real-time.
- **Predictive Analytics:** By integrating machine learning models into the database, organizations can perform predictive analytics directly on the data stored in the database. This allows for real-time predictions and insights without the need to export data to separate analytics platforms.
- **Data Classification:** Machine learning algorithms can assist in classifying data into different categories or groups based on the content or patterns in the data. This classification can be used for data organization, reporting, and analysis.

- **Personalization:** Databases can use machine learning to offer personalized recommendations and content based on users' preferences and behaviors. This is commonly used in e-commerce, content recommendation systems, and targeted marketing.
- **Data Cleaning and Deduplication:** Machine learning techniques can aid in automatically cleaning and deduplicating data within the database, improving data quality and reducing redundancy.
 - Data deduplication is a technique used to identify and eliminate duplicate copies of data within a dataset or storage system. The goal of deduplication is to reduce data redundancy and optimize storage utilization by storing only unique data and eliminating unnecessary copies.
- It's essential to note that machine learning in databases requires careful consideration of data privacy, security, and ethical concerns (privacy preserving techniques).
 - Privacy-preserving techniques refer to a set of methods and practices designed to protect sensitive information while still allowing data analysis and processing.



Graphic Era

(Deemed to be University)

Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture-3

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Database System Concepts and Architecture

- In a basic client/server DBMS architecture, the system functionality is distributed between two types of modules.
 - A **client module** is typically designed so that it will run on a user workstation or personal computer.
 - The other kind of module, called a **server module**, typically handles data storage, access, search, and other functions.

Data Models, Schemas, and Instances

- **Data abstraction:**

Generally refers to the suppression of details of data organization and storage, and the highlighting of the essential features for an improved understanding of data.

- One of the main characteristics of the database approach is to support data abstraction so that different users can perceive data at their preferred level of detail.

❑ **data model**—a collection of concepts that can be used to describe the structure of a database—provides the necessary means to achieve the abstraction.

- ❑ By structure of a database we mean the data types, relationships, and constraints that apply to the data.

Categories of Data Models

- **High-level or conceptual** data models provide concepts that are close to the way many users perceive data, whereas **low-level or physical data** models provide concepts that describe the details of how data is stored on the computer storage media.
- Between these two extremes is a class of **representational (or implementation) data models**.
 - which provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage.

- Conceptual data models use concepts such as entities, attributes, and relationships.
 - An entity represents a real-world object or concept, such as an employee or a project from the mini-world that is described in the database.
 - An attribute represents some property of interest that further describes an entity, such as the employee's name or salary.
 - A relationship among two or more entities represents an association among the entities, for example, a works-on relationship between an employee and a project.

- Physical data models describe how data is stored as files in the computer by representing information such as record formats, record orderings, and access paths.
 - **An access path** is a structure that makes the search for particular database records efficient.
 - An index is an example of an access path that allows direct access to data using an index term or a keyword.

Schemas, Instances, and Database State

- The description of a database is called the database schema, which is specified during database design and is not expected to change **frequently**.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

Schema

Data base

- the diagram displays the structure of each record type but not the actual instances of records.
- We call each object in the schema—such as STUDENT or COURSE—a **schema construct**.
- The actual data in a database may change quite frequently. For example, the database changes every time we add a new student or enter a new grade.
 - The data in the database at a particular moment in time is called a **database state or snapshot**. It is also called the **current set of occurrences or instances** in the database.
 - Every time we insert or delete a record or change the value of a data item in a record, we change one state of the database into another state.

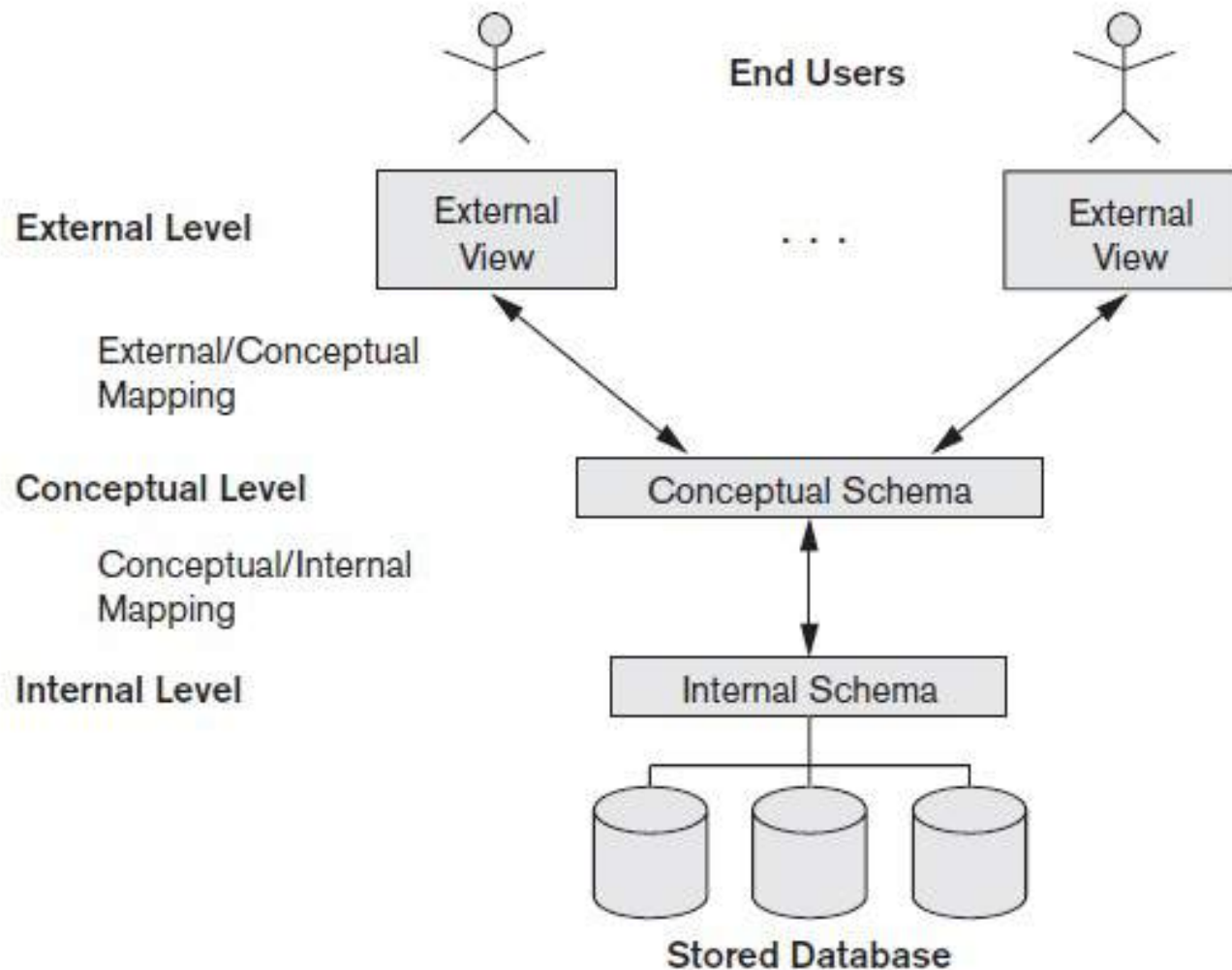
- When we define a new database, we specify its database schema only to the DBMS.
- At this point, the corresponding database state is the *empty state* with no data.
- We get the *initial state* of the database when the database is first **populated or loaded** with the initial data.
- From then on, every time an update operation is applied to the database, we get another database state. At any point in time, the database has a current state or?

- The DBMS is partly responsible for ensuring that every state of the database is a **valid state**—that is, a state that satisfies the structure and constraints specified in the schema.
- The DBMS stores the descriptions of the schema constructs and constraints—also called the **meta-data**—in the DBMS catalog so that DBMS software can refer to the schema whenever it needs to.

Three-Schema Architecture and Data Independence

- Three of the four important characteristics of the database approach-
 - Self-describing nature of a database system
 - Insulation between programs and data, and data abstraction
 - Support of multiple views of the data
 - Sharing of data and multiuser transaction processing
- In this section we specify an architecture for database systems, called the **three-schema architecture** that was proposed to help achieve and visualize these characteristics.

The Three-Schema Architecture



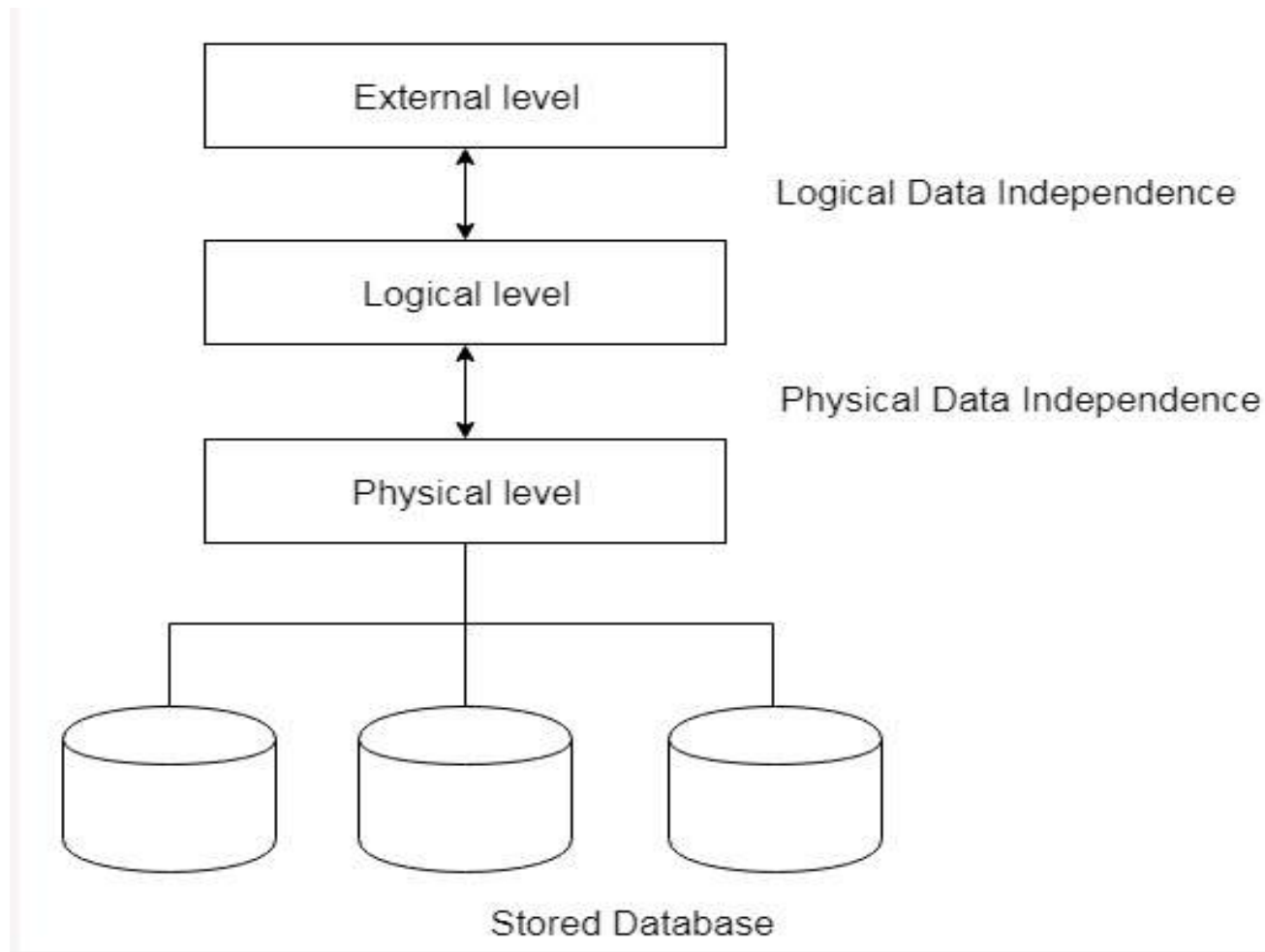
- The **internal level** has an internal schema, which describes the physical storage structure of the database.
 - describes the complete details of data storage and access paths for the database.
- The **conceptual level** has a conceptual schema, which describes the structure of the whole database for a community of users.
 - The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
- The **external or view level** includes a number of external schemas or user views.
 - Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

- The processes of transforming requests and results between levels are called **mappings**.

Data Independence

- can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level. Two types:
 - Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs.
 - Physical data independence is the capacity to change the internal schema without having to change the conceptual schema.

- Data independence occurs because when the schema is changed at some level, the schema at the next higher level remains unchanged; only the *mapping* between the two levels is changed.



Database Languages and Interfaces

- In many DBMSs where no strict separation of levels is maintained, one language, called the data definition language (DDL), is used by the **DBA** and **by database designers** to define both schemas.
- The DBMS will have a DDL compiler whose function is to process DDL statements in order to identify descriptions of the schema constructs and to store the schema description in the DBMS catalog.

- Once the database schemas are compiled and the database is populated with data, users must have some means to manipulate the database.
- Typical manipulations include retrieval, insertion, deletion, and modification of the data.
- The DBMS provides a set of operations or a language called the data manipulation language (**DML**) for these purposes.

Database Interfaces

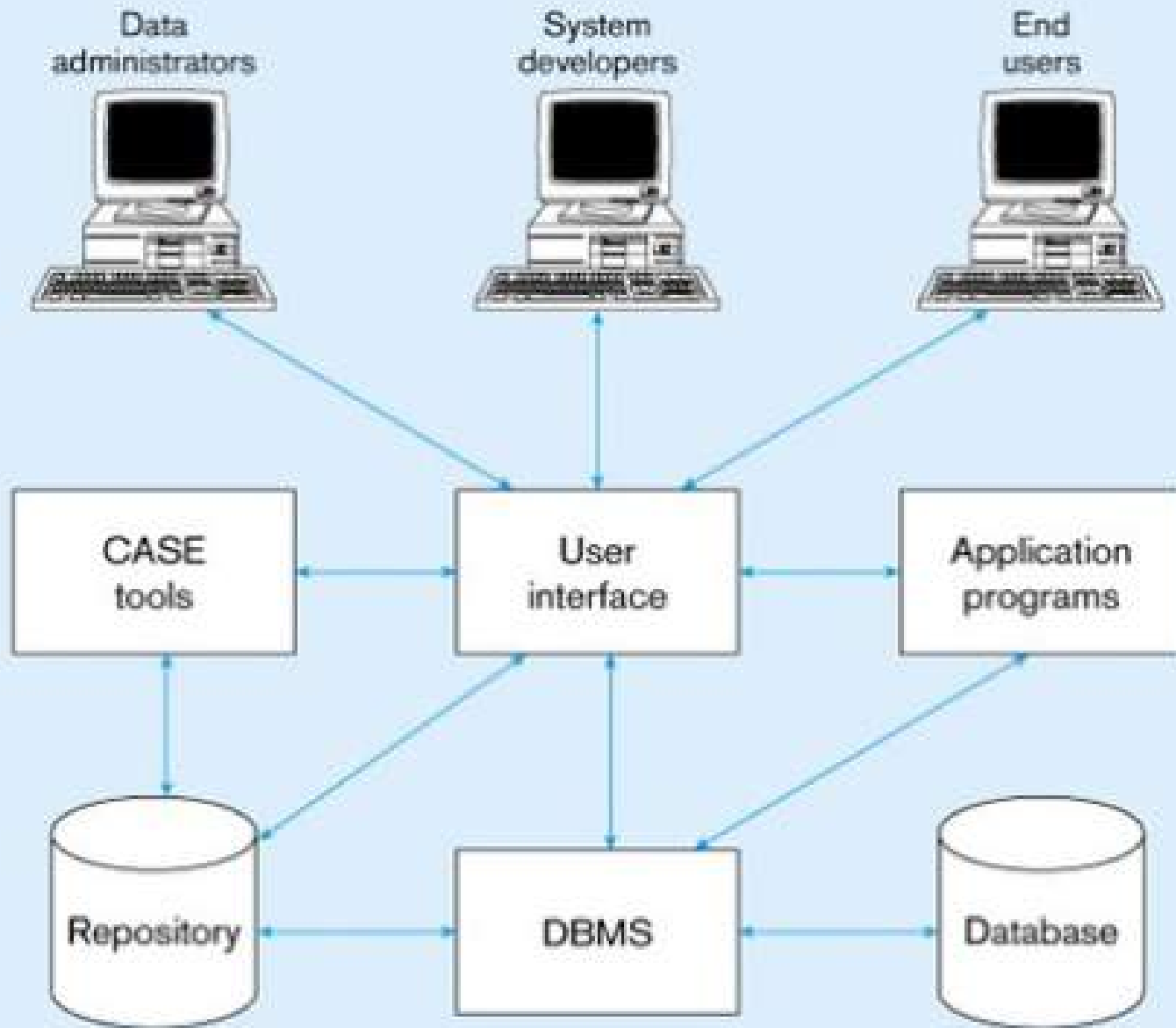
- **Menu-Based Interfaces for Web Clients or Browsing.**
 - Pull-down menus are a very popular technique in Web-based user interfaces.
 - These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request. Menus do away with the need to memorize the specific commands and syntax of a query language; rather, the query is composed step-by-step by picking options from a menu that is displayed by the system.

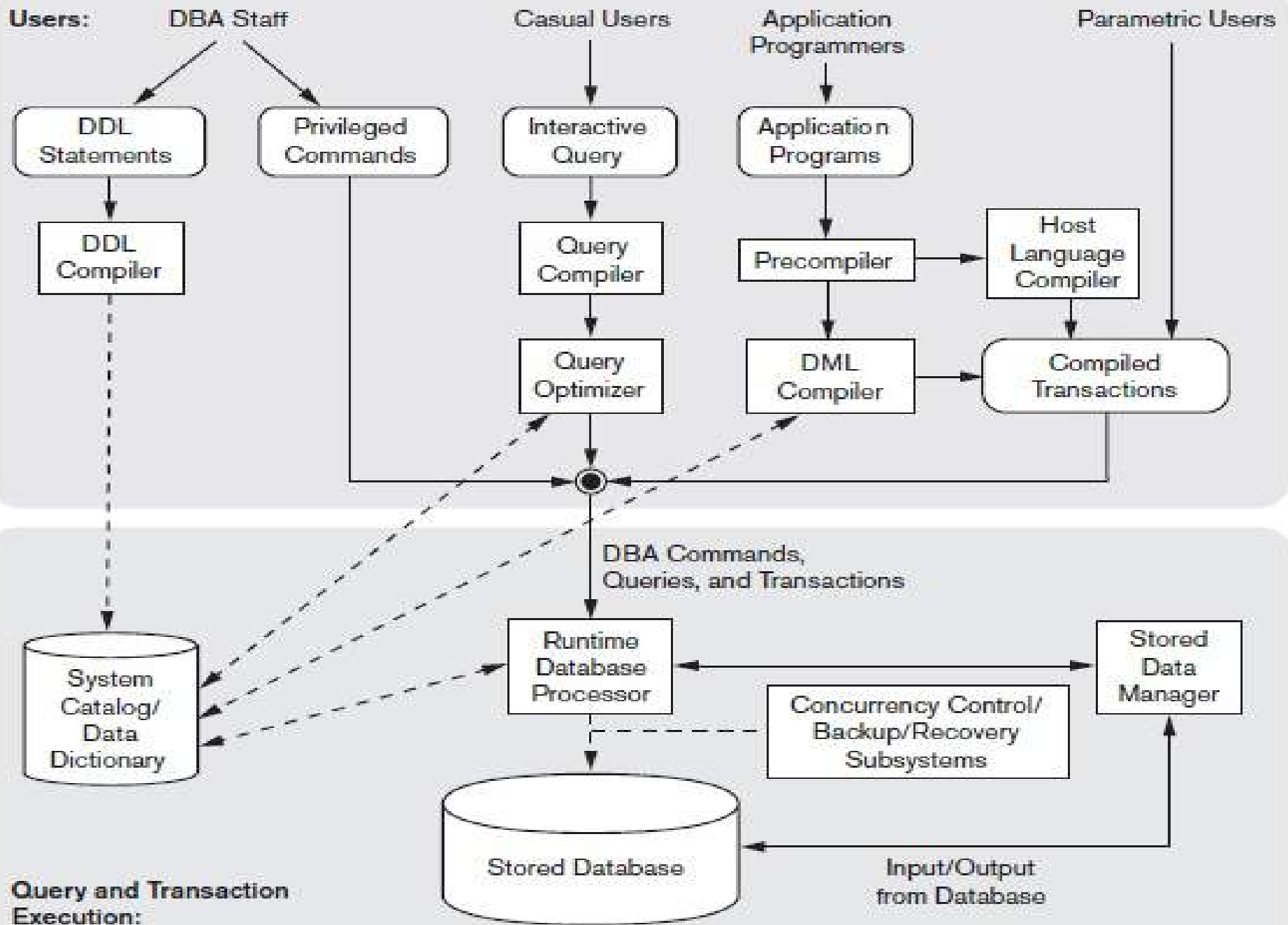
- **Forms-Based Interfaces:** A forms-based interface displays a form to each user. Users can fill out all of the form entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries.
- **Graphical User Interfaces:** GUI typically displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. Most GUIs use a pointing device such as a mouse, to select certain parts of the displayed schema diagram.

- A **natural language interface** is a user interface in which the user and the system communicate via a natural (human) language. The user provides input via speech or some other method, and the system generates responses in the form of utterances delivered by speech, text or some other method.
 - Eg: SIRI, ALEXA, etc,.

DBMS Component Modules

- The system catalog consists of tables and views that describe the structure of the database. Sometimes called the data dictionary, these table objects contain everything that the database knows about itself.





Steps

- The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the **operating system (OS)**, which schedules disk read/write.
- Many DBMSs have their own **buffer management** module to schedule disk read/write, because this has a considerable effect on performance. Reducing disk read/write improves performance considerably.
- the top part of Figure shows interfaces for the **DBA staff**, **casual users** who work with interactive interfaces to formulate queries, **application programmers** who create programs using some host programming languages, and **parametric users** who do data entry work by supplying parameters to predefined transactions.

- The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog.
- The **catalog** includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.
- Casual users and persons with occasional need for information from the database interact using some form of interface, which we call the **interactive query** interface.
- We have not explicitly shown any menu-based or form-based interaction that may be used to generate the interactive query automatically.

- a **query compiler** that compiles them into an internal form.
- Among other things, the **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution.
- Application programmers write programs in host languages such as Java, C, or C++ that are submitted to a precompiler. The **precompiler** extracts DML commands from an application program written in a host programming language.
- These commands are sent to the DML compiler for compilation into object code for database access. The rest of the program is sent to the host language compiler.

- Canned transactions are executed repeatedly by parametric users, who simply supply the parameters to the transactions. Each execution is considered to be a separate transaction.
 - An example is a bank withdrawal transaction where the account number and the amount may be supplied as parameters.
- the **runtime database processor** executes
 - (1) the privileged commands,
 - (2) the executable query plans, and
 - (3) the canned transactions with runtime parameters.
- It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory.

- Here, **concurrency control** and **backup and recovery systems** shown separately as a module in the figure. They are integrated into the working of the runtime database processor for purposes of transaction management.
- Figure is not meant to describe a specific DBMS; rather, it illustrates typical DBMS modules.
 - The DBMS interacts with the operating system when disk accesses—to the database or to the catalog—are needed.
 - If the computer system is shared by many users, the OS will schedule DBMS disk access requests and DBMS processing along with other processes.
 - On the other hand, if the computer system is mainly dedicated to running the database server, the DBMS will control main memory buffering of disk pages.
 - The DBMS also interfaces with compilers for general purpose host programming languages, and with application servers and client programs running on separate machines through the system network interface.

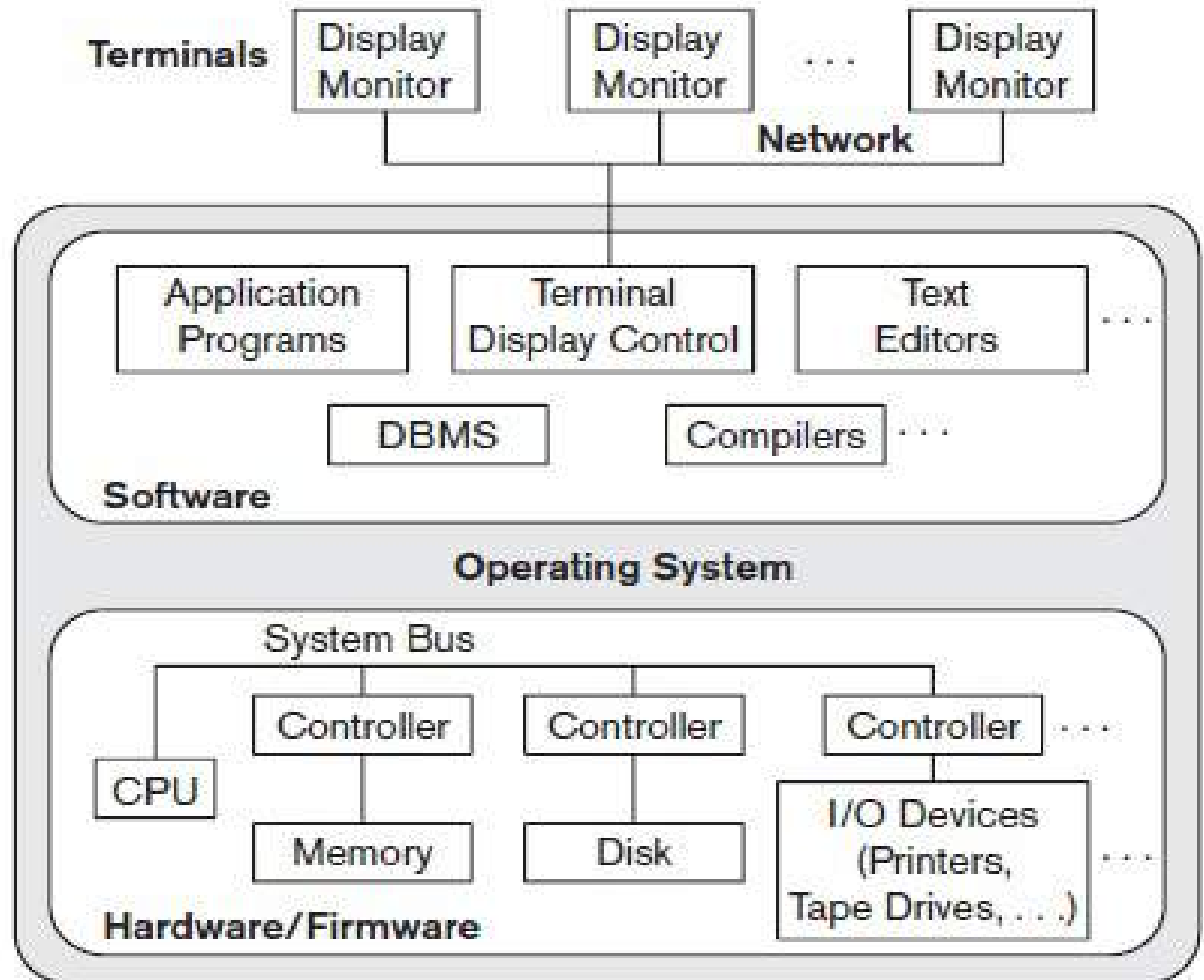
Tools, Application Environments, and Communications Facilities

- **CASE tools** are used in the design phase of database systems. Another tool that can be quite useful in large organizations is an **expanded data dictionary (or data repository) system**.

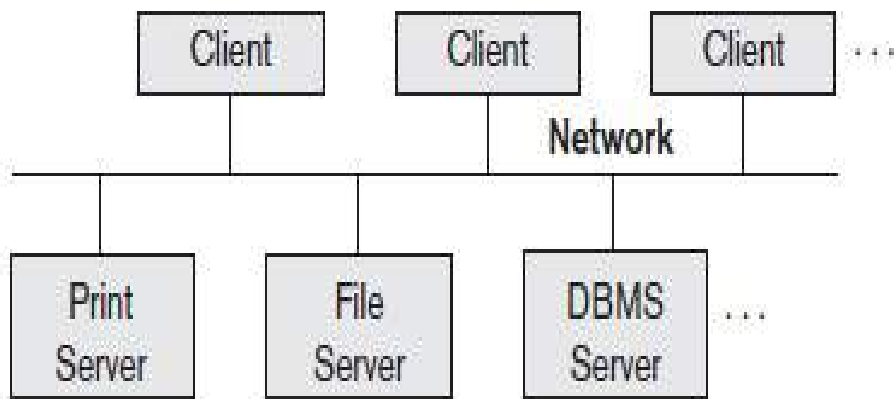
- The DBMS also needs to interface with **communications software**, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers.
 - In addition, some distributed DBMSs are physically distributed over multiple machines. In this case, communications networks are needed to connect the machines. These are often **local area networks (LANs)**, but they can also be other types of networks.

Centralized and Client/Server Architectures for DBMSs

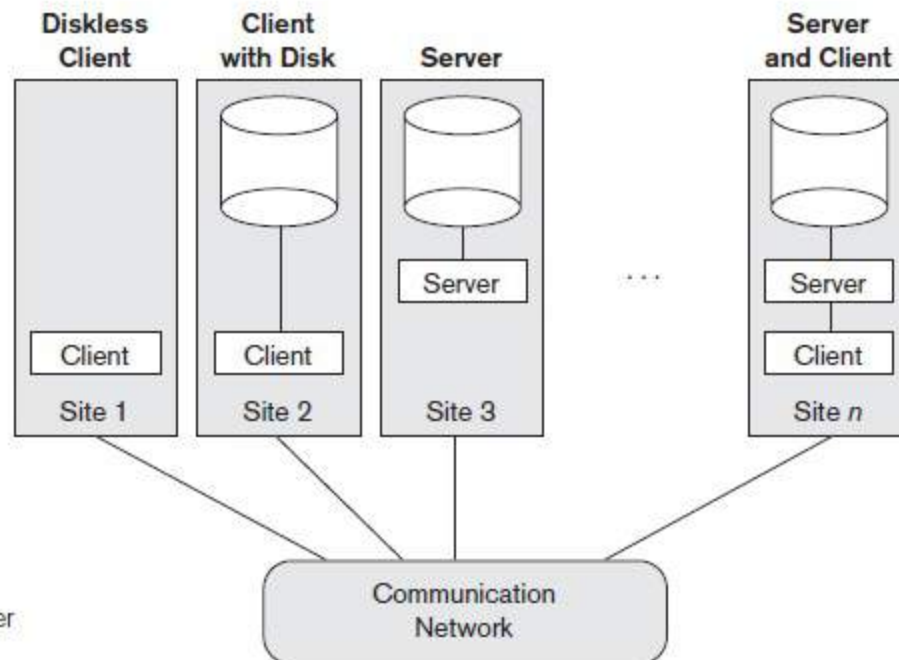
- Architectures for DBMSs have followed trends similar to those for general computer system architectures.
 - Earlier architectures used mainframe computers.
 - main processing for all system functions, including user application programs and user interface programs, as well as all the DBMS functionality.
 - The reason was that most users accessed such systems via computer terminals that did not have processing power and only provided display capabilities.
 - were connected to the central computer via various types of communications networks.



- At first, database systems used these computers similarly to how they had used display terminals, so that the DBMS itself was still a **centralized** DBMS in which all the DBMS functionality, application program execution, and user interface processing were carried out on one machine.
- **C/S:**
 - it is possible to connect a number of PCs or small workstations as clients to a **file server** that maintains the files of the client machines. Another machine can be designated as a **printer server** by being connected to various printers; all print requests by the clients are forwarded to this machine.
 - The **client machines** provide the user with the appropriate interfaces to utilize these servers, as well as with local processing power to run local applications.



Logical two-tier
client/server
architecture.



Physical two-tier client/server
architecture.

- The concept of client/server architecture assumes an underlying framework that consists of many PCs and workstations as well as a smaller number of mainframe machines, connected via LANs and other types of computer networks.
- A **client** in this framework is typically a user machine that provides user interface capabilities and local processing.
- A **server** is a system containing both hardware and software that can provide services to the client machines, such as file access, printing, archiving, or database access.

Two-Tier Client/Server Architectures for DBMSs

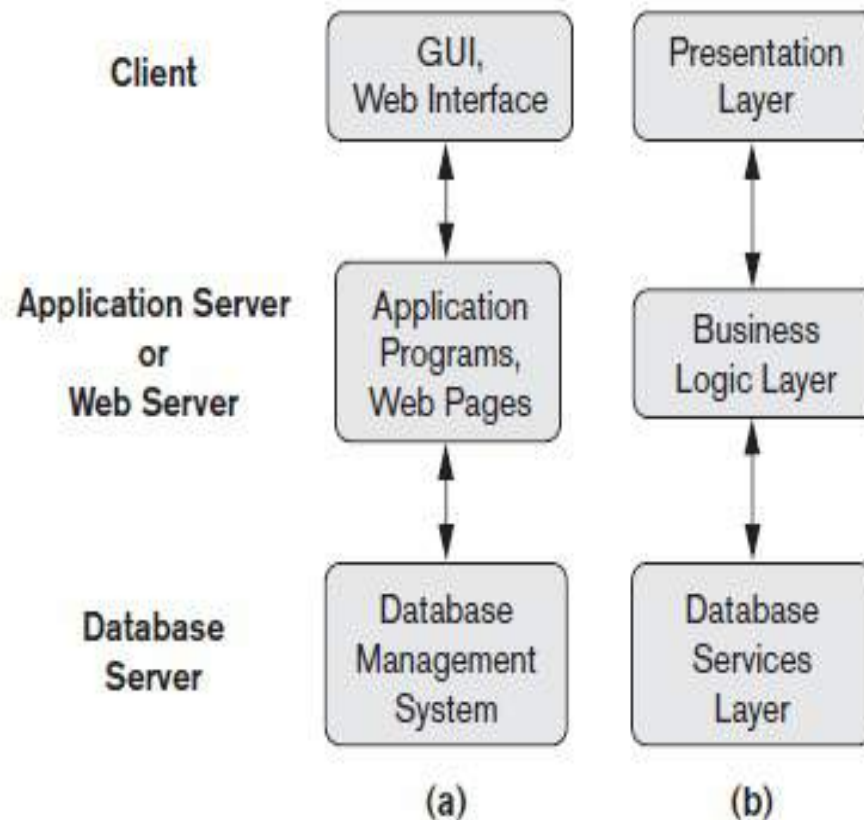
- the query and transaction functionality related to SQL processing remained on the server side. In such an architecture, the server is often called a **query server** or **transaction server** because it provides these two functionalities. In an RDBMS, the server is also often called an **SQL server**.

- The user interface programs and application programs can run on the client side.
- When DBMS access is required, the program establishes a connection to the DBMS (which is on the server side);
- once the connection is created, the client program can communicate with the DBMS.
 - standard called **Open Database Connectivity (ODBC)** provides an **application programming interface (API)**, which allows client-side programs to call the DBMS, as long as both client and server machines have the necessary software installed.
 - JDBC

- The different approach to two-tier client/server architecture was taken by some object-oriented DBMSs, where the software modules of the DBMS were divided between client and server in a more integrated way.
- the **server level** may include the part of the DBMS software responsible for handling data storage on disk pages, local concurrency control and recovery, buffering and caching of disk pages, and other such functions.
- Meanwhile, the **client level** may handle the user interface; data dictionary functions; DBMS interactions with programming language compilers; global query optimization, concurrency control, and recovery across multiple servers; structuring of complex objects from the data in the buffers; and other such functions.

Three-Tier and n-Tier Architectures for Web Applications

- Many Web applications use an architecture called the **three-tier architecture**, which adds an intermediate layer between the client and the database server.



- This intermediate layer or **middle tier** is called the **application server** or the **Web server**, depending on the application. This server plays an intermediary role by running application programs and storing business rules (procedures or constraints) that are used to access data from the database server.
- It can also improve database security by checking a client's credentials before forwarding a request to the database server.
- Steps:
 - The intermediate server accepts requests from the client
 - Processes the request and sends database queries and commands to the database server,
 - then acts as a conduit for passing (partially) processed data from the database server to the clients,
 - where it may be processed further and filtered to be presented to users in GUI format.
- Thus, the *user interface*, *application rules*, and *data access* act as the three tiers.

- It is possible to divide the layers between the user and the stored data further into finer components, thereby giving rise to n -tier architectures, where n may be four or five tiers.
- Typically, the business logic layer is divided into multiple layers. Besides distributing programming and data throughout a network, n -tier applications afford the advantage that any one tier can run on an appropriate processor or operating system platform and can be handled independently.
- Vendors of ERP (enterprise resource planning) and CRM (customer relationship management) packages often use a *middleware layer*, which accounts for the front-end modules (clients) communicating with a number of back-end databases (servers).

- It is possible to divide the layers between the user and the stored data further into finer components, thereby giving rise to n -tier architectures, where n may be four or five tiers.
- Typically, the business logic layer is divided into multiple layers. Besides distributing programming and data throughout a network, n -tier applications afford the advantage that any one tier can run on an appropriate processor or operating system platform and can be handled independently.
- Vendors of ERP (enterprise resource planning) and CRM (customer relationship management) packages often use a *middleware layer*, which accounts for the front-end modules (clients) communicating with a number of back-end databases (servers).

- Advances in encryption and decryption technology make it safer to transfer sensitive data from server to client in encrypted form, where it will be decrypted.

References

- Garcia-Molina, H., Ullman, J. D., & Widom, J. (2009). Database systems: The complete book (2nd ed.). Upper Saddle River, NJ: Prentice Hall.
- Elmasri, R., & Navathe, S. B. (2010). Fundamentals of database systems (6th ed.). Boston: Pearson.



Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

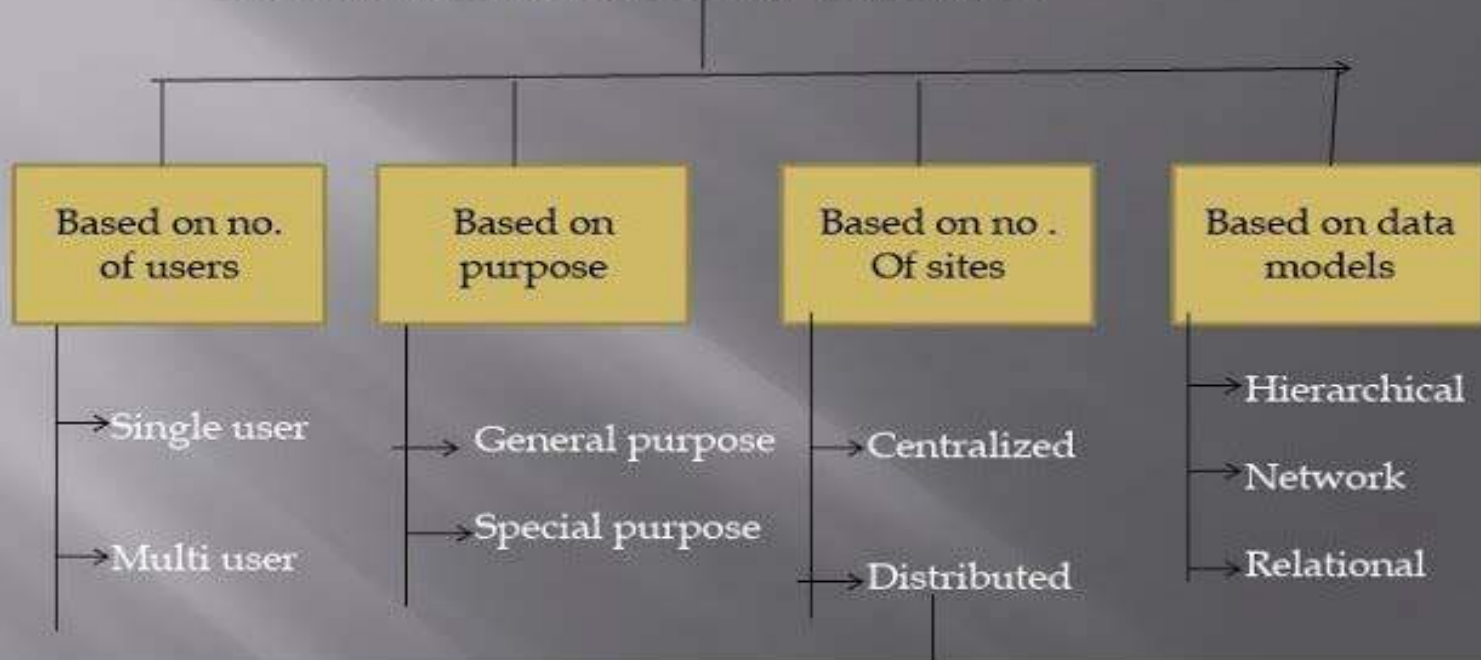
Database Management System (TCS 503) Lecture- 4

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Classification of Database Management Systems

Classification of DBMS



Classification of Database Management Systems

- There are several types of database management systems that are commonly used in the industry. Each type has its own strengths and weaknesses, and choosing the right type depends on the specific needs of the organization.
 - **Relational Database Management Systems**
 - **Hierarchical Database Management Systems**
 - **Network Database Management Systems**
 - **Object-Oriented Database Management Systems**

- Relational databases are the most common type of database management system. They are based on the relational model, which organizes data into tables with rows and columns.
- Hierarchical databases organize data in a tree-like structure, with each record having one parent and multiple children.
- Network databases are similar to hierarchical databases, but they allow records to have multiple parents.
- Object-oriented databases store data as objects, which contain both data and the methods used to manipulate the data.

RDBMS

- A relational database management systems is a type of database management system that stores and manages data in a tabular form.
- It uses a structured query language (SQL) to manipulate the data in the tables. The data is organized into tables, with each table consisting of rows and columns.
- The columns represent the attributes or characteristics of the data, while the rows represent the individual records or instances of the data.
- One of the main advantages of using an RDBMS is its ability to enforce data integrity and consistency through the use of constraints such as primary keys, foreign keys, and unique constraints.

- In an RDBMS, data is stored in tables that are related to each other through common attributes or fields. For example, a customer table may be related to an orders table through a common customer ID field. This allows for efficient querying of the data using SQL, which can retrieve data from multiple tables at once.



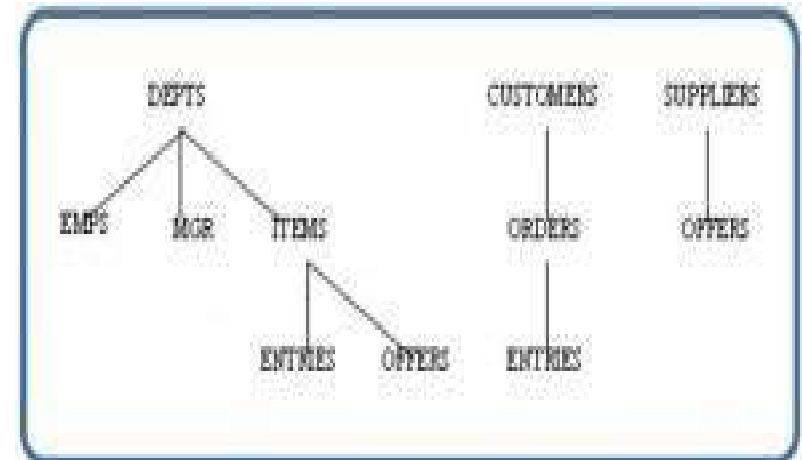
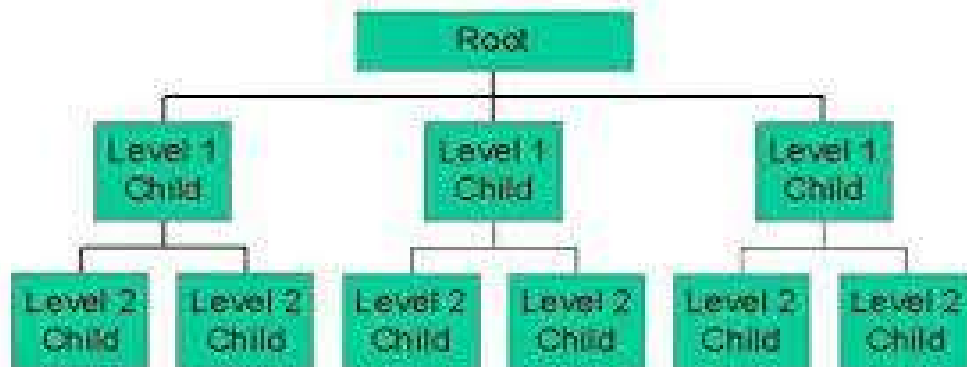
- Another advantage of using an RDBMS is its ability to handle complex transactions involving multiple tables, ensuring that all changes are made atomically and consistently.
- However, one disadvantage is that it can be slower than other types of databases when dealing with large amounts of unstructured data.

Hierarchical Database Management Systems

- A hierarchical database management system is a type of database management system that organizes data in a tree-like structure.
- In this structure, each record has a parent and one or more children, with the exception of the root record, which has no parent.
- This type of system is best suited for applications where data is naturally organized in a hierarchical manner, such as file systems or organizational charts.

- One advantage of using a hierarchical database management system is that it can be very efficient when dealing with large amounts of data that have a clear hierarchy.
- However, one disadvantage is that it can be difficult to modify the structure of the database once it has been established.

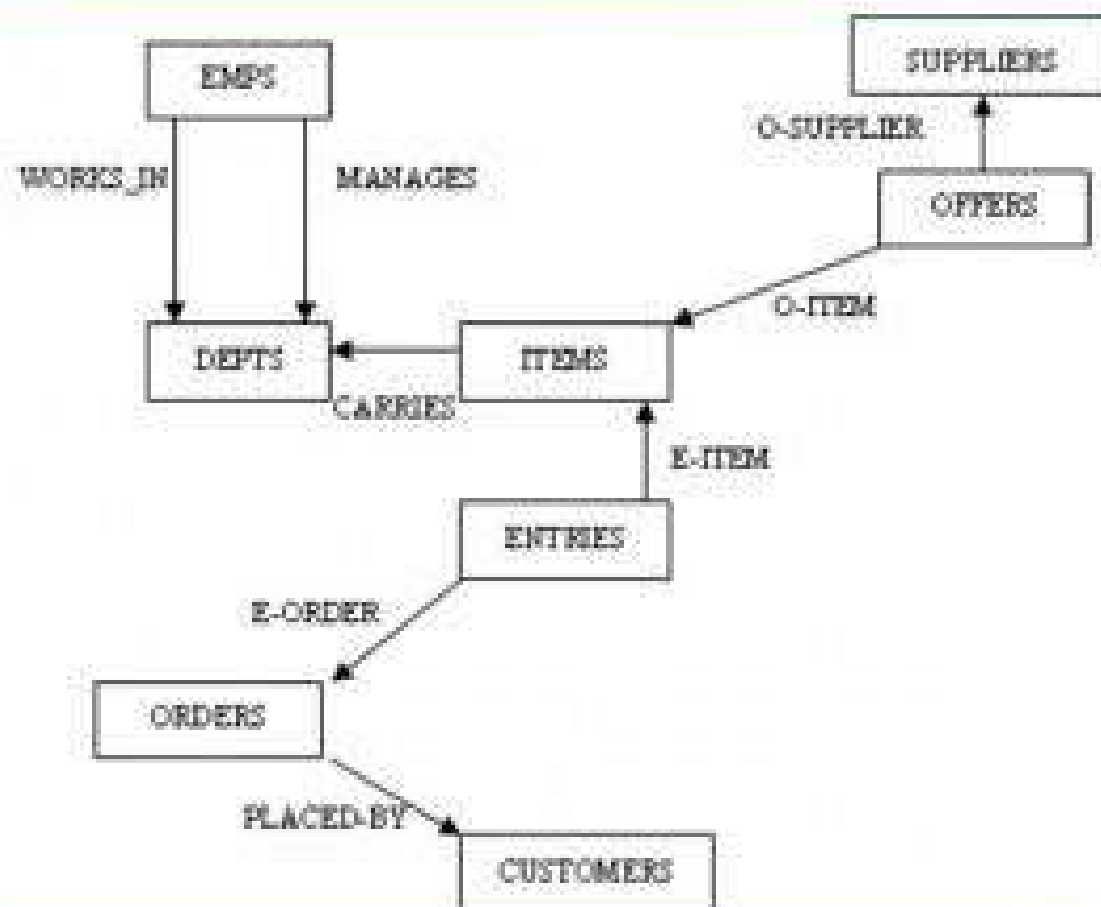
Hierarchical database model



Network Database Management Systems

- A network database management system (NDBMS) is a type of database management system that stores data in a network model. This means that data is organized into records, which are connected to one another through relationships.
- Each record can have multiple parent and child records, creating a complex web-like structure. NDBMSs are often used in large-scale applications where data needs to be accessed quickly and efficiently.

- One advantage of using an NDBMS is its ability to handle complex relationships between data. Because records can have multiple parents and children, it is easier to represent complex relationships than with other types of database management systems.
- However, one disadvantage of using an NDBMS is its complexity. The network model can be difficult to understand and manage, making it challenging for developers and administrators to work with.



Object-Oriented Database Management Systems

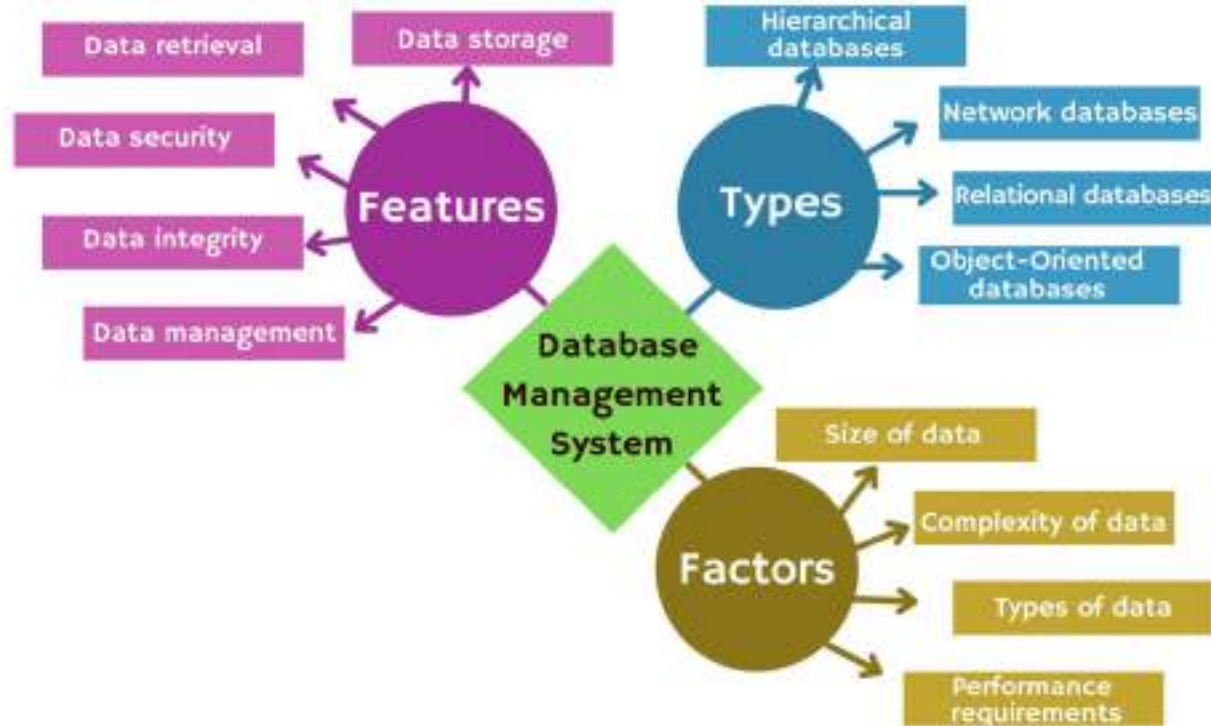
- An object-oriented database management system (OODBMS) is a type of database management system that uses an object-oriented data model.
- In this model, data is represented as objects, which can contain attributes and methods. The main advantage of using an OODBMS is that it allows for more natural representation of complex data structures. This makes it easier to work with data that has many-to-many relationships or hierarchical structures.

Comparison of Database Management Systems

- Relational database management systems (RDBMS) are the most widely used type of database management system. They use a table-based structure to store data and are best suited for applications that require complex queries and transactions.
- Hierarchical database management systems (HDBMS) are structured like a tree, with each record having only one parent. They are best suited for applications that require fast read and write access to large amounts of data.

Comparison of Database Management Systems (CONTINUED)

- Network database management systems (NDBMS) are similar to hierarchical databases, but they allow records to have multiple parents. They are best suited for applications that require complex relationships between data.
- Object-oriented database management systems (OODBMS) store data as objects, which can contain both data and procedures. They are best suited for applications that require complex data structures and support for object-oriented programming languages.



A database model is a way of organizing data in a database. There are several types of database models, each with its own advantages and disadvantages. The most common database models include the relational model, the hierarchical model, the network model, and the object-oriented model.

SQL

- Structured Query Language, commonly known as SQL, is a programming language used to manage and manipulate data in a relational database. SQL allows users to insert, update, delete, and retrieve data from a database in an efficient and organized manner.
- SQL queries are used to perform actions on a database, such as retrieving specific data or updating existing data.
 - Overall, SQL is a powerful tool for managing and manipulating data in a relational database, and its versatility makes it a valuable skill for anyone working with databases.

Database Design

- Considerations that need to be taken into account include scalability, performance, security, and ease of use.
- The goal is to create a database that meets the needs of the organization while also being easy to maintain and update.

Normalization

- Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.
- It involves breaking down a large table into smaller tables that are related to each other through common fields. Normalization ensures that each piece of data is stored only once, which reduces the risk of inconsistencies and errors in the data.

- There are several levels of normalization, with each level building on the previous one.
 - For example, first normal form (1NF) requires that each table have a primary key and that each column contain atomic values.
 - Second normal form (2NF) requires that every non-key attribute be fully dependent on the primary key.
 - Third normal form (3NF) requires that every non-key attribute be dependent only on the primary key and not on any other non-key attributes.
- By following these normalization rules, one can create a well-structured database that is easy to maintain and update.

Data Integrity

- There are several ways to ensure data integrity in a database.
 - One way is to use constraints, such as unique constraints, primary key constraints, and foreign key constraints.
 - These constraints ensure that data is entered correctly and that there are no duplicate entries.
 - Another way to ensure data integrity is through validation checks, which verify that the data entered meets certain criteria.
 - For example, a validation check could ensure that a date entered is within a certain range or that an email address entered is in a valid format.

Database Security

- Database security is a critical aspect of database management systems. It involves protecting the data stored in a database from unauthorized access, modification, and destruction.
 - Authentication and Authorization
 - **Authentication** mechanisms aim to answer the question, "Who are you?" **Authorization** mechanisms aim to answer the question, "What are you allowed to do?"
- There are several measures that can be taken to ensure the security of a database, including implementing access controls, encrypting sensitive data, regularly backing up data, and monitoring database activity for suspicious behavior

Database Management System Trends

- Cloud-based database management systems are becoming increasingly popular due to their flexibility and scalability. With cloud-based systems, data can be accessed from anywhere at any time, making it easier for businesses to manage their data and collaborate with others. In addition, cloud-based systems often have lower upfront costs and require less maintenance than traditional on-premise systems.
- NoSQL databases are another trend in database management systems. These databases are designed to handle unstructured data, such as social media posts and sensor data, which traditional relational databases struggle to manage. NoSQL databases are often used in big data applications, where large volumes of data need to be processed quickly.

Real-World Applications of Database Management Systems

- Database management systems are used in a wide variety of real-world applications, from e-commerce websites to healthcare systems. One common use of database management systems is in e-commerce websites, where they are used to store customer information, product data, and transaction details. This allows businesses to efficiently manage their inventory, process orders, and track customer behavior.
- Social media platforms also rely heavily on database management systems to store user profiles, posts, and interactions. These systems allow for quick and easy retrieval of data, which is essential for providing users with personalized content and recommendations. In the healthcare industry, database management systems are used to store patient records, medical histories, and treatment plans. This helps healthcare providers to deliver more accurate diagnoses and treatments, while also ensuring the privacy and security of patient data.

Challenges of Database Management Systems

- Managing a database management system can come with its fair share of challenges.
 - One of the biggest challenges is ensuring data security. With sensitive information stored in databases, it's important to have measures in place to prevent unauthorized access and protect against cyber attacks. This can include implementing strong passwords, encryption, and firewalls.
- Another challenge is maintaining data integrity.
 - This means ensuring that the data in the database is accurate, consistent, and reliable. To address this challenge, regular backups and data validation checks can be performed. Additionally, having clear data entry guidelines and training for employees can help prevent errors and inconsistencies.

Components

- A database management system (DBMS) consists of several components. Each component plays very important role in the database management system environment. The major components of database management system are:
 - Software
 - Hardware
 - Data
 - Procedures
 - Users (Application Programmers, DBA, End Users)

Data Dictionary

- A data dictionary is a centralized repository of information about data such as meaning, relationships to other data, origin, usage, and format. A data dictionary is used in the development of a relational database system.
- A data dictionary stores metadata that defines and describes data so that it can be easily understood by anyone who would like to use it or analyze it at a later date.

References

- Garcia-Molina, H., Ullman, J. D., & Widom, J. (2009). Database systems: The complete book (2nd ed.). Upper Saddle River, NJ: Prentice Hall.
- Elmasri, R., & Navathe, S. B. (2010). Fundamentals of database systems (6th ed.). Boston: Pearson.



Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 5

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Referential Integrity constraints in DBMS

- Referential Integrity is a concept in database management systems (DBMS) that ensures the consistency and validity of relationships between tables in a relational database.
- It involves maintaining the accuracy and integrity of data by enforcing certain rules regarding how data in one table can reference data in another table.

Here are the key concepts associated with Referential Integrity constraints:

- **CASCADE:** When a DELETE or UPDATE operation is performed on a primary key value that is referenced by foreign keys in other tables, the CASCADE option allows the DBMS to automatically perform corresponding actions on the related records in the referencing tables. For example, if a referenced record is deleted, all related records in other tables can be automatically deleted as well.

- The primary purpose of these constraints is to maintain the integrity and correctness of data, preventing actions that could lead to inconsistencies or errors.
 - **Primary Key Constraint:** A primary key is a unique identifier for a record in a table. It ensures that each record is unique and can be used as a reference in other tables. When a primary key is defined in a table, it automatically enforces Referential Integrity because any references to this primary key must point to valid, existing records.
 - **Foreign Key Constraint:** A foreign key is a column (or columns) in one table that is linked to the primary key in another table. It establishes a relationship between two tables and ensures that values in the foreign key column(s) correspond to valid primary key values in the referenced table. This prevents orphaned or dangling records, where a record references a non-existent record in another table.



Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 6

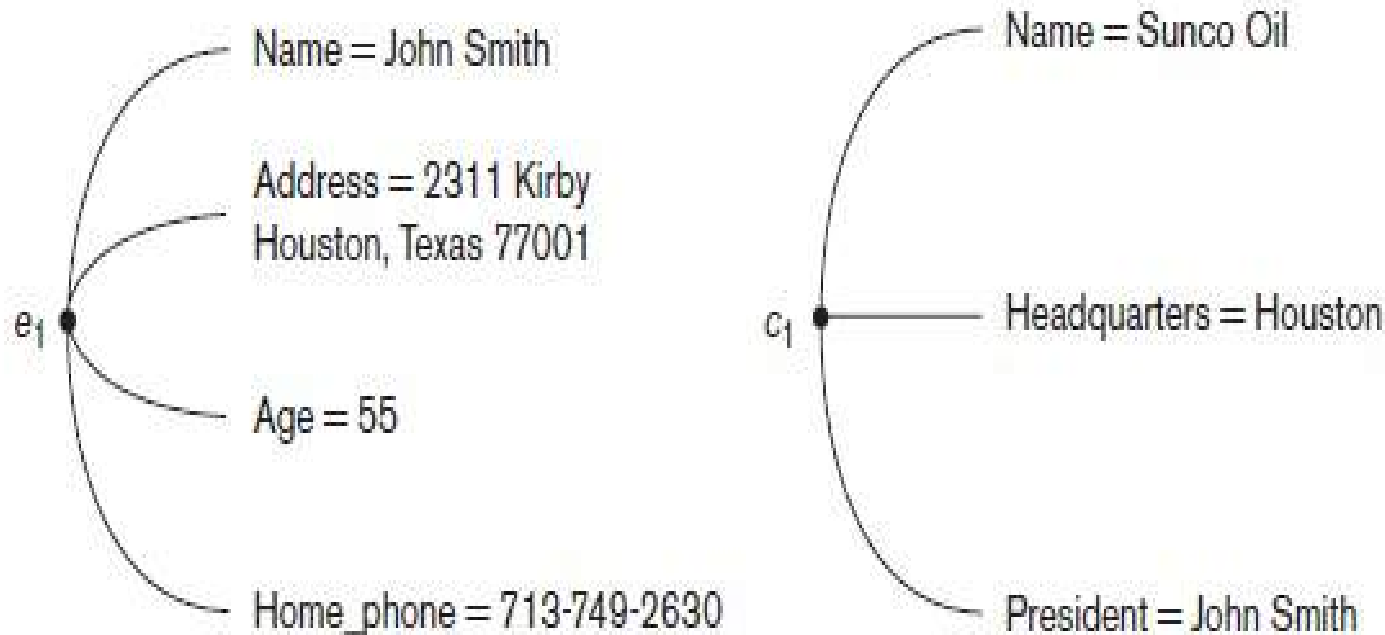
B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

E-R MODEL

- The Entity-Relationship (E-R) model is a conceptual data modeling technique used in database design to visually represent the structure of a database system.
- The ER model describes data as *entities*, *relationships*, and *attributes*.
- **Entities and Their Attributes.** The basic object that the ER model represents is an **entity**, which is a *thing* in the real world with an independent existence.

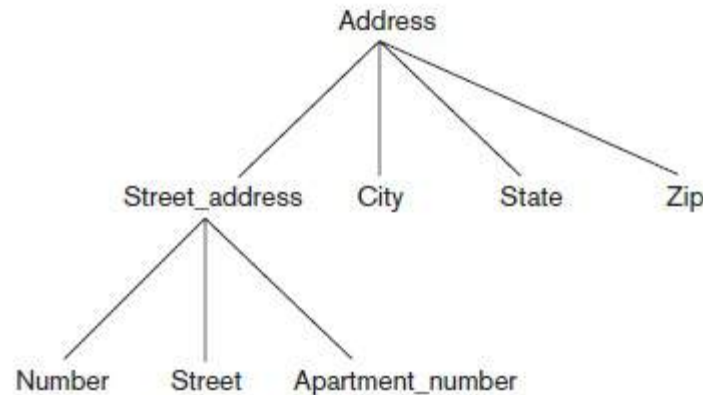
- Figure below shows two entities and the values of their attributes. (e_1 and c_1)



- An entity may be an object with a physical existence (for example, a particular person, car, house, or employee) or it may be an object with a conceptual existence (for instance, a company, a job, or a university course).
- Each entity has **attributes**—the particular properties that describe it.
 - For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.

Composite versus Simple (Atomic) Attributes.

- **Composite attributes** can be divided into smaller subparts, which represent more basic attributes with independent meanings. For example, the Address attribute of the EMPLOYEE entity shown in Figure can be subdivided into Street_address, City, State, and Zip, with the values '2311 Kirby', 'Houston', 'Texas', and '77001.'



Single-Valued versus Multivalued Attributes.

- Most attributes have a single value for a particular entity; such attributes are called **single-valued**. For example, Age is a single-valued attribute of a person.
- In some cases an attribute can have a set of values for the same entity—for instance, a Colors attribute for a car, or College_degrees attribute for a person.
 - Cars with one color have a single value, whereas two-tone cars have two color values. Similarly, one person may not have a college degree, another person may have one, and a third person may have two or more degrees; therefore, different people can have different *numbers of values* for the College_degrees attribute. Such attributes are called **multivalued**.

- A multivalued attribute may have lower and upper bounds to constrain the *number of values* allowed for each individual entity. For example, the Colors attribute of a car may be restricted to have between one and three values, if we assume that a car can have three colors at most.

Stored versus Derived Attributes.

- In some cases, two (or more) attribute values are related—for example, the Age and Birth_date attributes of a person. For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date. The Age attribute is hence called a **derived attribute** and is said to be **derivable from** the Birth_date attribute, which is called a **stored attribute**.

NULL Values

- In some cases, a particular entity may not have an applicable value for an attribute.
 - College_degrees attribute applies only to people with college degrees. For such situations, a special value called NULL is created.

Relationships

- Relationships represent associations between entities. They indicate how two or more entities are related to each other in the real world.
- Relationships can have various degrees, such as one-to-one, one-to-many, or many-to-many. For example, a "Student" entity might be related to a "Course" entity through an "Enrollment" relationship, which signifies that students are enrolled in courses.

Key Attributes

- Key attributes are attributes that uniquely identify an entity within an entity set. They are used to ensure the uniqueness of entities and are essential for building relationships between entities.

Weak Entities

- Weak entities are entities that cannot be uniquely identified by their attributes alone and depend on a strong entity (owner entity) for their identification. They have a partial key and are usually represented with double rectangles.

“E-R modeling is a crucial step in the database design process as it helps to organize and visualize the structure of the data, allowing designers to create a clear blueprint for the database schema before the actual implementation in a DBMS (Database Management System).”

Cardinality

- Cardinality describes the number of occurrences of one entity that can be associated with a single occurrence of another entity in a relationship.
- Cardinality in the context of the Entity-Relationship (E-R) model refers to the numerical relationships between entities in a database.

Cardinality (Concept)

- The cardinality of a set refers to the number of distinct elements or members within that set. The cardinality of a set A is denoted by $|A|$.
 - If you have a set $A = \{1, 2, 3, 4, 5\}$, then the cardinality of set A is $|A| = 5$, because there are five distinct elements in the set.
 - If you have a set $B = \{\text{apple}, \text{banana}, \text{orange}\}$, then the cardinality of set B is $|B| = 3$, because there are three distinct fruits in the set.
 - The cardinality of an empty set ($\emptyset$) is 0, because it has no elements.

Examples

- Cardinality of power set of $\{0,1,2,\dots,20\}$
 - The cardinality of the power set of a set A with n elements is 2^n .
 - In your case, the set $A = \{0, 1, 2, \dots, 20\}$ has 21 elements (from 0 to 20).
 - Therefore, the cardinality of the power set $P(A)$ is 2^{21} , which is equal to **2097152**.
 - So, $|P(A)| = 2^{21} = 2097152$.

Example	Cardinality
$A = \{5\}$	$ A = 1$
$B = \{7, 2\}$	$ B = 2$
$C = \{1, 3, 4\}$	$ C = 3$
$D = \{9, 1, 5, 8\}$	$ D = 4$
$E = \{5, 5, 5, 5, 5\}$	$ E = 1$

- cardinality of $\{1, 2, 3, 4, 4, 5, 6, 7, 7, 8, 9, 9, 9\}$
 - The set $\{1, 2, 3, 4, 4, 5, 6, 7, 7, 8, 9, 9, 9\}$ contains 9 distinct elements: 1, 2, 3, 4, 5, 6, 7, 8, and 9.
 - The repeated elements (like 4, 7, and 9) are counted only once in terms of the set's cardinality, as sets only contain distinct elements. Therefore, the cardinality of this set is 9.
 - Mathematically: $|\{1, 2, 3, 4, 4, 5, 6, 7, 7, 8, 9, 9, 9\}| = 9$.

- **One-to-One (1:1) Cardinality:** In a one-to-one relationship, each instance of one entity is associated with exactly one instance of another entity, and vice versa.
 - For example, in a "Person" and "Passport" relationship, each person can have only one passport, and each passport belongs to only one person.

Example of 1:1

- A 1:1 (one-to-one) relationship, also known as a bijection, is a type of relationship between two sets in which each element in the first set is uniquely paired with exactly one element in the second set, and vice versa.
- Example to illustrate a 1:1 relationship:
 - Consider two sets: Set A and Set B.
 - Set A: {apple, banana, orange, grape, cherry}
 - Set B: {red, yellow, orange, purple, red}

- Let's say we establish a relationship between the elements of Set A and Set B based on the colors of the fruits.
- We can see that:
 - "apple" in Set A corresponds to "red" in Set B.
 - "banana" in Set A corresponds to "yellow" in Set B.
 - "orange" in Set A corresponds to "orange" in Set B.
 - "grape" in Set A corresponds to "purple" in Set B.
 - "cherry" in Set A corresponds to "red" in Set B.
- In this example, every element in Set A is paired with a unique element in Set B, and vice versa. There is no repetition or ambiguity in the pairing. This is a 1:1 relationship (bijection) between the sets.

- **One-to-Many (1:N) Cardinality:** In a one-to-many relationship, each instance of one entity can be associated with multiple instances of another entity, while each instance of the other entity is associated with at most one instance of the first entity.

-

A 1:M (one-to-many) relationship is a type of relationship between two sets where each element in the first set can be associated with multiple elements in the second set, but each element in the second set is associated with only one element in the first set.

- For instance, in a "Department" and "Employee" relationship, each department can have many employees, but each employee belongs to only one department.

Example

- example using a "Department" and "Employee" scenario
 - Consider two sets: Set A and Set B.
 - Set A: {HR Department, Sales Department, Marketing Department}
 - Set B: {Employee 1, Employee 2, Employee 3, Employee 4, Employee 5}
- In this example, Set A represents different departments within a company, and Set B represents employees. Each department can have multiple employees, but each employee works in only one department.
- This represents a 1:M relationship.

– For instance:

- HR Department can have Employee 1 and Employee 2.
 - Sales Department can have Employee 3 and Employee 4.
 - Marketing Department can have Employee 5.
-
- In this 1:M relationship, one department can have multiple employees, but each employee belongs to only one department.
 - This type of relationship is common in real-world scenarios such as organizational structures, where a single entity (department) has multiple associated entities (employees).

- **Many-to-Many (N:M) Cardinality:** In a many-to-many relationship, each instance of one entity can be associated with multiple instances of another entity, and vice versa. This type of relationship requires an intermediary or associative entity to bridge the connection between the two entities.
 - For example, a "Student" and "Course" relationship might involve an "Enrollment" entity to represent the fact that each student can enroll in multiple courses, and each course can have multiple students.

- Here's an example using a "Student" and "Course" scenario:
 - Consider two sets: Set A and Set B.
 - Set A: {Alice, Bob, Carol, David}
 - Set B: {Math, Science, History, Art}
- In this example, Set A represents students and Set B represents courses. Since students can enroll in multiple courses and each course can have multiple students, we need an intermediary to manage the relationship.

- To establish the N:M relationship, we introduce a junction table (sometimes called an enrollment table) that connects students with courses:
 - Junction Table (Enrollments):
 - Alice is enrolled in Math and Science.
 - Bob is enrolled in Science and History.
 - Carol is enrolled in Math.
 - David is enrolled in Art.
- Here, Alice is associated with both Math and Science courses, and Science is associated with both Alice and Bob. This demonstrates a many-to-many relationship because multiple students can be associated with multiple courses.
- This type of relationship often requires additional data structures, like the junction table in this example, to properly model the connections between the sets.

Difference between composite and multi-valued attributes in ER Diagram

- A composite attribute is an attribute that can be further divided into smaller sub-parts, each representing a simpler attribute with independent meanings. Composite attributes are depicted as ovals connected to the main entity rectangle by a dotted line.
 - For example, consider an entity "Address" with composite attributes like "Street," "City," and "Postal Code." Each of these sub-parts has its own individual meaning and can be understood independently. Composite attributes are used to represent a more structured piece of information.

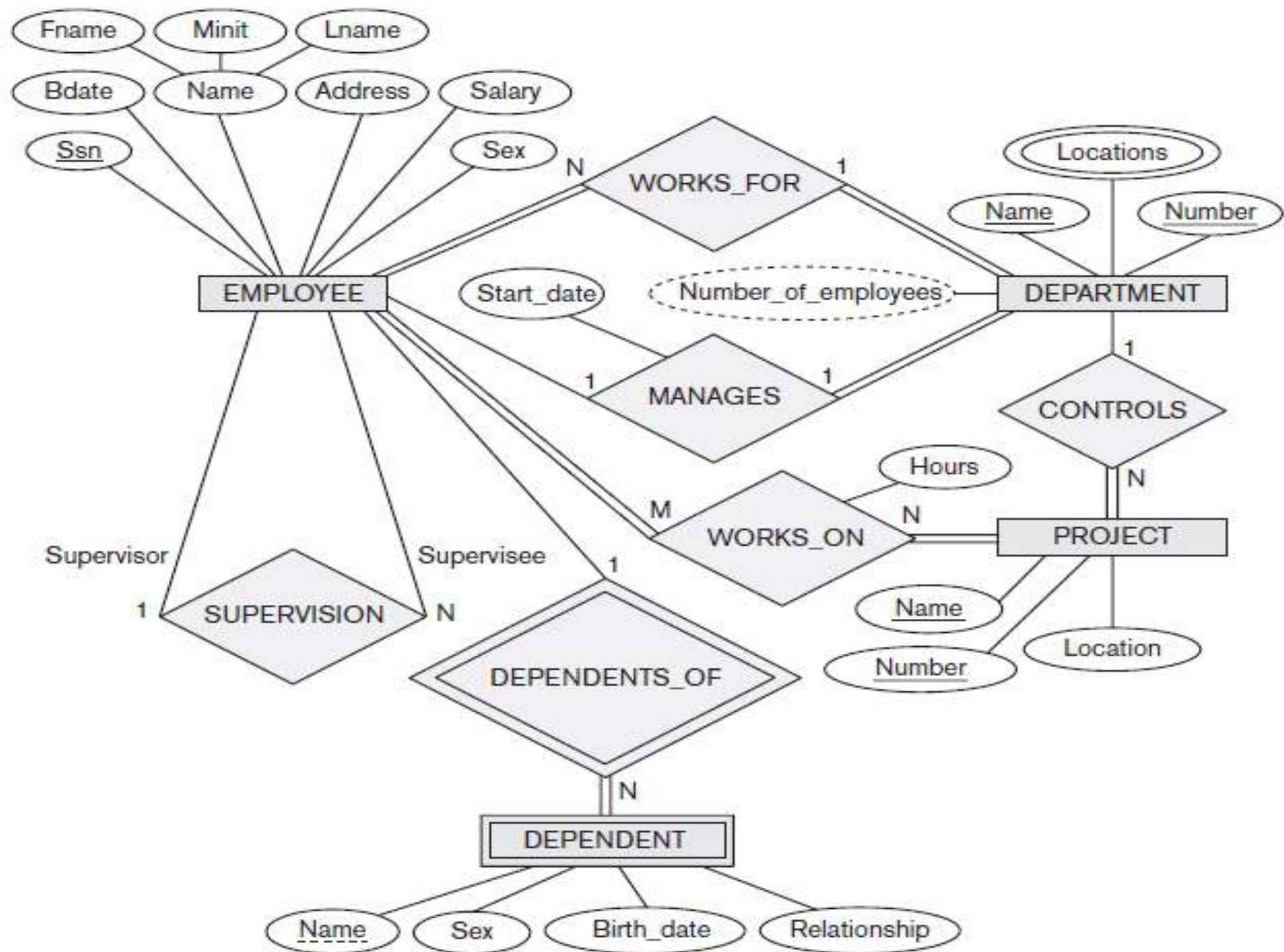
- A multivalued attribute is an attribute that can hold multiple values for a single entity occurrence. It's used when an entity can possess more than one value for a specific attribute. Multivalued attributes are depicted as double ovals connected to the main entity rectangle by a dotted line.
 - For instance, consider an entity "Person" with a multivalued attribute "Phone Numbers." A person can have multiple phone numbers associated with them, and each of these phone numbers is distinct but related to the same entity.

A Sample DB Application

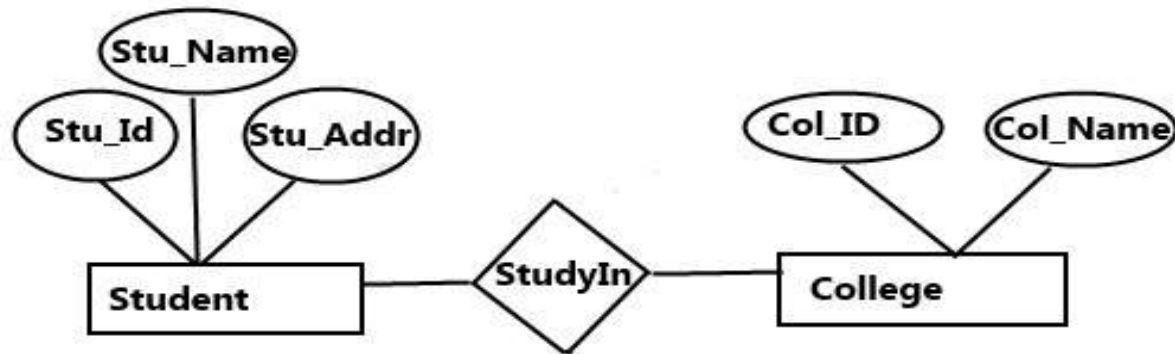
- The company is organized into departments.
- Each department has a unique name, a unique number, and a particular employee who manages the department.
- Keeping track of the start date when that employee began managing the department.
- A department may have several locations.

- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- Storing each employee's name, Social Security number, address, salary, (gender), and birth date.
- An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department.

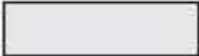
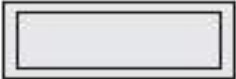
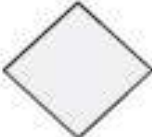
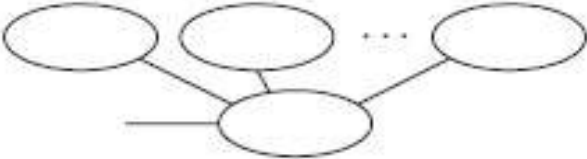
- keeping track of the current number of hours per week that an employee works on each project.
- keeping track of the direct supervisor of each employee (who is another employee).
- keeping track of the dependents of each employee for insurance purposes.
- keeping each dependent's first name, sex, birth date, and relationship to the employee.



- In the following diagram we have two entities Student and College and their relationship.
- The relationship between Student and College is many to one as a college can have many students however a student cannot study in multiple colleges at the same time.
- Student entity has attributes such as Stu_Id, Stu_Name & Stu_Addr and College entity has attributes such as Col_ID & Col_Name.

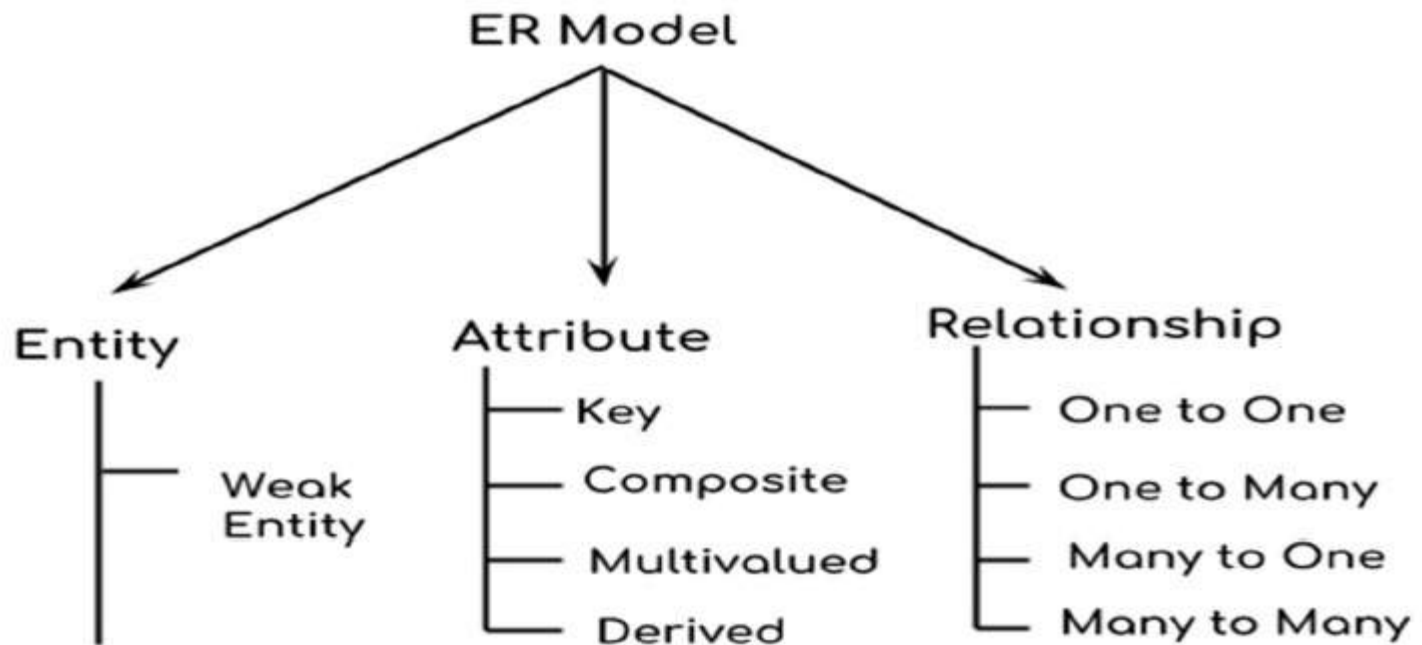


- **Rectangle:** Represents Entity sets.
- **Ellipses:** Attributes
- **Diamonds:** Relationship Set
- **Lines:** They link attributes to Entity Sets and Entity sets to Relationship Set
- **Double Ellipses:** Multivalued Attributes
- **Dashed Ellipses:** Derived Attributes
- **Double Rectangles:** Weak Entity Sets

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute

- An **identifying relationship** is a type of relationship in a relational database model where a weak entity depends on a strong entity for its existence and is uniquely identified by the combination of its attributes and the attributes of the associated strong entity. In other words, the weak entity cannot exist without the strong entity, and its identity is tied to the strong entity it's related to.
- The identifying relationship ensures that the combination of attributes from both the strong and weak entities forms a composite key that uniquely identifies each instance of the weak entity.

Components of a ER Diagram



Weak Entity

- An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle. A weak entity, in the context of a relational database, is an entity that cannot be uniquely identified by its own attributes alone.
- For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.
- A weak entity is a type of entity which doesn't have its key attribute. It can be identified uniquely by considering the primary key of another entity. For that, weak entity sets need to have participation.
- A weak entity type normally has a partial key, which is the attribute that can uniquely identify weak entities that are related to the same owner entity.



- In a modeling scenario, a weak entity doesn't have its own primary key, but it has a partial key that, in combination with the primary key of the associated strong entity, forms a unique identifier for the weak entity.
- For example, consider a database for a library.
 - The "Book" entity might be a strong entity with attributes like ISBN, title, and publication year.
 - However, a "Copy" entity could be a weak entity, as each copy of a book isn't uniquely identifiable by its own attributes.
 - Copies depend on the book they belong to for identification.
 - The combination of the book's ISBN and a copy number could be used as a composite key to uniquely identify copies.

Entity Types, Entity Sets, Keys, and Value Sets

- ER model describes data in terms of:
 - Entities and entity sets
 - Objects
 - Relationships and relationship sets
 - Connections between objects
 - Attributes
 - Properties that characterize or describe entities or relationships

Entity Types and Entity Sets

- A database usually contains groups of entities that are similar.
- For example, a company employing hundreds of employees may want to store similar information concerning each of the employees.
- These employee entities share the same attributes, but each entity has its *own value(s)* for each attribute.
- An **entity type** defines a *collection* (or *set*) of entities that have the same attributes.

En

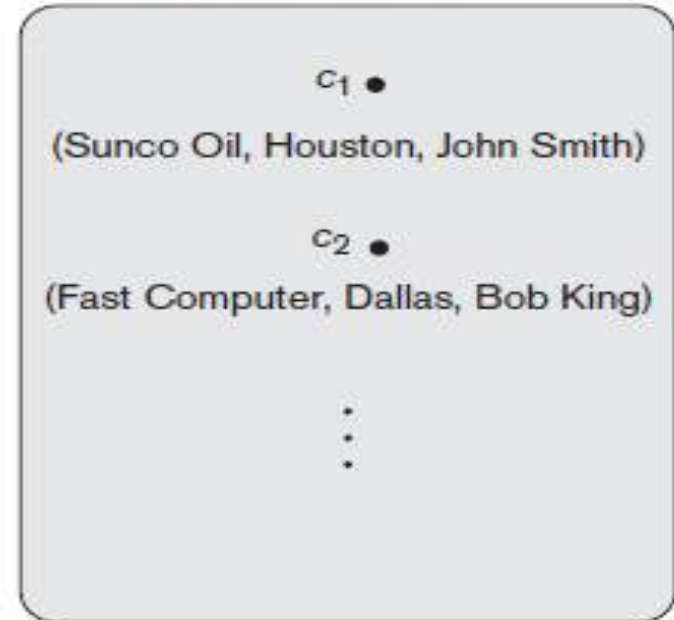
Entity Type Name:

EMPLOYEE

COMPANY

Name, Age, Salary

Name, Headquarters, President

En
(EEntity Set:
(Extension)

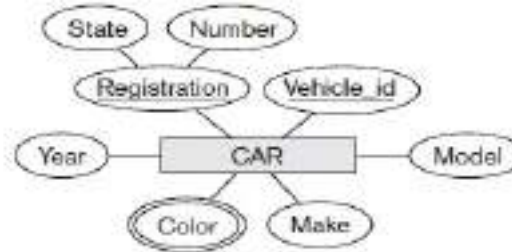
- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set**;
- The entity set is usually referred to using the same name as the entity type.
- For example, EMPLOYEE refers to both a *type of entity* as well as the current set of *all employee entities* in the database.
- An entity type describes the **schema** for a *set of entities* that share the same structure.

Key Attributes of an Entity Type

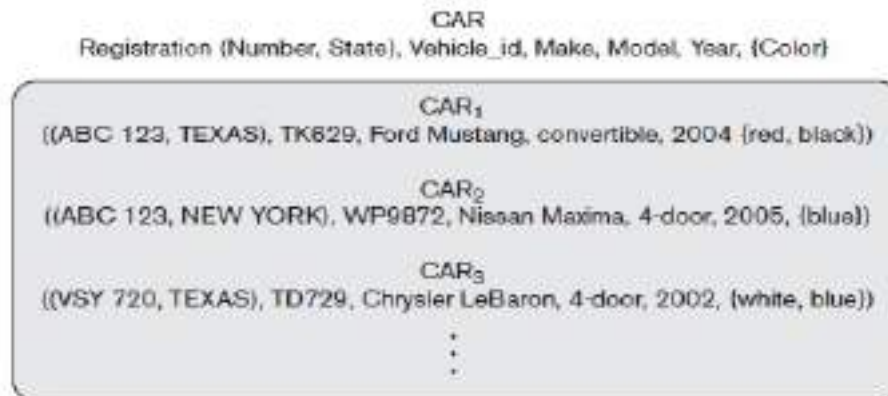
- An important constraint on the entities of an entity type is the **key** or **uniqueness constraint** on attributes.
- An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.

The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation, (b) Entity set with three entities.

(a)



(b)



- Some entity types have *more than one* key attribute.
- For example, each of the ***Vehicle_id and Registration*** attributes of the entity type CAR is a key in its own right. The Registration attribute is an example of a composite key formed from two simple component attributes, *State and Number*, neither of which is a key on its own.
- An entity type may also have *no key*, in which case it is called a ***weak entity type***

- An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.
- For example, the Name attribute is a key of the COMPANY entity type because no two companies are allowed to have the same name.

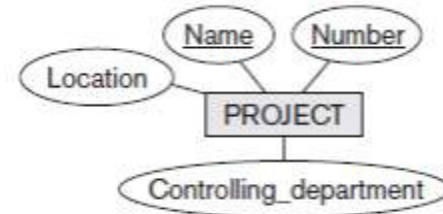
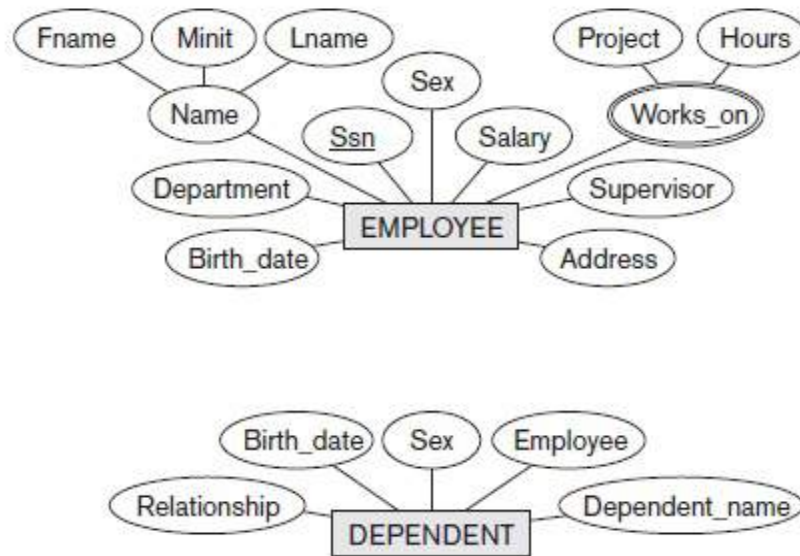
Value Sets (Domains) of Attributes

- Each simple attribute of an entity type is associated with a **value set** (or **domain** of values), which specifies the set of values that may be assigned to that attribute for each individual entity.
- We refer to the value of attribute A for entity e as $A(e)$.
- Mathematically, an attribute A of entity set E whose value set is V can be defined as a **function** from E to the power set $P(V)$ of V :
 - $A : E \rightarrow P(V)$

Initial Conceptual Design of the COMPANY Database

- Value sets are not displayed in ER diagrams, and are typically specified using the basic **data types** available in most programming languages, such as integer, string, Boolean, and so on.
- An entity type DEPARTMENT with attributes Name, Number, Locations, Manager, and Manager_start_date.
- Locations is the only multivalued attribute. We can specify that both Name and Number are (separate) key attributes because each was specified to be unique.
- An entity type PROJECT with attributes Name, Number, Location, and Controlling_department. Both Name and Number are (separate) key attributes.

- An entity type EMPLOYEE with attributes Name, Ssn, Sex, Address, Salary, Birth_date, Department, and Supervisor.
- Both Name and Address may be composite attributes; however, this was not specified in the requirements. We must go back to the users to see if any of them will refer to the individual components of Name—First_name, Middle_initial, Last_name—or of Address.
- An entity type DEPENDENT with attributes Employee, Dependent_name, Sex, Birth_date, and Relationship (to the employee).



- Four entity types
- Most attributes are simple, single-valued, and stored
 - Works_on and Locations are multivalued
 - Employee's Name is composite
- Employee has one key
- department and project have two keys,
- Dependent has none

Relationship Types, Relationship Sets, Roles, and Structural Constraints

- whenever an attribute of one entity type refers to another entity type, some relationship exists.
- For example,
 - the attribute Manager of DEPARTMENT refers to an employee who manages the department;
 - the attribute Controlling_department of PROJECT refers to the department that controls the project.
 - the attribute Supervisor of EMPLOYEE refers to another employee (the one who supervises this employee);
- **Relationship Sets:**
 - In the ER model, these references should not be represented as attributes but as **relationships**,
 - A relationship set is a collection of similar relationships.
 - It is an instance of a relationship type and represents a set of relationships between instances of entity types.
 - For example, if you have an entity type "Student" and an entity type "Course," the relationship set between them could be "Enrollment," representing the relationships where students are enrolled in courses.

Structural Constraints

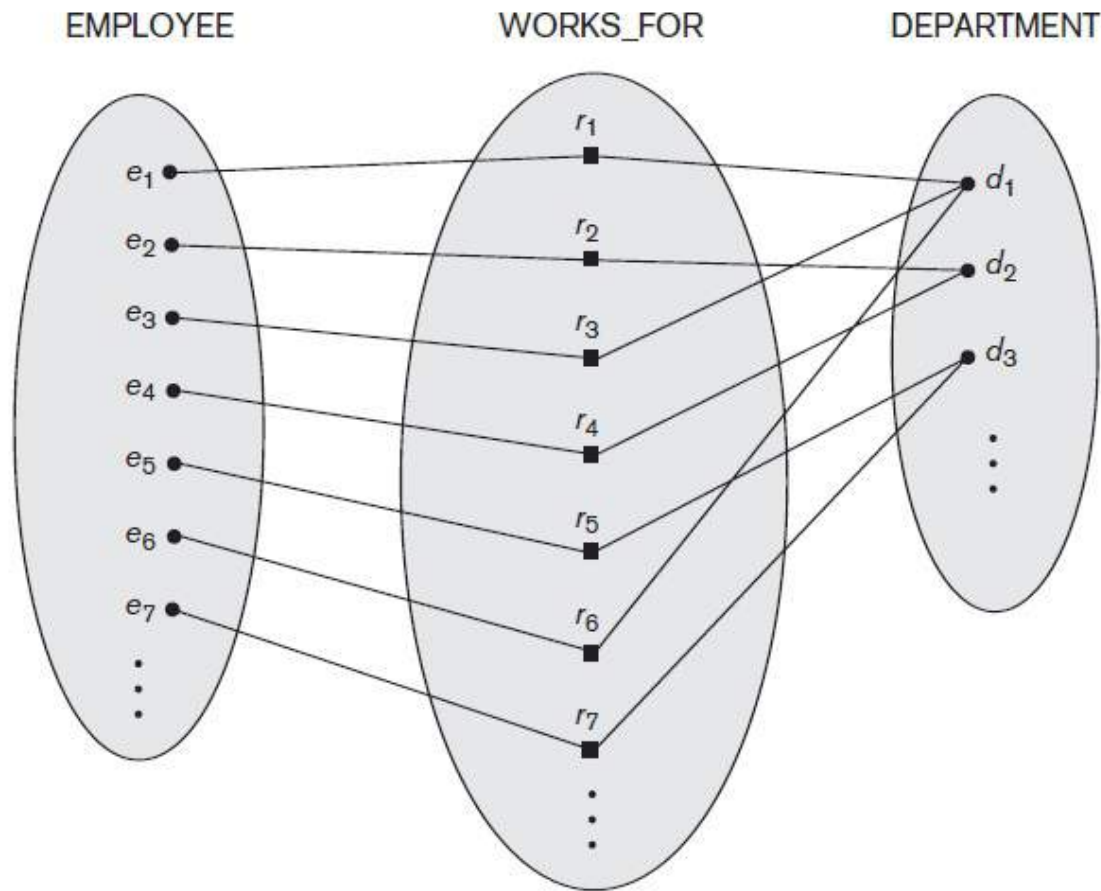
Structural constraints, also known as cardinality ratios, define the number of instances of one entity type that can be associated with the number of instances of another entity type through a specific relationship. These constraints help define the minimum and maximum participation of entities in a relationship. The common cardinality ratios are:

- **One-to-One (1:1):** Each instance of one entity is related to only one instance of another entity, and vice versa.
- **One-to-Many (1:N):** Each instance of one entity is related to multiple instances of another entity, but each instance of the latter is related to only one instance of the former.
- **Many-to-One (N:1):** The opposite of "One-to-Many"; each instance of one entity is related to only one instance of another entity, but each instance of the latter can be related to multiple instances of the former.
- **Many-to-Many (N:N):** Multiple instances of one entity can be related to multiple instances of another entity.

Relationship

- A **relationship type** R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations— or a **relationship set**—among entities from these entity types.
- Mathematically, the relationship set R is a set of **relationship instances** r_i , where each r_i associates n individual entities $(e_1, e_2, \dots, e_n)$, and each entity e_j in r_i is a member of entity set E_j , $1 \leq j \leq n$.

- For example, consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department for which the employee works in the corresponding entity set.
- Each relationship instance in the relationship set WORKS_FOR associates one EMPLOYEE entity and one DEPARTMENT entity.

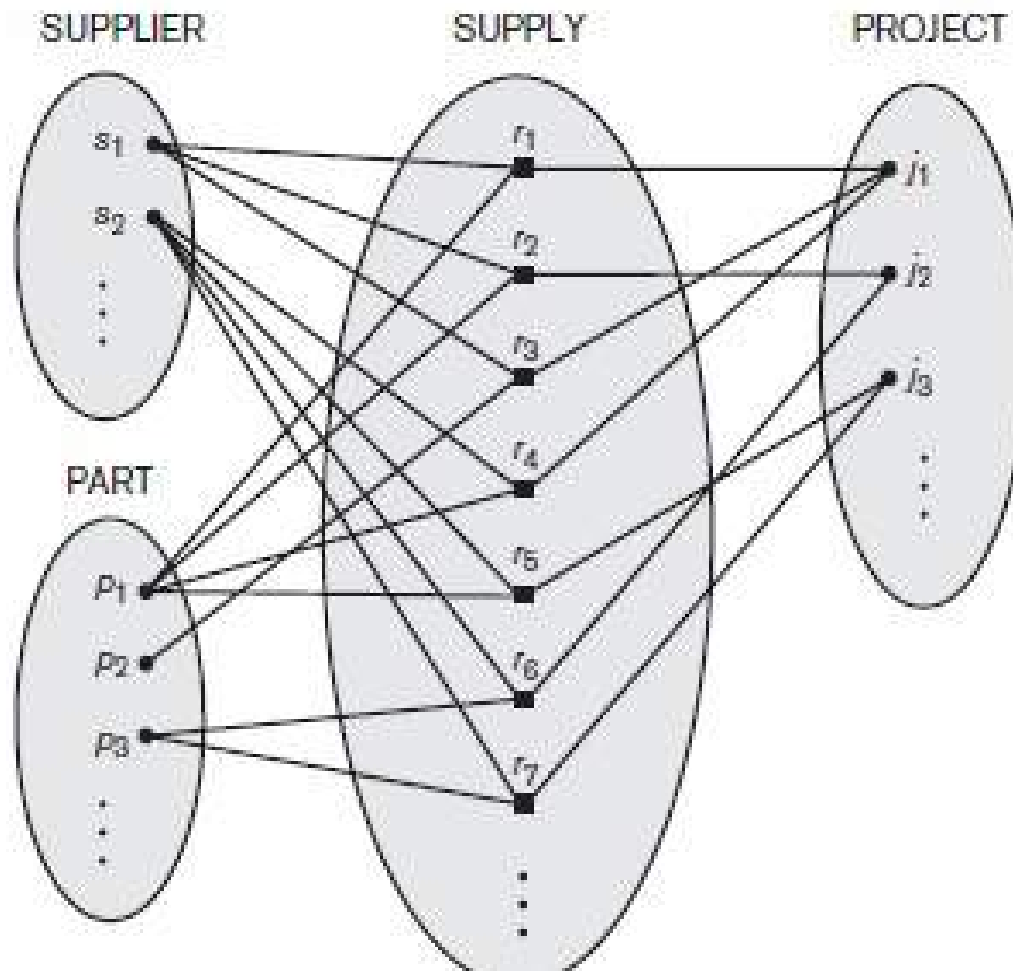


In the Figure , employees e_1, e_3 , and e_6 work for department d_1 ; employees e_2 and e_4 work for department d_2 ; and employees e_5 and e_7 work for department d_3 .

Relationship Degree

- **Degree of a Relationship Type.** The **degree** of a relationship type is the number of participating entity types. Hence, the WORKS_FOR relationship is of degree two.
- A relationship type of degree two is called **binary**, and one of degree three is called **ternary**.

An example of a ternary relationship is SUPPLY, shown in Figure below, where each relationship instance ri associates three entities—a supplier s , a part p , and a project j —whenever s supplies part p to project j . Relationships can generally be of any degree, but the ones most common are binary relationships. Higher degree relationships are generally more complex than binary relationships.



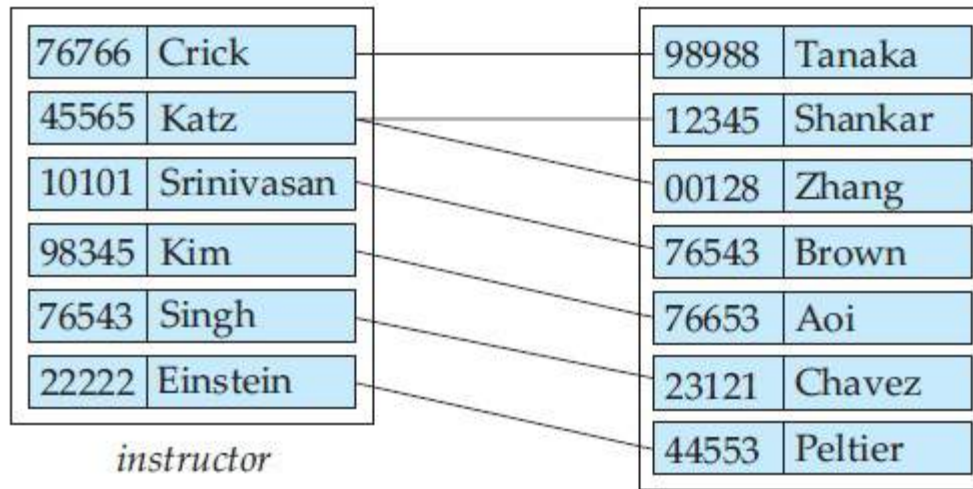
Entity Set (Instructor, student), Relationship set (Advisor)

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

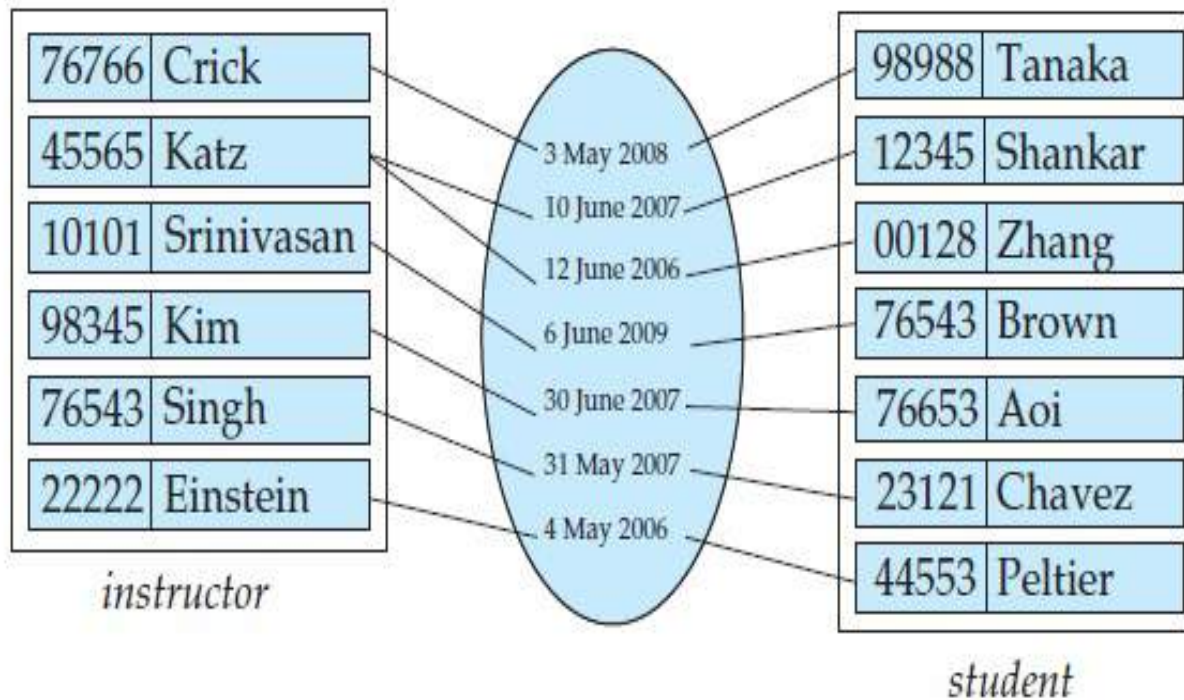
98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student



instructor

student



Constraints on Binary Relationship Types

- Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set.

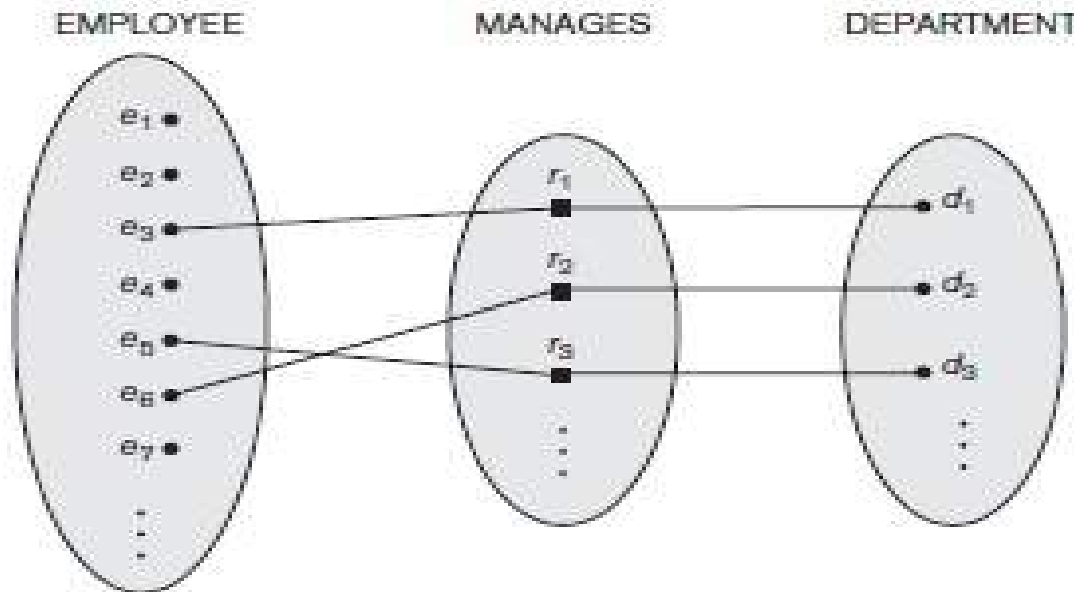
weak entity V/S Strong Entity

Strong Entity Set	Weak Entity Set
Strong entity set always has a primary key.	It does not have enough attributes to build a primary key.
It is represented by a rectangle symbol.	It is represented by a double rectangle symbol.
It contains a Primary key represented by the underline symbol.	It contains a Partial Key which is represented by a dashed underline symbol.
The member of a strong entity set is called as dominant entity set.	The member of a weak entity set called as a subordinate entity set.
Primary Key is one of its attributes which helps to identify its member.	In a weak entity set, it is a combination of primary key and partial key of the strong entity set.
In the ER diagram the relationship between two strong entity set shown by using a diamond symbol.	The relationship between one strong and a weak entity set shown by using the double diamond symbol.
The connecting line of the strong entity set with the relationship is single.	The line connecting the weak entity set for identifying relationship is double.

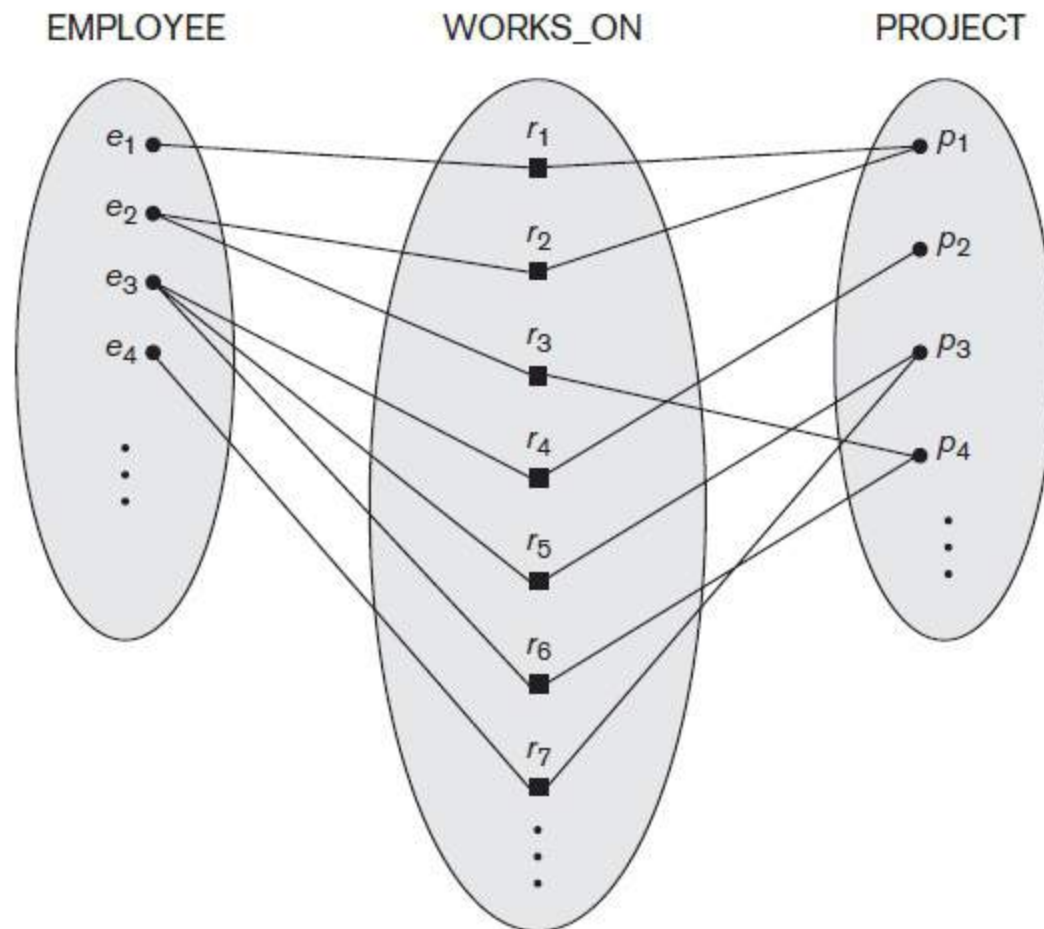
- **Cardinality Ratios for Binary Relationships:**
 - The **cardinality ratio** for a binary relationship specifies the *maximum* number of relationship instances that an entity can participate in.
 - For example, in the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio 1:N, meaning that each department can be related to (that is, employs) any number of employees, but an employee can be related to (work for) only one department.

- This means that for this particular relationship WORKS_FOR, a particular department entity can be related to any number of employees (N indicates there is no maximum number). On the other hand, an employee can be related to a maximum of one department.
- The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N.

- An example of a 1:1 binary relationship is MANAGES, which relates department entity to the employee who manages that department.
- This represent, employee can manage one department only and a department can have one manager only.



- The relationship type WORKS_ON is of cardinality ratio M:N, because an employee can work on several projects and a project can have several employees.



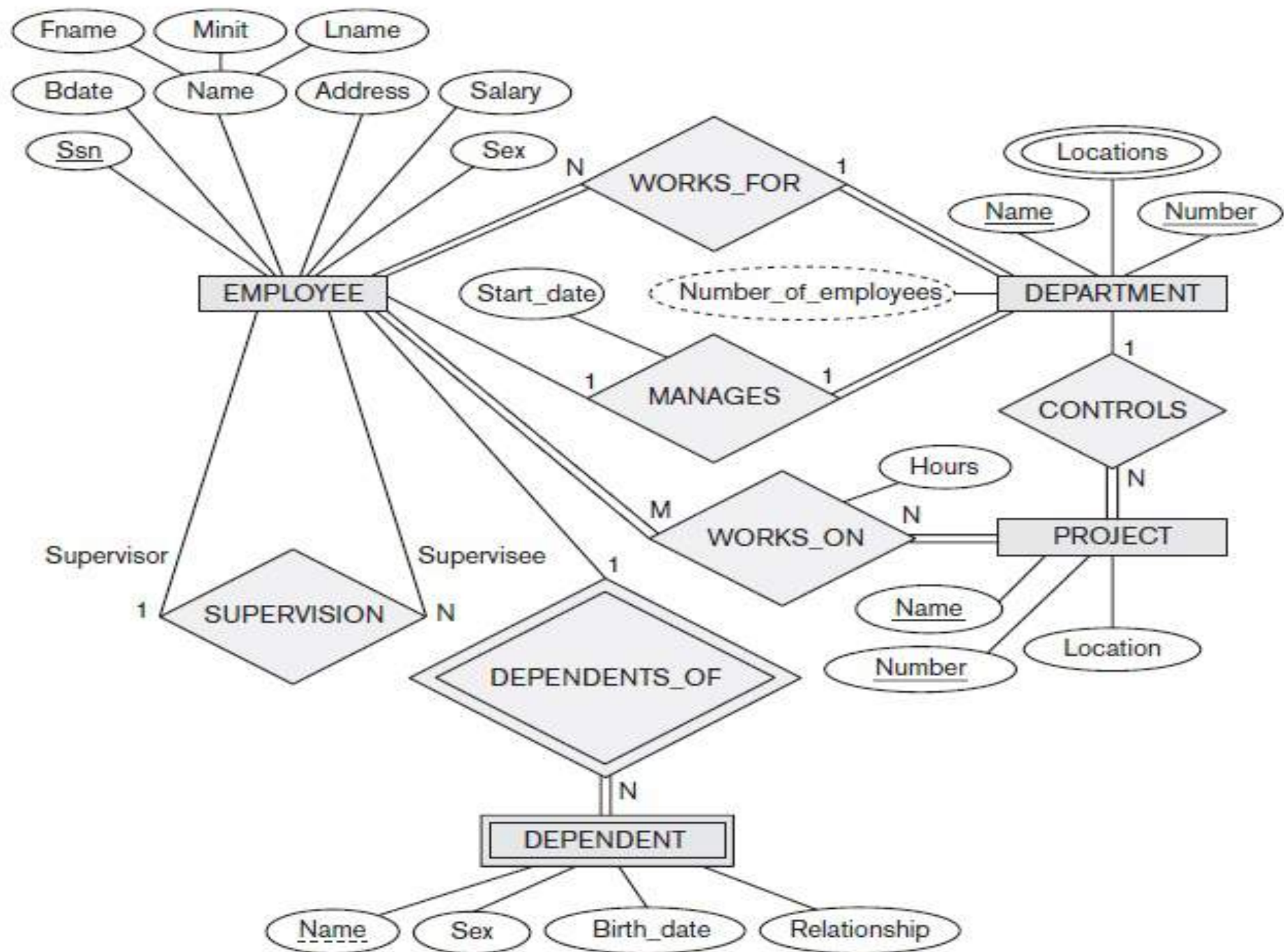
- Cardinality ratios for binary relationships are represented on ER diagrams by displaying 1, M, and N on the diamonds.

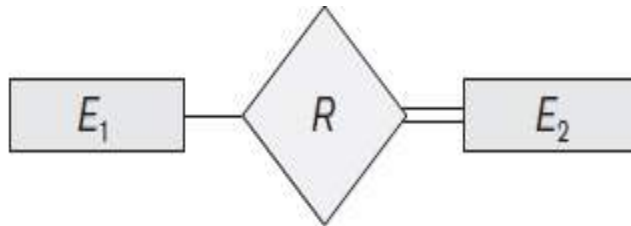
Participation Constraints and Existence Dependencies

- The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type.
- This constraint specifies the *minimum* number of relationship instances that each entity can participate in, and is sometimes called the **minimum cardinality constraint**.

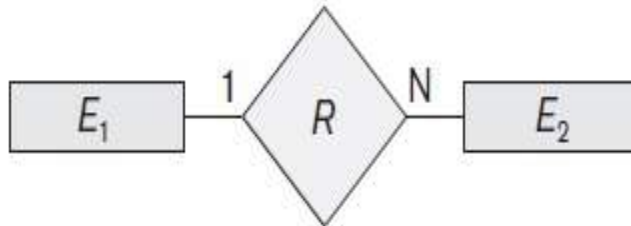
- There are two types of participation constraints—total and partial.
- Example: If a company policy states that *every* employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance. Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation**.
- meaning that every entity in *the total set* of employee entities must be related to a department entity via WORKS_FOR.
- Total participation is also called **existence dependency**.

- Now, we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**, meaning that *some* or *part of the set of* employee entities are related to some department entity via MANAGES, but not necessarily all.
- In ER diagrams, total participation (or existence dependency) is displayed as a ***double line*** connecting the participating entity type to the relationship, whereas partial participation is represented by a ***single line***.
- A weak entity type always has a *total participation constraint* (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity.

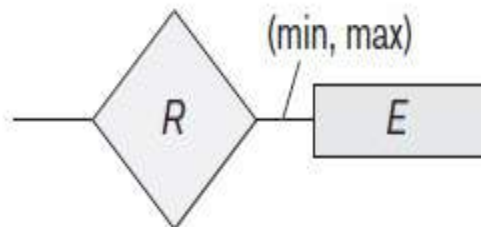




Total Participation of E_2 in R



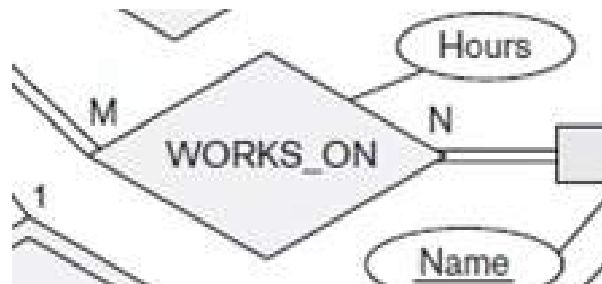
Cardinality Ratio 1: N for $E_1:E_2$ in R



Structural Constraint (min, max)
on Participation of E in R

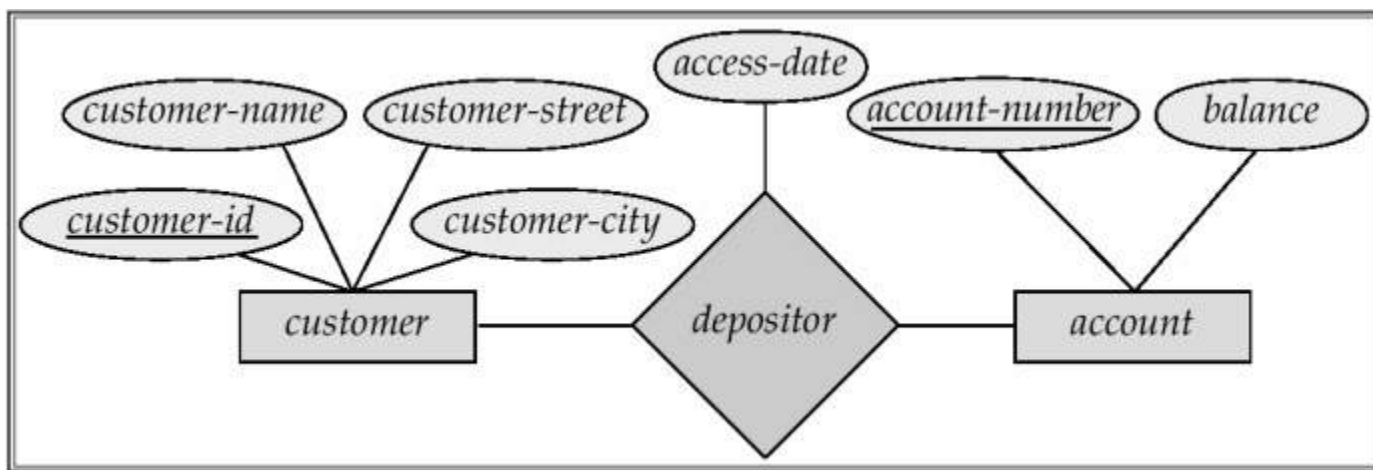
Attributes of Relationship Types

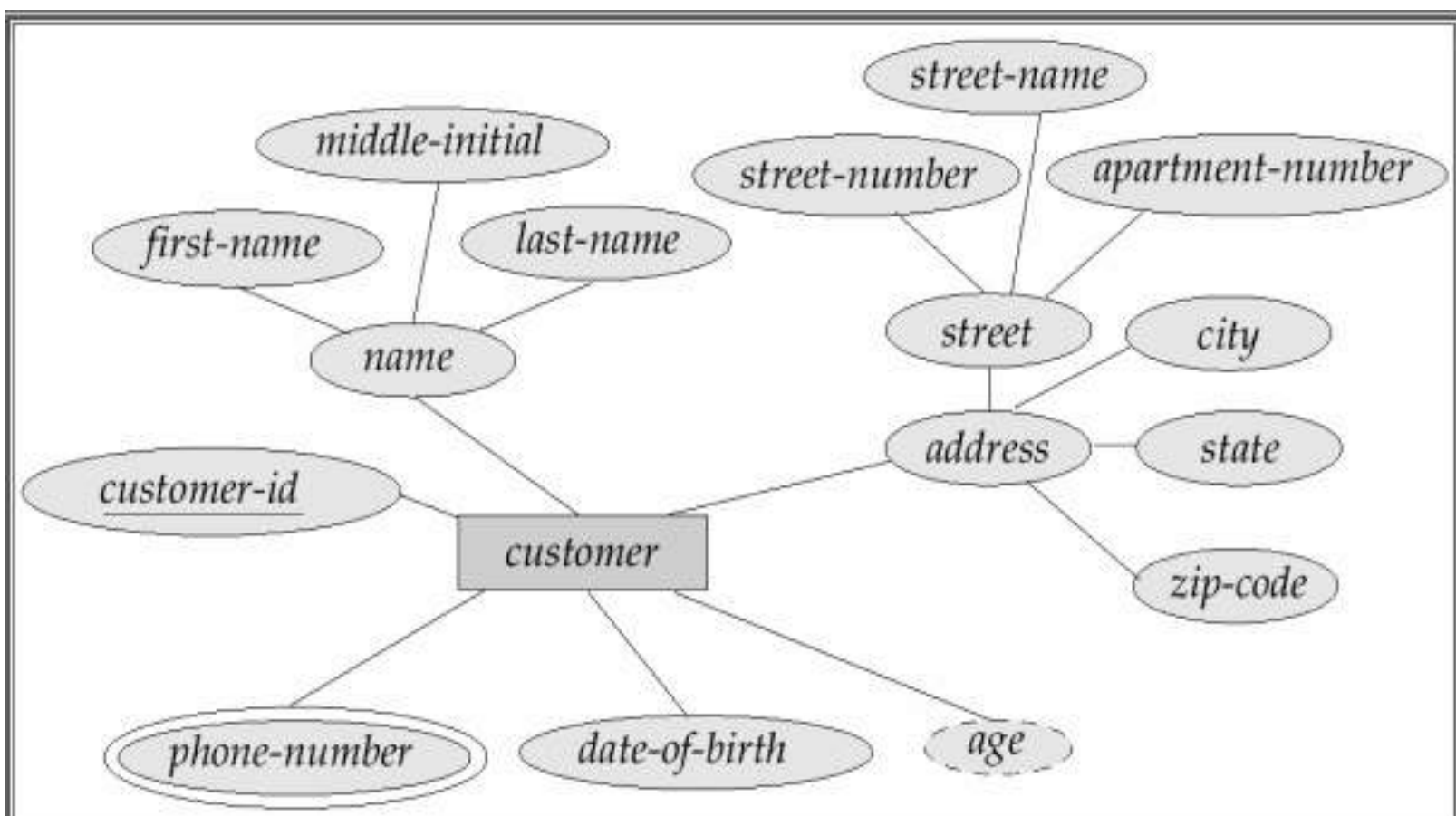
- Relationship types can also have attributes, similar to those of entity types. For example, to record the number of hours per week that an employee works on a particular project, we can include an attribute Hours for the WORKS_ON relationship type in Figure.



Relationship Types of Degree Higher than Two

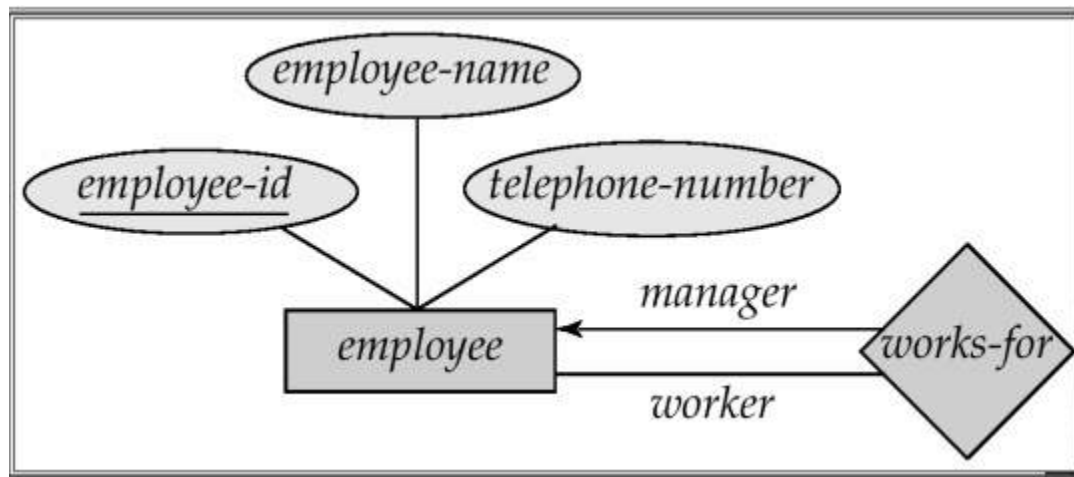
- It is defined the **degree** of a relationship type as the number of participating entity types and called a relationship type of degree two *binary* and a relationship type of degree three *ternary*.





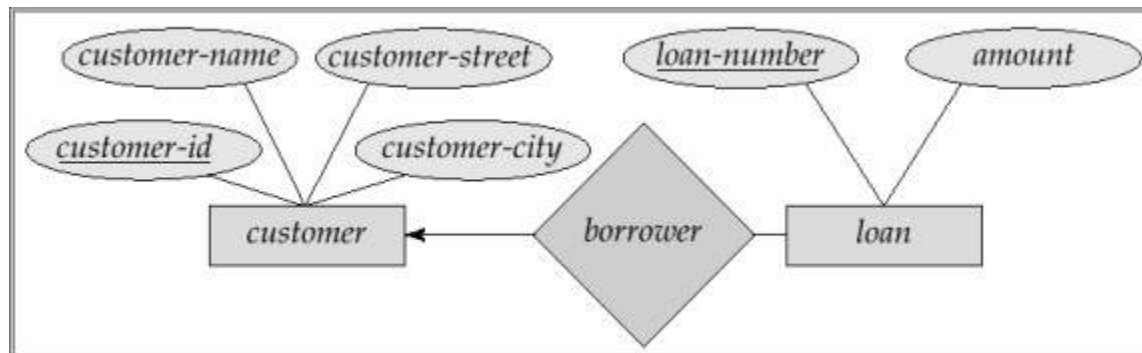
Roles

- Entity sets of a relationship need not be distinct.
- The labels “manager” and “worker” are called roles; they specify how employee entities interact via the works-for relationship set.
- Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- Role labels are optional, and are used to clarify semantics of the relationship

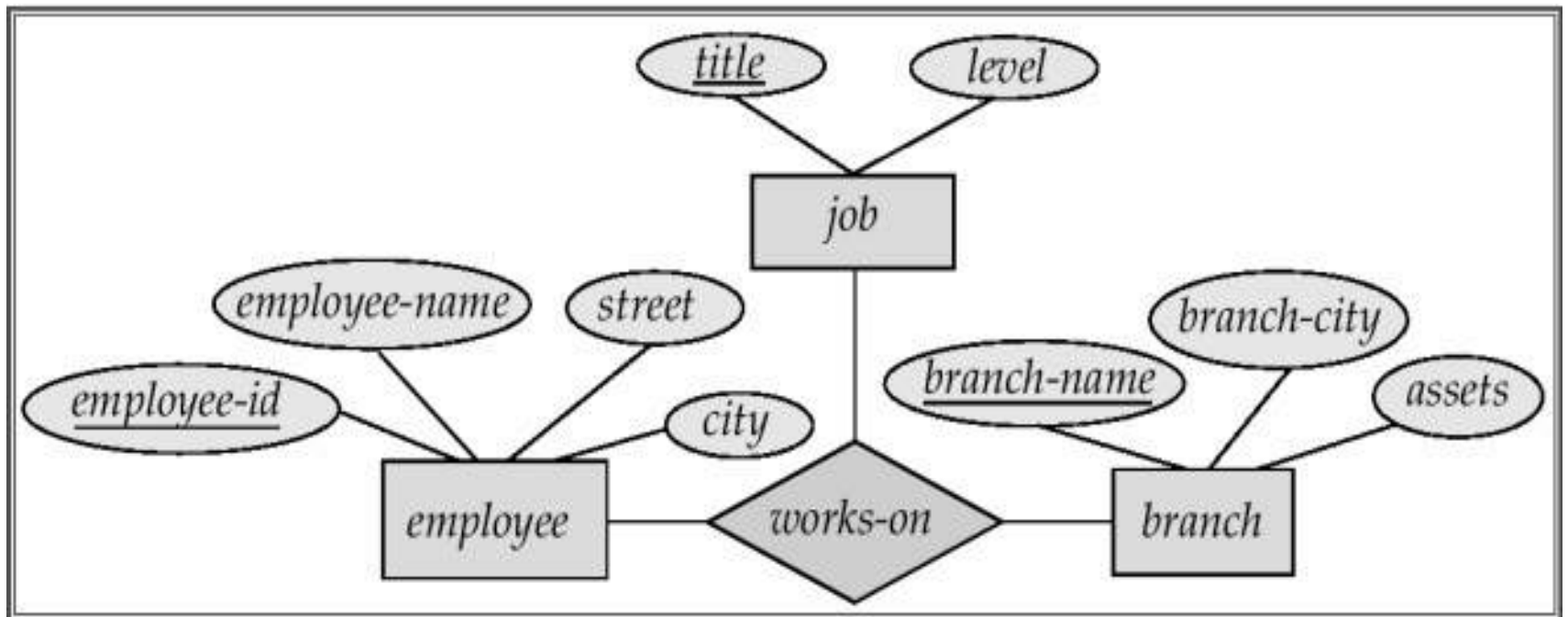


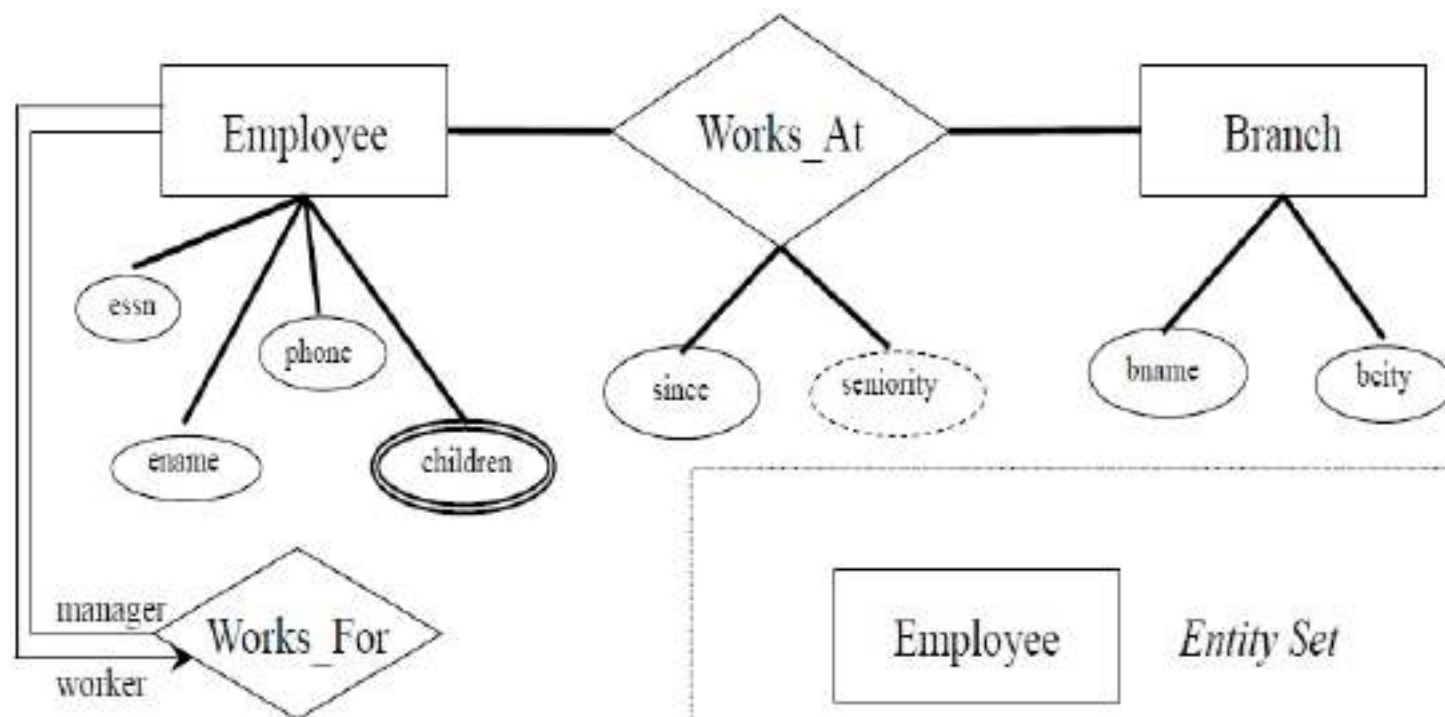
One to many relationship:

In the one-to-many relationship a loan is associated with at most one customer via borrower, a customer is associated with several (including 0) loans via borrower

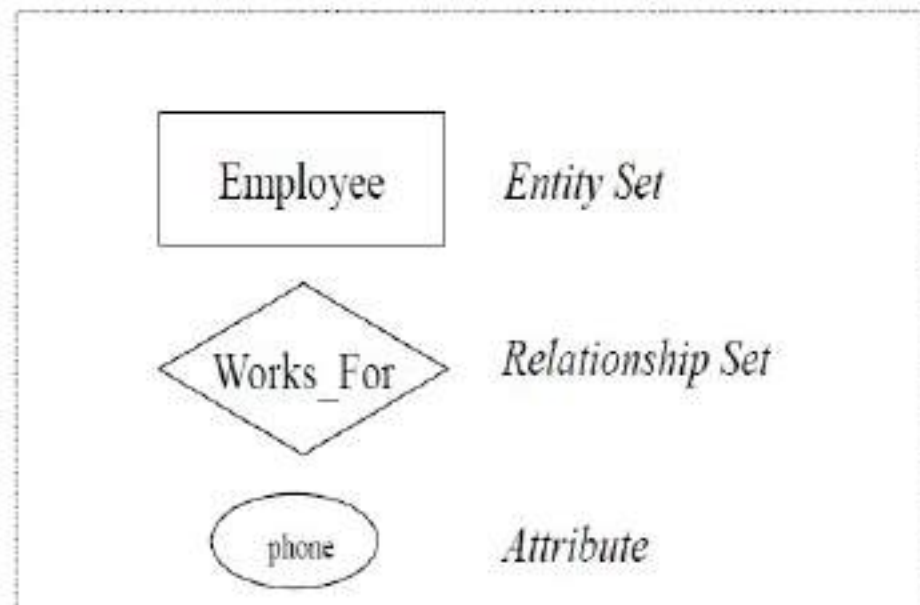


E-R Diagram with a Ternary Relationship





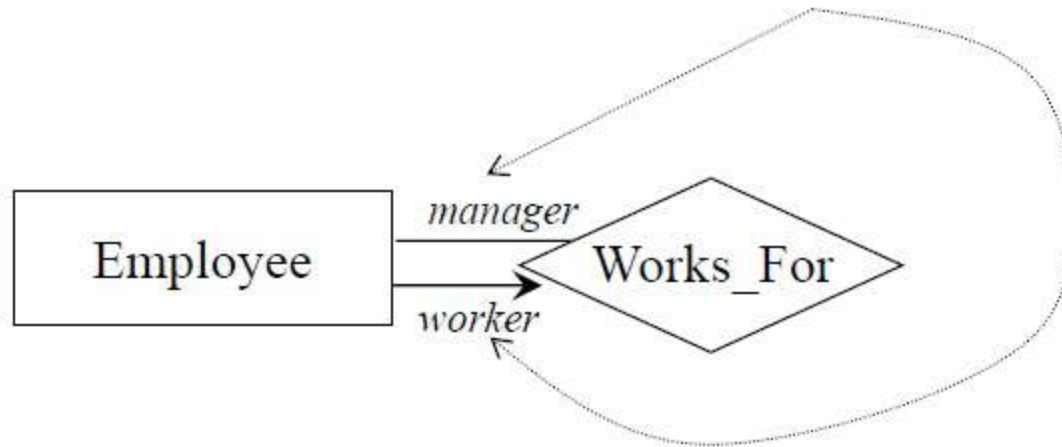
Lots of notation to come.

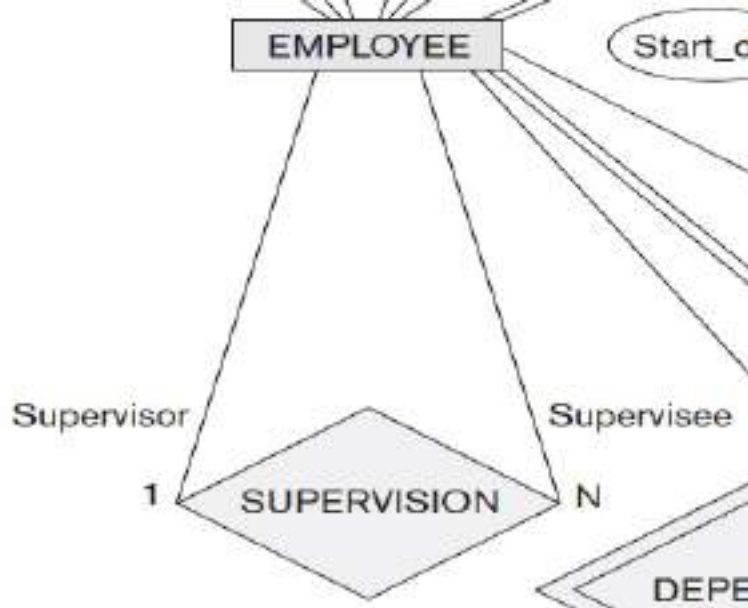


Recursive Relationship

In some cases the *same* entity type participates more than once in a relationship type in *different roles*. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called **recursive relationships**.

Recursive relationships: Must be declared with roles

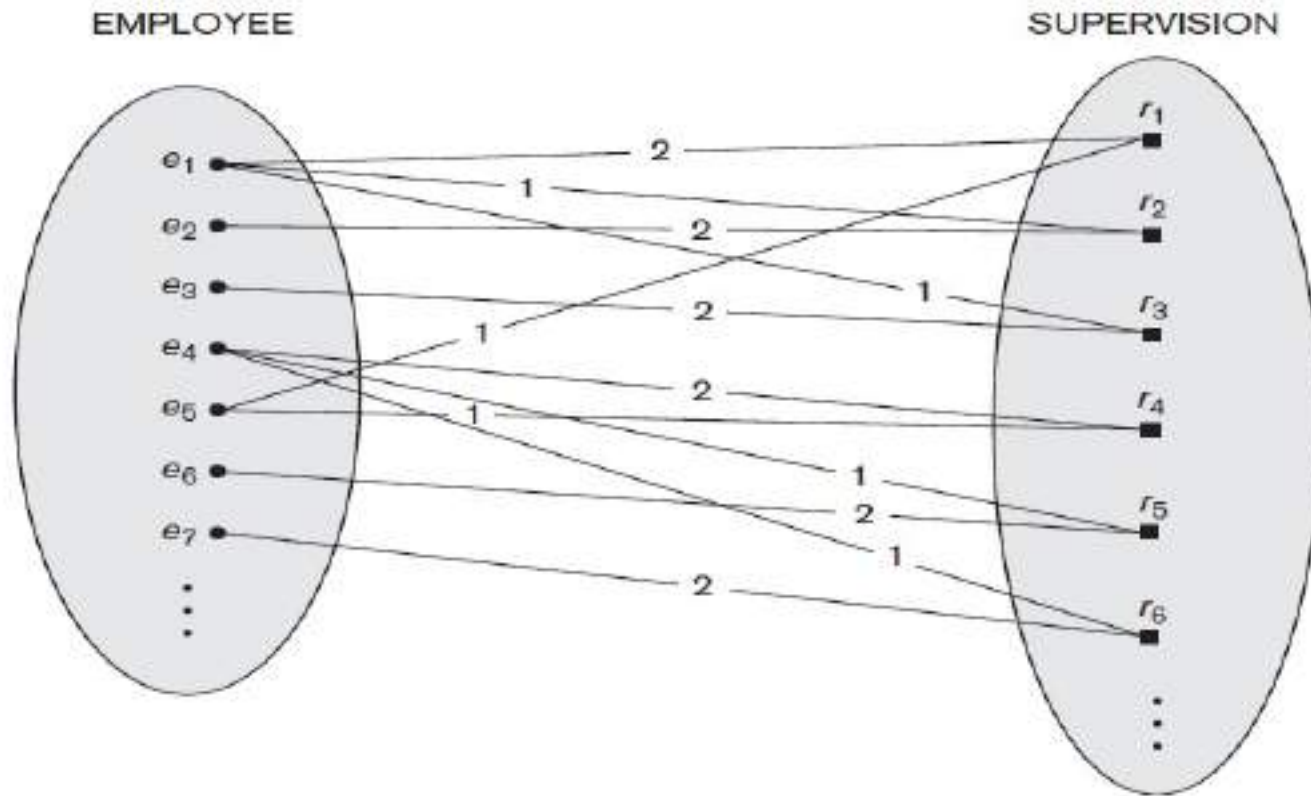


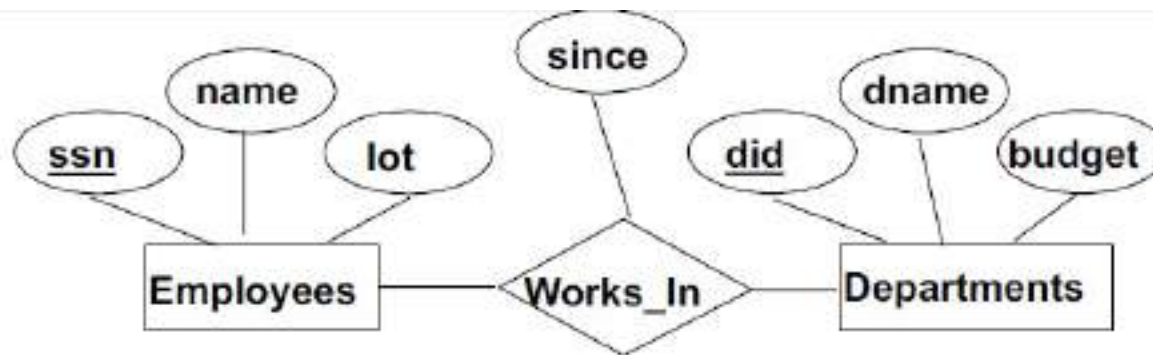


The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set.

Hence, the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or *boss*), and once in the role of *supervisee* (or *subordinate*).

- Each relationship instance r_i in SUPERVISION associates two employee entities e_j and e_k , one of which plays the role of supervisor and the other the role of supervisee.
- In Figure, the lines marked '1' represent the supervisor role, and those marked '2' represent the supervisee role.
- hence, e_1 supervises e_2 and e_3 , e_4 supervises e_6 and e_7 , and e_5 supervises e_1 and e_4 . In this example, each relationship instance must be connected with two lines, one marked with '1' (supervisor) and the other with '2' (supervisee).





```
CREATE TABLE Works_In(  
  ssn CHAR(11),  
  did INTEGER,  
  since DATE,  
  PRIMARY KEY (ssn, did),  
  FOREIGN KEY (ssn)  
    REFERENCES Employees,  
  FOREIGN KEY (did)  
    REFERENCES Departments)
```

ssn	did	since
123-22-3666	51	1/1/91
123-22-3666	56	3/3/93
231-31-5368	51	2/2/92

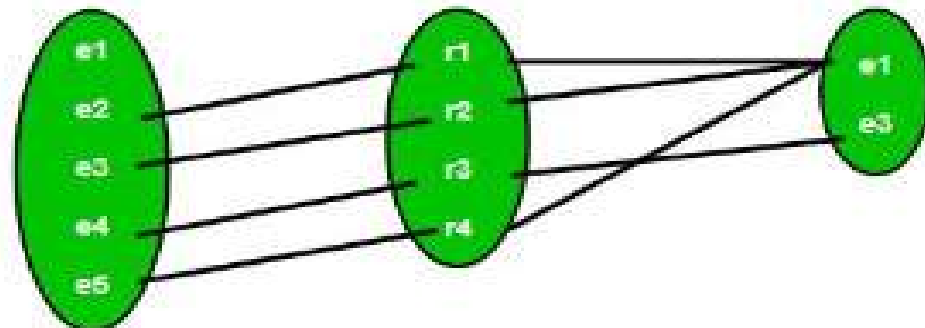
Maybe managed
by



Employee

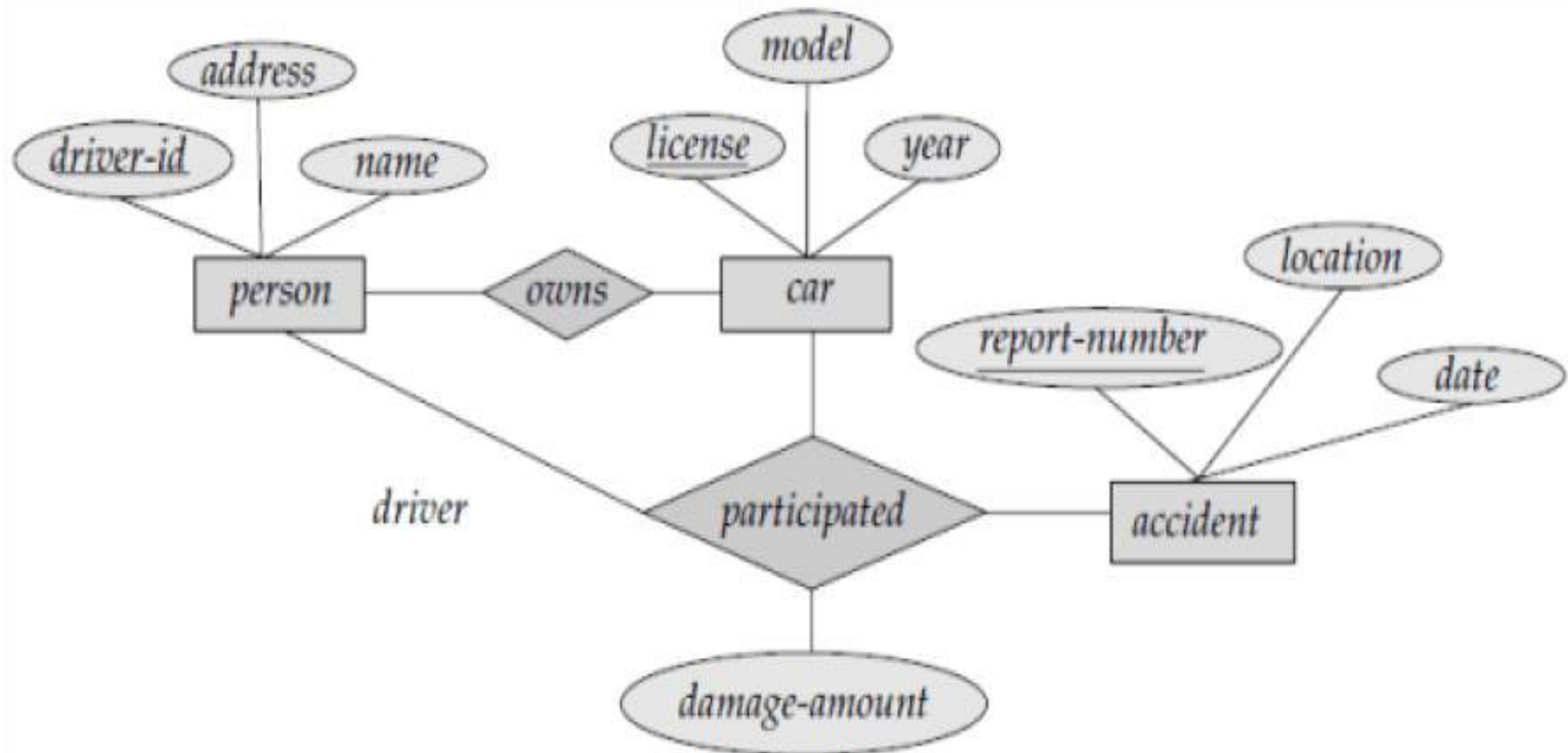
May
manage

- **Example:** Let us suppose that we have an employee table. A manager supervises a subordinate. Every employee can have a supervisor except the CEO and there can be at most one boss for each employee. One employee may be the boss of more than one employee. Let's suppose that REPORTS_TO is a recursive relationship on the Employee entity type where each Employee plays two roles.
 - Supervisor
 - Subordinate



- Supervisors and subordinates are called “**Role Names**”. Here the degree of the REPORTS_TO relationship is 1 i.e. a unary relationship.

Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents.



Appropriate tables for the above ER Diagram

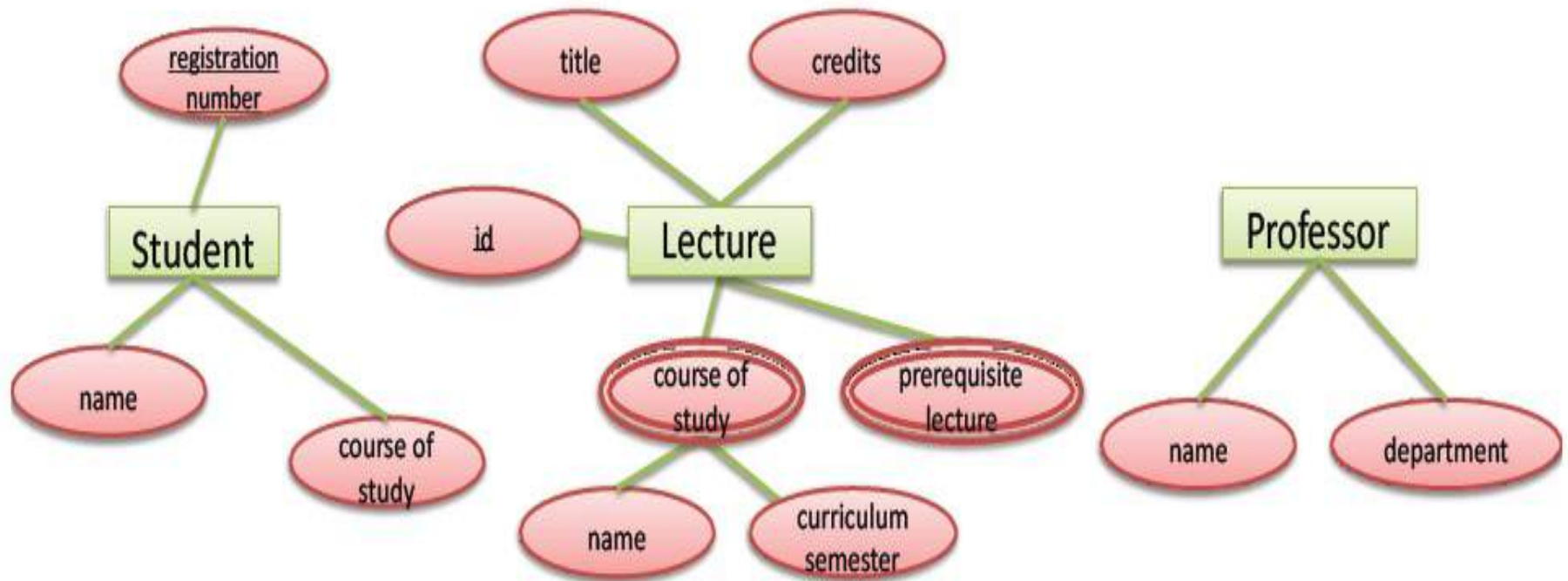
- **person (driver-id, name, address)**
- **car (license, year, model)**
- **accident (report-number, date, location)**
- **participated(driver-id, license, report-number, damage-amount)**

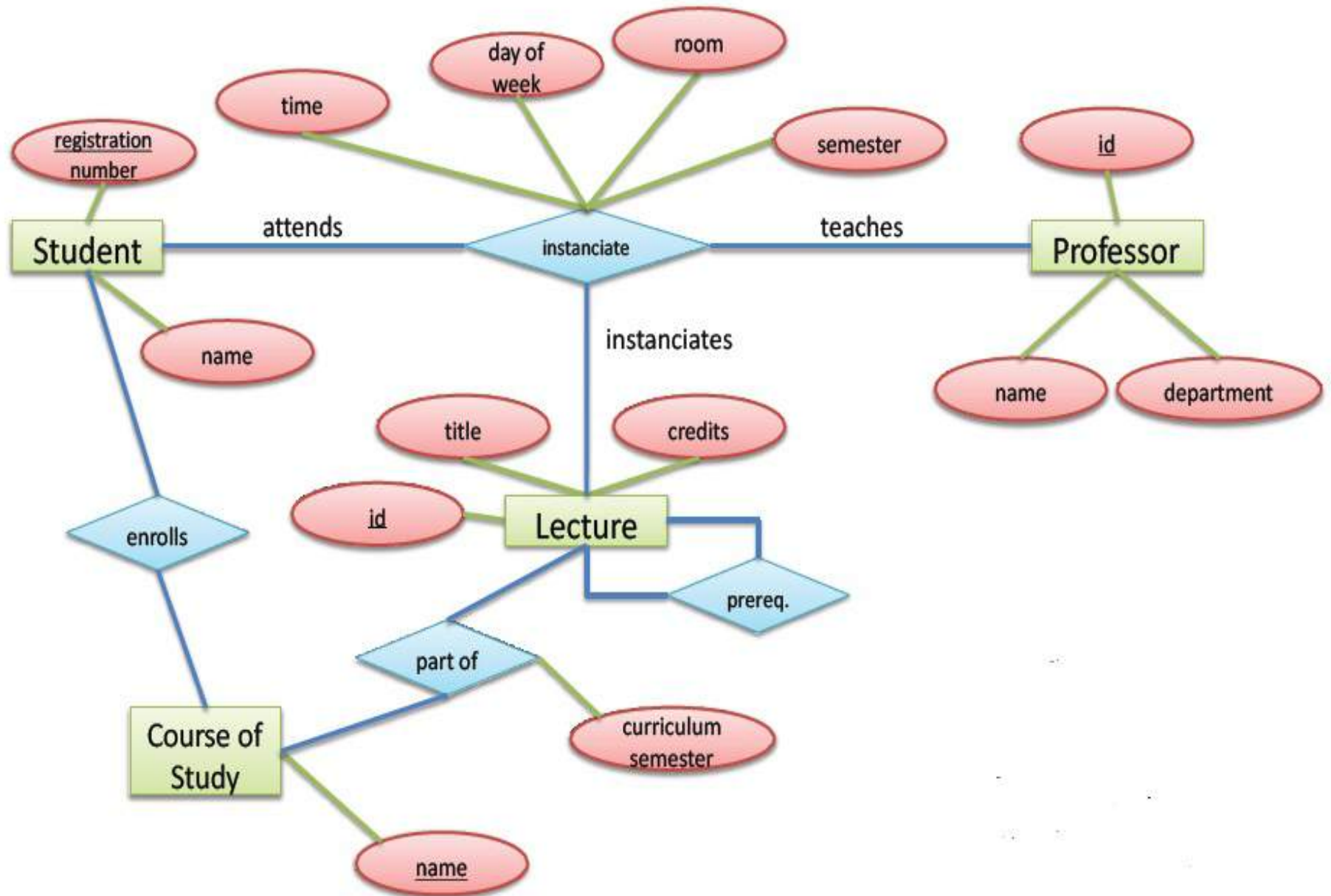
STUDENT

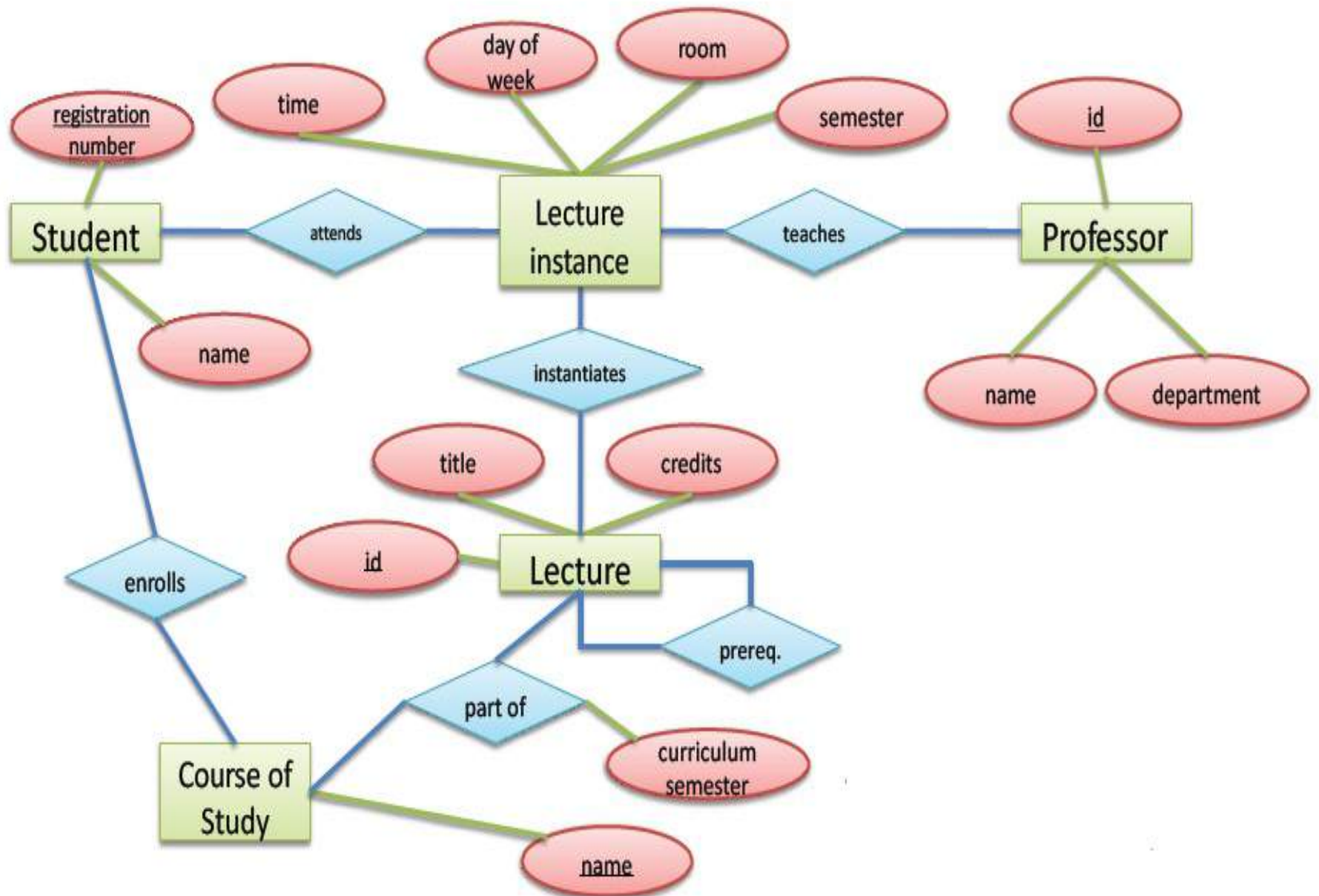
LECTURE

PROFESSOR

- What attributes are there?
 - In our database, we have students. They have a *name, a registration number and a course of study*.
 - The university offers lectures. Each lecture may be part of some course of study in a certain semester. Lectures may have other lectures as prerequisites. They have a title, provide a specific number of credits and have *unique ID*
 - Professors have a *name and are member of a specific department*.





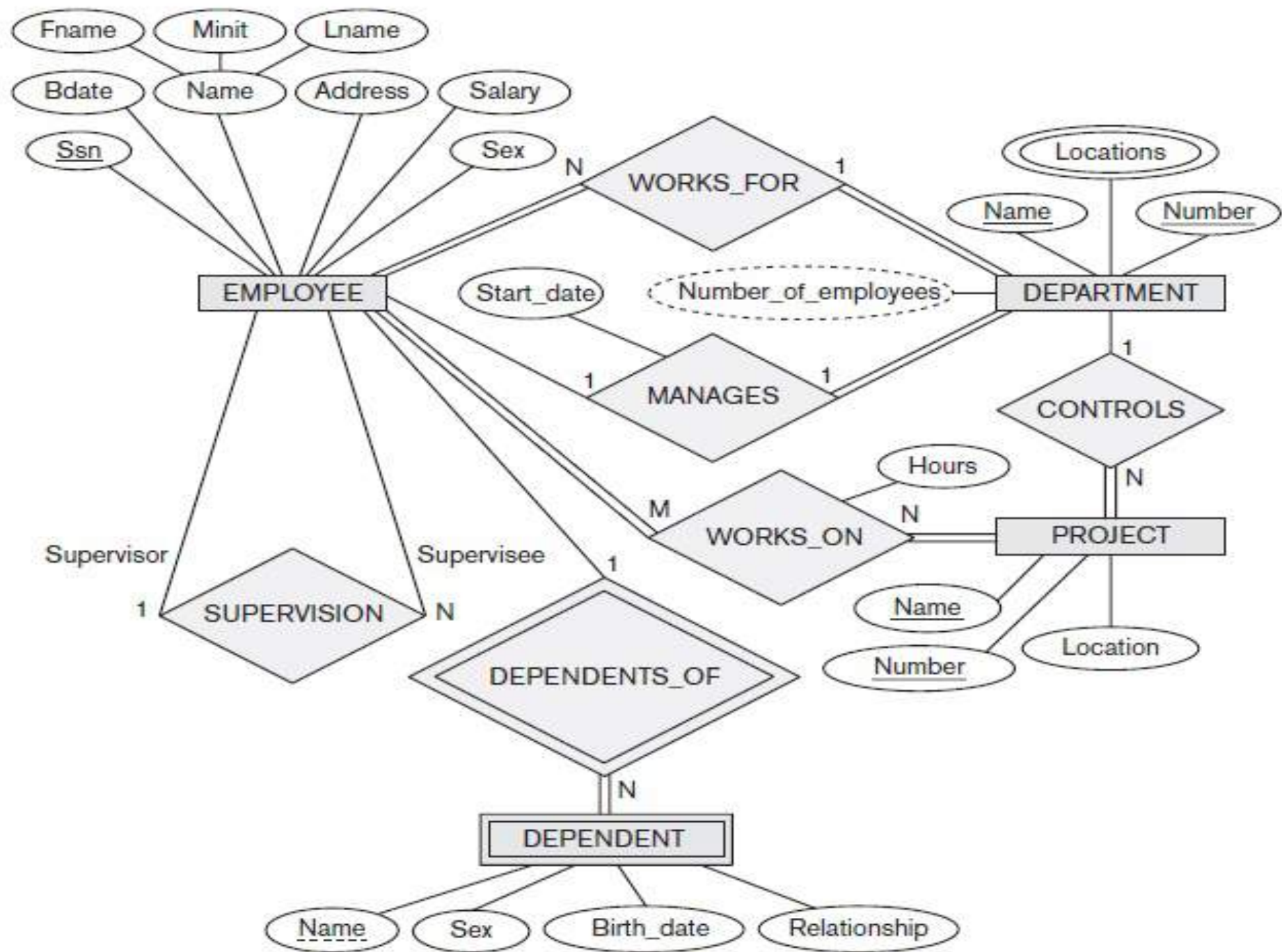


- Modeling is not that simple
- Many possible (and also correct) ways of modeling the same mini world.
 - Some are more elegant, some are less elegant
- Models alone are not enough, they need to be documented
 - What are the meanings of the attributes? The meanings of the relationships?

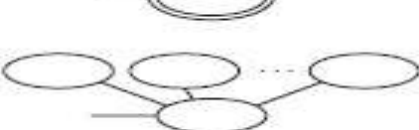
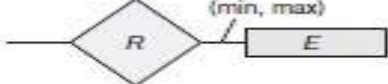
Proper Naming of Schema Constructs

- When designing a database schema, the choice of names for entity types, attributes, relationship types, and (particularly) roles is not always straightforward.
- One should choose names that convey, as much as possible, the meanings attached to the different constructs in the schema.
- We choose to use *singular names* for entity types, rather than plural ones, because the entity type name applies to each individual entity belonging to that entity type.
- In our ER diagrams, we will use the convention that entity type and relationship type names are uppercase letters, attribute names have their initial letter capitalized, and role names are lowercase letters.

- As a general practice, given a narrative description of the database requirements, the *nouns* appearing in the narrative tend to give rise to entity type names, and the *verbs* tend to indicate names of relationship types.
- Another naming consideration involves choosing binary relationship names to make the ER diagram of the schema readable from left to right and from top to bottom.
- We have generally followed this guideline in Figure, To explain this naming convention further, we have one exception to the convention in Figure—the DEPENDENTS_OF relationship type, which reads from bottom to top. When we describe this relationship, we can say that the DEPENDENT entities (bottom entity type) are DEPENDENTS_OF (relationship name) an EMPLOYEE (top entity type).



Design Choices for ER Conceptual Design

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1: N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Alternative Notations for ER Diagrams

- In this section, we describe one alternative ER notation for specifying structural constraints on relationships, which replaces the cardinality ratio (1:1, 1:N, M:N) and single/double line notation for participation constraints.
- This notation involves associating a pair of integer numbers (min, max) with each *participation* of an entity type E in a relationship type R , where $0 \leq \min \leq \max$ and $\max \geq 1$.
- The numbers mean that for each entity e in E , e must participate in at least *min* and at most *max* relationship instances in R *at any point in time*. In this method, $\min = 0$ implies partial participation, whereas $\min > 0$ implies total participation.
- Let's say you have two entities, "Department" and "Employee," and you want to represent the relationship between them. If each department can have zero or one employees and each employee must belong to exactly one department, you would use the notation (0,1) on the Department side and (1,1) on the Employee side.

- Note that a 1-N relationship would have the values reversed using the (min,max) notation:

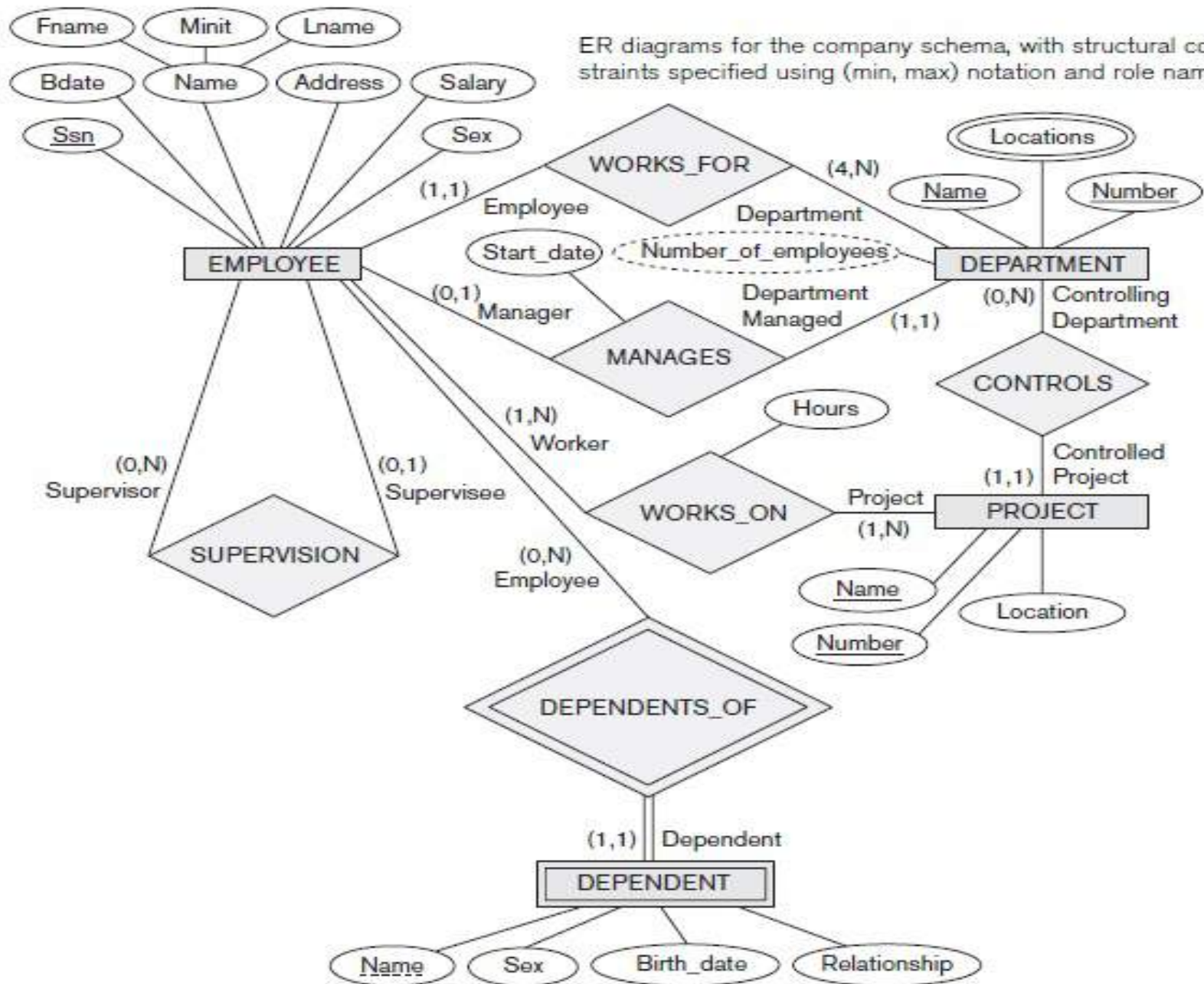
DEPARTMENT ===1=== employs ===N=== EMPLOYEE // 1 dept = N employees

DEPARTMENT -----(1,N)----- employs --- (1,1)----- EMPLOYEE // dept can have 1..N employees

The second line, above, means that a given department can appear multiple times in the EMPLOYS relationship; i.e., a department can have multiple employees.

An employee can appear only once; that is, can work for only a single department. Every employee must appear at least once, and every department.

ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.





- Double line between the entity set “Student” and relationship set “Enrolled in” signifies total participation.
- It specifies that each student must be enrolled in at least one course.
- Single line between the entity set “Course” and relationship set “Enrolled in” signifies partial participation.
- It specifies that there might exist some courses for which no enrollments are made.



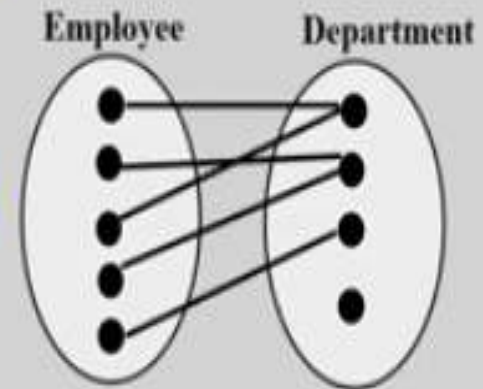
Each student can enroll minimum **1** and maximum **3** courses, and each course can have maximum **25** students.

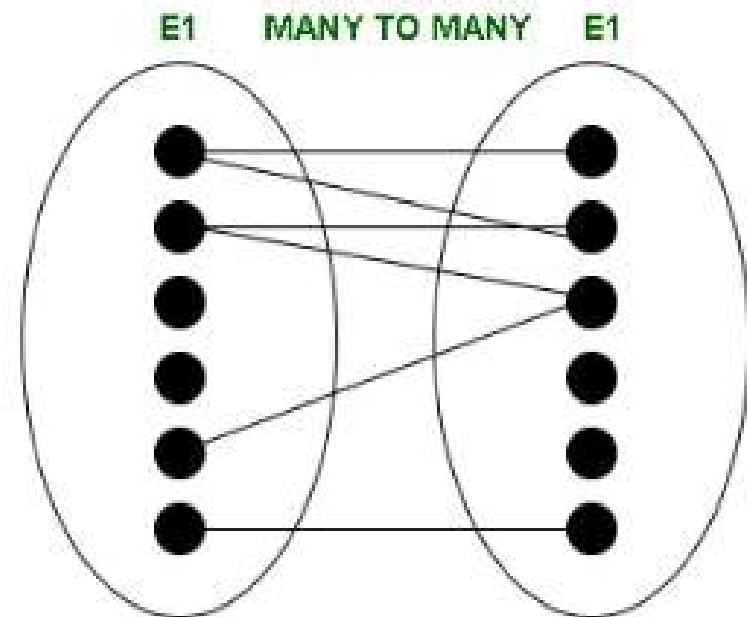
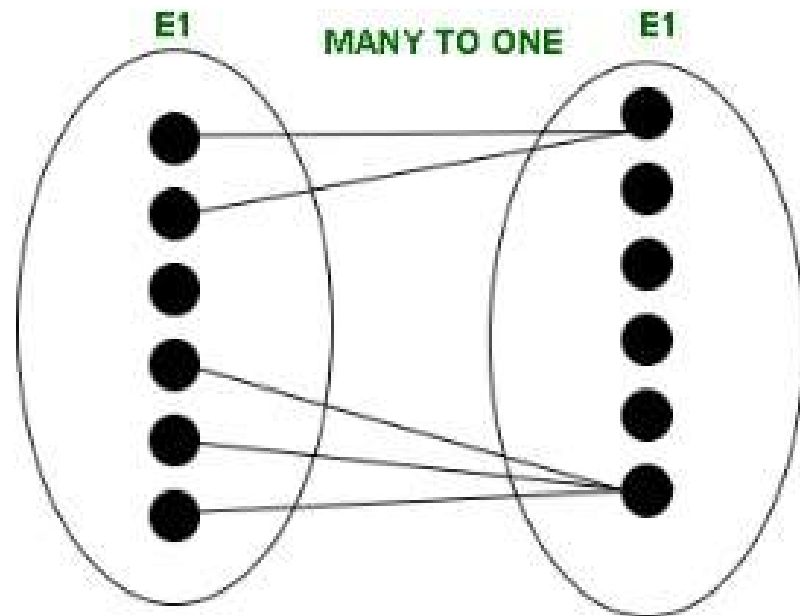
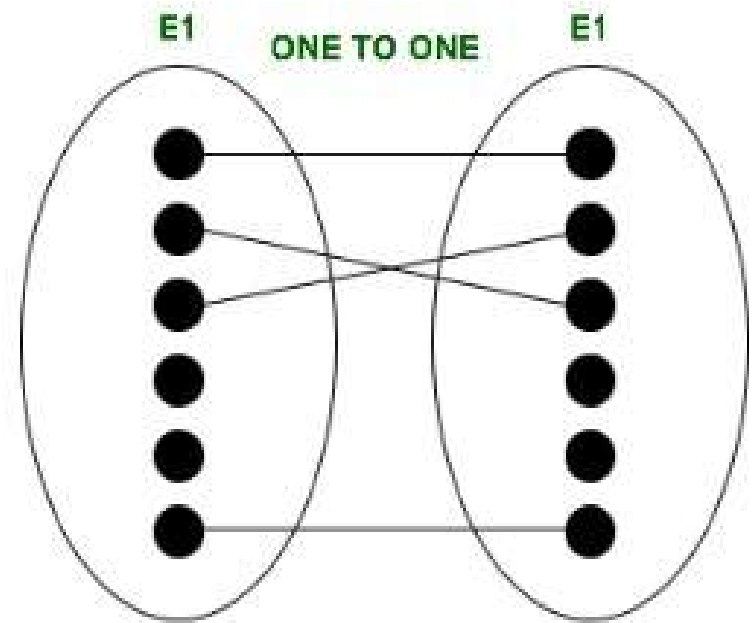
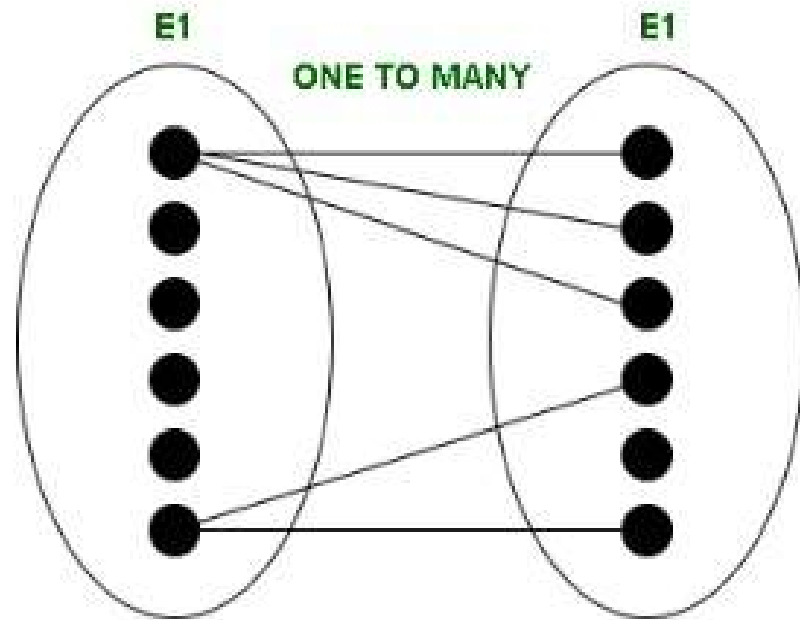
- This is a pair of numbers (m, n) that appear on the connecting line between the entities and their relationships. The minimum number of times an entity can appear in a relation is represented by m whereas, the maximum time it is available is denoted by n .
- If m is 0 it signifies that the entity is participating in the relation partially, whereas, if m is either greater than or equal to 1, it denotes total participation of the entity.

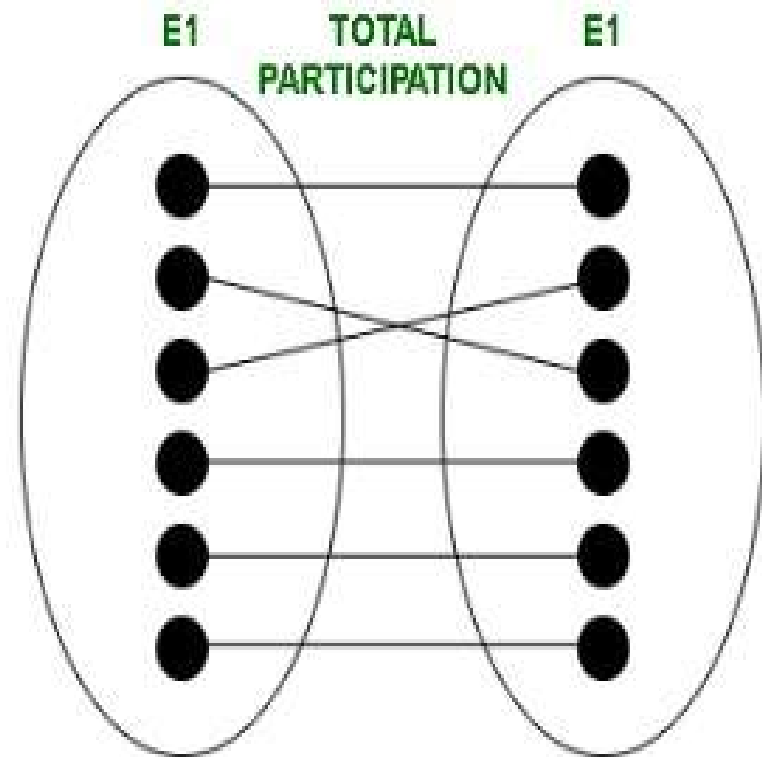
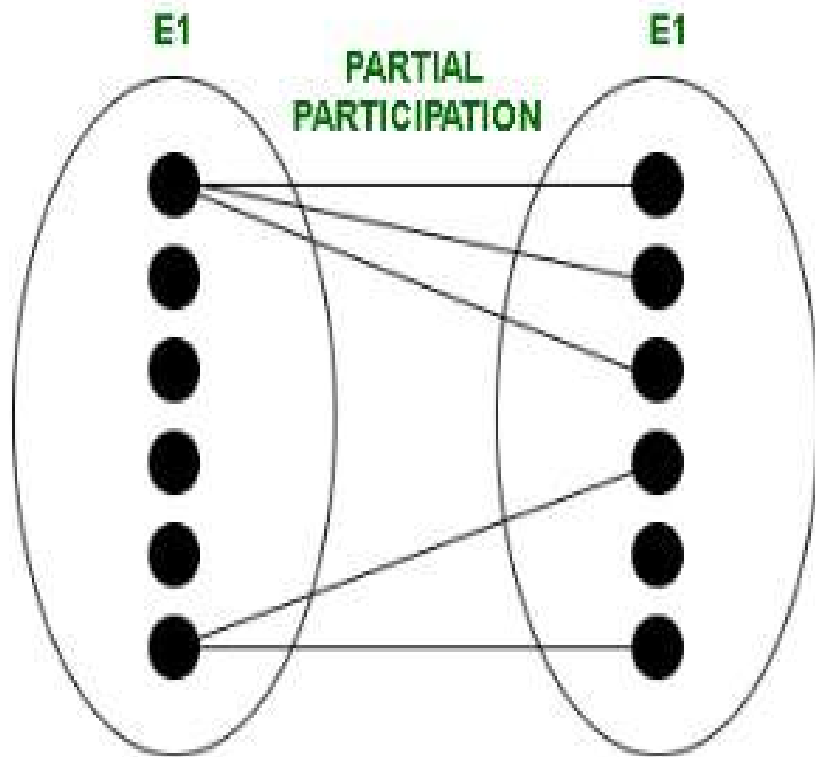
Example

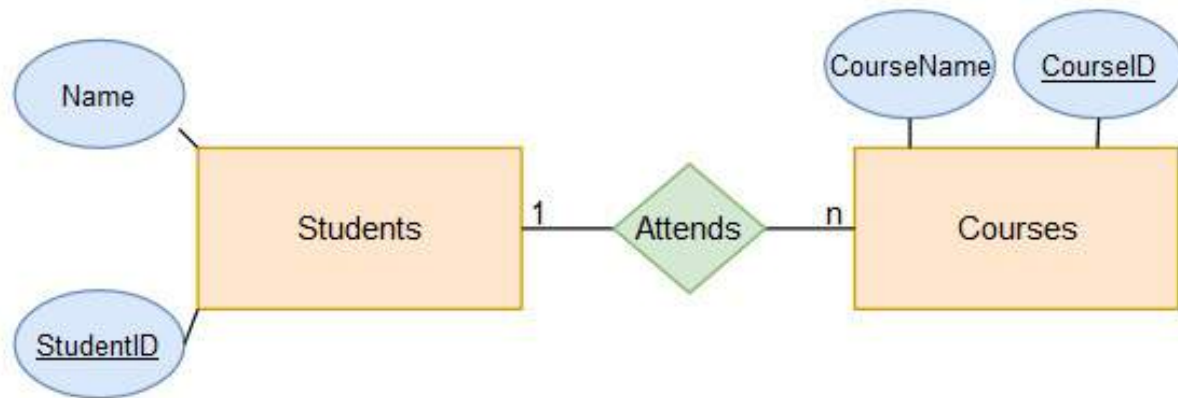
In (min , max) notation based on the participation we represent min value and based on cardinality we represent the max value.

Every employee works for a department and a department can have many employees. New dept. need not have any employee.

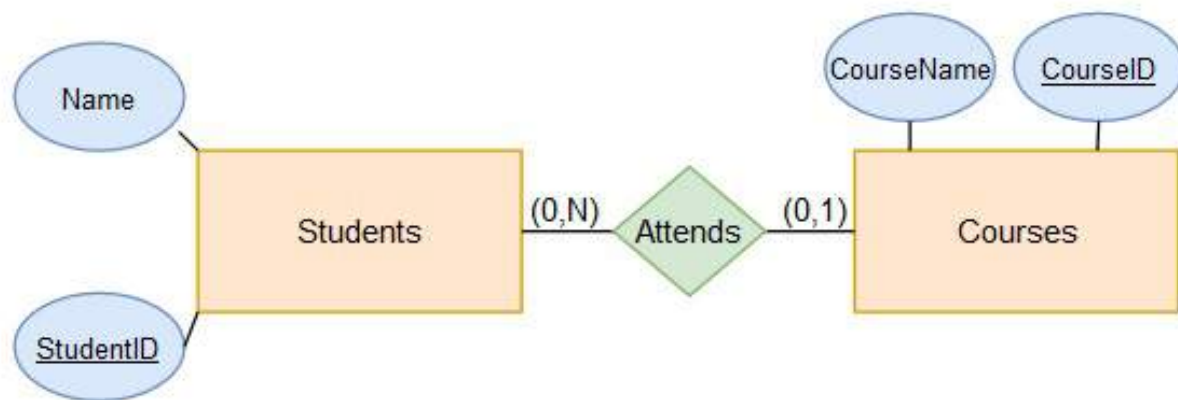


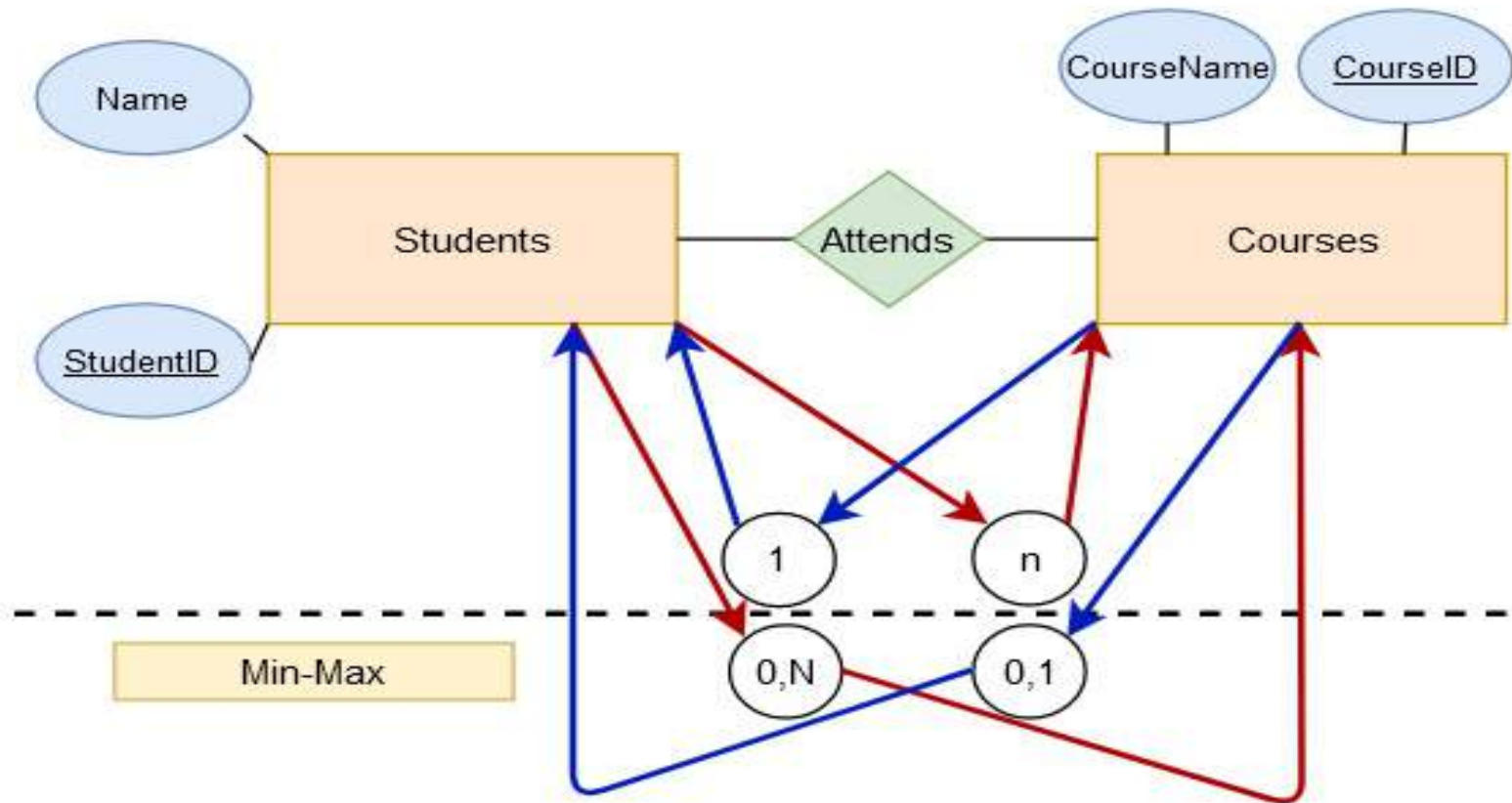






Min-Max





If you follow the red arrows, you will see that each student can enroll in ***n*** courses. No course can be taken twice by a student (blue arrows).

The Min-Max notation is written exactly the other way around than the normal notation (which still includes additional lower and upper bounds as information).

Problem 1

Problem: Each of the following is either an entity type and/or an entity. If it's an entity type, then provide examples of entities; if it's an entity, define a possible entity type.

- A municipal bond from Detroit
- Ford
- Clothes
- Employee

Solution: The municipal bond is an entity; *bond* is a possible entity type.

Ford is an entity; *manufacturer* is a possible entity type.

Clothes could be either: 1) a type if the entities are pants, shirts, etc.; 2) an entity if the type is product.

Employee is an entity type; Angela and Natalie are example entities.

Problem 2

Problem: What is the relationship between departments and managers? Fill in the blanks with either a *one* or a *many*:

- For each department there can be, at most, _____ managing employee(s).
- For each managing employee there can be, at most, _____ department(s).

Solution:



The ER diagram now represents much more information than it did above:

Any one division can contain many departments. Any one department can be contained in, at most, one division.

Any department can have, at most, one managing employee (or manager). Any manager can manage, at most, one department.

Problem 3

- We have three entities: car, customer, and salesperson.
- **Consider the following situations:** There is a ternary relationship called *sell* relating all three entities. Sells: “Customer *cust* bought car *car* from salesperson *SP*.”

Sells		
Car	Cust	SP
Cobra	Sam	Don
Mustang	Sam	Don
Mustang	Sam	Sharon
Cobra	Jenn	Sharon
Mustang	Jenn	Sharon

Question: Using this data, who sold Sam the Cobra?

- We have three binary relations as shown below:

Buys		Buys from		Sells	
Cust	Car	Cust	SP	SP	Car
Sam	Mustang	Sam	Don	Don	Cobra
Sam	Cobra	Sam	Sharon	Don	Mustang
Jenn	Mustang	Jenn	Sharon	Sharon	Cobra
Jenn	Cobra			Sharon	Mustang

Buys

Customer *cust* buys car *car*.

buys from

Customer *cust* buys from salesperson *SP*.

Sells

Salesperson *SP* has sold car *car*.

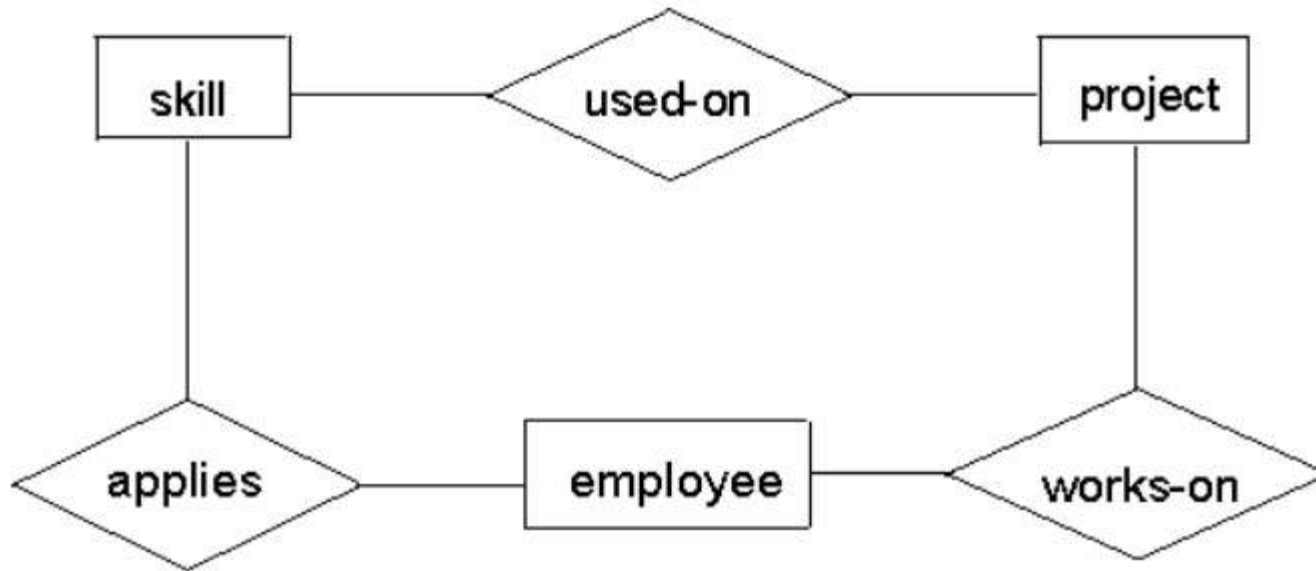
Question: Using this data, who sold Sam the Cobra?

- In situation #1 you can see from the first line that Don sold Sam the Cobra.
- In situation #2, looking at **buys** you can see that Sam did actually buy a Cobra (second line).
 - Looking at **buys from** you can see that both Don and Sharon sold Sam cars.
 - Looking at **sells** you can see that both Don and Sharon have sold Cobras. So the answer is either Don or Sharon sold Sam the Cobra.
 - This is not good enough. This demonstrates that having three binary relationships does not capture the same information that one ternary relationship does.

Degree of a relationship

- Relationships can be classified by the number of entity types involved. This is referred to as the *degree* of a relationship.
 - Binary: This is a relationship between two entity types.
 - Ternary: This is a relationship between three entity types.
 - Recursive: This is a relationship involving only one entity type.

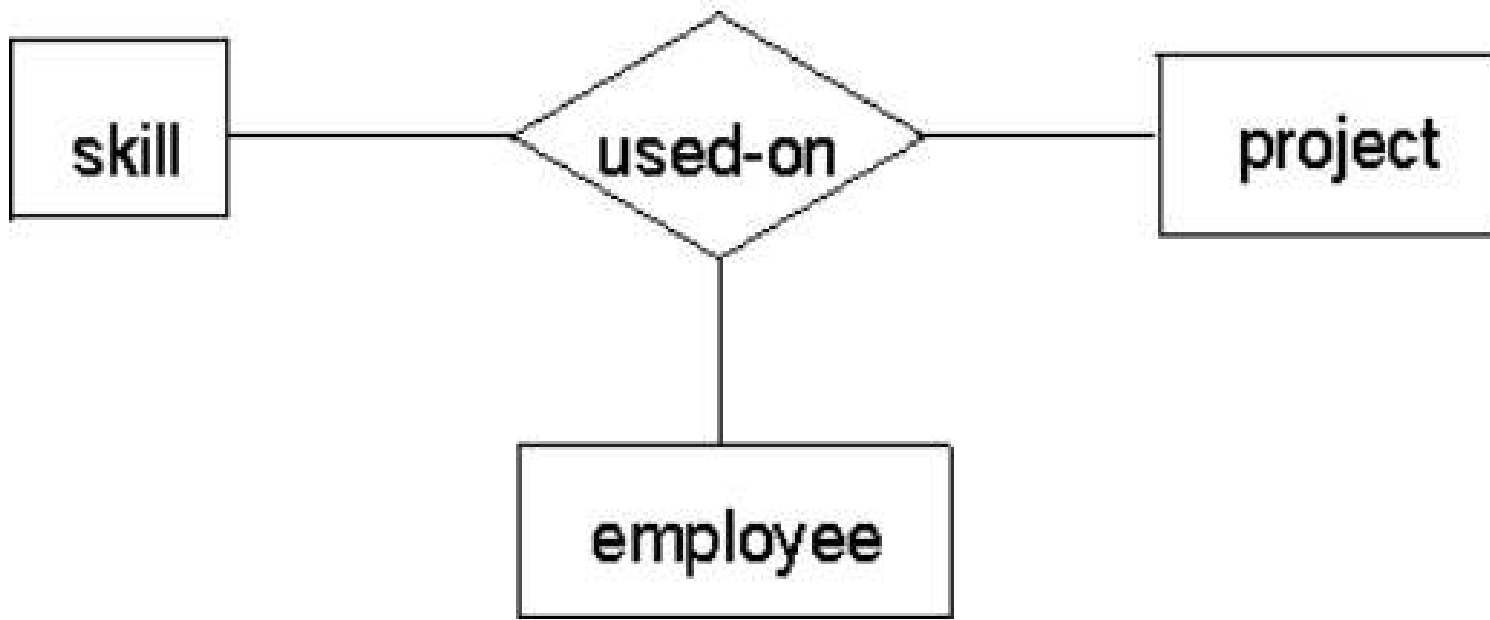
- For example, suppose that we want to capture which employees use which *skills* on which *project*. We might try to represent this data in a database as **three binary** relationships between skills and project, project and employee, and employee and skill.



The *applies* relationship indicates which employee applies which skill.
The *used on* relationship indicates which skill is used on which project.
The *works on* relationship indicates which employee works on which project.
But this is not enough to specify which employee uses which skill on which project.

- works-on
 - Lindsey and Mackenzie have worked on projects *A* and *B*.
- Applies
 - Lindsey has used skills *interface design* and *database design* while Mackenzie only used her *database design* skill.
- used on
 - Both skills have been used on both projects.

Given this information, it is impossible to figure out on which projects Lindsey used which skills. She could have used *interface design* on project *B* and *database design* on project *A*---or the other way around. Or she might have used both skills on both projects. The database simply does not give us enough information.



The *used on* relationship captures information three pieces at a time. It stores facts such as:

Lindsey used *interface design* skill on project A.

Mackenzie used *database design* skill on project A.

Lindsey used *interface design skill* on project B.

Lindsey used *database design skill* on project B.

Mackenzie used *database design* skill on project B.

- On an ER diagram, if the end of a relationship is straight, it represents 1, while a "crow's foot" end represents many.
- A one to one relationship - a man can only marry one woman, and a woman can only marry one man, so it is a one to one (1:1) relationship

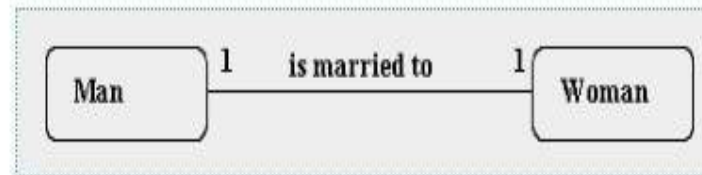


Figure : One to One relationship example

- A one to many relationship - one manager manages many employees, but each employee only has one manager, so it is a one to many (1:n) relationship



Figure : One to Many relationship example

- A many to one relationship - many students study one course. They do not study more than one course, so it is a many to one (m:1) relationship



Figure : Many to One relationship example

- A many to many relationship - One lecturer teaches many students and a student is taught by many lecturers, so it is a many to many (m:n) relationship

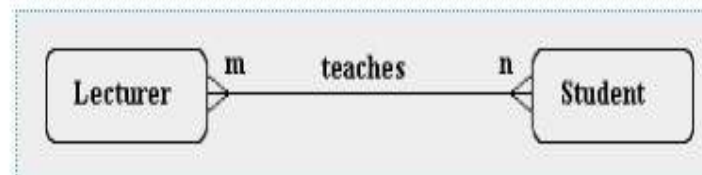


Figure : Many to Many relationship example

Redundant relationships

- Some ER diagrams end up with a relationship loop.
- check to see if it is possible to break the loop without losing info
- Given three entities A, B, C, where there are relations A-B, B-C, and C-A, check if it is possible to navigate between A and C via B. If it is possible, then A-C was a redundant relationship.
- Always check carefully for ways to simplify your ER diagram. It makes it easier to read the remaining information.

- **Country Bus Company**

- A Country Bus Company owns a number of busses.
- Each bus is allocated to a particular route, although some routes may have several busses.
- Each route passes through a number of towns.
- One or more drivers are allocated to each stage of a route, which corresponds to a journey through some or all of the towns on a route.
- Some of the towns have a garage where busses are kept and each of the busses are identified by the registration number and can carry different numbers of passengers, since the vehicles vary in size and can be single or double-decked.
- Each route is identified by a route number and information is available on the average number of passengers carried per day for each route.
- Drivers have an employee number, name, address, and sometimes a telephone number.

- **Entities**

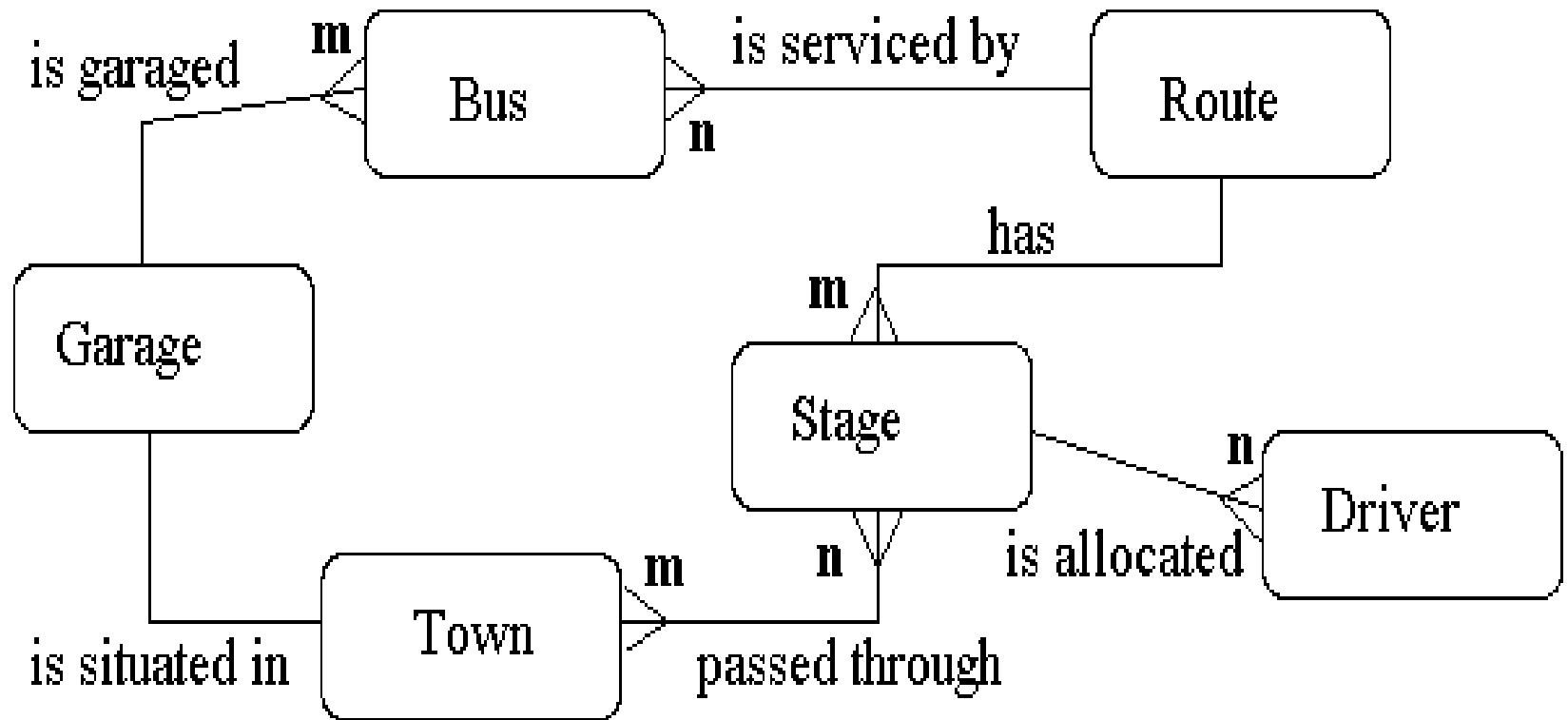
- Bus - Company owns busses and will hold information about them.
- Route - Buses travel on routes and will need described.
- Town - Buses pass through towns and need to know about them
- Driver - Company employs drivers, personnel will hold their data.
- Stage - Routes are made up of stages
- Garage - Garage houses buses, and need to know where they are.

- **Relationships**

- A bus is allocated to a route and a route may have several buses.
- Bus-route (m:1) is serviced by
- A route comprises of one or more stages.
- route-stage (1:m) comprises
- One or more drivers are allocated to each stage.
- driver-stage (m:1) is allocated
- A stage passes through some or all of the towns on a route.
- stage-town (m:n) passes-through
- A route passes through some or all of the towns
- route-town (m:n) passes-through
- Some of the towns have a garage
- garage-town (1:1) is situated
- A garage keeps buses and each bus has one 'home' garage
- garage-bus (m:1) is garaged

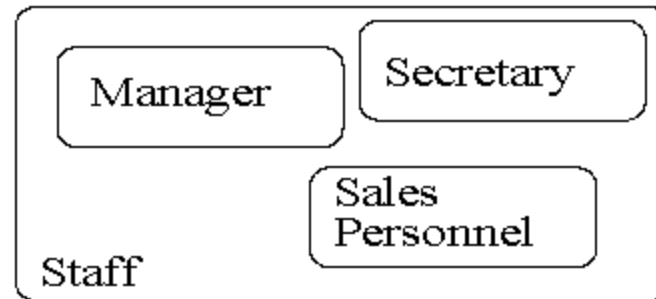
- **Attributes**

- Bus (reg-no,make,size,deck,no-pass)
- Route (route-no,avg-pass)
- Driver (emp-no,name,address,tel-no)
- Town (name)
- Stage (stage-no)
- Garage (name,address)



Extended E-R Features

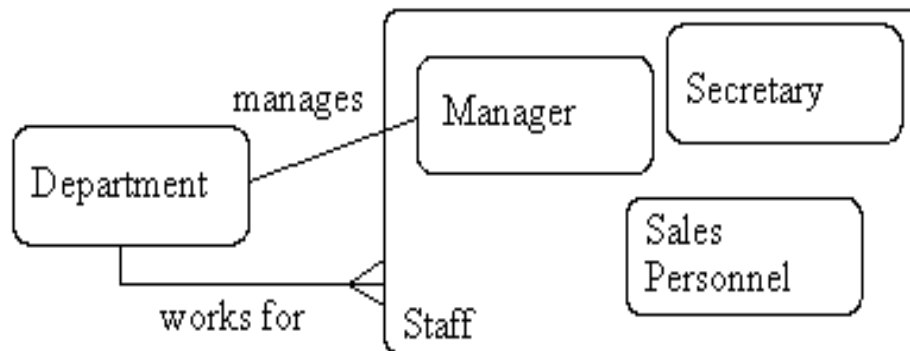
- Specialization
- Generalization
- Categorization
- Aggregation
 - Superclass - an entity type that includes distinct subclasses that require to be represented in a data model.
 - Subclass - an entity type that has a distinct role and is also a member of a superclass.



Subclasses need not be mutually exclusive; a member of staff may be a manager and a sales person.

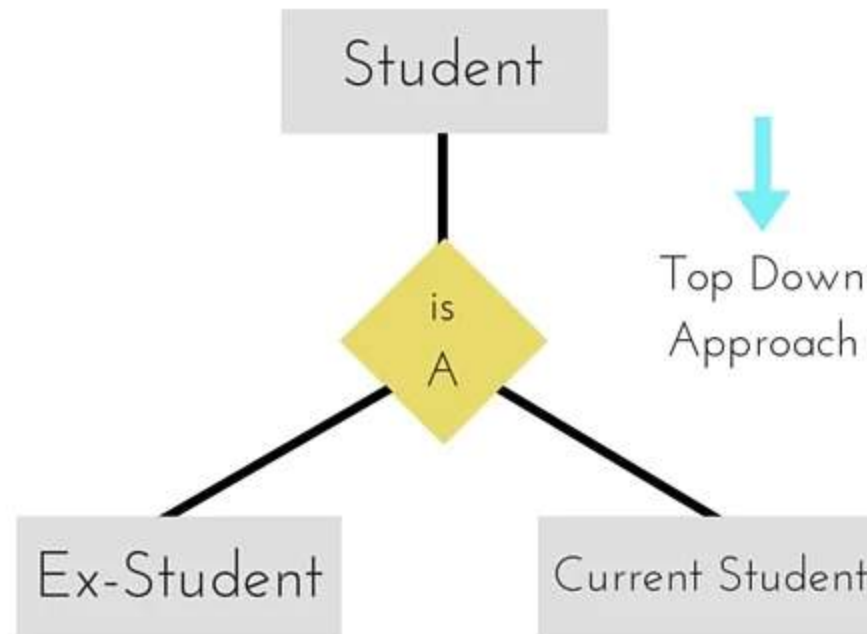
Specialisation

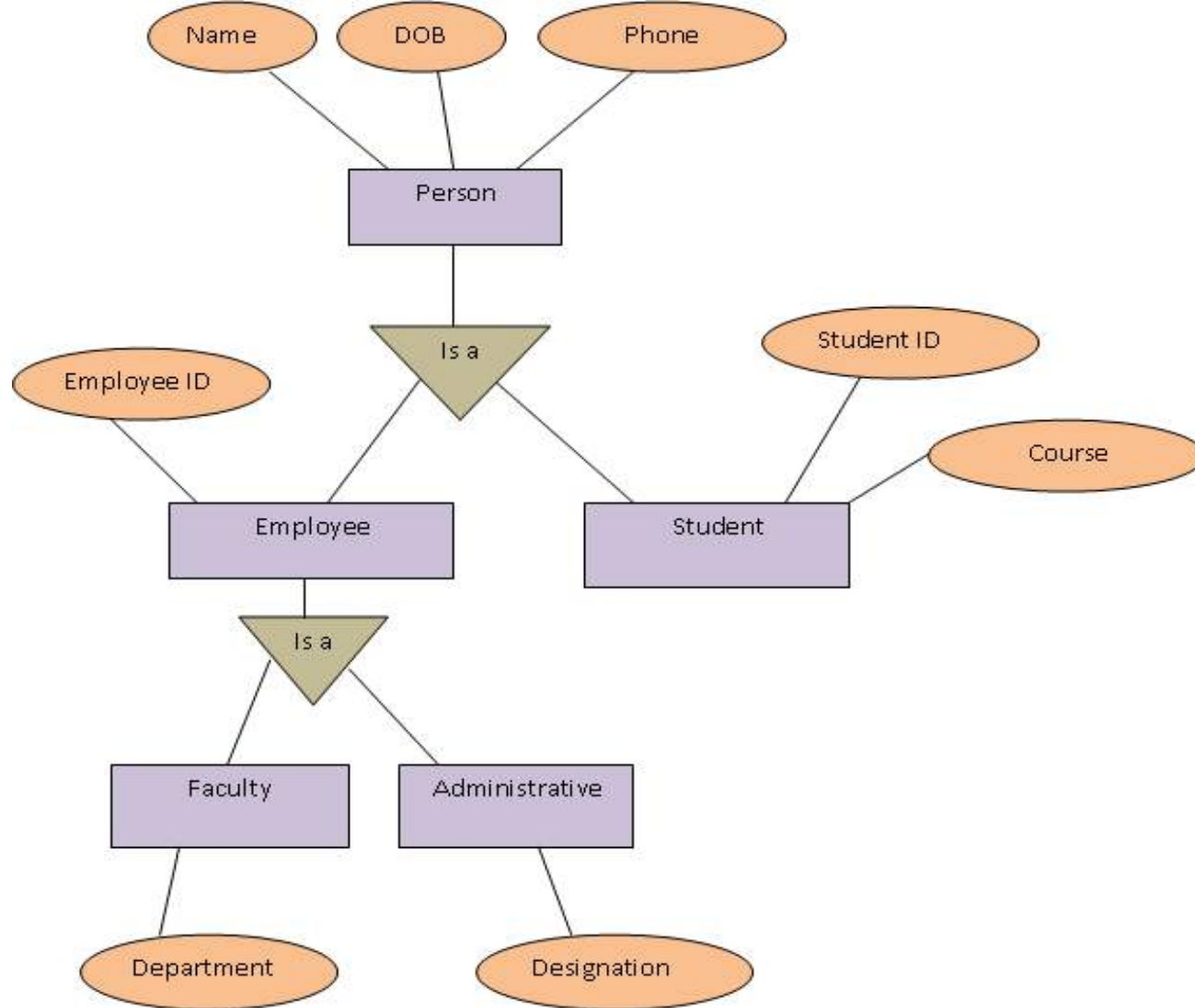
- This is the process of maximising the differences between members of an entity by identifying their distinguishing characteristics.
 - Staff(staff_no,name,address,dob)
 - Manager(bonus)
 - Secretary(wp_skills)
 - Sales_personnel(sales_area, car_allowance)



- Here we have shown that the ***manages*** relationship is only applicable to the Manager subclass, whereas the ***works_for*** relationship is applicable to all staff.
- It is possible to have subclasses of subclasses.

- **Specialization** is opposite to Generalization. It is a top-down approach in which one higher level entity can be broken down into two lower level entity. In specialization, a higher level entity may not have any lower-level entity sets, it's possible.





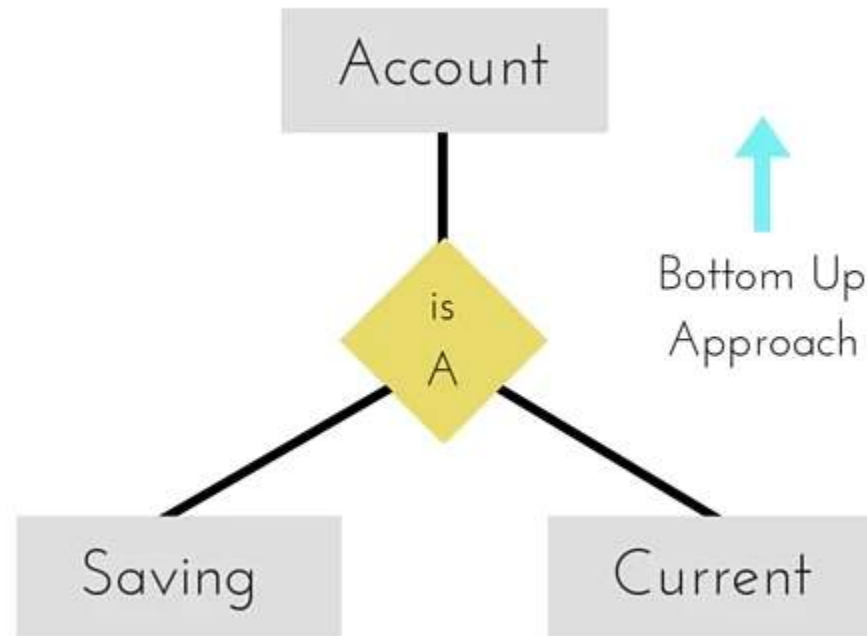
In Specialization hierarchy, Person is the Super class and Employee and Student are the sub classes of Person. Faculty and Administrative are Sub classes of Employee entity type. **Sub Classes (entities) inherit the attributes of Super Class and add some attributes of their own.**

- While converting an ER diagram into tables we write respective schemas. But schemas against all entities are not formed.
- **Schemas are formed against only leaf entities.**
- We do not write schema for Person entity because Person *is an* employee or a Student.
- Further employee *is a* Faculty or Administrative.
- So, we make schemas of only Student, Faculty and Administrative entity types.
- **Schemas:**
 - Student (Name, DOB, Phone, Student ID, Course)
 - Faculty (Name, , DOB, Phone, Employee ID, Department)
 - Administrative (Name, , DOB, Phone, Employee ID, Designation)
- **Here, Student schema inherits attributes of Person. Faculty and Administrative entities inherit the attributes of Person and Employee.**

GENERALIZATION

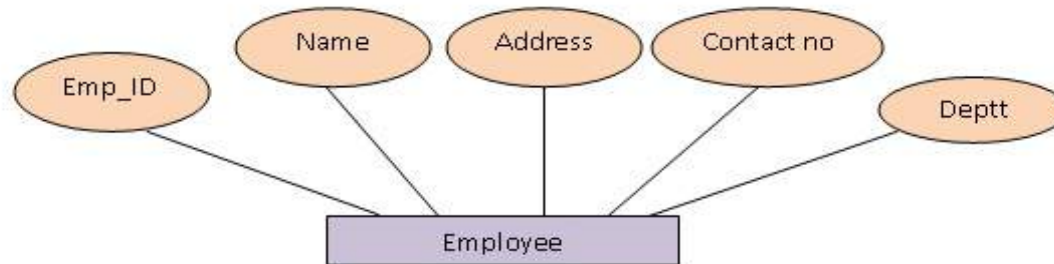
- Generalisation is the process of minimizing the differences between entities by identifying common features.
- Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it.
- This is the identification of a generalized superclass from the original subclasses. This is the process of identifying the common attributes and relationships.
 - For instance, taking:
 - `car(regno,colour,make,model,numSeats)`
 - `motorbike(regno,colour,make,model,hasWindshield)`
- And forming:
 - `vehicle(regno,colour,make,model,numSeats,hasWindshield)`

- **Generalization** is a bottom-up approach in which two lower level entities combine to form a higher level entity. In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.

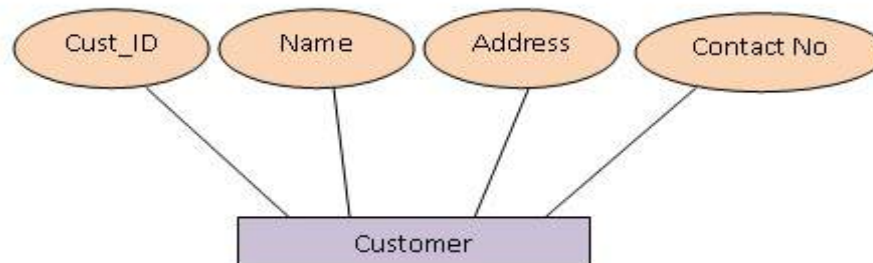


- Example:
 - We start with two entity types

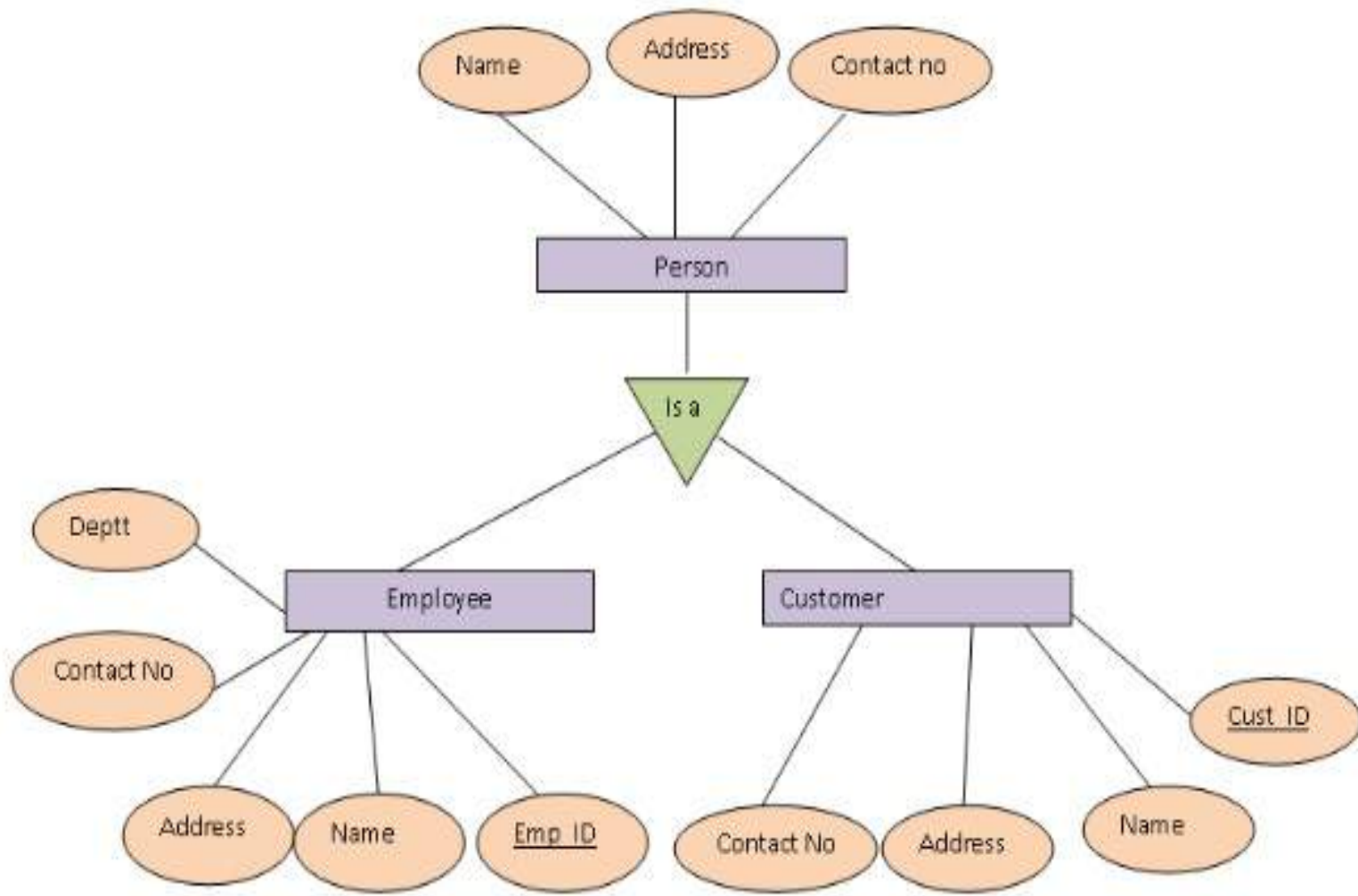
Employee (Emp_id, Name, Address, Contact no)



Customer (Cust id, Name, Address, Contact no).

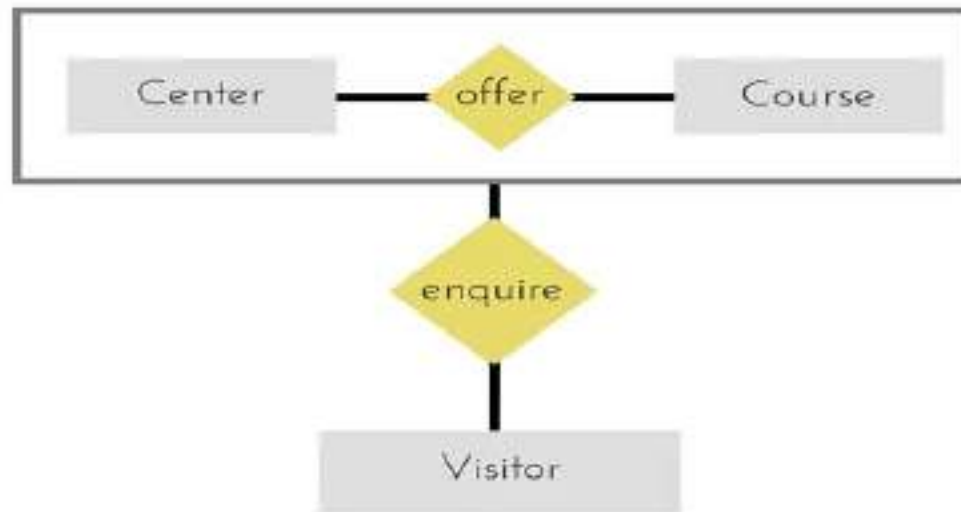


These two are special entities types. We can merge these two entities into one entity type named Person.



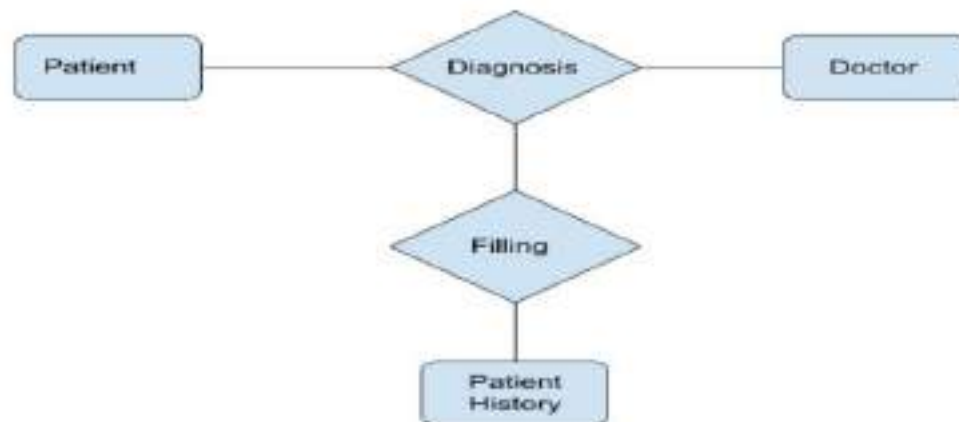
Aggregation

Aggregation is a process when relation between two entities is treated as a **single entity**.

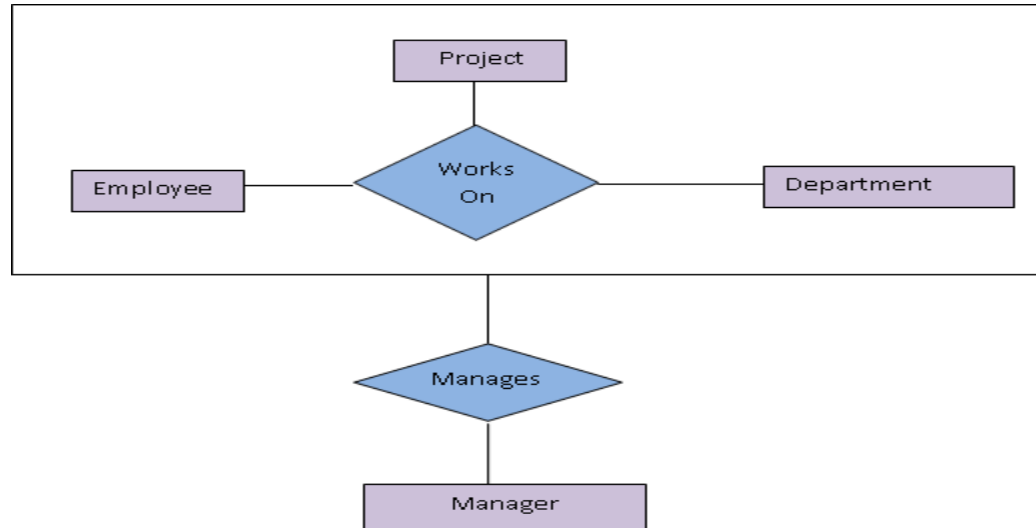


In the diagram above, the relationship between **Center** and **Course** together, is acting as an Entity, which is in relationship with another entity **Visitor**. Now in real world, if a Visitor or a Student visits a Coaching Center, he/she will never enquire about the center only or just about the course, rather he/she will ask enquire about both.

- Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.
- In this case, the patient is unable to work on his own. In order to receive a diagnosis, he must establish a relationship with the doctor . A diagnosis cannot be made by the doctor without the patient. In the future, the doctor will require information regarding the patient's history, which he will have to collect from a filing system.



- Suppose in an organization the Manager entity manages set of employee in a department working on specific projects.



- Employee, Department and Projects are entity types having a ternary relationship called **Works On**.
- Manager manages employee, manager also manages department and manager also manages Projects.
- It is difficult to represent this complicated relationship representation in ER diagram.
- It will be quite complicated to involve entity types into a relationship when they already are participating in a relationship.

Aggregation is used to solve such complication representations. In aggregation we represent same Employee, Department and Project entities and their relationship as a higher level entity by aggregating. Here we have another entity Manager who creates a relationship with this higher level entity.

Characteristics of Aggregation

- **Reduction of data:** Aggregation reduces the complexity of data by combining multiple data points into a single summary value or representation.
- **Loss of detail:** Since aggregation combines multiple data points into a single value, some of the details of individual data points may be lost.
- **Increases interpretability:** Aggregation makes it easier to interpret and understand data.
- **Improves efficiency:** Aggregation can improve the efficiency of data processing by reducing the amount of data that needs to be processed.

Removing Redundant Attributes in Entity Sets

- A good entity-relationship design does not contain redundant attributes.
- Once the entities and their corresponding attributes are chosen, the relationship sets among the various entities are formed.
- These relationship sets may result in a situation where attributes in the various entity sets are redundant and need to be removed .

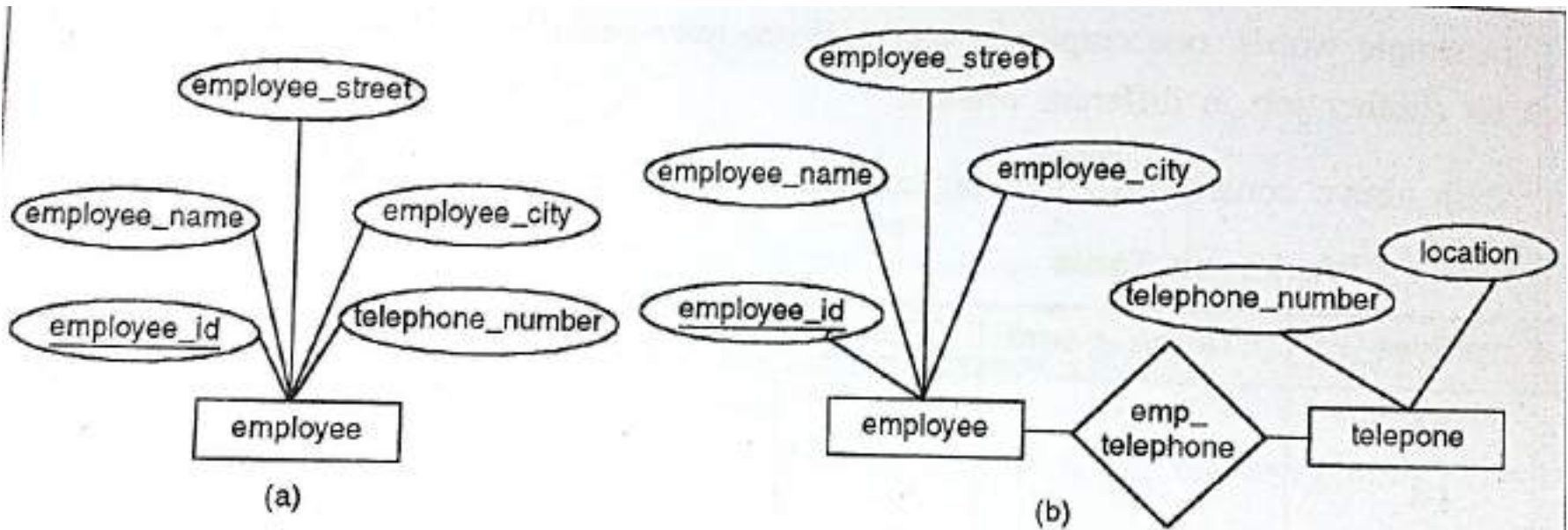
- Consider the entity sets ***instructor*** and ***department***
 - Instructor includes the attributes ID, name, dept_name, and salary with ID as primary key
 - Department includes the attributes dept_name, building, and budget with dept_name as primary key
- Attribute ***dept_name*** appears in both entity sets. It is primary key for entity department, it is redundant in the entity set instructor and needs to be removed.
- If both entities have ***one to one*** relationship then we can remove it from instructor table as it will get added up in the relational schema of department.

Design Issues

- There are number of different ways to define set of entities and relationship among them. We examine basic issues in the design of an E-R model. These are:
 - Use of entity sets vs. attributes
 - Use of entity sets vs. relationship sets
 - Binary versus n-ary relationship sets
 - Placement of relationship attributes

Design Issues: Use of entity sets vs. attributes

- Treating telephone as an attribute telephone-number implies that employees have exactly one telephone number each
- Treating telephone as an entity permits employees to have several telephone numbers associated with them.
- The distinctions to consider an object as attribute or entity depends on the structure of real-world enterprise being modeled.



Relationship Sets

- A relationship is an association among several entities.

Example:

44553 (Peltier)
student entity

advisor
relationship set

22222 (Einstein)
instructor entity

- A relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets.

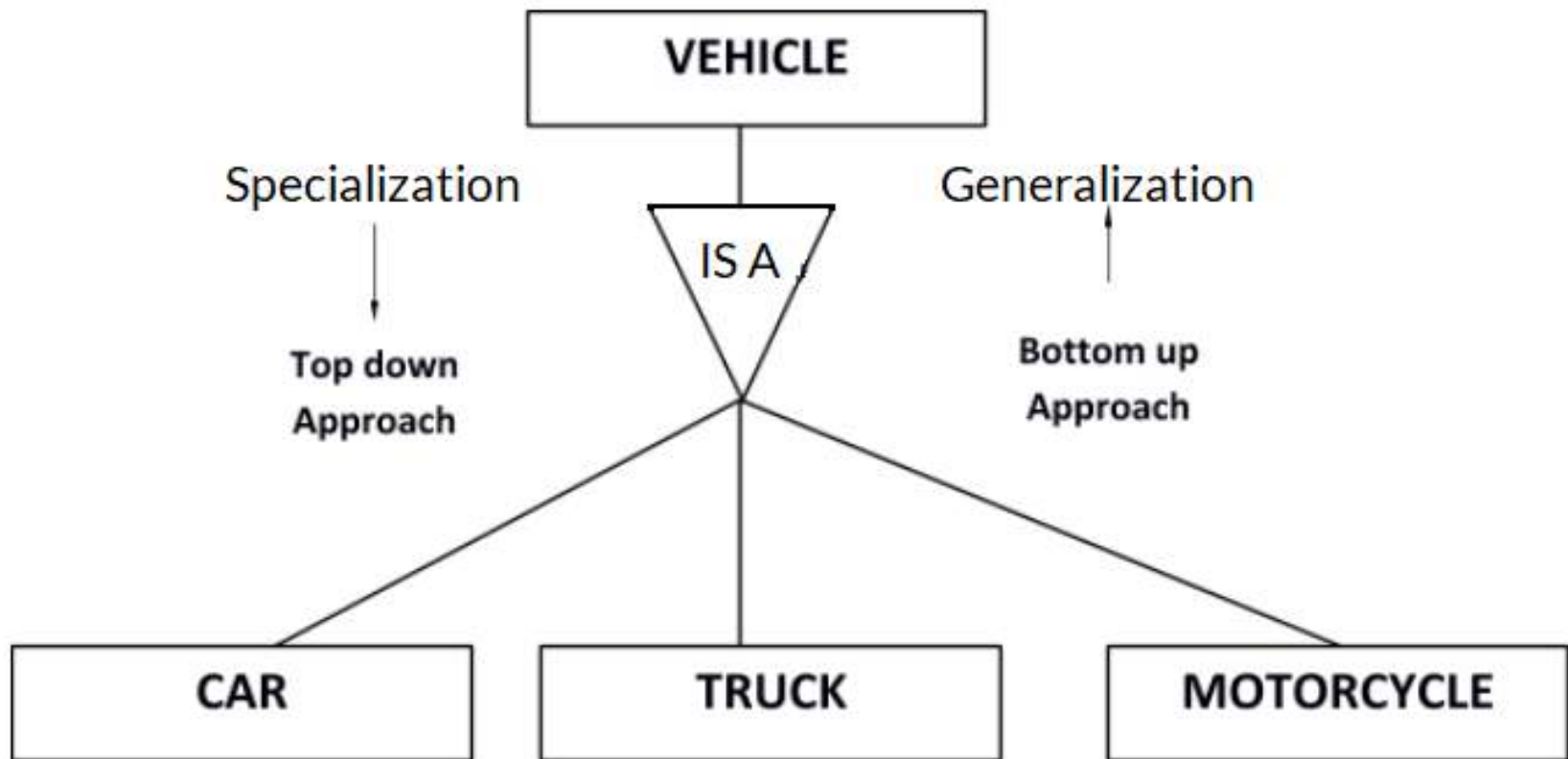
$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

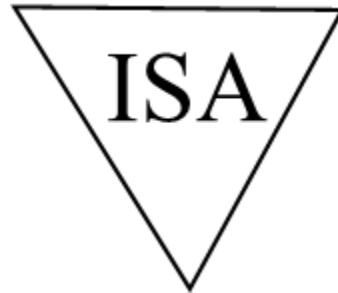
- Example:

$(44553, 22222) \in \text{advisor}$

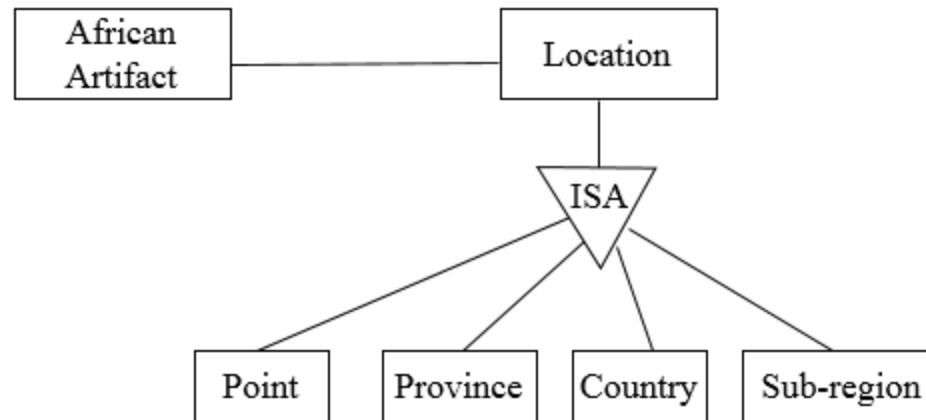
Constraints on specialization/generalization



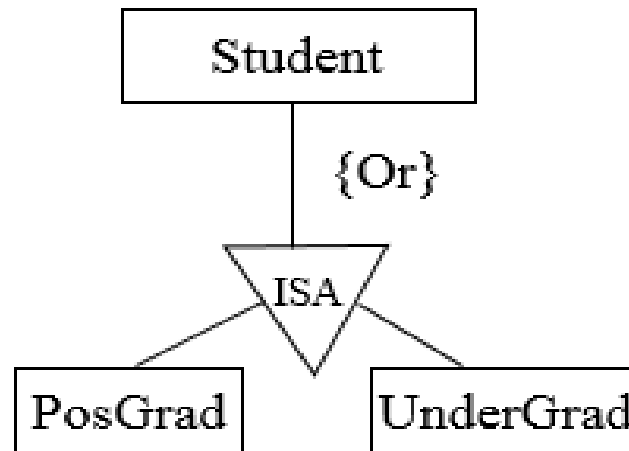
A diamond notation is a common representation of specialization/generalization relationships in ER diagrams.



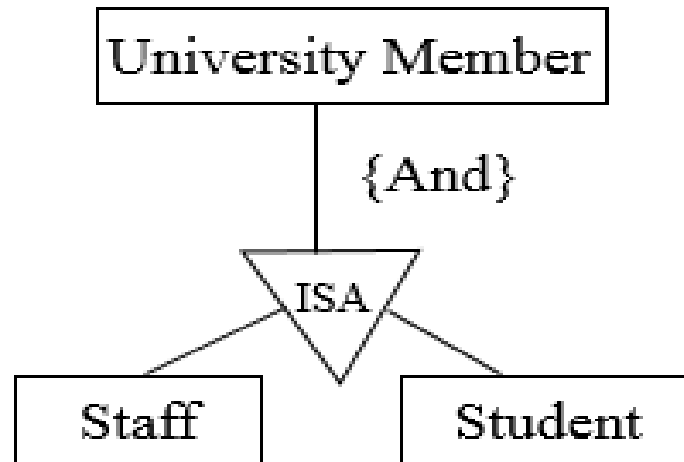
- The scenario has a specialization relationship between the location and different specific types of locations (i.e. point, province, country and sub-region). This specialization relationship is represented in the ER diagram below.



Disjoint: The disjoint constraint only applies when a superclass has more than one subclass. If the subclasses are disjoint, then an entity occurrence can be a member of only one of the subclasses, e.g. postgrads or undergrads 'OR' you cannot be both. To represent a disjoint superclass/subclass relationship, OR is used.

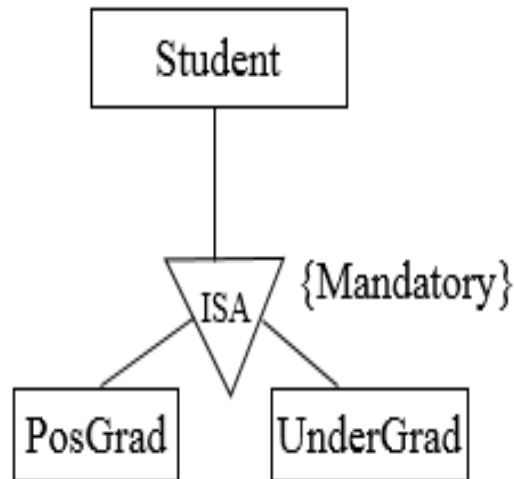


Overlapping: This applies when an entity occurrence may be a member of more than one subclass, e.g. student and staff and some people are both. 'And' is used to represent the overlapping specialization/generalization relationship in the ER diagram.

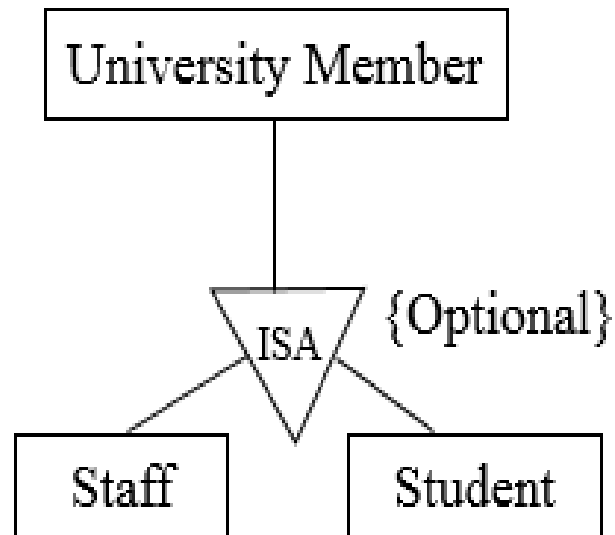


Completeness constraints

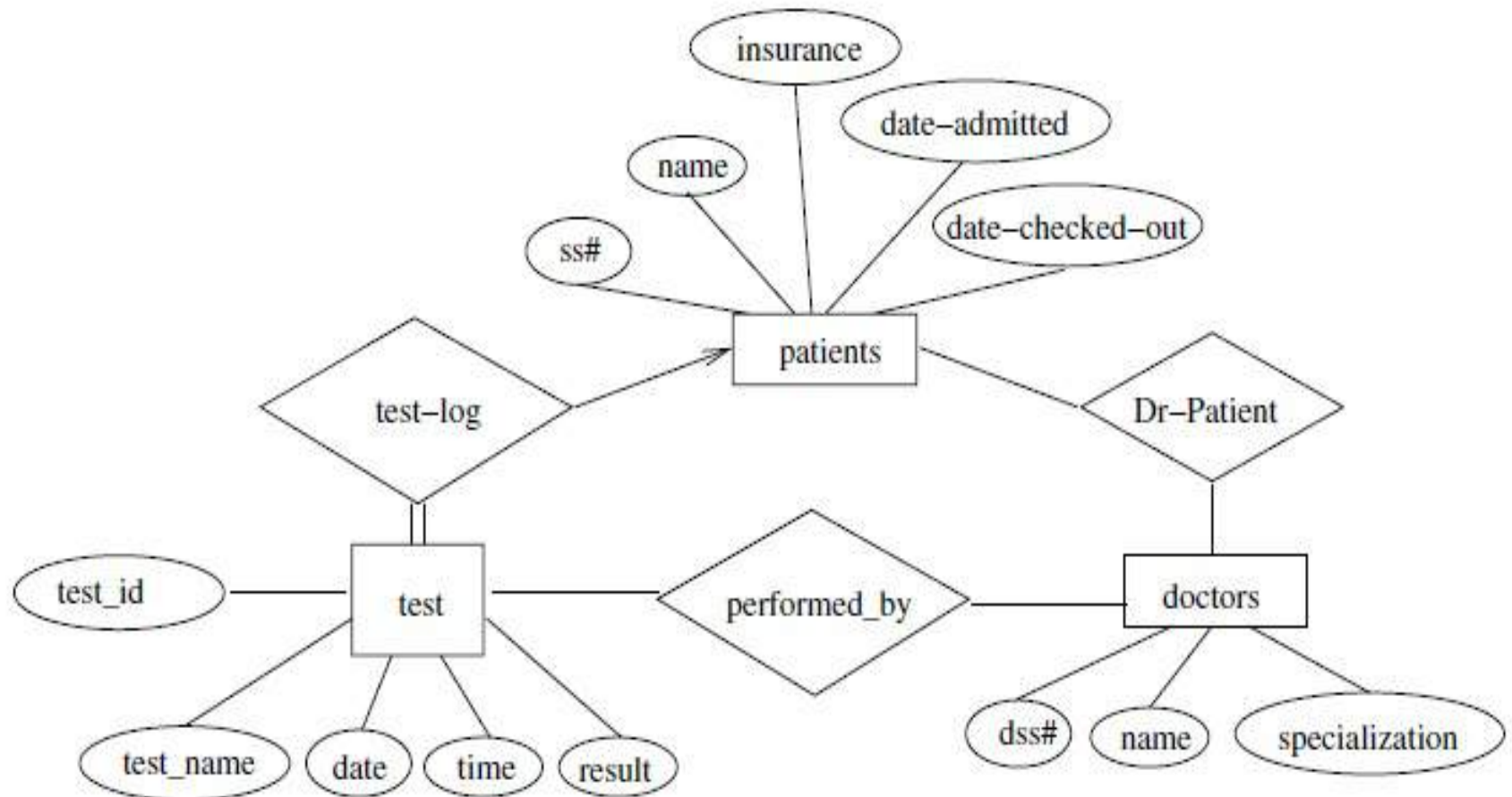
- **Total:** Each superclass (higher-level entity) must belong to subclasses (lower-level entity sets), e.g. a student must be postgrad or undergrad. To represent completeness in the specialization/generalization relationship, the keyword 'Mandatory' is used.



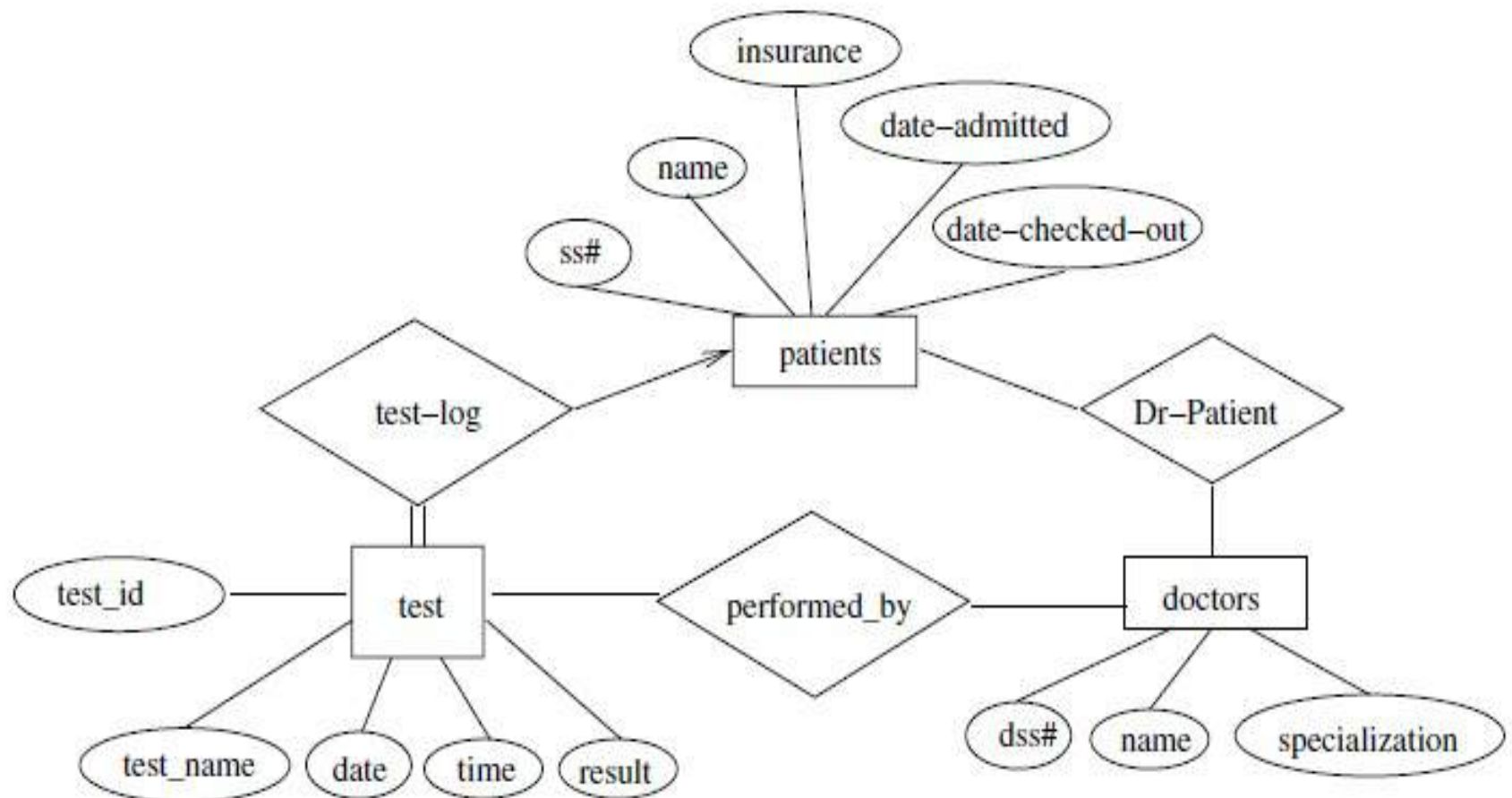
Partial: Some superclasses may not belong to subclasses (lower-level entity sets), e.g. some people at UCT are neither student nor staff. The keyword 'Optional' is used to represent a partial specialization/generalization relationship



- Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted.

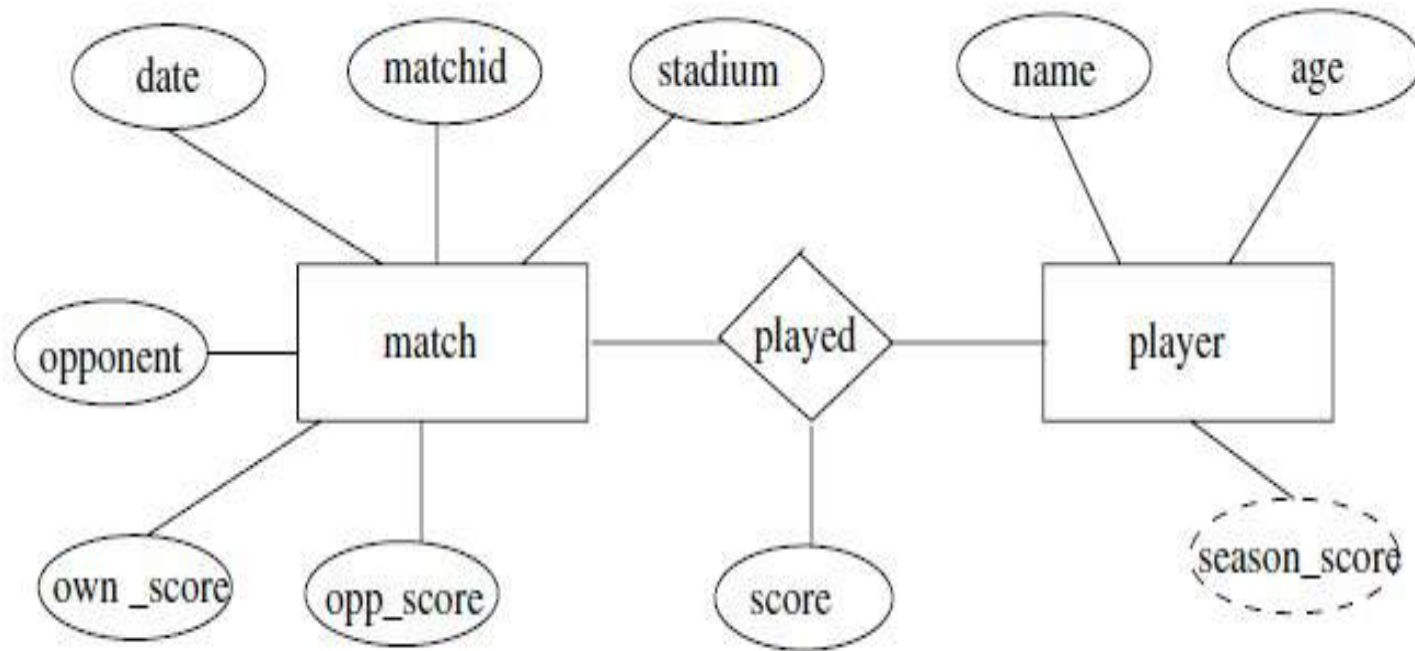


- Construct appropriate tables for each of the E-R diagram.



- Hospital tables:
 - patients (patient-id, name, insurance, date-admitted, date-checked-out)
 - doctors (doctor-id, name, specialization)
 - test (testid, testname, date, time, result)
 - doctor-patient (patient-id, doctor-id)
 - test-log (testid, patient-id) performed-by (testid, doctor-id)

- Design an E-R diagram for keeping track of the exploits of your favorite sports team. You should store the matches played, the scores in each match, the players in each match and individual player statistics for each match. Summary statistics should be modeled as derived attributes.





Graphic Era

(Deemed to be University)

Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 7

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

The *instructor* relation.

$\sigma_{dept_name = \text{"Physics"}} (instructor)$

$\sigma_{salary > 90000} (instructor)$

In general, we allow comparisons using $=$, $\neq$, $<$, $\leq$, $>$, and $\geq$ in the selection predicate. Furthermore, we can combine several predicates into a larger predicate by using the connectives *and* ($\wedge$), *or* ($\vee$), and *not* ($\neg$). Thus, to find the instructors in Physics with a salary greater than \$90,000, we write:

$\sigma_{dept_name = \text{"Physics"} \wedge salary > 90000} (instructor)$

RESULT:

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

SECTION
RELATION →

Consider a query to find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both.

Solution: $R1 = \pi_{course\ id} (\sigma_{semester = "Fall" \wedge year = 2009} (section))$

$R2 = \pi_{course\ id} (\sigma_{semester = "Spring" \wedge year = 2010} (section))$

RESULT: $R1 \cup R2$

<i>course_id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

- Notice that there are 8 tuples in the result, even though there are 3 distinct courses offered in the Fall 2009 semester and 6 distinct courses offered in the Spring 2010 semester.
- Since relations are sets, duplicate values such as CS-101, which is offered in both semesters, are replaced by a single occurrence.

Set difference

- Query: find all the courses taught in the Fall 2009 semester but not in Spring 2010 semester.
- Solution:
 - $R1 = \pi_{course\ id}(\sigma_{semester = "Fall" \wedge year = 2009}(section))$
 - $R2 = \pi_{course\ id}(\sigma_{semester = "Spring" \wedge year = 2010}(section))$
 - RESULT: $R1 - R2$

<i>course_id</i>
CS-347
PHY-101

Set difference

- Query: find all the courses taught in the SPRING 2010 semester but not in FALL 2009 semester.
- Solution:
 - $R1 = \pi_{course\ id}(\sigma_{semester = "Spring" \wedge year = 2010}(section))$
 - $R2 = \pi_{course\ id}(\sigma_{semester = "Fall" \wedge year = 2009}(section))$
 - RESULT: $R1 - R2$

NOTE: $R1 - R2$ is not equal to $R2 - R1$.

Course_id
Cs-351
Cs-319
FIN-201
HIS-351
MU-199

Renaming

- The **rename operator** ' ρ ' (*rho*) is one of the unary operators in relational algebra and is used to rename relations in a DBMS.

$$\rho_{new_relation_name(new_attribute_name_list)}(R)$$

- Three following types of renaming can be done:
 - Renaming a relation
 - Renaming an attribute
 - Renaming both

- **Rename a relation**

- Suppose we have a relation named *Students* and we want to change it to *FinalYrStudents*, the rename operation works as follows:

- $\rho_{FinalYrStudents}(Students)$

- **Rename an attribute**

- Suppose we have a relation named *Students* and we want to change its attributes *StudentID*, *StudentName* to *SID* and *SName*, the rename operation works as follows

- $\rho(SID, SName)(Students)$

- **Rename both**

- Next, we'll change both the relation name and attributes of the *Students* class:

- $\rho_{FinalYrStudents(SID, SName)}(Students)$

IMPORTANT

- There are actually three forms of the operation:

– *left outer join*, denoted 

– *right outer join*, denoted 

– *full outer join*, denoted 

union

R

Cust_name	Cust_status
Sham	Good
Rahul	Excellent
Mohan	Bad
Sachin	Excellent
Dinesh	Bad

S

Cust_name	Cust_status
Karan	Bad
Sham	Good
Sachin	Excellent
Rohan	Average

- R U S

Cust_name	Cust_status
Sham	Good
Rahul	Excellent
Mohan	Bad
Sachin	Excellent
Dinesh	Bad
Karan	Bad
Rohan	Average

- INTERSECTION

Cust_name	Cust_status
Sham	Good
Sachin	Excellent

QUESTION

Suppose there is a banking database which comprises following tables :

- Customer(Cust_name, Cust_street, Cust_city)
- Branch(Branch_name, Branch_city, Assets)
- Account (Branch_name, Account_number, Balance)
- Loan(Branch_name, Loan_number, Amount)
- Depositor(Cust_name, Account_number)
- Borrower(Cust_name, Loan_number)

Query : Find the names of all the customers who have taken a loan from the bank and also have an account at the bank.

- names of customers who have taken loan, indicates “borrowers” information.
 - So R 1= Borrower.
- Customer Name and Account number which can be obtained from “Depositor” relation.
 - R2=Depositor

- **Column 1 :** Cust_name from Borrower
 - $\pi_{Customer_name}(borrower)$
- **Column 2 :** Cust_name from Depositor
 - $\pi_{Customer_name}(Deositor)$
- We need to find out the **names of customers** who are present in **both Borrower** table and **Depositor** table. Hence, operator to be used “Intersection”.
 - $\pi_{Customer_name}(borrower) \cap \pi_{Customer_name}(Deositor)$
 -



Graphic Era

(Deemed to be University)
Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 7

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Relational Model and Relational Algebra

Relational Model Concepts

- The relational model represents the database as a collection of *relations*.
- When a relation is thought of as a **table** of values, each row in the table represents a collection of related data values.
- A row represents a fact that typically corresponds to a real-world entity or relationship.
- The table name and column names are used to help to interpret the meaning of the values in each row.

- the table below, is called STUDENT because each row represents facts about a particular student entity.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

- The column names—Name, Student_number, Class, and Major—specify how to interpret the data values in each row, based on the column each value is in.
- All values in a column are of the same data type.

- In the formal relational model terminology,
 - a row is called a *tuple*,
 - a column header is called an *attribute*,
 - and the table is called a *relation*.
- The data type describing the types of values that can appear in each column is represented by a *domain* of possible values.

Domains, Attributes, Tuples, and Relations

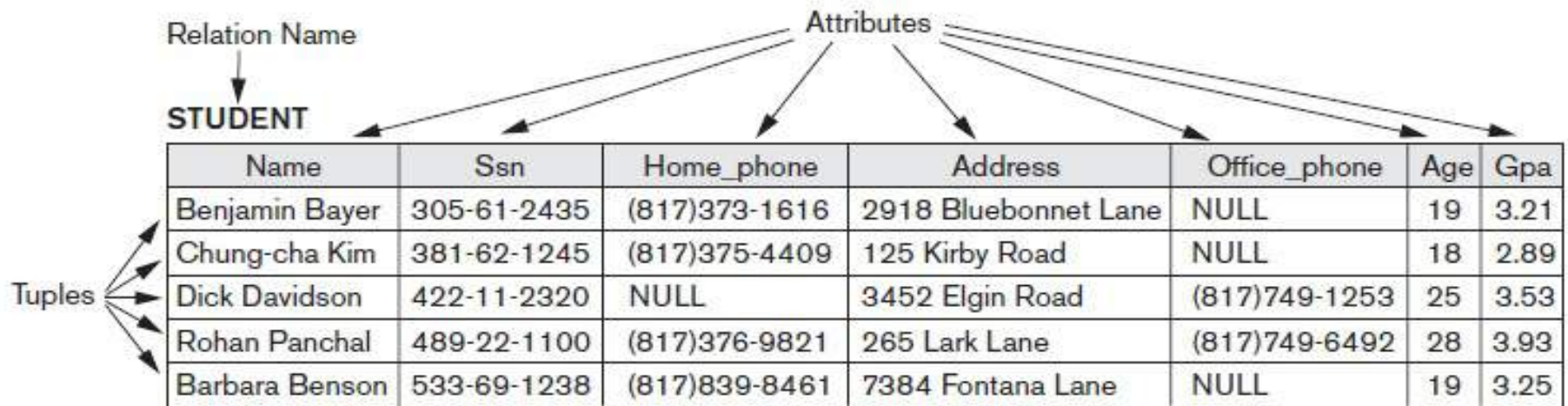
- A **domain** D is a set of atomic values. By **atomic** we mean that each value in the domain is indivisible as far as the formal relational model is concerned.
- A common method of specifying a domain is to specify a data type from which the data values forming the domain are drawn.

- Some examples of domains follow:
 - ***indian_phone_numbers***: The set of ten-digit phone numbers valid in INDIA.
 - ***Names***: The set of character strings that represent names of persons.
 - ***Employee_ages***: Possible ages of employees in a company; each must be an integer value between 15 and 80.
 - ***Academic_department_names***: The set of academic department names in a university, such as Computer Science, Economics, and Physics.

- A **data type** or **format** is also specified for each domain.
 - For example, the data type for the domain `indian_phone_numbers` can be declared as a character string of the form *(ddd)dddddddddd* where each *d* is a numeric (decimal) digit and the first three digits form a valid telephone area code.

- A **relation schema** (or relation scheme) R , denoted by $R(A_1, A_2, \dots, A_n)$, is made up of a relation name R and a list of attributes, $A_1, A_2, \dots, A_n$.
- Each **attribute** A_i is the name of a role played by some domain D in the relation schema R . D is called the **domain** of A_i and is denoted by **dom**(A_i).
- The **degree** (or **arity**) of a relation is the number of attributes n of its relation schema.

- A relation of degree ***seven***, which stores information about university students, would contain seven attributes describing each student as follows:
 - STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)
- Using the data type of each attribute, the definition is sometimes written as:
 - STUDENT(Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)



- Figure shows an example of a STUDENT relation, which corresponds to the STUDENT schema.
- Each tuple in the relation represents a particular student entity (or object).
- We display the relation as a table, where each tuple is shown as a *row* and each attribute corresponds to a *column header* indicating a role or interpretation of the values in that column.
- *NULL values* represent attributes whose values are unknown or do not exist for some individual STUDENT tuple.

Characteristics of Relations

- **Ordering of Tuples in a Relation.**
 - A relation is defined as a *set* of tuples. Mathematically, elements of a set have *no order* among them; hence, tuples in a relation do not have any particular order.
 - In other words, a relation is not sensitive to the ordering of tuples. However, in a file, records are physically stored on disk (or in memory), so there always is an order among the records.

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

- Hence, the relation displayed in Figure below is considered *identical* to the one shown in Figure above.
- When a relation is implemented as a file or displayed as a table, a particular ordering may be specified on the records of the file or the rows of the table.

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

- Values and NULLs in the Tuples
 - Each value in a tuple is an **atomic** value; that is, it is not divisible into components within the framework of the basic relational model.

Atomic values

- An *atomic value* is an instance of one of the built-in atomic data types that are defined by Schema. These data types include strings, integers, decimals, dates, and other atomic types. These types are described as atomic because they cannot be subdivided.
- ***Atomic*** means ***data*** which cannot be divided further.
Rule of *atomicity*:
 - **Means:** a column with ***atomic data*** can't have several ***values*** of the same type of ***data*** in the same column.

★ Values & Nulls in a Tuple:

- Each value in a tuple is an **atomic value**.

STUDENT	ROLL_NO	NAME	ADDRESS
	2	Ben	Bengaluru, Karnataka-560051

STUDENT	ROLL_NO	NAME	CITY	STATE	PINCODE
	2	Ben	Bengaluru	Karnataka	560051

- An important concept is that of NULL values, which are used to represent the values of attributes that may be unknown or may not apply to a tuple.
- special value, called NULL, is used in these cases.
 - For example, in Figure, some STUDENT tuples have NULL for their office phones because they do not have an office (that is, office phone *does not apply* to these students).

- A domain is a named set of scalar values, all of the same type.
 - Example 1: Domain of P# is the set of character strings of length 6.
 - Example 2: Domain of WEIGHT is the set of small integers less than 10,000.
 - Example 3: Domain of QTY is the set of integers less than one billion.
- **Therefore, a "domain" is a "data type".**

NAME	ROLL_NO	PHONE_NO	ADDRESS	AGE
Ram	14795	7305758992	Noida	24
Shyam	12839	9026288936	Delhi	35
Laxman	33289	8583287182	Gurugram	20
Mahesh	27857	7086819134	Ghaziabad	27
Ganesh	17282	9028 9i3988	Delhi	40

- In the given table, NAME, ROLL_NO, PHONE_NO, ADDRESS, and AGE are the attributes.
- The instance of schema STUDENT has 5 tuples.
- t3 = <Laxman, 33289, 8583287182, Gurugram, 20>

- Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.
 - **Domain:** It contains a set of atomic values that an attribute can take.
 - **Attribute:** It contains the name of a column in a particular table. Each attribute A_i must have a domain, $\text{dom}(A_i)$
 - **Relational instance:** In the relational database system, the relational instance is represented by a finite set of tuples. Relation instances do not have duplicate tuples.

- Degree (or arity) means the number of attributes.
- Cardinality means the number of tuples.

- **Relational schema:** A relational schema contains the name of the relation and name of all columns or attributes.
- **Relational key:** In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

- **Ordering of Values within a Tuple and an Alternative Definition of a Relation:**

According to the preceding definition of a relation, an n -tuple is an ordered list of n values, so the ordering of values in a tuple—and hence of attributes in a relation schema—is important.

★ Ordering of Values within a Tuple:

- A tuple $\rightarrow$ set of ($\langle$ attribute $\rangle$, $\langle$ value $\rangle$) pair, then ordering of attributes is not important.

$\dagger = \langle (\text{RollNo}, 2), (\text{Name}, \text{Ben}), (\text{Age}, 22) \rangle$

- **Interpretation (Meaning) of a Relation.** The relation schema can be interpreted as a **declaration** or a type of **assertion**.

For example, the schema of the STUDENT relation asserts that, in general, a student entity has a Name, Ssn, Home_phone, Address, Office_phone, Age, and Gpa. Each tuple in the relation can then be interpreted as a **fact** or a particular instance of the assertion.

Relational Model Constraints and Relational Database Schemas

- Constraints that are inherent in the data model. We call these **inherent model-based constraints** or **implicit constraints**. (eg. Duplicate tuples and rows are not allowed in relation)
- Constraints that can be directly expressed in schemas of the data model, typically by specifying them in the DDL (data definition language). **We call these schema-based constraints or explicit constraints.** (Eg. $16 < \text{Age} < 70$.)
- **Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs. We call these application-based or semantic constraints.**

Schemas constraints

- Domain Constraints:
 - Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain $\text{dom}(A)$.
 - Data type check.

STUDENT	RollNo	Name	Phone	Age
	1	Jeremy	2563251425	14
	2	Charles	1234568978	14
	3	Weston	3625368914	A

Violates Domain Constraints

- Key Constraints

- no two tuples in any relation state r of R should have the same combination of values for the attributes.
 - $t1[SK] \neq t2[SK]$
- Any such set of attributes SK is called a **superkey** of the relation schema R .
- A superkey SK specifies a *uniqueness constraint* that no two distinct tuples in any state r of R can have the same value for SK.

Name	Roll no	<u>Age</u>
Rahul	1	19
Rohit	2	19
Aakash	3	20

KEY



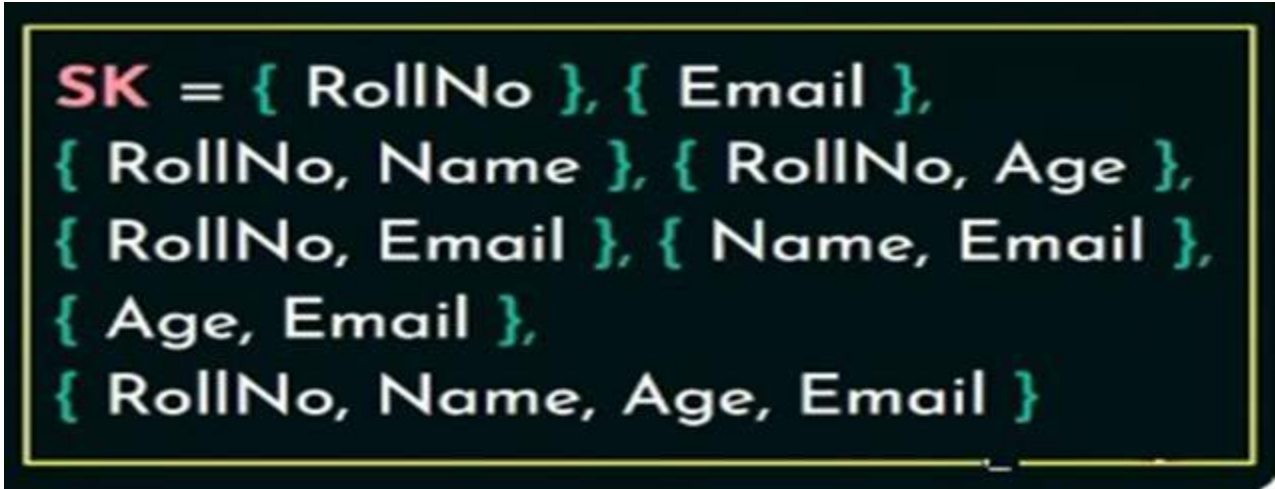
STUDENT	<u>RollNo</u>	Name	Age	Email
	1	Jeremy	14	jeremyl6@gmail.com
	2	Josh	14	josh25@gmail.com
	3	Charles	15	charly01@gmail.com

SK = { RollNo }, { Email },
{ RollNo, Name }, { RollNo, Age },
{ RollNo, Email }, { Name, Email },
{ Age, Email },
{ RollNo, Name, Age, Email }

- Hence, a key satisfies two properties:
 - Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key.This first property also applies to a superkey.

STUDENT	<u>RollNo</u>	Name	Age	Email
	1	Jeremy	14	jeremy16@gmail.com
	2	Josh	14	josh25@gmail.com
	3	Charles	15	charly01@gmail.com
Not possible	3	Alicia	13	alicia22@gmail.com

- It is a *minimal superkey*—that is, a superkey from which we cannot remove any attributes and still have the uniqueness constraint in condition 1 hold. This property is not required by a superkey.



SK = { RollNo }, { Email },
{ RollNo, Name }, { RollNo, Age },
{ RollNo, Email }, { Name, Email },
{ Age, Email },
{ RollNo, Name, Age, Email }

- In general, a relation schema may have more than one key. In this case, each of the keys is called a **candidate key**.
 - For example, the CAR relation below has two candidate keys: License_number and Engine_serial_number.

CAR

<u>License_number</u>	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

- It is common to designate one of the candidate keys as the **primary key** of the relation.
- Notice that when a relation schema has several candidate keys the choice of one to become the primary key is somewhat arbitrary; however, it is usually better to choose a primary key with a single attribute or a small number of attributes. The other candidate keys are designated as **unique keys**, and are not underlined.

Entity Integrity, Referential Integrity, and Foreign Keys

- The **entity integrity constraint** states that no primary key value can be NULL.

★ Constraints on Null Values:

- Specifies whether null values are permitted or not (**NOT NULL**).

STUDENT	<u>RollNo</u>	Name	Age	Grade
	1	Jeremy	14	A
	2	Charles	14	A
	3	Charles	13	B

★ Entity Integrity Constraint:

- States that **no primary key value can be null**.

STUDENT	<u>RollNo</u>	Name	Age	Grade
	1	Jeremy	14	A
	2	Charles	14	A
	null	Charles	13	B

- Referential integrity constraints
 - Key constraints and entity integrity constraints are specified on individual relations.
 - The **referential integrity constraint** is specified between two relations and is used to maintain the consistency among tuples in the two relations.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

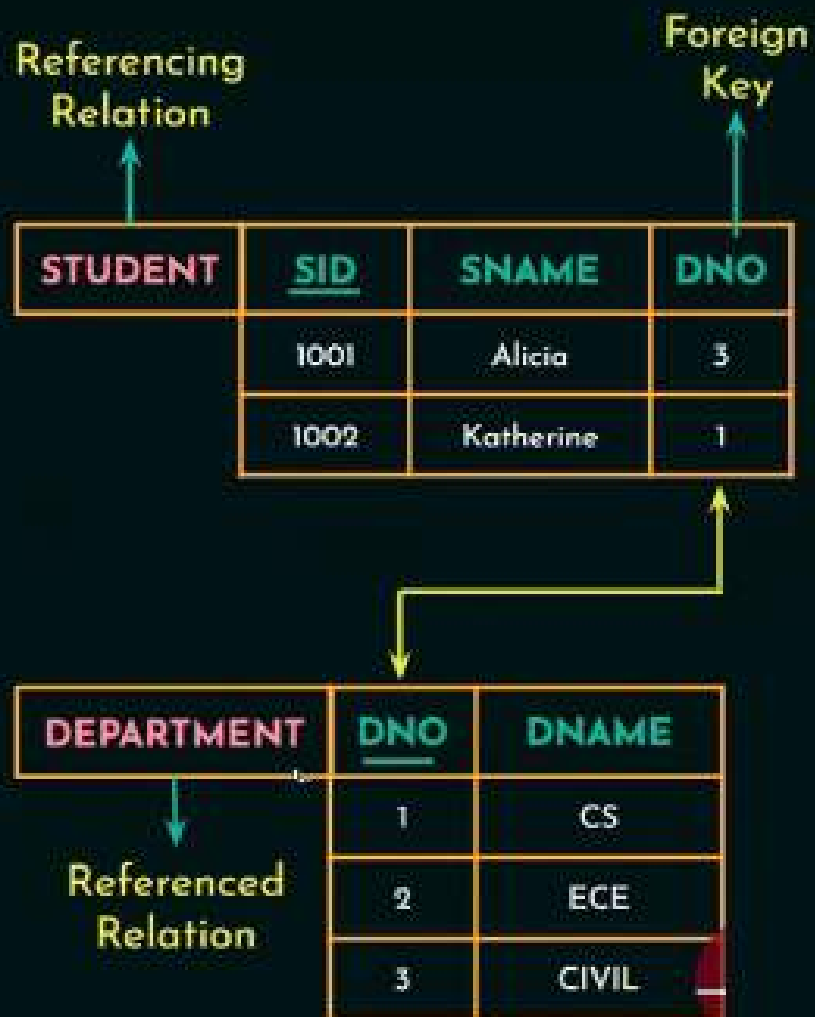
Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

the attribute ***Dno*** of EMPLOYEE gives the department number for which each employee works; hence, its value in every EMPLOYEE tuple must match the Dnumber value of some tuple in the DEPARTMENT relation.

- To define referential integrity more formally, first we define the concept of a ***foreign key***.
- The conditions for a foreign key, given below, specify a referential integrity constraint between the two relation schemas $R1$ and $R2$. A set of attributes FK in relation schema $R1$ is a **foreign key** of $R1$ that **references** relation $R2$ if it satisfies the following rules:
 - The attributes in FK have the same domain(s) as the primary key attributes PK of $R2$; the attributes FK are said to **reference** or **refer to** the relation $R2$.
 - we have $t_1[FK] = t_2[PK]$, and we say that the tuple t_1 **references** or **refers to** the tuple t_2 .



Update operation

- The operations of the relational model can be categorized into *retrievals* and *updates*.
- There are three basic operations that can change the states of relations in the database:
 - Insert, Delete, and Update (or Modify).
- They insert new data, delete old data, or modify existing data records. **Insert** is used to insert one or more new tuples in a relation, **Delete** is used to delete tuples, and **Update** (or **Modify**) is used to change the values of some attributes in existing tuples.

The Insert Operation

- The **Insert** operation provides a list of attribute values for a new tuple t that is to be inserted into a relation R .
- Insert can violate any of the four types of constraints discussed in the previous section.
 - Domain constraints can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type.

EMPLOYEE	<u>EmpID</u>	Name	Age	Phone	DNo
	1330	Benjamin	29	6562356521	2
	1331	Charles	31	4523651852	1
	1332	Harry	25	652562514	A

Violates Domain Constraint

- Key constraints can be violated if a key value in the new tuple t already exists in another tuple in the relation $r(R)$.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

- Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> into EMPLOYEE.
- *Result:* This insertion violates the **key constraint** because another tuple with the same **Ssn** value already exists in the EMPLOYEE relation, and so it is rejected.

- Entity integrity can be violated if any part of the primary key of the new tuple t is NULL.

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

- Insert <'Cecilia', 'F', 'Kolonsky', NULL, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.
- *Result:* This insertion violates the **entity integrity constraint** (NULL for the primary key Ssn), so it is rejected.

- Referential integrity can be violated if the value of any foreign key in t refers to a tuple that does not exist in the referenced relation.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

- Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.
- Result:* This insertion violates the **referential integrity constraint** specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.

- *Operation:*
 - Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.
 - ***Result:*** This insertion satisfies all constraints, so it is acceptable.

If an insertion violates one or more constraints, the default option is to reject the insertion.

DELETE Operation

EMPLOYEE	<u>EmpID</u>	Name	Age	Phone
	1330	Benjamin	29	6562356521
	1331	Charles	31	4523651852

PROJECT	<u>PNo</u>	ProjectName	DNo
	1	Global Payment	2
	2	Discover	1

WORKS_ON	<u>EmpID</u>	<u>PNo</u>	Hours
	1331	1	40
	1331	2	56
	1330	1	

- **Ex: (i)** Delete the EMPLOYEE tuple with EmpID = '1331' (Violates referential integrity)
- **(ii)** Delete the WORKS_ON tuple with EmpID = '1331' and PNo = 1 (acceptable).

- Several options are available if a deletion operation causes a violation.
 - The first option, called **restrict**, is to *reject the deletion*.
 - The second option, called **cascade**, is to *attempt to cascade (or propagate) the deletion* by deleting tuples that reference the tuple that is being deleted.

The Update operation

- The **Update** (or **Modify**) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation R .

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Operation:

Update the salary of the EMPLOYEE tuple with Ssn = '999887777' to 28000.

Result: Acceptable.

Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 1.

Result: Acceptable.

Operation:

Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.

Result: Unacceptable, because it violates referential integrity.

- *Operation:*

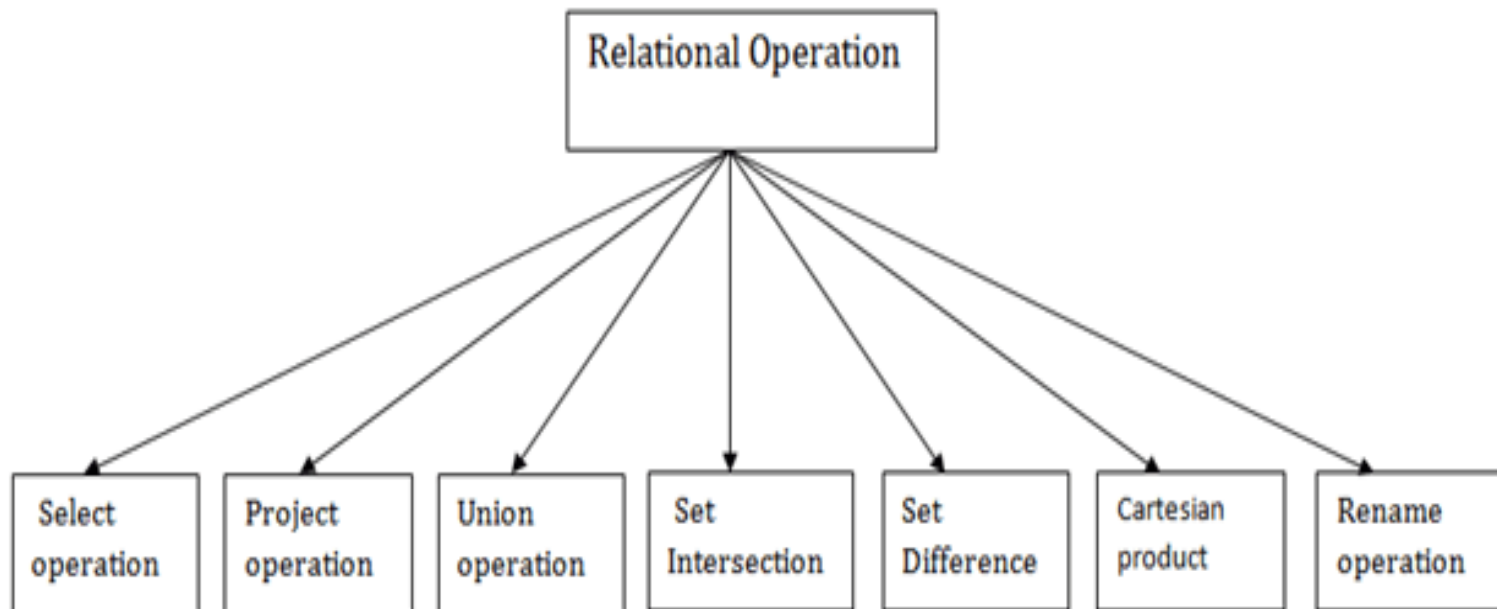
Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.

- *Result:* Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

Relational Algebra

- Relational algebra is a procedural query language. It gives a step by step process to obtain the result of the query. It uses operators to perform queries.

- **Types of relational Algebra**



Unary Relational Operations:

SELECT and PROJECT

- The SELECT operation is used to choose a *subset* of the tuples from a relation that satisfies a **selection condition**. One can consider the SELECT operation to be *filter* that keeps only those tuples that satisfy a qualifying condition.
- we can consider the SELECT operation to *restrict* the tuples in a relation to only those tuples that satisfy the condition.

- The SELECT operation can also be visualized as a *horizontal partition* of the relation into two sets of tuples—those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are discarded.
- **For example**, to select the EMPLOYEE tuples whose department is 4, or those whose salary is greater than \$30,000, we can individually specify each of these two conditions with a SELECT operation as follows:

— $\sigma_{\text{Dno}=4}(\text{EMPLOYEE})$

— $\sigma_{\text{Salary}>30000}(\text{EMPLOYEE})$

- In general, the SELECT operation is denoted by

— $\sigma_{\langle \text{selection condition} \rangle}(R)$

- where the symbol σ (sigma) is used to denote the SELECT operator and the selection condition is a Boolean expression (condition) specified on the attributes of relation R .

- The Boolean expression specified in <selection condition> is made up of a number of **clauses** of the form
 - <attribute name> <comparison op> <constant value>
or
 - <attribute name> <comparison op> <attribute name>
- where <attribute name> is the name of an attribute of R,
<comparison op> is normally one of the operators {=, <, ≤, >, ≥, ≠},
and <constant value> is a constant value from the attribute domain.

- For example, to select the tuples for all employees who either work in department 4 and make over \$25,000 per year, or work in department 5 and make over \$30,000, we can specify the following SELECT operation:

EMPLOYEE									
Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

$$-\sigma_{(Dno=4 \text{ AND } Salary>25000) \text{ OR } (Dno=5 \text{ AND } Salary>30000)}(EMPLOYEE)$$

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

- In SQL, the SELECT condition is typically specified in the WHERE clause of a query. For example, the following operation:

$\sigma_{\text{Dno}=4 \text{ AND Salary} > 25000}(\text{EMPLOYEE})$

would correspond to the following SQL query:

SELECT *

FROM EMPLOYEE

WHERE Dno=4 AND Salary>25000;

PROJECT Operation

- The SELECT operation chooses some of the **rows** from the table while discarding other rows.
- The **PROJECT** operation, on the other hand, selects certain **columns** from the table and discards the other columns.
- Therefore, the result of the **PROJECT** operation can be visualized as a *vertical partition* of the relation into two relations: one has the needed columns (attributes) and contains the result of the operation, and the other contains the discarded columns.

- For example, to list each employee's first and last name and salary, we can use the PROJECT operation as follows:

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

— $\pi_{\text{Lname, Fname, Salary}}(\text{EMPLOYEE})$

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

- The general form of the PROJECT operation is
 - $\pi_{\langle \text{attribute list} \rangle}(R)$
 - where π (pi) is the symbol used to represent the PROJECT operation, and $\langle \text{attribute list} \rangle$ is the desired sublist of attributes from the attributes of relation R .
- If the attribute list includes only ***non-key*** attributes of R , duplicate tuples are likely to occur. The PROJECT operation *removes any duplicate tuples*, so the result of the PROJECT operation is a set of distinct tuples, and hence a valid relation. This is known as **duplicate elimination**.

- For example, consider the following PROJECT operation:

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

$\pi_{\text{Sex, Salary}}(\text{EMPLOYEE})$

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

Notice that the tuple <'F', 25000> appears only once in Figure , even though this combination of values appears twice in the EMPLOYEE relation.

- In SQL, the PROJECT attribute list is specified in the SELECT clause of a query. For example, the following operation:

$$- \pi_{\text{Sex, Salary}}(\text{EMPLOYEE})$$

would correspond to the following SQL query:

```
SELECT DISTINCT Sex, Salary  
FROM EMPLOYEE
```

Sequences of Operations

- We can write a single relational algebra expression, also known as an **in-line expression**, as follows:

EMPLOYEE									
Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

— $\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

Relational Algebra Operations from Set Theory

- **The UNION, INTERSECTION, and MINUS Operations**

- The next group of relational algebra operations are the standard mathematical operations on sets.
- For example, to retrieve the Social Security numbers (ssn) of all employees who either work in department 5 or directly supervise an employee who works in department 5, we can use the UNION operation as follows:

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

- $\pi_{\text{Ssn}} (\sigma_{\text{Dno}=5} (\text{EMPLOYEE})) \cup \pi_{\text{Super_ssn}} (\sigma_{\text{Dno}=5} (\text{EMPLOYEE}))$

- Can also be written as:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$$

$$\text{RESULT1} \leftarrow \pi_{\text{Ssn}}(\text{DEP5_EMPS})$$

$$\text{RESULT2}(\text{Ssn}) \leftarrow \pi_{\text{Super_ssn}}(\text{DEP5_EMPS})$$

$$\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$$

- The relation **RESULT1** has the **Ssn** of all employees who work in **department 5**, whereas **RESULT2** has the **Ssn** of all employees who directly **supervise** an employee who works in **department 5**. The UNION operation produces the tuples that are in either **RESULT1** or **RESULT2** or both (see Figure below), while eliminating any duplicates.
- Thus, the Ssn value '333445555' appears only once in the result.

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Result of the UNION operation
 $\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Result of the UNION operation
 $RESULT \leftarrow RESULT1 \cup RESULT2$.

- We can define the three operations UNION, INTERSECTION, and SET DIFFERENCE on two union-compatible relations R and S as follows:
- UNION: The result of this operation, denoted by $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S . Duplicate tuples are eliminated.
- INTERSECTION: The result of this operation, denoted by $R \cap S$, is a relation that includes all tuples that are in both R and S .
- SET DIFFERENCE (or MINUS): The result of this operation, denoted by $R - S$, is a relation that includes all tuples that are in R but not in S .

- The relations STUDENT and INSTRUCTOR in Figure below are union compatible and their tuples represent the names of students and the names of instructors, respectively.

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

UNION

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

- The result of the INTERSECTION operation (Figure below) includes only those who are both students and instructors.

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

INTERSECTION



Fn	Ln
Susan	Yao
Ramesh	Shah

STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

- STUDENT – INSTRUCTOR

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

- INSTRUCTOR – STUDENT

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

- Notice that both UNION and INTERSECTION are *commutative operations*; that is,

$$R \cup S = S \cup R \text{ and } R \cap S = S \cap R$$

- Both UNION and INTERSECTION can be treated as *n*-ary operations applicable to any number of relations because both are also *associative operations*; that is,

$$R \cup (S \cup T) = (R \cup S) \cup T \text{ and } (R \cap S) \cap T = R \cap (S \cap T)$$

- The MINUS operation is *not commutative*; that is, in general,

$$R - S \neq S - R$$

The CARTESIAN PRODUCT (CROSS PRODUCT) Operation

- The **CARTESIAN PRODUCT** operation—also known as **CROSS PRODUCT** or **CROSS JOIN**—which is denoted by $\times$.
- In general, the result of $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$ is a relation Q with degree $n + m$ attributes $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The resulting relation Q has one tuple for each combination of tuples—one from R and one from S . Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.
- SELF JOIN

- A *Cartesian product* is the result of joining every row in one table with every row in another table. This occurs when there is no WHERE clause to restrict rows.
- Consider two relations student(sno, fname, lname) and DETAIL(rollno, age) below:

SNO	FNAME	LNAME
1	Albert	Singh
2	Nora	Fatehi

ROLLNO	AGE
5	18
9	21

- On applying CROSS PRODUCT on STUDENT and DETAIL:
 - STUDENT $\times$ DETAILS

SNO	FNAME	LNAME
1	Albert	Singh
2	Nora	Fatehi

ROLLNO	AGE
5	18
9	21

Output:

SNO	FNAME	LNAME	ROLLNO	AGE
1	Albert	Singh	5	18
1	Albert	Singh	9	21
2	Nora	Fatehi	5	18
2	Nora	Fatehi	9	21

We can observe that the number of tuples in STUDENT relation is 2, and the number of tuples in DETAIL is 2. So the number of tuples in the resulting relation on performing CROSS PRODUCT is $2 \times 2 = 4$.

- Important points on CARTESIAN PRODUCT(CROSS PRODUCT) Operation:

1. The cardinality (number of tuples) of resulting relation from a Cross Product operation is equal to the number of attributes(say m) in the first relation multiplied by the number of attributes in the second relation(say n).

- $\text{Cardinality} = m * n$

2. The Cross Product of two relation $A(R_1, R_2, R_3, \dots, R_p)$ with degree p , and $B(S_1, S_2, S_3, \dots, S_n)$ with degree n , is a relation $C(R_1, R_2, R_3, \dots, R_p, S_1, S_2, S_3, \dots, S_n)$ with degree $p + n$ attributes.

- $\text{Degree} = p + n$

3. CROSS PRODUCT is a binary set operation means, at a time we can apply the operation on two relations.

EXAMPLE

- Cartesian product operation: It combines R1 and R2 without any condition. It is denoted by X.
 - Degree of $R1 \times R2$ = degree of R1 + degree of R2
{degree = total no of columns}

Consider R1 table –

RegNo	Branch	Section
1	CSE	A
2	ECE	B
3	CIVIL	A
4	IT	B

Table R2

Name	RegNo
Bhanu	2
Priya	4

R1 X R2

RegNo	Branch	Section	Name	RegNo
1	CSE	A	Bhanu	2
1	CSE	A	Priya	4
2	ECE	B	Bhanu	2
2	ECE	B	Priya	4
3	CIVIL	A	Bhanu	2
3	CIVIL	A	Priya	4
4	IT	B	Bhanu	2
4	IT	B	Priya	4

Union Example

- UNION is symbolized by 'U' symbol. It includes all tuples that are in tables A or in B.
- It also eliminates duplicate tuples.
- For a union operation to be valid, the following conditions must hold –
 - R and S must be the same number of attributes.
 - Attribute domains need to be compatible.
 - Duplicate tuples should be automatically removed.

Table A		Table B	
column 1	column 2	column 1	column 2
1	1	1	1
1	2	1	3

UNION OPERATION

A $\cup$ B gives

Table A $\cup$ B	
column 1	column 2
1	1
1	2
1	3

EXAMPLE

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input: $\sigma_{\text{BRANCH_NAME}=\text{"perryride"}}$ (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

CARTESIAN PRODUCT EXAMPLE

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

Binary Relational Operations: JOIN and DIVISION

- The **JOIN** operation, denoted by ' $\bowtie$ ', is used to combine *related tuples* from two relations into single “longer” tuples.
- This operation is very important for any relational database with more than a single relation because it allows us to process relationships among relations.

- Common attribute (or column)
- Cross product and condition

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

- To illustrate JOIN, suppose that we want to retrieve the name of the manager of each department.
- To get the manager's name, we need to combine each department tuple with the employee tuple whose 'Ssn' value matches the 'Mgr_ssn' value in the department tuple.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

- We do this by using the JOIN operation and then projecting the result over the necessary attributes, as follows:

$$\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{Mgr_ssn}=\text{Ssn}} \text{EMPLOYEE}$$

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

- Note that ***Mgr_ssn*** is a foreign key of the DEPARTMENT relation that references ***Ssn***, the primary key of the EMPLOYEE relation.
- This referential integrity constraint plays a role in having matching tuple in the referenced relation EMPLOYEE.

NATURAL JOIN

EMPLOYEE

EMP_NO	EMP_NAME	LOCATION
1	rohit	bangalore
2	ajay	dehradun
3	arun	delhi
4	vikas	chennai

DEPARTMENT

DEPT_NO	DEPT_NAME	EMP_NO
D1	CSE	1
D2	IT	2
D3	ME	4

FIND THE EMPLOYEE NAMES WHO IS WORKING IN A DEPARTMENT?

EMP_NO	EMP_NAME	LOCATION	DEPT_NO	DEPT_NAME	EMP_NO
1	rohit	bangalore	D1	CSE	1 ★
1	rohit	bangalore	D2	IT	2
1	rohit	bangalore	D3	ME	4
2	ajay	dehradun	D1	CSE	1
2	ajay	dehradun	D2	IT	2 ★
2	ajay	dehradun	D3	ME	4
3	arun	delhi	D1	CSE	1
3	arun	delhi	D2	IT	2
3	arun	delhi	D3	ME	4
4	vikas	chennai	D1	CSE	1
4	vikas	chennai	D2	IT	2
4	vikas	chennai	D3	ME	4 ★

```

SELECT EMP_NAME FROM EMPLOYEE, DEPARTMENT WHERE EMPLOYEE.EMP_NO= DEPARTMENT.EMP_NO;

```

EMP_NO	EMP_NAME	LOCATION	DEPT_NO	DEPT_NAME	EMP_NO
1	rohit	bangalore	D1	CSE	1
2	ajay	dehradun	D2	IT	2
4	vikas	chennai	D3	ME	4

output

EMP_NAME
rohit
ajay
vikas

SELECT EMP_NAME **FROM** EMPLOYEE, DEPATMENT **WHERE** EMPLOYEE.EMP_NO= DEPARTMENT.EMP_NO

SELECT EMP_NAME **FROM** EMPLOYEE NATURAL JOIN DEPARTMENT;

IN TERMS OF LIVE ORACLE SQL

- `select * from hr.employees;`
- `select * from hr.departments;`
- `select FIRST_NAME from hr.employees natural
join hr.departments;`

- A NATURAL JOIN is a [JOIN operation](#) that creates an implicit join clause for you based on the common columns in the two tables being joined. Common columns are columns that have the same name in both tables.
- A NATURAL JOIN can be an INNER join, a LEFT OUTER join, or a RIGHT OUTER join. The default is INNER join.

EQUI JOIN

EMPLOYEE

EMP_NO	EMP_NAME	POSTAL_ADDRESS
1	rohit	delhi
2	ajay	dehradun
3	arun	bangalore
4	vikas	chennai

DEPARTMENT

DEPT_NO	DEPT_LOCATION	EMP_NO
D1	delhi	1
D2	haldwani	2
D3	haridwar	4

FIND THE EMPLOYEE NAMES WHO IS WORKING IN A DEPARTMENT HAVING DEPARTMENT LOCATION SAME AS THEIR POSTAL ADDRESS?

EMP_NO	EMP_NAME	POSTAL_ADD RESS	DEPT_NO	DEPT_LOCATI ON	EMP_NO
1	rohit	delhi	D1	delhi	1
1	rohit	delhi	D2	haldwani	2
1	rohit	delhi	D3	haridwar	4
2	ajay	dehradun	D1	delhi	1
2	ajay	dehradun	D2	haldwani	2
2	ajay	dehradun	D3	haridwar	4
3	arun	banglore	D1	delhi	1
3	arun	banglore	D2	haldwani	2
3	arun	banglore	D3	haridwar	4
4	vikas	chennai	D1	delhi	1
4	vikas	chennai	D2	haldwani	2
4	vikas	chennai	D3	haridwar	4

SELECT EMP_NAME **FROM** EMPLOYEE, DEPARTMENT **WHERE** EMPLOYEE.EMP_NO= DEPARTMENT.EMP_NO
AND EMPLOYEE.POSTAL_ADDRESS = DEPT_LOCATION;

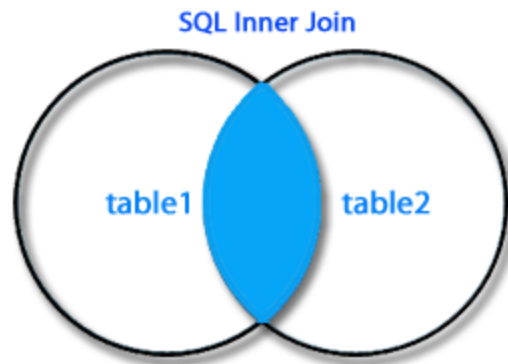
EMP_NO	EMP_NAME	POSTAL_ADDRESS	DEPT_NO	DEPT_LOCATION	EMP_NO
1	rohit	delhi	D1	delhi	1

OUTPUT

EMP_NAME
rohit

INNER JOIN

- The INNER JOIN selects all rows from both participating tables as long as there is a match between the columns. An SQL INNER JOIN is same as JOIN clause, combining rows from two or more tables.



- SELECT *

FROM table1 INNER JOIN table2

ON table1.column_name = table2.column_name;

FOOD			
ITEM_ID	ITEM_NAME	ITEM_UNIT	COMPANY_ID
1	Chex Mix	Pcs	16
6	Cheez-It	Pcs	15
2	BN Biscuit	Pcs	15
3	Mighty Munch	Pcs	17
4	Pot Rice	Pcs	15
5	Jaffa Cakes	Pcs	18
7	Salt n Shake	Pcs	

COMPANY		
COMPANY_ID	COMPANY_NAME	COMPANY_CITY
18	Order All	Boston
15	Jack Hill Ltd	London
16	Akas Foods	Delhi
17	Foodies.	London
19	sip-n-Bite.	New York

- `SELECT foods.item_name,foods.item_unit,
company.company_name,company.company_city
FROM foods INNER JOIN company
ON foods.company_id =company.company_id;`

ITEM_NAME	ITEM_	COMPANY_NAME	COMPANY_CITY
Chex Mix	Pcs	Akas Foods	Delhi
Cheez-It	Pcs	Jack Hill Ltd	London
BN Biscuit	Pcs	Jack Hill Ltd	London
Mighty Munch	Pcs	Foodies.	London
Pot Rice	Pcs	Jack Hill Ltd	London
Jaffa Cakes	Pcs	Order All	Boston

Exists in both
left and right tables
company and foods

Left Table
company

Right Table
foods

(18, Order All, Boston)
(18, Jaffa Cakes, Pcs)
(15, Jack Hill Ltd, London)
(15, Cheez-It, Pcs)
(15, Jack Hill Ltd, London)
(15, BN Biscuit, Pcs)
(15, Jack Hill Ltd, London)
(15, Pot Rice, Pcs)
(16, Akas Foods, Delhi)
(16, Chex Mix, Pcs)
(17, Foodies., London)
(17, Mighty Munch, Pcs)

- Inner Joins do not have to use equality to join the fields
- Can use < ,>, <>.

- `SELECT * FROM TableA INNER JOIN TableB ON TableA.PK > TableB.PK`
 - **ON TableA.PK > TableB.PK:** This is the join condition. It specifies that you want to join rows from TableA and TableB where the value of the "PK" column in TableA is greater than the value of the "PK" column in TableB. The "PK" column is usually a primary key, which is used to uniquely identify rows in a table.

EXAMPLE

ORDERS

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

CUSTOMERS

CustomerID	CustomerName	ContactName	Country
1	Alfreds Futterkiste	Maria Anders	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mexico

- SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate

FROM Orders INNER JOIN Customers

ON Orders.CustomerID=Customers.CustomerID;

OUTPUT

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	9/18/1996
10365	Antonio Moreno Taquería	11/27/1996
10383	Around the Horn	12/16/1996
10355	Around the Horn	11/15/1996
10278	Berglunds snabbköp	8/12/1996

EXAMPLE

EMPLOYEE

EMP_ID	EMP_NAME	CITY	SALARY	AGE
1	Angelina	Chicago	200000	30
2	Robert	Austin	300000	26
3	Christian	Denver	100000	42
4	Kristen	Washington	500000	29
5	Russell	Los angels	200000	36
6	Marry	Canada	600000	48

PROJECT

PROJECT_NO	EMP_ID	DEPARTMENT
101	1	Testing
102	2	Development
103	3	Designing
104	4	Development

- SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT

FROM EMPLOYEE INNER JOIN PROJECT

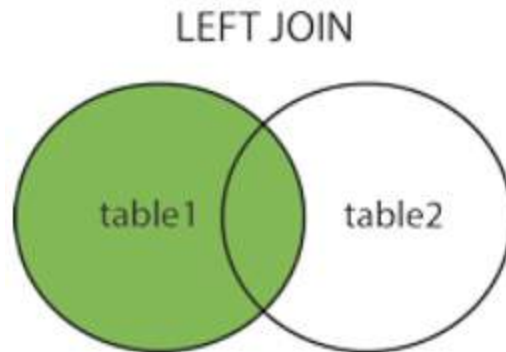
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development

LEFT JOIN or LEFT OUTER JOIN

- The SQL left join returns all the values from left table and the matching values from the right table. If there is no matching join value, it will return NULL.



- SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT

FROM EMPLOYEE LEFT JOIN PROJECT

ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;

Output

EMP_NAME	DEPARTMENT
Angelina	Testing
Robert	Development
Christian	Designing
Kristen	Development
Russell	NULL
Marry	NULL

EXAMPLE

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

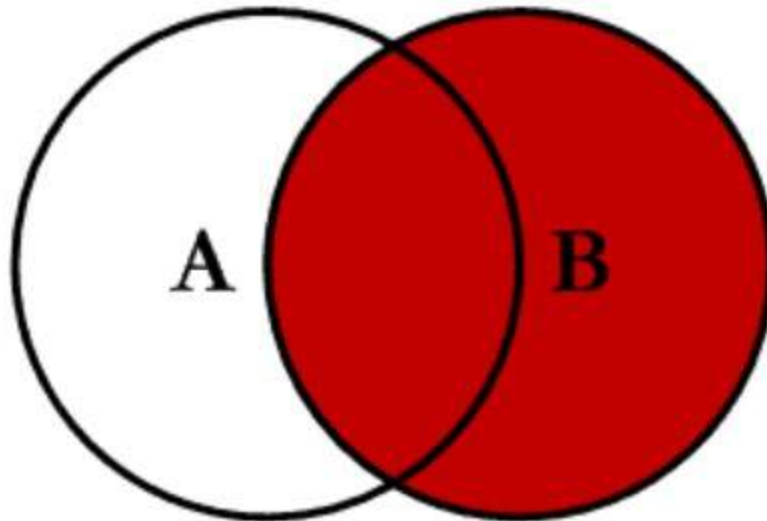
- `SELECT Student.NAME, StudentCourse.COURSE_ID
FROM Student LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;`

Output:

NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL

RIGHT JOIN

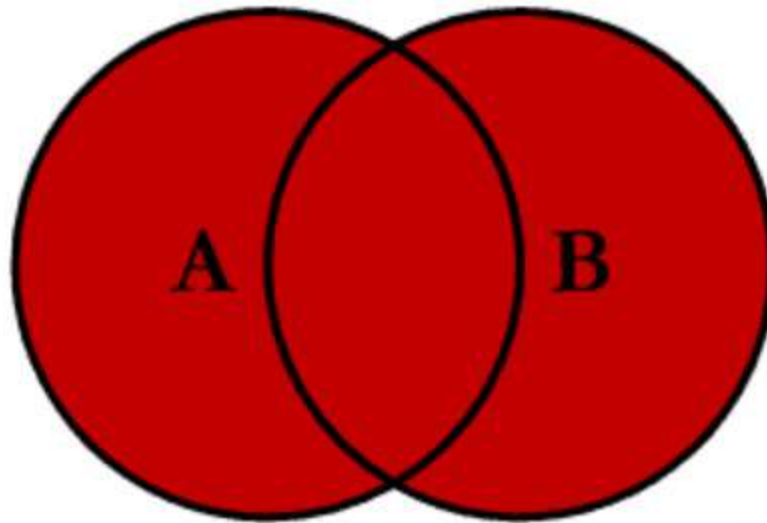
- In SQL, RIGHT JOIN returns all the values from the values from the rows of right table and the matched values from the left table. If there is no matching in both tables, it will return NULL.



TableA Value	PK	TableB PK	Value
FOX	1	1	TROT
COP	2	2	CAR
TAXI	3	3	CAB
WASHINGTON	6	6	MONUMENT
DELL	7	7	PC
NULL	NULL	8	MICROSOFT
NULL	NULL	9	APPLE
NULL	NULL	11	SCOTCH

Full Outer Join

- Full outer join produces the set of all records in Table A and Table B, with matching records from both sides where available. If there is no match, the missing side will contain null.

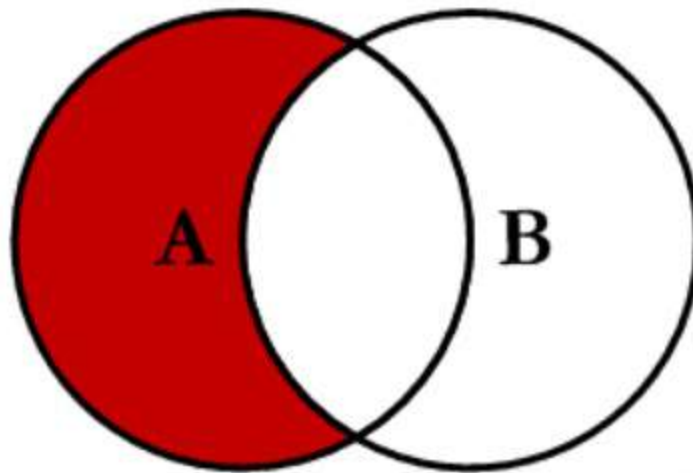


TableA Value	PK	TableB PK	Value
FOX	1	1	TROT
COP	2	2	CAR
TAXI	3	3	CAB
LINCOLN	4	NULL	NULL
ARIZONA	5	NULL	NULL
WASHINGTON	6	6	MONUMENT
DELL	7	7	PC
LUCENT	10	NULL	NULL
NULL	NULL	8	MICROSOFT
NULL	NULL	9	APPLE
NULL	NULL	11	SCOTCH

- MySQL doesn't have FULL OUTER JOIN
- Simulate using UNION, LEFT and RIGHT JOINS
 - SELECT * FROM TableA LEFT JOIN TableB ON TableA.PK = TableB.PK UNION SELECT * FROM TableA RIGHT JOIN TableB ON TableA.PK = TableB.PK

Left Join Excluding Inner Join

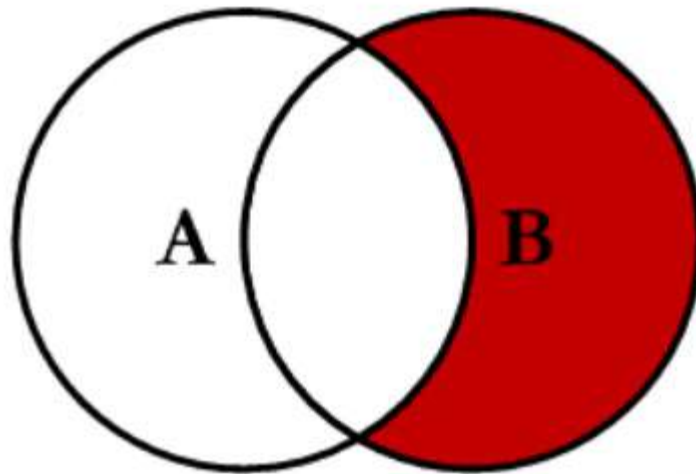
- This query will return all of the records in the left table (table A) that do not match any records in the right table (table B).



- `SELECT * FROM TableA LEFT JOIN TableB ON TableA.PK = TableB.PK WHERE TableB.PK IS NULL`
- Perform left outer join, then exclude the records we don't want from the right side via a where clause

Right Join Excluding Inner Join

- This query will return all of the records in the right table (table B) that do not match any records in the left table (table A).



- `SELECT * FROM TableA RIGHT JOIN TableB ON TableA.PK = TableB.PK WHERE TableA.PK IS NULL`
- Perform right outer join, then exclude the records we don't want from the left side via a where clause.

Cross Join

- A cross join is a Cartesian Product join – it is every record in Table A combined with every record in Table B.
 - `SELECT * from TableA CROSS JOIN TableB`
 - `SELECT * from TableA, TableB`

supplier_id	supplier_name
10000	IBM
10001	Hewlett Packard
10002	Microsoft
10003	NVIDIA

order_id	supplier_id	order_date
500125	10000	2003/05/12
500126	10001	2003/05/13
500127	10004	2003/05/14

- `SELECT suppliers.supplier_id, suppliers.supplier_name, orders.order_date
FROM suppliers INNER JOIN orders ON suppliers.supplier_id =
orders.supplier_id;`

supplier_id	name	order_date
10000	IBM	2003/05/12
10001	Hewlett Packard	2003/05/13

supplier_id	supplier_name
10000	IBM
10001	Hewlett Packard
10002	Microsoft
10003	NVIDIA

order_id	supplier_id	order_date
500125	10000	2003/05/12
500126	10001	2003/05/13
500127	10004	2003/05/14

- `SELECT suppliers.supplier_id, suppliers.supplier_name, orders.order_date
FROM suppliers LEFT OUTER JOIN orders ON suppliers.supplier_id =
orders.supplier_id;`

supplier_id	supplier_name	order_date
10000	IBM	2003/05/12
10001	Hewlett Packard	2003/05/13
10002	Microsoft	<null>
10003	NVIDIA	<null>

supplier_id	supplier_name
10000	IBM
10001	Hewlett Packard
10002	Microsoft
10003	NVIDIA

order_id	supplier_id	order_date
500125	10000	2003/05/12
500126	10001	2003/05/13
500127	10004	2003/05/14

- SELECT suppliers.supplier_id, suppliers.supplier_name, orders.order_date
FROM suppliers **FULL OUTER JOIN** orders ON suppliers.supplier_id =
orders.supplier_id;

supplier_id	supplier_name	order_date
10000	IBM	2013/08/12
10001	Hewlett Packard	2013/08/13
10002	Microsoft	<null>
10003	NVIDIA	<null>
<null>	<null>	2013/08/14

DIVISION METHOD in Relational Algebra

- The division operator is used for queries which involve the 'all'.
- $R1 \div R2$ = tuples of R1 associated with all tuples of R2.

Name	Course
System	Btech
Database	Mtech
Database	Btech
Algebra	Btech



Course
Btech
Mtech



NAME
Database

ENROLLED

STUDENT_ID	COURSE_ID
Student_1	DBMS
Student_2	DBMS
Student_1	OS
Student_3	OS

COURSE

COURSE_ID
DBMS
OS

- $\text{ENROLLED}(\text{STUDENT_ID}, \text{COURSE_ID}) / \text{COURSE}(\text{COURSE_ID})$

This will return the following relation as output.

STUDENT_ID
Student_1

EXAMPLE 1: RELATIONAL ALGEBRA

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

$\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{Dname}='Research'}(\text{DEPARTMENT})$

$\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{Dnumber}=\text{Dno}} \text{EMPLOYEE})$

$\text{RESULT} \leftarrow \pi_{\text{Fname}, \text{Lname}, \text{Address}}(\text{RESEARCH_EMPS})$

- As a single in-line expression, this query becomes:

$\pi_{\text{Fname, Lname, Address}} (\sigma_{\text{Dname='Research', Dnumber=Dno}} (\text{DEPARTMENT EMPLOYEE}))$



Graphic Era

(Deemed to be University)

Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 8

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

SQL Data Definition and Data Types

- SQL is a comprehensive database language: It has statements for data definitions, queries, and updates. Hence, it is both a DDL *and* a DML.
- SQL uses the terms **table**, **row**, and **column** for the formal relational model terms *relation*, *tuple*, and *attribute*, respectively.
 - The main SQL command for data definition is the **CREATE** statement, which can be used to create schemas, tables (relations), and domains (as well as other constructs such as views, assertions, and triggers).

The CREATE TABLE Command in SQL

- The **CREATE TABLE** command is used to specify a new relation by giving it a name and specifying its attributes and initial constraints.
- The attributes are specified first, and each attribute is given a name, a data type to specify its domain of values, and any attribute constraints, such as NOT NULL.
- The key, entity integrity, and referential integrity constraints can be specified within the CREATE TABLE statement after the attributes are declared, or they can be added later using the ALTER TABLE command
 - **CREATE TABLE** EMPLOYEE (.....;

- The relations declared through CREATE TABLE statements are called **base tables** (or base relations); this means that the relation and its tuples are actually created and stored as a file by the DBMS.
- Base relations are distinguished from **virtual relations**, created through the CREATE VIEW statement, which may or may not correspond to an actual physical file.
- In SQL, the attributes in a base table are considered to be *ordered in the sequence in which they are specified* in the CREATE TABLE statement. However, rows (tuples) are not considered to be ordered within relation.

```

CREATE TABLE EMPLOYEE
( Fname          VARCHAR(15)          NOT NULL,
  Minit          CHAR,
  Lname         VARCHAR(15)          NOT NULL,
  Ssn           CHAR(9)             NOT NULL,
  Bdate         DATE,
  Address       VARCHAR(30),
  Sex           CHAR,
  Salary        DECIMAL(10,2),
  Super_ssn     CHAR(9),
  Dno           INT                 NOT NULL,
  PRIMARY KEY (Ssn),
  FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
  FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber) );

CREATE TABLE DEPARTMENT
( Dname          VARCHAR(15)          NOT NULL,
  Dnumber        INT                 NOT NULL,
  Mgr_ssn       CHAR(9)             NOT NULL,
  Mgr_start_date DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );

```

It is important to note that in table above, there are some *foreign keys that may cause errors* because they are specified either via circular references or because they refer to a table that has not yet been created.

For example, the foreign key ***Super_ssn*** in the EMPLOYEE table is a circular reference because it refers to the table itself.

The foreign key ***Dno*** in the EMPLOYEE table refers to the DEPARTMENT table, which has not been created yet.

To deal with this type of problem, these constraints can be left out of the initial CREATE TABLE statement, and then added later using the ALTER TABLE statement.

Attribute Data Types and Domains in SQL

- **Numeric:** **Numeric** data types include integer numbers of various sizes (INTEGER or INT, and SMALLINT) and floating-point (real) numbers of various precision (FLOAT or REAL, and DOUBLE PRECISION).
 - Formatted numbers can be declared by using DECIMAL(*i,j*)—or DEC(*i,j*) or NUMERIC(*i,j*)—where *i*, the *precision*, is the total number of decimal digits and *j*, the *scale*, is the number of digits after the decimal point.
- **Character-string**
- **DATE:** The **DATE** data type has ten positions, and its components are YEAR, MONTH, and DAY in the form YYYY-MM-DD.
- **Boolean:** A **Boolean** data type has the traditional values of TRUE or FALSE.
- **A timestamp**

Specifying Constraints in SQL

- SQL allows NULLs as attribute values, a *constraint* NOT NULL may be specified if NULL is not permitted for a particular attribute.
- This is always implicitly specified for the attributes that are part of the *primary key* of each relation.
- It is also possible to define a *default value* for an attribute by appending the clause
 - **DEFAULT** <value> to an attribute definition.
- The default value is included in any new tuple if an explicit value is not provided for that attribute.

Set default value of college_country column to 'US'

```
CREATE TABLE College ( college_id INT PRIMARY  
KEY, college_code VARCHAR(20), college_country  
VARCHAR(20) DEFAULT 'US' );
```

- Here, the default value of the college_country column is **US**. If we try to store a NULL value in the college_country column, its value will be **US** by default.

- Another type of constraint can restrict attribute or domain values using the **CHECK** clause following an attribute or domain definition.
 - **WHERE (AGE >20)**

Specifying Key and Referential Integrity Constraints

- Keys and referential integrity constraints are very important, there are special clauses within the CREATE TABLE statement to specify them.
- The **PRIMARY KEY** clause specifies one or more attributes that make up the primary key of a relation. (**NOT NULL** and **UNIQUE**)
- Referential integrity is specified via the **FOREIGN KEY** clause.
- A referential integrity constraint can be violated when tuples are inserted or deleted, or when a foreign key or primary key attribute value is modified. (SET NULL, cascade)

Specifying Constraints on Tuples

Using CHECK

- In addition to key and referential integrity constraints, which are specified by special keywords, other *table constraints* can be specified through additional CHECK clauses at the end of a CREATE TABLE statement.
- These can be called **tuple-based** constraints because they apply to each tuple *individually* and are checked whenever a tuple is inserted or modified.
- For example, suppose that the DEPARTMENT table had an additional attribute Dept_create_date, which stores the date when the department was created. Then we could add the following CHECK clause at the end of the CREATE TABLE statement for the DEPARTMENT table to make sure that manager's start date is later than the department creation date.
- **CHECK** (Dept_create_date <= Mgr_start_date);

Basic Retrieval Queries in SQL

- SQL has one basic statement for retrieving information from a database: the **SELECT** statement.
- The SELECT statement *is not the same as* the SELECT operation of relational algebra.

SELECT <attribute list>

FROM <table list>

WHERE <condition>;

where

- <attribute list> is a list of attribute names whose values are to be retrieved by the query.
- <table list> is a list of the relation names required to process the query.
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

Example

- Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

SELECT Bdate, Address

FROM EMPLOYEE

WHERE Fname='John' **AND** Minit='B' **AND** Lname='Smith';

<u>Bdate</u>	<u>Address</u>
1965-01-09	731Fondren, Houston, TX

Ambiguous Attribute Names, Aliasing, Renaming, and Tuple Variables

- In SQL, the same name can be used for two (or more) attributes as long as the attributes are in *different relations*.
- If this is the case, and a multi-table query refers to two or more attributes with the same name, we *must qualify* the attribute name with the relation name to prevent ambiguity. This is done by *prefixing* the relation name to the attribute name and separating the two by a **period “.”**.
 - **SELECT** Fname, EMPLOYEE.Name, Address
FROM EMPLOYEE, DEPARTMENT

- An alias can follow the keyword **AS**.
- for example, by writing EMPLOYEE E in the FROM clause.

```
SELECT E.Fname, E.Lname  
FROM EMPLOYEE AS E
```

- It is also possible to **rename** the relation attributes within the query in SQL by giving them aliases.
- For example, if we write

```
EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno)
```

in the FROM clause, Fn becomes an alias for Fname, Mi for Minit, Ln for Lname, and so on.

Recursive query

- **Query 8.** For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname  
FROM EMPLOYEE AS E, EMPLOYEE AS S  
WHERE E.Super_ssn=S.Ssn;
```

- we can think of E and S as two *different copies* of the EMPLOYEE relation; the first, E, represents employees in the role of supervisees or subordinates; the second, S, represents employees in the role of supervisors.
- We can now join the two copies. Of course, in reality there is *only one* EMPLOYEE relation, and the join condition is meant to join the relation with itself by matching the tuples that satisfy the join condition $E.Super_ssn = S.Ssn$.

Unspecified WHERE Clause and Use of the Asterisk

- A *missing* WHERE clause indicates no condition on tuple selection; hence, *all tuples* of the relation specified in the FROM clause qualify and are selected for the query result. If more than one relation is specified in the FROM clause and there is no WHERE clause, then the CROSS PRODUCT—*all possible tuple combinations*—of these relations is selected.

- **Q1: SELECT Ssn**
FROM EMPLOYEE;
- **Q2: SELECT Ssn, Dname**
FROM EMPLOYEE, DEPARTMENT;
- For example, Query q selects all EMPLOYEE Ssns, and Query 10 selects all combinations of an EMPLOYEE Ssn and a DEPARTMENT Dname, regardless of whether the employee works for the department or not.
 - Notice that Q2 is similar to a CROSS PRODUCT operation followed by a PROJECT

- To retrieve all the attribute values of the selected tuples, we do not have to list the attribute names explicitly in SQL; we just specify an *asterisk* (*), which stands for *all the attributes*.

Tables as Sets in SQL

- As we mentioned earlier, SQL usually treats a table not as a set but rather as a **multiset**; *duplicate tuples can appear more than once* in a table, and in the result of a query.
- If we *do want* to eliminate duplicate tuples from the result of an SQL query, we use the keyword **DISTINCT** in the SELECT clause, meaning that only distinct tuples should remain in the result.
- In general, a query with SELECT DISTINCT eliminates duplicates, whereas a query with SELECT ALL does not.

- There are set union (UNION), set difference (**EXCEPT**), and set intersection (**INTERSECT**) operations.

Substring Pattern Matching and Arithmetic Operators

- The first feature allows comparison conditions on only parts of a character string, using the **LIKE** comparison operator. This can be used for string **pattern matching**.
- Partial strings are specified using two reserved characters: % replaces an arbitrary number of zero or more characters, and the underscore (_) replaces a single character.

- The standard arithmetic operators for addition (+), subtraction (−), multiplication (*), and division (/) can be applied to numeric values or attributes with numeric domains.
- For string data types, the concatenate operator || can be used in a query to append two string values.
- Another comparison operator, which can be used for convenience, is **BETWEEN**,

- **SELECT ***
FROM EMPLOYEE
WHERE (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
- The condition (Salary **BETWEEN** 30000 **AND** 40000) in Q14 is equivalent to the condition ((Salary >= 30000) **AND** (Salary <= 40000)).

Ordering of Query Results

- SQL allows the user to order the tuples in the result of a query by the values of one or more of the attributes that appear in the query result, by using the **ORDER BY** clause.
- The default order is in ascending order of values. We can specify the keyword **DESC** if we want to see the result in a descending order of values. The keyword **ASC** can be used to specify ascending order explicitly.

- A *simple* retrieval query in SQL can consist of up to four clauses, but only the first two—SELECT and FROM—are mandatory.
 - **SELECT** <attribute list>
FROM <table list>
[**WHERE** <condition>]
[**ORDER BY** <attribute list>];
- Two additional clauses GROUP BY and HAVING. These can be used to provide additional power to aggregate functions; and various types of joins that can combine records from various tables in different ways.

INSERT, DELETE, and UPDATE

Statements in SQL

- In SQL, three commands can be used to modify the database: INSERT, DELETE, and UPDATE.
 - In its simplest form, INSERT is used to add a single tuple to a relation.
 - The values should be listed *in the same order* in which the corresponding attributes were specified in the CREATE TABLE command.
 - **INSERT INTO** EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard', 'Marini', 4, '653298653');

- The DELETE command removes tuples from a relation. It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted.

- **DELETE FROM** EMPLOYEE
WHERE Lname='Brown';

- **DELETE FROM** EMPLOYEE
WHERE Ssn='123456789';

- The **UPDATE** command is used to modify attribute values of one or more selected tuples.
- As in the DELETE command, a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation. However, updating a primary key value may propagate to the foreign key values of tuples in other relations if such a *referential triggered action* is specified in the referential integrity constraints of the DDL.

Comparisons Involving NULL and Three-Valued Logic

- SQL has various rules for dealing with NULL values.
- NULL is used to represent a missing value, but that it usually has one of three different interpretations
 - value *unknown* (exists but is not known),
 - value *not available* (exists but is purposely withheld),
or
 - value *not applicable* (the attribute is undefined for this tuple).

- **Unknown value.** A person's date of birth is not known, so it is represented by NULL in the database.
- **Unavailable or withheld value.** A person has a home phone but does not want it to be listed, so it is withheld and represented as NULL in the database.
- **Not applicable attribute.** An attribute Last_College_Degree would be NULL for a person who has no college degrees because it does not apply to that person.
 - When a NULL is involved in a comparison operation, the result is considered to be UNKNOWN (it may be TRUE or it may be FALSE).

- SQL uses a three-valued logic with values TRUE, FALSE, and UNKNOWN instead of the standard two-valued (Boolean) logic with values TRUE or FALSE.
- It is therefore necessary to define the results (or truth values) of three-valued logical expressions when the logical connectives AND, OR, and NOT are used.

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN
OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN
NOT			
TRUE	FALSE		
FALSE	TRUE		
UNKNOWN	UNKNOWN		

- For example, the result of (FALSE **AND** UNKNOWN) is FALSE, whereas the result of (FALSE **OR** UNKNOWN) is UNKNOWN. The result of the NOT logical operation. Notice that in standard Boolean logic, only TRUE or FALSE values are permitted; there is no UNKNOWN value.

- **SELECT** Fname, Lname
FROM EMPLOYEE
WHERE Super_ssn **IS** NULL;

SQL allows queries that check whether an attribute value is **NULL**.

NESTED QUERIES

- **SELECT** **DISTINCT** Pnumber
FROM PROJECT
WHERE Pnumber **IN**
(**SELECT** Pnumber
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE Dnum=Dnumber **AND**
Mgr_ssn=Ssn **AND** Lname='Smith')

OR

Pnumber **IN**
(**SELECT** Pno
FROM WORKS_ON, EMPLOYEE
WHERE Essn=Ssn **AND** Lname='Smith');

- The first nested query selects the project numbers of projects that have an employee with last name 'Smith' involved as manager, while the second nested query selects the project numbers of projects that have an employee with last name 'Smith' involved as worker.



Graphic Era

(Deemed to be University)

Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 9

B. Tech- CSE
3rd Year (5th semester)

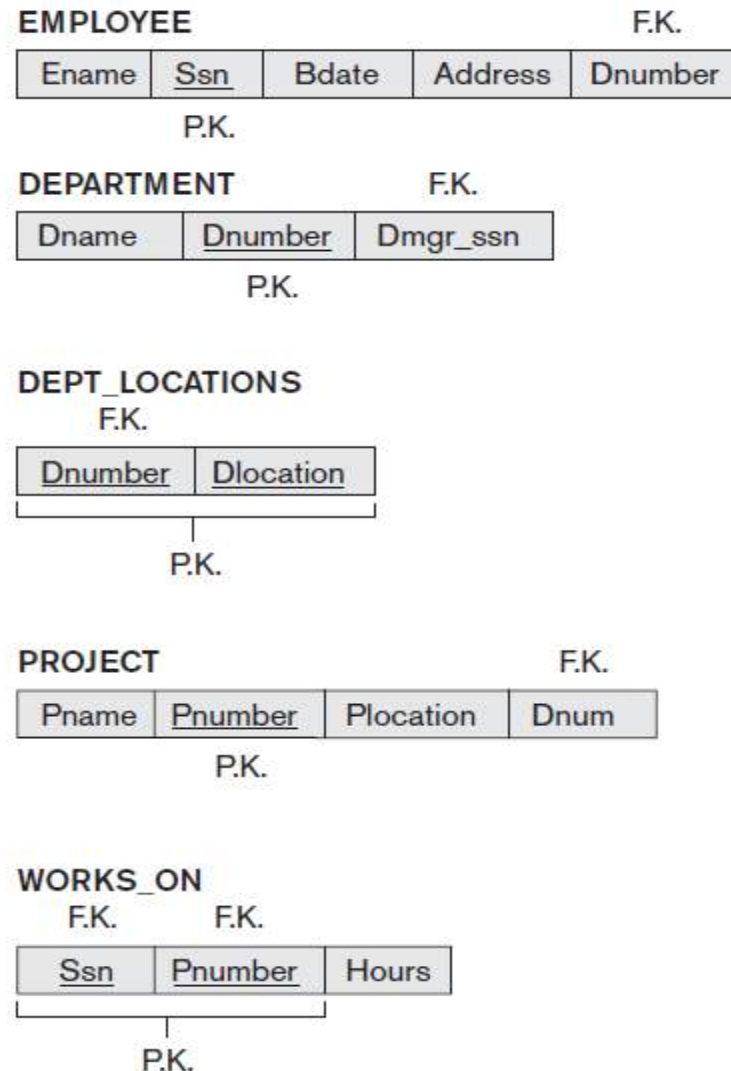
REHAN, Assistant Professor
+91-9548283808

Unit 4: Database Design

Informal Design Guidelines for Relation Schemas

- Four *informal guidelines* that may be used as *measures to determine the quality* of relation schema design:
 - Making sure that the semantics of the attributes is clear in the schema
 - Reducing the redundant information in tuples
 - Reducing the NULL values in tuples
 - Disallowing the possibility of generating spurious tuples

- In general, the easier it is to explain the semantics of the relation, the better the relation schema design will be.
- To illustrate this, consider Figure below, a simplified version of the COMPANY relational database schema



Example:

Employee number	Employee Name	Salary	City
1	Dana	50000	San Francisco
2	Francis	38000	London
3	Andrew	25000	Tokyo

In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number.

Functional dependencies help ensure data integrity and play a crucial role in normalization.

Functional dependencies

X	Y
a	1
b	2
c	3

Functional Dependencies: $X \longrightarrow Y$

A functional dependency between two attributes, often denoted as X and Y , states that for every unique value of X , there is a unique value of Y . In other words, knowing the value of X will allow you to determine the value of Y .

- **Definition.** A **functional dependency**, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a *constraint* on the possible tuples that can form a relation state r of R . The constraint is that, for any two tuples $t1$ and $t2$ in r that have $t1[X] = t2[X]$, they must also have $t1[Y] = t2[Y]$.
- Means, using a value of one attribute, the value of another attribute can be searched.

for any two tuples $t1$ and $t2$ in r that have $t1[X] = t2[X]$, they must also have $t1[Y] = t2[Y]$.

R

X	Y
a	1
b	2
c	3

- This means that the values of the Y component of a tuple in r depend on, or are *determined by*, the values of the X component;
- We also say that there is a functional dependency from X to Y , or that Y is **functionally dependent** on X .

TRIVIAL

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}**. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

For example,

Let us now consider the same <Section> table with two separate attributes to comprehend the trivial dependency's concept. Here is a trivial functional dependency because SecId is a subset of SecId and SecName.

$\{\text{Sec_Id}, \text{Sec_Name}\} \rightarrow \text{Sec_Id}$

Non-TRIVIAL

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **roll_no** $\rightarrow$ **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**. Similarly, **{roll_no, name}** $\rightarrow$ **age** is also a non-trivial functional dependency, since **age** is **not a subset of** **{roll_no, name}**

For example,

Sec_Id $\rightarrow$ Sec_Name

The FD given above is a functional dependency that is of a non-trivial type since SecName is not a subset of SecId.

EXAMPLE

Figure 15.8

A relation $R(A, B, C, D)$
with its extension.

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Here, the following FDs *may or may not hold*

$B \rightarrow C$;

$C \rightarrow B$;

$\{A, B\} \rightarrow C$;

$\{A, B\} \rightarrow D$; and

$\{C, D\} \rightarrow B$. **(because the four tuples in the current extension have no violation of these constraints)**

However,

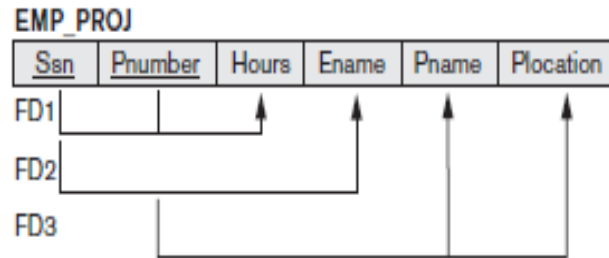
$A \rightarrow B$ (tuples 1 and 2 violate this constraint);

$B \rightarrow A$ (tuples 2 and 3 violate this constraint);

$D \rightarrow C$ (tuples 3 and 4 violate it).

(do not hold because we already have violations of them in the given extension)

- a. $Ssn \rightarrow Ename$
- b. $Pnumber \rightarrow \{Pname, Plocation\}$
- c. $\{Ssn, Pnumber\} \rightarrow Hours$



- These functional dependencies specify that:
 - (a) the value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename),
 - (b) the value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation), and
 - (c) a combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours).
- Alternatively, we say that Ename is functionally determined by (or functionally dependent on) Ssn, or *given a value of Ssn, we know the value of Ename*, and so on.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

- A relation state of TEACH with a *possible* functional dependency $TEXT \rightarrow COURSE$. However, $TEACHER \rightarrow COURSE$, $TEXT \rightarrow TEACHER$ and $COURSE \rightarrow TEXT$ are ruled out.
- At first glance we may think that $Text \rightarrow Course$, we cannot confirm this unless we know that it is true *for all possible legal states* of TEACH. It is, however, sufficient to demonstrate *a single counterexample* to disprove a functional dependency. For example, because 'Smith' teaches both 'Data Structures' and 'Data Management,' we can conclude that Teacher *does not* functionally determine Course.

EXAMPLE 2

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

- roll_no $\rightarrow$ { name, dept_name, dept_building }
- dept_name $\rightarrow$ dept_building
- name $\rightarrow$ dept_name
- dept_building $\rightarrow$ dept_name

- **Armstrong's axioms/properties of functional dependencies:**
 - **Reflexivity:** If Y is a subset of X, then $X \rightarrow Y$ holds by reflexivity rule
Example: {roll_no, name} $\rightarrow$ name is valid.
 - **Augmentation:** If $X \rightarrow Y$ is a valid dependency, then $XZ \rightarrow YZ$ is also valid by the augmentation rule.
Example: {roll_no, name} $\rightarrow$ dept_building **is valid**, hence {roll_no, name, dept_name} $\rightarrow$ {dept_building, dept_name} is also valid.
 - **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$ are both valid dependencies, then $X \rightarrow Z$ is also valid by the Transitivity rule.
Example: roll_no $\rightarrow$ dept_name & dept_name $\rightarrow$ dept_building, then roll_no $\rightarrow$ dept_building **is also valid**.

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

SECONDARY RULES

- **Union:** IF $X \rightarrow Y$ && $X \rightarrow Z$
THEN $X \rightarrow YZ$.
- **Decomposition:** IF $X \rightarrow YZ$.
Then $X \rightarrow Y$ && $X \rightarrow Z$
- **Pseudo transitivity:** IF $X \rightarrow Y$ && $WY \rightarrow Z$
THEN $WX \rightarrow Z$.

(NOTE: applied on a right hand side)

EXAMPLE

- Consider the following table of data $r(R)$ of the relation schema $R(ABCDE)$ shown in Table R:

A	B	C	D	E
a1	b1	c1	d1	e1
a2	b1	c2	d2	e1
a3	b2	c1	d1	e1
a4	b2	c2	d2	e1
a5	b3	c3	d1	e1

Table R

- What kind of dependencies can we observe among the attributes in Table R?*
 - Since the values of A are unique (a1, a2, a3, etc.), it follows from the FD definition that:
 - $A \rightarrow B$, $A \rightarrow C$, $A \rightarrow D$, $A \rightarrow E$
 - It also follows that $A \rightarrow BC$ (or any other subset of ABCDE).
 - This can be summarized as $A \rightarrow BCDE$.
 - From our understanding of primary keys, A is a primary key.
- Other observations:
 - Combinations of BC are unique, therefore $BC \rightarrow ADE$.
 - Combinations of BD are unique, therefore $BD \rightarrow ACE$.

Functional dependencies play a crucial role in database design and optimization for several reasons:

- **Data Integrity:** Functional dependencies ensure the integrity of the data in a relational database.
- **Normalization:** Functional dependencies are the foundation of the normalization process in database design. By identifying and addressing these dependencies, you can break down large, complex tables into smaller, more manageable ones, reducing data redundancy and improving data organization.

- **Reduction of Data Redundancy:** Functional dependencies help eliminate data redundancy by ensuring that data is stored in the most efficient manner. By organizing data into separate tables based on their functional dependencies, you avoid duplicate information and make updates or inserts less error-prone.
- **Anomaly Prevention:** Functional dependencies help prevent data anomalies, such as insertion, update, and deletion anomalies.
- **Query Optimization:** Understanding functional dependencies can lead to more efficient query design.
- **Schema Design:** Functional dependencies guide the design of the database schema. They help determine which attributes belong in which tables, what should be a primary key, and how tables should be related to each other.
- **Candidate Key Selection:** Functional dependencies help identify candidate keys in a relation. Candidate keys are crucial for defining the uniqueness of records and ensuring data integrity.

EXAMPLE

A	B	C
1	4	2
1	5	3
1	6	3
3	2	2

1. $AB \rightarrow C \ \&\& \ C \rightarrow B$
2. $BC \rightarrow A \ \&\& \ B \rightarrow C$
3. $BC \rightarrow A \ \&\& \ A \rightarrow C$
4. $AC \rightarrow B \ \&\& \ B \rightarrow C$

EXAMPLE

X	Y	Z
1	2	4
3	5	4
3	7	2
1	4	2

1. $X \rightarrow Y \ \&\& \ YZ \rightarrow X$
2. $Z \rightarrow Y \ \&\& \ ZX \rightarrow Y$
3. $Y \rightarrow Z \ \&\& \ XY \rightarrow Z$
4. $X \rightarrow Z \ \&\& \ YZ \rightarrow X$

Closure set of attributes

- Attribute closure, also known as the closure of an attribute set, is a fundamental concept in database theory and plays a crucial role in determining keys and superkeys in a relational database.
- **Definition:** The closure of an attribute set X , denoted as X^+ , is the set of all attributes that can be determined by X using a given set of functional dependencies.

- $R(XYZ)$
 - $X \rightarrow Y$
 - $Y \rightarrow Z$
 - $(X^+) =$
 - X
 - XY
 - XYZ
 - XYZ is the closure of X

- $R(UVWXYZ)$
 - $U \rightarrow V$
 - $W \rightarrow XY$
 - $UW \rightarrow Z$
 - $X \rightarrow UZ$
 - $Y \rightarrow WZ$
 - $(X^+) =$
 - X
 - UXZ
 - $UVXZ$

$UVXZ$ IS A CLOSURE SET OF X .

- (XY^+)
 - UXY
 - $UWXYZ$
 - $UVWXYZ$

- $R(ABCDEFG)$
 - $A \rightarrow B$
 - $BC \rightarrow D$
 - $E \rightarrow F$
 - $F \rightarrow G$
- Calculate the closure of the attribute set $\{A\}$ based on the provided functional dependencies.
 - $\{A, B\}$.
- Determine the closure of the attribute set $\{C\}$ using the given functional dependencies.
 - $\{C\}$
- Find the closure of the attribute set $\{E, G\}$ with respect to the functional dependencies.
 - $\{E, F, G\}$
- $\text{Closure}(\{E\}) = \{E, F, G\}$
- $\text{Closure}(\{F\}) = \{F, G\}$
- $\text{Closure}(\{G\}) = \{G\}$
- $\text{Closure}(\{BC\}) = \{BC, D\}$
- $\text{Closure}(\{AC\}) = \{A, B, C, D\}$
- $\text{Closure}(\{A, E\}) = \{A, E, F, B, G\}$

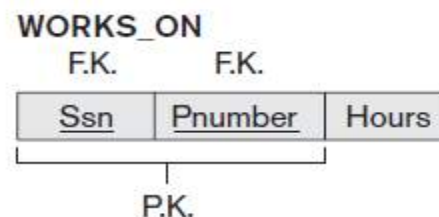
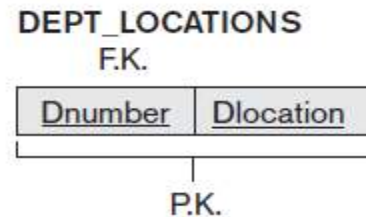
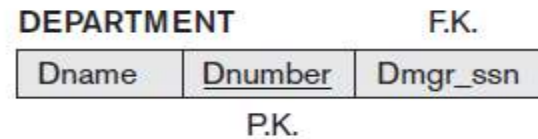
example

- $R(ABCDEFGG)$
 - $A \rightarrow B$
 - $BC \rightarrow DE$
 - $AEG \rightarrow G$
 - $\text{FIND}(AC^+) =$
 - $ABCDE$

KEYS

- CANDIDATE KEY
- PRIMARY KEY
- SUPER KEY
- The difference between a key and a superkey is that a key has to be *minimal*;
- If a relation schema has more than one key, each is called a **candidate key**. One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called secondary keys.

- {Ssn} is a **key** for EMPLOYEE, whereas {Ssn}, {Ssn, Ename}, {Ssn, Ename, Bdate}, and any set of attributes that includes Ssn are all **superkeys**.



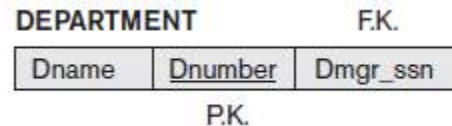
Normal Forms Based on Primary Keys

- Normalization is a process that minimizes data redundancy, reduces anomalies, and ensures data integrity by organizing data in a structured and efficient manner.
- Normal forms are a set of rules or guidelines that determine the degree to which a table is normalized. These rules are based on functional dependencies and are often related to the use of primary keys.

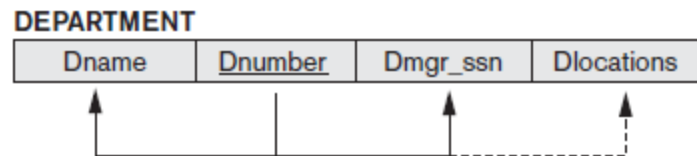
- The most common normal forms based on primary keys are:
 - **First Normal Form (1NF):**
 - Each column in a table contains only atomic (indivisible) values.
 - Each column has a unique name.
 - The order of the rows and columns does not matter.
 - There are no duplicate rows in the table.

EXAPLANATION FOR 1NF

- Consider the DEPARTMENT relation schema shown in Figure.



- suppose that we extend it by including the 'Dlocations' attribute as shown in Figure



- Figure below As we can see, this is not in 1NF because Dlocations is not an atomic attribute,

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

- 1NF version of the same relation with redundancy.

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

SOLUTION 1

- Remove the attribute Dlocations that violates 1NF and place it in a separate relation DEPT_LOCATIONS along with the primary key Dnumber of DEPARTMENT.
- The primary key of this relation is the combination {Dnumber, Dlocation}, as shown in Figure. **(COMPOSITE (PRIMARY) KEY)**

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

SOLUTION 2

- Expand the key so that there will be a separate tuple in the original DEPARTMENT relation for each location of a DEPARTMENT, as shown in Figure.
- In this case, the primary key becomes the combination {Dnumber, Dlocation}. This solution has the disadvantage of introducing *redundancy* in the relation.

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

SOLUTION 3

- If a *maximum number of values* is known for the attribute—for example, if it is known that *at most three locations* can exist for a department—replace the Dlocations attribute by three atomic attributes: Dlocation1, Dlocation2, and Dlocation3. This solution has the disadvantage of introducing *NULL values* if most departments have fewer than three locations.

- Of the three solutions above, the first is generally considered best because it does not suffer from redundancy and it is completely general, having no limit placed on a maximum number of values.

- First normal form also disallows multivalued attributes that are themselves composite. These are called **nested relations** because each tuple can have a relation *within it*.
- The schema of this EMP_PROJ relation can be represented as follows:
 - EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)})

EMP_PROJ

EMP_PROJ		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

- To normalize this into 1NF, we remove the nested relation attributes into a new relation and *propagate the primary key* into it; the primary key of the new relation will combine the partial key with the primary key of the original relation. Decomposition and primary key propagation yield the schemas EMP_PROJ1 and EMP_PROJ2, as shown in Figure.

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

EXAMPLE

- As an example, consider the following non-1NF relation: (From this)

CANDIDATE (Ssn, Name, {JOB_HIST (Company, Highest_position, {SAL_HIST (Year, Max_sal)}}))

- To this

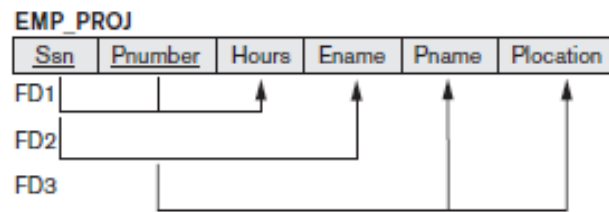
CANDIDATE_1 (Ssn, Name)
CANDIDATE_JOB_HIST (Ssn, Company, Highest_position)
CANDIDATE_SAL_HIST (Ssn, Company, Year, Max-sal)

2NF

- The Second Normal Form (2NF) is a crucial concept in Database Management Systems (DBMS).
- It is a level of normalization that ensures a relational database structure's efficiency and integrity.
- Second Normal Form (2NF) is based on the concept of full functional dependency.

- **Rules Governing the 2nd Normal Form in DBMS**
 - A relation must meet two conditions to be in 2NF:
 - It must be in 1NF.
 - It must not contain any partial dependency (every non-prime attribute of the relation is fully functionally dependent on each candidate key).

- NON-PRIME attributes: The attributes which are not participating in the formation of candidate key.



- The test for 2NF involves testing for functional dependencies whose left-hand side attributes are part of the primary key. If the primary key contains a single attribute, the test need not be applied at all.
- The EMP_PROJ relation in Figure above is in 1NF but is not in 2NF.
- The nonprime attribute **Ename** violates 2NF because of FD2, as do the nonprime attributes **Pname** and **Plocation** because of FD3.
- The functional dependencies FD2 and FD3 make **Ename, Pname, and Plocation** partially dependent on the primary key {Ssn, Pnumber} of EMP_PROJ, thus violating the 2NF test.

- the functional dependencies FD1, FD2, and FD3 in Figure lead to the decomposition of EMP_PROJ into the three relation schemas EP1, EP2, and EP3 shown in Figure below each of which is in 2NF.

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



Explanation for 2 NF

<u>ST_NO</u>	<u>COUR_NO</u>	COUR_FEE
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

Check:

$\text{COUR_FEE} \rightarrow \text{COUR_NO}$

$\text{COUR_FEE} \rightarrow \text{ST_NO}$

$\{\text{COUR_FEE}, \text{ST_NO}\} \rightarrow \text{COUR_NO}$

$\{\text{COUR_FEE}, \text{COUR_NO}\} \rightarrow \text{ST_NO}$

Hence, **COUR_FEE** would be a non-prime attribute, as it does not belong to the **one only** candidate key $\{\text{ST_NO}, \text{COUR_NO}\}$

$\{\text{COUR_NO}\} \rightarrow \text{COUR_FEE}$ (**COUR_FEE** is dependent on **COUR_NO**, which is a proper subset of the candidate key.)

Non-prime attribute COURSE_FEE is dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF.

- To convert the above relation to 2NF, we need to split the table into two tables such as :
 - Table 1: ST_NO, COUR_NO
 - Table 2: COUR_NO, COUR_FEE

<u>ST_NO</u>	<u>COUR_NO</u>
1	C1
2	C2
1	C4
4	C3
4	C1
2	C5

<u>COUR_NO</u>	<u>COUR_FEE</u>
C1	1000
C2	1500
C3	1000
C4	2000
C5	2000

Note – 2NF tries to reduce the redundant data getting stored in memory. For instance, if there are 100 students taking C1 course, we don't need to store its Fee as 1000 for all the 100 records, instead once we can store it in the second table as the course fee for C1 is 1000.

- R(ABCD)

- $AB \rightarrow D$

- $B \rightarrow C$

Candidate key: AB

PRIME: A,B

Non-Prime: C,D

$B \rightarrow C$ (Partial dependency) **(HENCE NOT IN 2NF)**

NOTE:

A	B	CHECK
1	NULL	OK
NULL	2	OK
NULL	NULL	NOT OK
3	4	OK

HENCE There might be a scenario where B can be null and we want to retrieve the value of C from B, then there'll be a problem. (PARTIAL DEPENDENCY)

- $R(ABCD)$
 - $R1(\underline{AB}D)$
 - $R2(\underline{B}C)$

Question

- R(ABCDEF)

- $C \rightarrow F$

- $E \rightarrow A$

- $EC \rightarrow D$

- $A \rightarrow B$

Candidate key=?

($EC \rightarrow ECDABF$)

EC is a candidate key.

PRIME: E, C

NON-PRIME: ABDF

PARTIAL DEPENDENCY: $E \rightarrow A$, $C \rightarrow F$

(HENCE, NOT IN 2 NF).

3NF

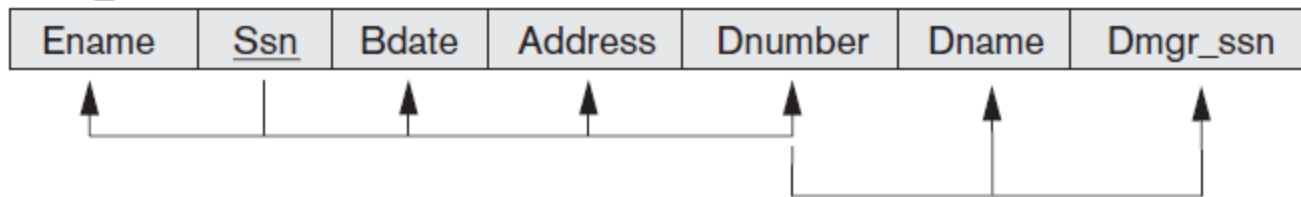
- A relation will be in 3NF if it is in 2NF and not contain any transitive dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

- A relation is in third normal form if it holds at least one of the following conditions for every non-trivial function dependency $X \rightarrow Y$.
 - X is a super key.
 - Y is a prime attribute, i.e., each element of Y is part of some candidate key.**(PD and TD, $P \rightarrow NP$ and $NP \rightarrow NP$ respectively, hence 3NF)**

- A functional dependency $X \rightarrow Y$ is called transitive dependency if X,Y are non-prime.
- **Third normal form (3NF)** is based on the concept of *transitive dependency*.
- A relation R is in 3NF if
 - It is in 2 NF
 - There should be no transitive dependency.

- The dependency ***Ssn* → *Dmgr_ssn*** is transitive through ***Dnumber*** in EMP_DEPT in Figure because both the dependencies ***Ssn* → *Dnumber*** and ***Dnumber* → *Dmgr_ssn*** hold *and* ***Dnumber*** is neither a key itself nor a subset of the key of EMP_DEPT.
- we can see that the dependency of ***Dmgr_ssn*** on ***Dnumber*** is undesirable in EMP_DEPT since ***Dnumber*** is not a key of EMP DEPT.

EMP_DEPT

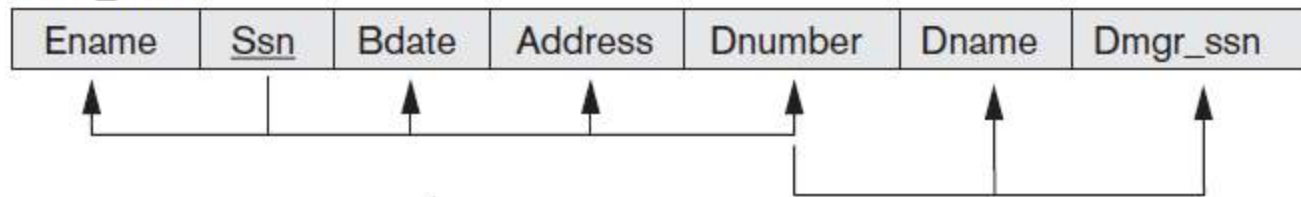


- The relation schema EMP_DEPT in Figure is in 2NF, since no partial dependencies on a key exist. However, EMP_DEPT is not in 3NF because of the transitive dependency of ***Dmgr_ssn*** (and also ***Dname***) on ***Ssn*** via ***Dnumber***.

SOLUTION

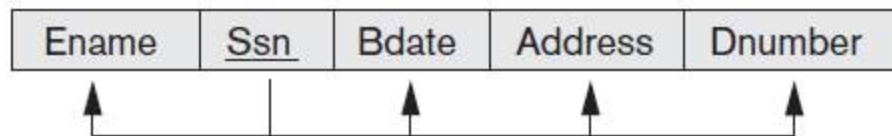
- We can normalize EMP_DEPT by decomposing it into the two 3NF relation schemas ED1 and ED2 shown in Figure.

EMP_DEPT

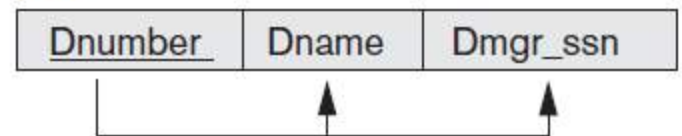


3NF Normalization

ED1



ED2



Example

– $R(\underline{A}BC)$

- $A \rightarrow B$
- $B \rightarrow C$

R

A	B	C
1	a	x
2	a	x
3	a	x
4	b	y
5	b	y
6	c	z
7	c	z

R1

<u>A</u>	B
1	a
2	a
3	a
4	b
5	b
6	c
7	c

R2

<u>B</u>	C
a	x
b	y
c	z

example

- $R(ABCDE)$
 - $A \rightarrow B$
 - $B \rightarrow E$
 - $C \rightarrow D$
 - $C_k = AC$
- $AC^+ \rightarrow ABCDE$
- $R1 \rightarrow ABE$
- $R11 \rightarrow AB$**
- $R12 \rightarrow BE$**
- $R2 \rightarrow CD$**
- $R3 \rightarrow AB$**

- Table below informally summarizes the three normal forms based on primary keys, the tests used in each case, and the corresponding *remedy* or normalization performed to achieve the normal form.

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

Canonical Cover of FDs

Question: Given a relational Schema $R(A, B, C, D)$ and set of Function Dependency $FD = \{ B \rightarrow A, AD \rightarrow BC, C \rightarrow ABD \}$.

Find the canonical cover?

Solution: Given $FD = \{ B \rightarrow A, AD \rightarrow BC, C \rightarrow ABD \}$, now decompose the FD using decomposition rule(Armstrong Axiom).

$B \rightarrow A$

$AD \rightarrow B$ (using decomposition inference rule on $AD \rightarrow BC$)

$AD \rightarrow C$ (using decomposition inference rule on $AD \rightarrow BC$)

$C \rightarrow A$ (using decomposition inference rule on $C \rightarrow ABD$)

$C \rightarrow B$ (using decomposition inference rule on $C \rightarrow ABD$)

$C \rightarrow D$ (using decomposition inference rule on $C \rightarrow ABD$)

The next step is to find closure of the left side of each of the given FD by including that FD and excluding that FD, if closure in both cases are same then that FD is redundant and we remove that FD from the given set, otherwise if both the closures are different then we do not exclude that FD.

- Calculating closure of all FD $\{ B \rightarrow A, AD \rightarrow B, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
 - **1a.** Closure $B^+ = BA$ using FD = $\{ B \rightarrow A, AD \rightarrow B, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
 - **1b.** Closure $B^+ = B$ using FD = $\{ AD \rightarrow B, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
 - From 1 a and 1 b, we found that both the Closure(by including $B \rightarrow A$ and excluding $B \rightarrow A$) are not equivalent, hence FD $B \rightarrow A$ is important and cannot be removed from the set of FD.

- **2 a.** Closure $AD^+ = ADBC$ using $FD = \{ B \rightarrow A, \mathbf{AD \rightarrow B}, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
- **2 b.** Closure $AD^+ = ADCB$ using $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
- **From 2 a and 2 b, we found that both the Closure (by including $AD \rightarrow B$ and excluding $AD \rightarrow B$) are equivalent, hence FD $AD \rightarrow B$ is not important and can be removed from the set of FD.**
- **Hence resultant $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$**

- 3 a. Closure $AD^+ = ADCB$ using $FD = \{ B \rightarrow A, \mathbf{AD} \rightarrow \mathbf{C}, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
- 3 b. Closure $AD^+ = AD$ using $FD = \{ B \rightarrow A, C \rightarrow A, C \rightarrow B, C \rightarrow D \}$
- From 3 a and 3 b, we found that both the Closure (by including $\mathbf{AD} \rightarrow \mathbf{C}$ and excluding $\mathbf{AD} \rightarrow \mathbf{C}$) are not equivalent, hence FD $AD \rightarrow C$ is important and cannot be removed from the set of FD.

- 4 a. Closure $C^+ = CABD$ using $FD = \{ B \rightarrow A, AD \rightarrow C, \mathbf{C \rightarrow A}, C \rightarrow B, C \rightarrow D \}$
- 4 b. Closure $C^+ = CBDA$ using $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, C \rightarrow D \}$
- From 4 a and 4 b, we found that both the Closure (by including $\mathbf{C \rightarrow A}$ and excluding $\mathbf{C \rightarrow A}$) are equivalent, hence FD $\mathbf{C \rightarrow A}$ is not important and can be removed from the set of FD.
- Hence resultant $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, C \rightarrow D \}$

- 5 a. Closure $C^+ = CBDA$ using $FD = \{ B \rightarrow A, AD \rightarrow C, \mathbf{C \rightarrow B}, C \rightarrow D \}$
- 5 b. Closure $C^+ = CD$ using $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow D \}$
- From 5 a and 5 b, we found that both the Closure (by including $\mathbf{C \rightarrow B}$ and excluding $\mathbf{C \rightarrow B}$) are not equivalent, hence FD $\mathbf{C \rightarrow B}$ is important and cannot be removed from the set of FD.

- 6 a. Closure $C^+ = CDBA$ using $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, \mathbf{C \rightarrow D} \}$
- 6 b. Closure $C^+ = CBA$ using $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B \}$
- From 6 a and 6 b, we found that both the Closure(by including $\mathbf{C \rightarrow D}$ and excluding $\mathbf{C \rightarrow D}$) are not equivalent, hence FD $\mathbf{C \rightarrow D}$ is important and cannot be removed from the set of FD.
- **Hence resultant $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, C \rightarrow D \}$**
- Since $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, C \rightarrow D \}$ is resultant FD.

- now we have checked the redundancy of attribute, since the left side of FD **$AD \rightarrow C$** has two attributes, let's check their importance, i.e. whether they both are important or only one.
- Closure $AD^+ = ADCB$ using $FD = \{ B \rightarrow A, \mathbf{AD \rightarrow C}, C \rightarrow B, C \rightarrow D \}$
- Closure $A^+ = A$ using $FD = \{ B \rightarrow A, \mathbf{AD \rightarrow C}, C \rightarrow B, C \rightarrow D \}$
- Closure $D^+ = D$ using $FD = \{ B \rightarrow A, \mathbf{AD \rightarrow C}, C \rightarrow B, C \rightarrow D \}$
- Since the closure of AD^+ , A^+ , D^+ that we found are not all equivalent, hence in FD $AD \rightarrow C$, both A and D are important attributes and cannot be removed.
- Hence resultant $FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow B, C \rightarrow D \}$ and we can rewrite as
- **$FD = \{ B \rightarrow A, AD \rightarrow C, C \rightarrow BD \}$ is Canonical Cover of $FD = \{ B \rightarrow A, AD \rightarrow BC, C \rightarrow ABD \}$.**

CANONICAL COVER of FD (EXAMPLE)

- Given a relational Schema $R(W, X, Y, Z)$ and set of Function Dependency $FD = \{ W \rightarrow X, Y \rightarrow X, Z \rightarrow WXY, WY \rightarrow Z \}$. Find the canonical cover?

FD	WITH	WITHOUT	SELECT/NOT SELECT
$W \rightarrow X$	WX	W	SELECT
$Y \rightarrow X$	YX	Y	SELECT
$Z \rightarrow W$	ZWXY	ZXY	SELECT
$Z \rightarrow X$	ZXWY	ZWYX	NOT SELECT
$Z \rightarrow Y$	ZYWX	ZWX	SELECT
$WY \rightarrow Z$	WYZX	WYX	SELECT

- Hence resultant $FD = \{ W \rightarrow X, Y \rightarrow X, Z \rightarrow W, Z \rightarrow Y, WY \rightarrow Z \}$

- Since $FD = \{ W \rightarrow X, Y \rightarrow X, Z \rightarrow W, Z \rightarrow Y, WY \rightarrow Z \}$ is resultant FD now, we have checked the redundancy of attribute, since the left side of FD $WY \rightarrow Z$ has two attributes at its left, let's check their importance, i.e. whether they both are important or only one.

FD	WITH
WY	WYZX
W	WX
Y	YX

- Since the closure of WY^+ , W^+ , Y^+ that we found are not all equivalent, hence in FD $WY \rightarrow Z$, both W and Y are important attributes and cannot be removed.
- Hence resultant FD = $\{ W \rightarrow X, Y \rightarrow X, Z \rightarrow W, Z \rightarrow Y, WY \rightarrow Z \}$ and we can rewrite as:
 - FD = $\{ W \rightarrow X, Y \rightarrow X, Z \rightarrow WY, WY \rightarrow Z \}$ is Canonical Cover of FD = $\{ W \rightarrow X, Y \rightarrow X, Z \rightarrow WXY, WY \rightarrow Z \}$.

EXAMPLE

- Given a relational Schema $R(V, W, X, Y, Z)$ and set of Function Dependency $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow VXZ \}$. Find the canonical cover?

FD	WITH	WITHOUT	S/NS
$V \rightarrow W$			
$VW \rightarrow X$			
$Y \rightarrow V$			
$Y \rightarrow X$			
$Y \rightarrow Z$			

FD	WITH	WITHOUT	S/NS
$V \rightarrow W$	VWX	V	S
$VW \rightarrow X$	VWX	VW	S
$Y \rightarrow V$	YVXZW	YXZ	S
$Y \rightarrow X$	YXVZW	YXVZW	NS
$Y \rightarrow Z$	YZVWX	YVWX	S

- Hence resultant $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$.

- Since $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$ is resultant FD now, we have checked the redundancy of attribute, since the left side of FD $VW \rightarrow X$ has two attributes at its left, let's check their importance, i.e. whether they both are important or only one.
- Closure $VW^+ = VWX$ using $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$
- Closure $V^+ = VWX$ using $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$
- Closure $W^+ = W$ using $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$
- Since the closure of VW^+ , V^+ , W^+ we found that all the Closures of VW and V are equivalent, hence in FD $VW \rightarrow X$, W is not at all an important attribute and can be removed.
- Hence resultant $FD = \{ V \rightarrow W, V \rightarrow X, Y \rightarrow V, Y \rightarrow Z \}$ and we can rewrite as
 - **$FD = \{ V \rightarrow WX, Y \rightarrow VZ \}$ is Canonical Cover of $FD = \{ V \rightarrow W, VW \rightarrow X, Y \rightarrow VXZ \}$.**

QUESTION

Q. Suppose a relational schema $R(w\ x\ y\ z)$, and set of functional dependency as following

FD : $x \rightarrow w$,
 $wz \rightarrow xy$,
 $y \rightarrow wxz$ }

Find the canonical cover F_c (Minimal set of functional dependency).

Solution: DECOMPOSITION

$x \rightarrow w$, $wz \rightarrow x$, $wz \rightarrow y$, $y \rightarrow w$, $y \rightarrow x$, $y \rightarrow z$

Important: Decomposition is always done at right side of FD. Never try to decompose at left side. It makes FDs invalid.

$AB \rightarrow CD$

We can decompose

$AB \rightarrow C$

$AB \rightarrow D$



$AB \rightarrow CD$

We can't decompose

$A \rightarrow CD$

$B \rightarrow CD$



FD	WITH	WITHOUT	S/NS
$x \rightarrow w$	xw	x	S
$wz \rightarrow x$	wzxy	wzyx	NS
$wz \rightarrow y$	wzyx	wz	S
$y \rightarrow w$	ywxz	yxwz	NS
$y \rightarrow x$	yxwz	yz	S
$y \rightarrow z$	yzxw	yxw	S

- Now FD set (all essential FDs)

- $x \rightarrow w$

- $wz \rightarrow y$

- $y \rightarrow x$

- $y \rightarrow z$

- Now we will check if there is any extra attribute at left side of FD.
- For this $wz \rightarrow y$ (only this FD has more than one attribute at left side, it may only contain extra attribute at left side)
- compute
 - $wz^+ = wzyx$
 - $w^+ = w$
 - $z^+ = z$

$wz \rightarrow y$ is essential because wz and w are different, wz and z are different .

Therefore, the minimal set of FD is $F_c : \{ x \rightarrow w, wz \rightarrow y, y \rightarrow xz \}$

BOYCE-CODD NF

- **Boyce-Codd normal form (BCNF)** was proposed as a simpler form of 3NF, but it was found to be stricter than 3NF.
- That is, every relation in BCNF is also in 3NF; however, a relation in 3NF is *not necessarily* in BCNF.
- The properties followed by BCNF in DBMS are as- It should already follow the properties of 3NF. For a functional dependency, $A \rightarrow B$, A must be a super key or candidate key.

- (PD and TD, $P \rightarrow NP$ (2nf) and $NP \rightarrow NP$ (3nf) respectively, hence 3NF)
- $P/NP \rightarrow P$? (BCNF deals with these cases.)
- $R(ABC)$
 - $AB \rightarrow C$
 - $C \rightarrow B$
 - 2NF and 3NF? (AB is a candidate key and super key)

- A relation is in BCNF if it holds the following conditions for every non-trivial function dependency $X \rightarrow Y$.
 - X is a super key.
- $R(ABC)$
 - $AB \rightarrow C$
 - $C \rightarrow B$
- Find CK=
 - AB and AC

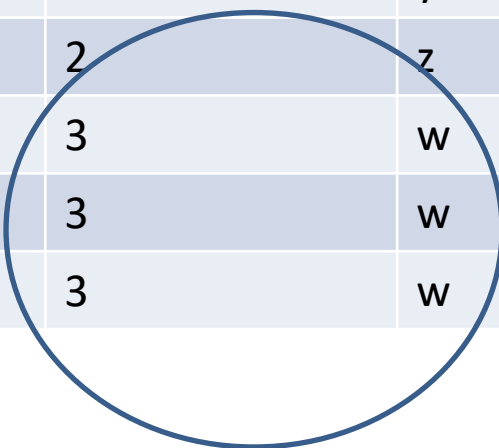
Explanation

– R(ABC)

- $AB \rightarrow C$
- $C \rightarrow B$

$Ch = AB, AC^R$

A	B	C
A	1	x
B	2	y
C	2	z
C	3	w
D	3	w
E	3	w



R1

A	C
A	x
B	y
C	z
C	w
D	w
E	w

R2

C	B
X	1
Y	2
Z	2
W	3

IDENTIFYING THE NF's

- R(ABCDEFGH)

- $AB \rightarrow C$
- $A \rightarrow DE$
- $B \rightarrow F$
- $F \rightarrow GH$

CK=AB

CHECK for every dependency: **BCNF**: $AB \rightarrow C$, rest are not in BCNF (NOT IN BCNF)

3NF: $AB \rightarrow C$, rest are not in 3NF (NOT IN 3NF)

2NF: $AB \rightarrow C$, rest are not in 2NF (NOT IN 2NF)

HENCE TABLE IS IN 1NF.

IDENTIFYING THE NF's

- R(ABCDE)

- $CE \rightarrow D$

- $D \rightarrow B$

- $C \rightarrow A$

CK=CE (why?)

CHECK for every dependency: **BCNF**: $CE \rightarrow D$, rest are not in BCNF (NOT IN BCNF)

3NF: $CE \rightarrow D$, rest are not in 3NF (NOT IN 3NF)

2NF: $CE \rightarrow D$, $D \rightarrow B$ rest are not in 2NF (NOT IN BCNF)

HENCE TABLE IS IN 1NF.

4NF

- In many cases, relations have constraints that cannot be specified as functional dependencies
- In this section, we discuss the concept of *multivalued dependency* (MVD) and define *fourth normal form*, which is based on this dependency.
- Multivalued dependencies are a consequence of first normal form (1NF), which disallows an attribute in a tuple to have a *set of values*, and the accompanying process of converting an unnormalized relation into 1NF.

- Depends on key and multi valued dependency.
- The relation is in 4 NF if:
 - In BCNF
 - No multi valued dependency. ($X \twoheadrightarrow Y$)

NOTE: (There's a difference between multi valued attribute and multi valued dependency)

CONDITIONS FOR MVD

- For MVD:
 - If $X \twoheadrightarrow Y$, for single value of X , more than one value of Y exists.
 - *Table should have at least 3 columns (attributes)*
 - *For this table with X , Y and Z , Y and Z should be independent.*

Problem

- For a dependency $A \rightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

<u>NAME</u>	<u>MOBILE</u>	<u>EMAIL</u>
REHAN	123	12@gmail.com
REHAN	123	34@gmail.com
REHAN	123	56@gmail.com
REHAN	456	12@gmail.com
REHAN	456	34@gmail.com
REHAN	456	56@gmail.com
REHAN	789	12@gmail.com
REHAN	789	34@gmail.com
REHAN	789	56@gmail.com

- The relation is called all-key relation (with key made of all attributes)
- Informally, whenever two *independent* 1:N relationships $A:B$ and $A:C$ are mixed in the same relation, $R(A, B, C)$, an MVD may arise.¹⁴

solution

<u>NAME</u>	<u>MOBILE</u>
REHAN	123
REHAN	456
REHAN	789

<u>NAME</u>	<u>EMAIL</u>
REHAN	12@gmail.com
REHAN	34@gmail.com
REHAN	56@gmail.com

EMP

<u>Ename</u>	<u>Pname</u>	<u>Dname</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

- The EMP relation with two MVDs: $\text{Ename} \twoheadrightarrow \text{Pname}$ and $\text{Ename} \twoheadrightarrow \text{Dname}$.

EMP_PROJECTS

<u>Ename</u>	<u>Pname</u>
Smith	X
Smith	Y

EMP_DEPENDENTS

<u>Ename</u>	<u>Dname</u>
Smith	John
Smith	Anna

- Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

A multivalued dependency $X \twoheadrightarrow Y$ specified on relation schema R , where X and Y are both subsets of R , specifies the following constraint on any relation state r of R : If two tuples t_1 and t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3 and t_4 should also exist in r with the following properties,¹⁵ where we use Z to denote $(R - (X \cup Y))$:

- $t_3[X] = t_4[X] = t_1[X] = t_2[X]$.
- $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$.
- $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$.

EMP

<u>Ename</u>	<u>Pname</u>	<u>Dname</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

- Whenever $X \twoheadrightarrow Y$ holds, we say that X **multidetermines** Y . Because of the symmetry in the definition, whenever $X \twoheadrightarrow Y$ holds in R , so does $X \twoheadrightarrow Z$.

An MVD $X \twoheadrightarrow Y$ in R is called a **trivial MVD** if

- (a) Y is a subset of X , or
- (b) $X \cup Y = R$.

For example, the relation EMP_PROJECTS in Figure has the trivial MVD $\text{Ename} \twoheadrightarrow \text{Pname}$. An MVD that satisfies neither (a) nor (b) is called a **nontrivial MVD**.

EMP_PROJECTS

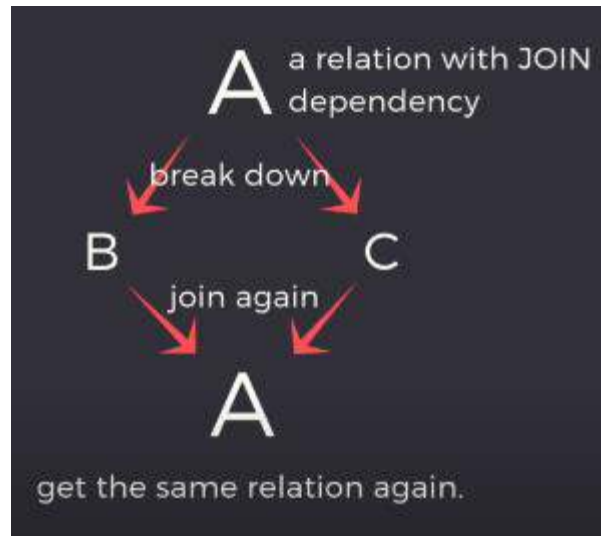
<u>Ename</u>	<u>Pname</u>
Smith	X
Smith	Y

EMP_DEPENDENTS

<u>Ename</u>	<u>Dname</u>
Smith	John
Smith	Anna

5NF

- 4NF and PJNF (Project Join NF) (lossless decomposition (non-additive decomposition)).

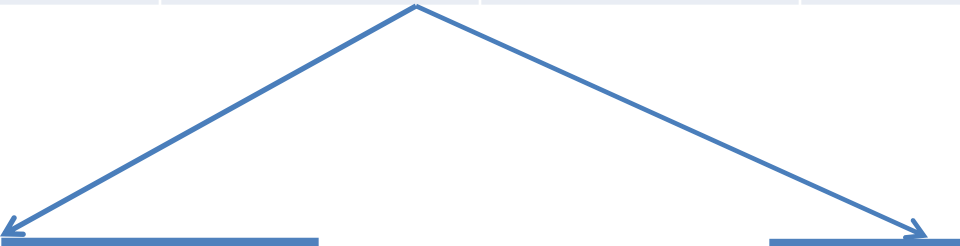


Conditions for lossless if:

- $\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$
- $\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$
- $\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R1)$ or $\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R2)$
 - (The common attribute must be a key for at least one relation (R1 or R2). i.e. suppose an attribute "A" is common in both R1 and R2 then either A should be key in R1 or A should be key in R2.)

- $\text{Att}(R1) \cup \text{Att}(R2) = \text{Att}(R)$

A	B	C	D
1	a	p	x
2	b	q	y

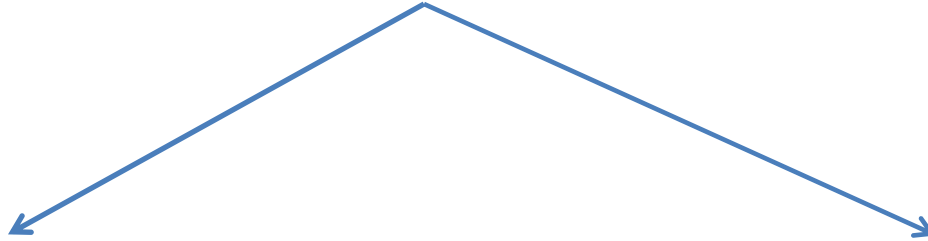


A	B
1	a
2	b

D
x
y

- $\text{Att}(R1) \cap \text{Att}(R2) \neq \Phi$ (NO COMMON ATTRIBUTE)

A	B	C	D
1	a	p	x
2	b	q	y



A	B
1	a
2	b

C	D
p	x
q	y

Cartesian product

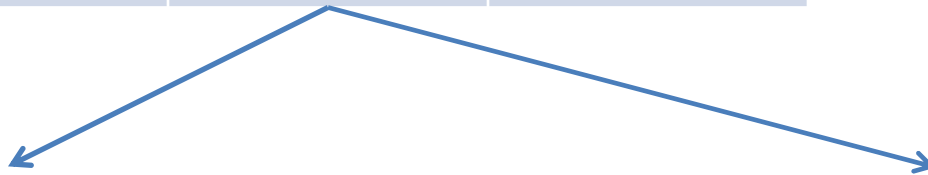
EXTRA



A	B	C	D
1	A	p	x
1	A	q	y
2	B	p	x
2	b	q	y

- $\text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R1) \text{ or } \text{Att}(R1) \cap \text{Att}(R2) \rightarrow \text{Att}(R2)$

A	B	C
1	a	p
2	b	q
3	a	r



A	B
1	a
2	b
3	a

Cartesian product

B	C
a	p
b	q
a	r

EXTRA



A	B	C
1	A	p
1	A	r
2	B	q
3	a	p
3	a	r

- Common attribute value should have distinct values (Candidate Key).

Question

- For Example, A relation $R(A, B, C, D)$ with FD set $\{A \rightarrow BC\}$ is decomposed into $R_1(ABC)$ and $R_2(AD)$ which is a lossless join decomposition as:
 - First condition holds true as $\text{Att}(R_1) \cup \text{Att}(R_2) = (ABC) \cup (AD) = (ABCD) = \text{Att}(R)$.
 - Second condition holds true as $\text{Att}(R_1) \cap \text{Att}(R_2) = (ABC) \cap (AD) \neq \Phi$
 - The third condition holds as $\text{Att}(R_1) \cap \text{Att}(R_2) = A$ is a key of $R_1(ABC)$ because $A \rightarrow BC$ is given.

- **Question: Consider a schema $R(X, Y, Z, W)$ and functional dependencies $FD = \{ X \rightarrow Y \text{ and } Z \rightarrow W \}$. Then the decomposition of relational schema R into relation $R1(X Y)$ and relation $R2(Z W)$ is**
 - Attribute ($R1$) $\cup$ Attribute ($R2$) = $X Y Z W$ = Attribute (R)
(which satisfies the first condition of lossless join decomposition hence we check the next condition of same.)
 - Attribute ($R1$) $\cap$ Attribute ($R2$) = NULL, (i.e. there is no common attribute in relation $R1$ and relation $R2$) which violates the condition of lossless join decomposition. Hence the decomposition is not lossless (LOSSY)

- Consider a relation schema $R(X\ Y\ Z\ W\ P)$ (above table R) is decomposed into $R1(X\ Y\ Z)$ and $R2(Z\ W\ P)$. determine whether the above $R1$ and $R2$ are Lossless or Lossy?
- Cond 1:** satisfied as $\text{Attribute}(R1) \cup \text{Attribute}(R2) = \text{Attribute}(R) = (X\ Y\ Z\ W\ P)$
- Cond 2:** satisfied as $\text{Attribute}(R1) \cap \text{Attribute}(R2) \neq \Phi = (Z)$
- Cond 3:** Not satisfied as common attribute Z is not key in any relation $R1$ or $R2$ (you can check from table values of column Z is repeating)

X	Y	Z	W	P
1	244	A	I	W
2	231	B	J	X
1	534	A	K	Y
3	988	C	I	Z

- Consider a relation schema $R(X Y Z W P)$ (above table R) is decomposed into $R_1(X Y)$ and $R_2(Z W)$. determine whether the above R_1 and R_2 are Lossless or Lossy?
- Not satisfied as $\text{Attribute}(R_1) \cup \text{Attribute}(R_2) (X Y Z W) \neq \text{Attribute}(R) = (X Y Z W P)$
- Since Condition 1 is not satisfied so we will not check condition 2 and 3

- Consider a relation schema $R(X Y Z W P)$ (above table R) is decomposed into $R1(X Y Z)$ and $R2(W P)$, determine whether the above $R1$ and $R2$ are Lossless or Lossy?
- **Cond 1:** satisfied as $\text{Attribute}(R1) \cup \text{Attribute}(R2) = \text{Attribute}(R) = (X Y Z W P)$
- **Cond 2:** Not satisfied as $\text{Attribute}(R1) \cap \text{Attribute}(R2) = \emptyset$
- Since Condition 2 is not satisfied so we will not check condition 3

- Consider a relation schema $R (A , B , C , D)$ with the functional dependencies $A \rightarrow B$ and $C \rightarrow D$. Determine whether the decomposition of R into $R_1 (A , B)$ and $R_2 (C , D)$ is lossless or lossy.

– According to condition-01

$$\begin{aligned} & R_1 (A , B) \cup R_2 (C , D) \\ &= R (A , B , C , D) \text{ (YES)} \end{aligned}$$

According to condition-02, intersection of both the sub relations must not be null.

So, we have-

$$\begin{aligned} & R_1 (A , B) \cap R_2 (C , D) \\ &= \Phi \text{ ANSWER: LOSSY} \end{aligned}$$

- Consider a relation schema $R (A , B , C , D)$ with the following functional dependencies-

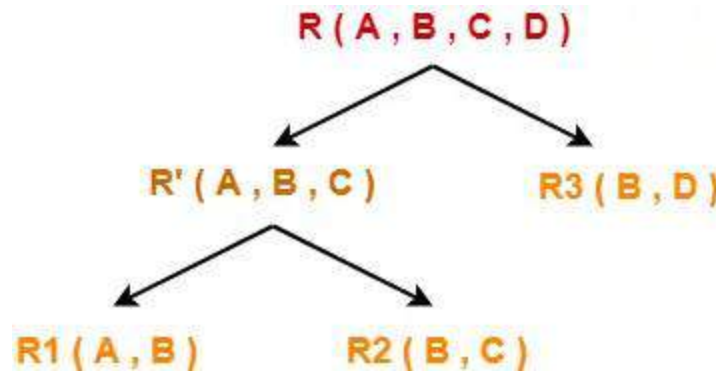
$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow D$

$D \rightarrow B$

- Determine whether the decomposition of R into $R_1 (A , B)$, $R_2 (B , C)$ and $R_3 (B , D)$ is lossless or lossy.



- $R' (A , B , C) \cup R_3 (B , D) = R (A , B , C , D)$ [condition-01 satisfies]
- $R' (A , B , C) \cap R_3 (B , D) = B$ [condition-02 satisfies]
- $R' (A , B , C) \cap R_3 (B , D) = B$
 - Now, the closure of attribute B is- $B^+ = \{ B , C , D \}$
 - Now, we see-
 - Attribute ' B ' can not determine attribute ' A ' of sub relation R' .
 - Thus, it is not a super key of the sub relation R' .
 - Attribute ' B ' can determine all the attributes of sub relation R_3 .
 - Thus, it is a super key of the sub relation R_3 .

- **Decomposition of $R'(A, B, C)$ into $R_1(A, B)$ and $R_2(B, C)$ -**
 - $R_1(A, B) \cup R_2(B, C) = R'(A, B, C)$
 - condition-01 satisfies
 - $R_1(A, B) \cap R_2(B, C) = B$ (condition-02 satisfies)
 - $R_1(A, B) \cap R_2(B, C) = B$
- Now, the closure of attribute B is-
- $B^+ = \{ B, C, D \}$
- Now, we see-
- Attribute 'B' can not determine attribute 'A' of sub relation R_1 .
- Thus, it is not a super key of the sub relation R_1 .
- Attribute 'B' can determine all the attributes of sub relation R_2 .
- Thus, it is a super key of the sub relation R_2 .
-
- **condition-03 satisfies.**
- **Thus, we conclude that the decomposition is lossless.**

X (P, Q, R)

P	Q	R
37	25	16
29	18	35
16	39	28

X1 (P, Q)

P	Q
37	25
29	18
16	39

X2 (Q, R)

Q	R
25	16
18	35
39	28

Dependency Preserving

- Decomposition of a relation is done when a relation in a relational model is not in appropriate normal form. Relation R is decomposed into two or more relations if decomposition is lossless join as well as dependency preserving.
- **Dependency preserving** decomposition is an important technique in DBMS for improving database efficiency while maintaining data consistency and integrity.

- **Lossless Join Decomposition**

- If we decompose a relation R into relations R_1 and R_2 ,
- Decomposition is lossy if $R_1 \bowtie R_2 \supset R$ Decomposition is lossless if $R_1 \bowtie R_2 = R$
- **To check for lossless join decomposition using the FD set, the following conditions must hold:**
 - $\text{Att}(R_1) \cup \text{Att}(R_2) = \text{Att}(R)$
 - $\text{Att}(R_1) \cap \text{Att}(R_2) \neq \Phi$
 - $\text{Att}(R_1) \cap \text{Att}(R_2) \rightarrow \text{Att}(R_1)$ or $\text{Att}(R_1) \cap \text{Att}(R_2) \rightarrow \text{Att}(R_2)$

Dependency Preserving Decomposition

- If we decompose a relation R into relations R_1 and R_2 , All dependencies of R either must be a part of R_1 or R_2 or must be derivable from a combination of functional dependency of R_1 and R_2 .
- For Example, A relation $R(A, B, C, D)$ with FD set $\{A \rightarrow BC\}$ is decomposed into $R_1(ABC)$ and $R_2(AD)$ which is dependency preserving because FD $A \rightarrow BC$ is a part of $R_1(ABC)$.

- Consider a schema $R(A,B,C,D)$ and functional dependencies $A \rightarrow B$ and $C \rightarrow D$ which is decomposed into $R1(AB)$ and $R2(CD)$
 - This decomposition is *dependency preserving decomposition* because
 - $A \rightarrow B$ can be ensured in $R1(AB)$
 - $C \rightarrow D$ can be ensured in $R2(CD)$

- Let a relation $R(A,B,C,D)$ and set a FDs $F = \{ A \rightarrow B , A \rightarrow C , C \rightarrow D \}$ are given.
A relation R is decomposed into –
 - $R_1 = (A, B, C)$ with FDs $F_1 = \{A \rightarrow B, A \rightarrow C\}$, and
 - $R_2 = (C, D)$ with FDs $F_2 = \{C \rightarrow D\}$.
 - $F' = F_1 \cup F_2 = \{A \rightarrow B, A \rightarrow C, C \rightarrow D\}$
 - so, $F' = F$. And so, $F'^+ = F^+$.
- **Thus, the decomposition is dependency preserving decomposition.**

Questions

$R(ABCD)$ $A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$

Decomposition $R1(AB)$ $R2(BC)$ $R3(BD)$

- $R1(AB)$: $A \rightarrow A, A \rightarrow B, B \rightarrow B, B \rightarrow A$ [$A \rightarrow B$ IS SELECTED] [only non-trivial]
(CALCULATE THE CLOSURE OF B^+)
- $R2(BC)$: $B \rightarrow C, C \rightarrow B$ [BOTH SELECTED]
(CALCULATE THE CLOSURE OF C^+)
- $R3(BD)$: $B \rightarrow D, D \rightarrow B$ [BOTH SELECTED]
- SELECTED: $A \rightarrow B, B \rightarrow C, C \rightarrow B, B \rightarrow D, D \rightarrow B$
- Now, check main FD's ($A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$) can be achieved using the currently derived dependencies ($A \rightarrow B, B \rightarrow C, C \rightarrow B, B \rightarrow D, D \rightarrow B$).
- In $C \rightarrow D$, calculate C^+ using new dependencies ($C^+ \rightarrow CBD$), hence all FD's can be derived, so dependency is preserved.

Questions

$R(ABCD) \ AB \rightarrow CD, D \rightarrow A$

Decomposition $R_1(AD) \ R_2(BCD)$

Questions

- The given set of FDs be $E : \{B \rightarrow A, D \rightarrow A, AB \rightarrow D\}$, find the minimal cover of E .
- The given set of FDs be $E : \{A \rightarrow BCDE, CD \rightarrow E\}$, find the minimal cover of E .

Question

- R(ABCD)
 - R1(ABC)
 - R2(CD)
 - FD's: $B \rightarrow C$, $AC \rightarrow D$. (Check whether dependency is preserved or not)

Question

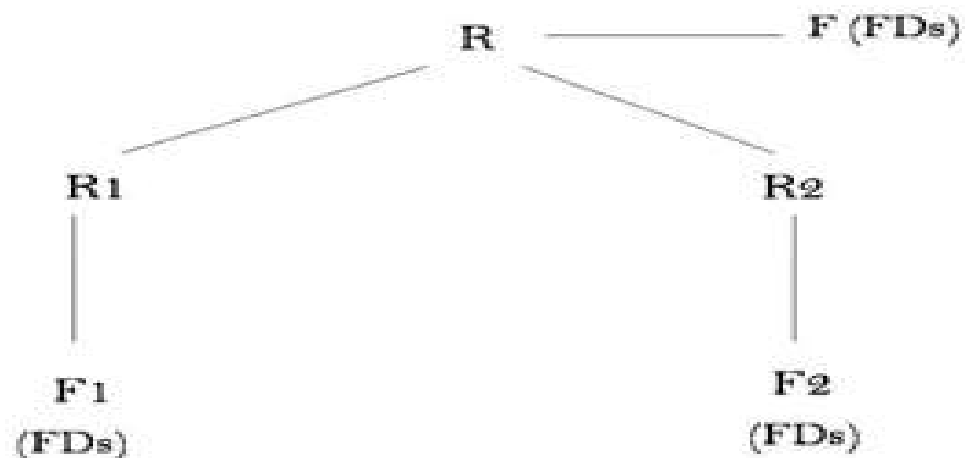
Let a relation **R (ABC)**

FDs = $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Decomposition (D) = $\{AB, BC\}$

Let a relation **R (ABCD)** and **FDs** = $\{A \rightarrow B, B \rightarrow C, A \rightarrow C, D \rightarrow C\}$ and **D** = $\{AB, BCD\}$

SOLUTION: It is dependency preserving.



If $R_1 \bowtie R_2 = R$ (**lossless decomposition**)

If $F_1 \cup F_2 = F$ (**dependency preserving**)

In general,

$$R_1 \bowtie R_2 \supseteq R$$

Here $R_1 \bowtie R_2 = R$ (**Lossless**)

$R_1 \bowtie R_2 \supset R$ (**Lossy**)

And

$$F_1 \cup F_2 \subseteq F$$

Here

$F_1 \cup F_2 = F$ (**dependency preserving**)

$F_1 \cup F_2 \subset F$ (**Non – dependency preserving**)



Graphic Era

(Deemed to be University)

Accredited by NAAC with Grade A

Database Management System (TCS 503) Lecture- 10

B. Tech- CSE
3rd Year (5th semester)

REHAN, Assistant Professor
+91-9548283808

Unit 5: Transaction Management

- Transaction processing systems are systems with large databases and hundreds of concurrent users executing database transactions.
- These systems require high availability and fast response time for hundreds of concurrent users.

- One criterion for classifying a database system is according to the number of users who can use the system **concurrently**.
- A DBMS is **single-user** if at most one user at a time can use the system, and it is **multiuser** if many users can use the system—and hence access the database—concurrently.

- A **transaction** is an executing program that forms a logical unit of database processing.
- A transaction includes one or more database access operations—these can include insertion, deletion, modification, or retrieval operations.
- The database operations that form a transaction can either be embedded within an application program or they can be specified interactively via a high-level query language such as SQL.

- If the database operations in a transaction do not update the database but only retrieve data, the transaction is called a **read-only transaction**; otherwise it is known as a **read write transaction**.
- A **database** is basically represented as a collection of *named data items*. The size of a data item is called its **granularity**.

- A transaction is a set of logically related operations.
- For example, you are transferring money from your bank account to your friend's account, the set of operations would be like this:
 - 1. Read your account balance
 - 2. Deduct the amount from your balance
 - 3. Write the remaining balance to your account
 - 4. Read your friend's account balance
 - 5. Add the amount to his account balance
 - 6. Write the new updated balance to his account.

This whole set of operations can be called a **transaction**.

- In DBMS, we write the above 6 steps transaction like this:
- Lets say your account is A and your friend's account is B, you are transferring 10000 from A to B, the steps of the transaction are:
 - 1. $R(A)$;
 - 2. $A = A - 10000$;
 - 3. $W(A)$;
 - 4. $R(B)$;
 - 5. $B = B + 10000$;
 - 6. $W(B)$;

In the above transaction R refers to the Read operation and W refers to the write operation.

- The main problem that can happen during a transaction is that the transaction can fail before finishing the all the operations in the set.
- This can happen due to power failure, system crash etc. This is a serious problem that can leave database in an **inconsistent state**.

- To solve this problem, we have the following two operations:
 - **Commit:** If all the operations in a transaction are completed successfully then commit those changes to the database permanently.
 - **Rollback:** If any of the operation fails then rollback all the changes done by previous operations.

BLIND WRITE

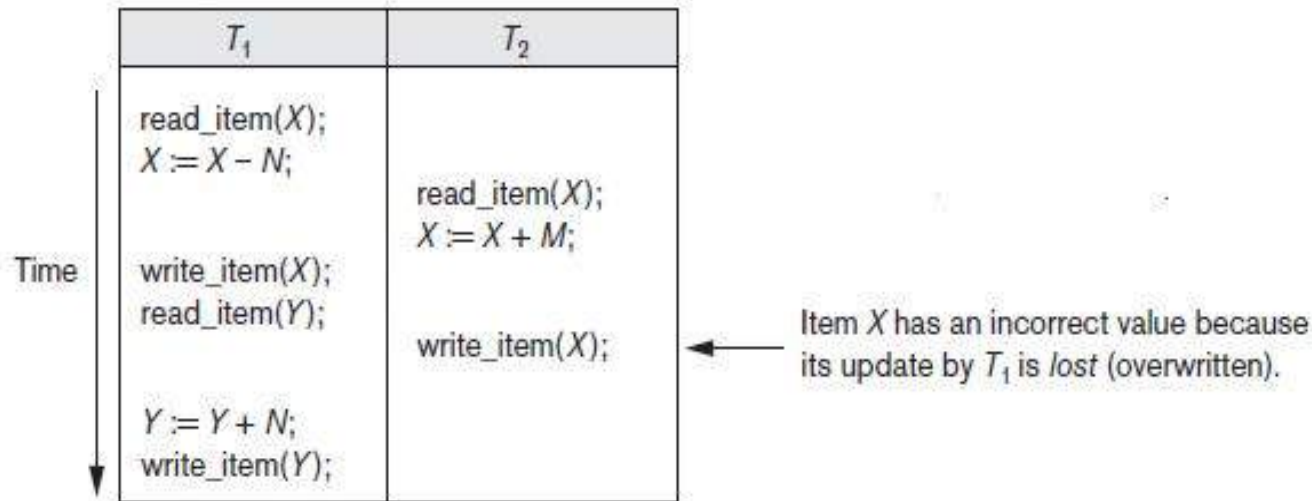
- If there is no read that happens prior to the first write then it is said to be a *blind write*.

T1	T2	T3
	$R_2(X)$	
$R_1(X)$		
		$W_3(X)$
	$W_2(X)$	

- $W_3(X)$ is a blind write, as there is no read before write [$R_3(X)$ before $W_3(X)$]
- $W_2(X)$ is not a blind write, as a read happens before write [$R_2(X)$ before $W_2(X)$]

Problems

- **The Lost Update Problem.** This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database items incorrect.



For example, if $X = 80$ at the start (originally there were 80 reservations on the flight), $N = 5$ (T_1 transfers 5 seat reservations from the flight corresponding to X to the flight corresponding to Y), and $M = 4$ (T_2 reserves 4 seats on X), the final result should be $X = 79$. **However**, in the interleaving of operations shown in Figure, it is $X = 84$ because the update in T_1 that removed the five seats from X was *lost*.

Consider the below diagram where two transactions T_x and T_y are performed on the same account A where the balance of account A is \$300.

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A - 50$	
t_3	—	READ (A)
t_4	—	$A = A + 100$
t_5	—	—
t_6	WRITE (A)	—
t_7		WRITE (A)

LOST UPDATE PROBLEM

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300 (only read).
- At time t_2 , transaction T_x deducts \$50 from account A that becomes \$250 (only deducted and not updated/write).
- Alternately, at time t_3 , transaction T_y reads the value of account A that will be \$300 only because T_x didn't update the value yet.
- At time t_4 , transaction T_y adds \$100 to account A that becomes \$400 (only added but not updated/write).
- At time t_6 , transaction T_x writes the value of account A that will be updated as \$250 only, as T_y didn't update the value yet.
- Similarly, at time t_7 , transaction T_y writes the values of account A, so it will write as done at time t_4 that will be \$400. It means the value written by T_x is lost, i.e., \$250 is lost.

2. Dirty Read Problems (W-R Conflict)

The dirty read problem occurs when one transaction updates an item of the database, and somehow the transaction fails, and before the data gets rollback, the updated database item is accessed by another transaction. There comes the Read-Write Conflict between both transactions.

Consider two transactions T_x and T_y in the below diagram performing read/write operations on account A where the available balance in account A is \$300:

Time	T_x	T_y
t_1	READ (A)	—
t_2	$A = A + 50$	—
t_3	WRITE (A)	—
t_4	—	READ (A)
t_5	SERVER DOWN ROLLBACK	—

DIRTY READ PROBLEM

- At time t_1 , transaction T_x reads the value of account A, i.e., \$300.
- At time t_2 , transaction T_x adds \$50 to account A that becomes \$350.
- At time t_3 , transaction T_x writes the updated value in account A, i.e., \$350.
- Then at time t_4 , transaction T_y reads account A that will be read as \$350.
- Then at time t_5 , transaction T_x rolls back due to server problem, and the value changes back to \$300 (as initially).
- But the value for account A remains \$350 for transaction T_y as committed, which is the dirty read and therefore known as the Dirty Read Problem.

3. Unrepeatable Read Problem (W-R Conflict)

Also known as Inconsistent Retrievals Problem that occurs when in a transaction, two different values are read for the same database item.

Consider two transactions, T_x and T_y , performing the read/write operations on account A, having an available balance = \$300. The diagram is shown below:

Time	T_x	T_y
t_1	READ (A)	—
t_2	—	READ (A)
t_3	—	$A = A + 100$
t_4	—	WRITE (A)
t_5	READ (A)	—

UNREPEATABLE READ PROBLEM

- At time t_1 , transaction T_x reads the value from account A, i.e., \$300.
- At time t_2 , transaction T_y reads the value from account A, i.e., \$300.
- At time t_3 , transaction T_y updates the value of account A by adding \$100 to the available balance, and then it becomes \$400.
- At time t_4 , transaction T_y writes the updated value, i.e., \$400.
- After that, at time t_5 , transaction T_x reads the available value of account A, and that will be read as \$400.
- It means that within the same transaction T_x , it reads two different values of account A, i.e., \$ 300 initially, and after updation made by transaction T_y , it reads \$400. It is an unrepeatable read and is therefore known as the Unrepeatable read problem.

Failures

- **Types of Failures.** Failures are generally classified as transaction, system, and media failures.
 - **A computer failure (system crash)**
 - **A transaction or system error**
 - **Local errors or exception conditions detected by the transaction.**
 - **Disk failure**
 - **Physical problems and catastrophes**

The ACID Properties

- A transaction is a single logical unit of work which accesses and possibly modifies the contents of a database.
- Transactions access data using read and write operations. In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called **ACID properties**.

Database ACID Properties

Atomicity

means either all successful or none.

Consistency

ensures bringing the database from one consistent state to another consistent state.
ensures bringing the database from one consistent state to another consistent state.

Isolation

ensures that transaction is isolated from other transaction.

Durability

means once a transaction has been committed, it will remain so, even in the event of errors, power loss etc.

Multiple statements make up a transaction. These statements can be broken down into the parts of the transaction like the input, data processing, checking, storage and output.

Transaction management component ensures **Atomicity** of database.

- **Assume 3 Processes A, B and C. For a transaction t to complete, all processes must be validated and have completed.**

Therefore, it must follow an AND condition:

$$t = A \wedge B \wedge C = 1$$

The transaction cannot be an OR condition:

$$t = A \vee B \vee C = 0$$

- The reason a transaction must be validated is in order to keep the database in a consistent state. Any partial saving of data can lead to inconsistencies that can severely affect the accuracy of information.
- Atomicity By this, we mean that either the entire transaction takes place at once or doesn't happen at all.
- There is no midway i.e. transactions do not occur partially. Each transaction is considered as one unit and either runs to completion or is not executed at all.
- It involves the following two operations.
 - Abort: If a transaction aborts, changes made to database are not visible.
 - Commit: If a transaction commits, changes made are visible. Atomicity is also known as the 'All or nothing rule'.

- Consider the following transaction T consisting of T1 and T2:
Transfer of 100 from account X to account Y.

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

- If the transaction fails after completion of T1 but before completion of T2.(say, after write(X) but before write(Y)), then amount has been deducted from X but not added to Y. This results in an **inconsistent database state**. Therefore, the transaction must be executed in entirety in order to ensure correctness of database state.

ISOLATION

- In order to prevent conflicts in the database which could result in inconsistencies, there must be process isolation. This ensures that process t1 and t2 don't attempt to execute at the same time and write data to the database.
- Transactions occur independently without interference.
- **Concurrency control component** take care of isolation.

- Let $X = 500$, $Y = 500$.

Consider two transactions **T** and **T''**.

T	T''
Read (X)	Read (X)
$X := X * 100$	Read (Y)
Write (X)	$Z := X + Y$
Read (Y)	Write (Z)
$Y := Y - 50$	
Write	

- Suppose T has been executed till Read (Y) and then T'' starts. As a result, interleaving of operations takes place due to which T'' reads correct value of X but incorrect value of Y and sum computed by
 - T'': $(X + Y = 50,000 + 500 = 50,500)$
- is thus not consistent with the sum at end of transaction:
 - T: $(X + Y = 50,000 + 450 = 50,450)$.

- This results in database inconsistency, due to a loss of 50 units. Hence, transactions must take place in isolation and changes should be visible only after they have been made to the main memory

CONSISTENCY

- This means that integrity constraints must be maintained so that the database is consistent before and after the transaction.
- It refers to the correctness of a database. Referring to the example below, The total amount before and after the transaction must be maintained.

Before: X : 500	Y: 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

- Total before T occurs = $500 + 200 = 700$.
- Total after T occurs = $400 + 300 = 700$.

Therefore, database is consistent. Inconsistency occurs in case T1 completes but T2 fails. As a result T is incomplete.

DURABILITY

- Persistent data storage and backup tests a database system's effectivity.
- DB on the network, either in local data centers or on the cloud. Making sure that the data is properly secured and managed by an administrator is the most important part of database administration.

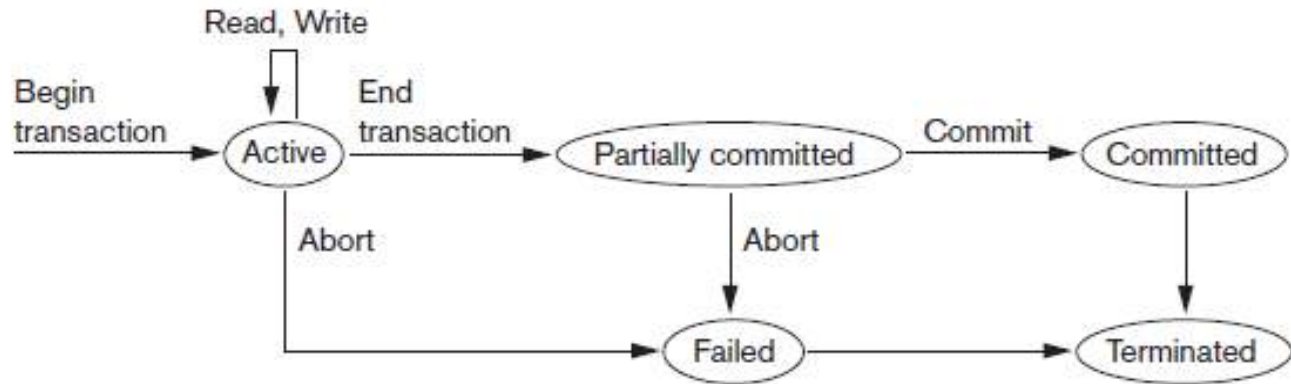
- This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs.
- These updates now become permanent and are stored in non-volatile memory. The effects of the transaction, thus, are never lost.

NOTE: The ACID properties, in totality, provide a mechanism to ensure correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations and updates that it makes are durably stored.

Transaction States

- A transaction is an atomic unit of work that should either be completed in its entirety or not done at all.
- The recovery manager of the DBMS needs to keep track of the following operations:
 - BEGIN_TRANSACTION. This marks the beginning of transaction execution.
 - READ or WRITE. These specify read or write operations on the database items that are executed as part of a transaction.
 - END_TRANSACTION. This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.
 - COMMIT_TRANSACTION. This signals a *successful end* of the transaction so that any changes (updates) executed by the transaction can be safely **committed** to the database and will not be undone.
 - ROLLBACK (or ABORT). This signals that the transaction has *ended unsuccessfully*, so that any changes or effects that the transaction may have applied to the database must be **undone**.

State transition diagram illustrating the states for transaction execution.



A transaction goes into an **active state** immediately after it starts execution, where it can execute its READ and WRITE operations. When the transaction ends, it moves to the **partially committed state** (At this point, some recovery protocols need to ensure that a system failure will not result in an inability to record the changes of the transaction permanently).

Once this check is successful, the transaction is said to have reached its commit point and enters the **committed state**.

However, a transaction can go to the **failed state** if one of the checks fails or if the transaction is aborted during its active state. The transaction may then have to be rolled back to undo the effect of its WRITE operations on the database. The **terminated state** corresponds to the transaction leaving the system.

Concurrency

- Advantages:
 - Waiting time reduced
 - Response time is also reduced.
 - Resource utilization is enhanced.
 - Efficiency is increased.

SCHEDULES

- A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions.
- Schedule is a process of lining the transactions and executing them one by one.
- When there are multiple transactions that are running in a concurrent manner and the order of operation is needed to be set so that the operations do not overlap each other, Scheduling is brought into play and the transactions are timed accordingly.

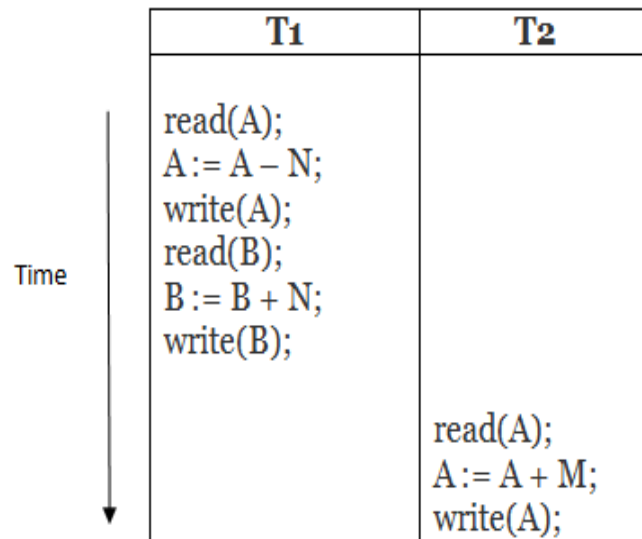
Serial Schedule

- The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed. (No context switching)
 - **For example:** Suppose there are two transactions T1 and T2 which have some operations. If it has no interleaving of operations, then there are the following two possible outcomes:
 - Execute all the operations of T1 which was followed by all the operations of T2.
 - Execute all the operations of T2 which was followed by all the operations of T1.

In the given (a) figure, Schedule A shows the serial schedule where T1 followed by T2.

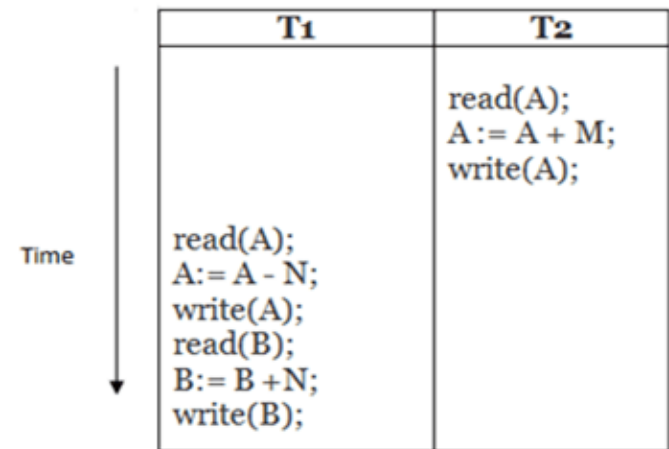
In the given (b) figure, Schedule B shows the serial schedule where T2 followed by T1.

(a)



Schedule A

(b)

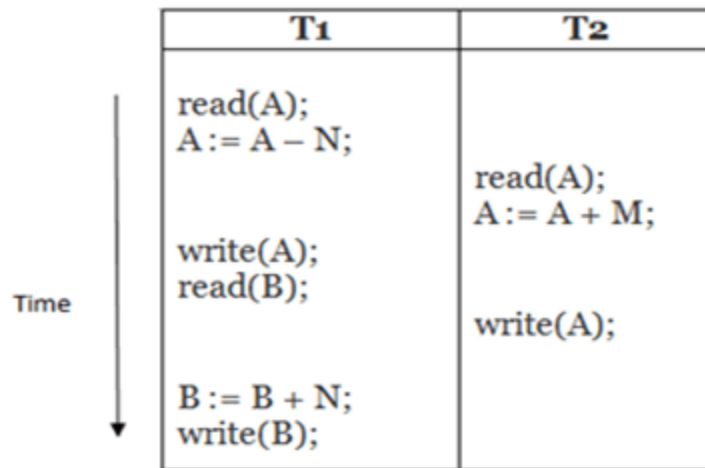


Schedule B

Non-serial Schedule

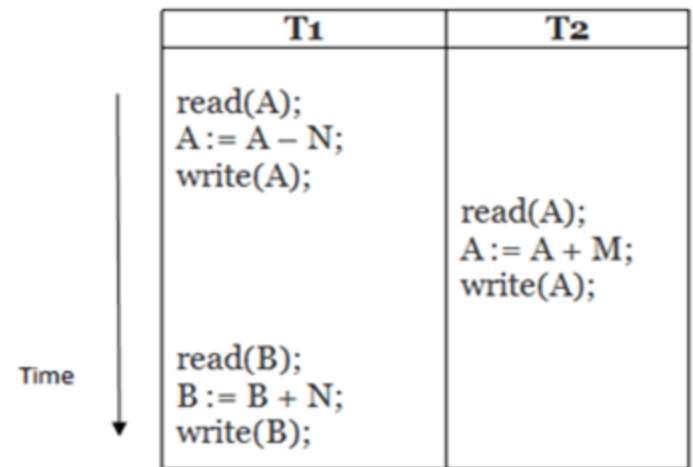
- If interleaving of operations is allowed, then there will be non-serial schedule (context switching)
- It contains many possible orders in which the system can execute the individual operations of the transactions.
- In the given figure (c) and (d), Schedule C and Schedule D are the non-serial schedules. It has interleaving of operations.

(c)



Schedule C

(d)



Schedule D

Serializable schedule

- The serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another.
- It identifies which schedules are correct when executions of the transaction have interleaving of their operations.
- A non-serial schedule will be serializable if its result is equal to the result of its transactions executed serially.

Conflict Serializability

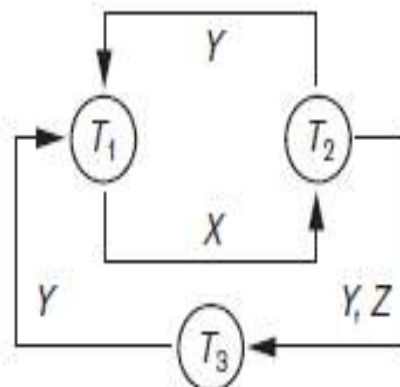
- **Algorithm:** Testing Conflict Serializability of a Schedule S

1. For each transaction T_i participating in schedule S , create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
3. For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
4. For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

- Conditions:
 - Different transactions
 - Same data item
 - One of them is a write operation

Time

Transaction T_1	Transaction T_2	Transaction T_3
$\text{read_item}(X);$ $\text{write_item}(X);$ $\text{read_item}(Y);$ $\text{write_item}(Y);$	$\text{read_item}(Z);$ $\text{read_item}(Y);$ $\text{write_item}(Y);$ $\text{read_item}(X);$ $\text{write_item}(X);$	$\text{read_item}(Y);$ $\text{read_item}(Z);$ $\text{write_item}(Y);$ $\text{write_item}(Z);$



Equivalent serial schedules

None

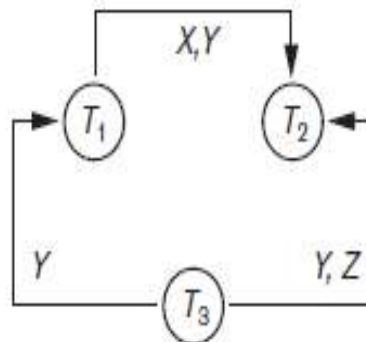
Reason

Cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$

Cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$

Time

Transaction T_1	Transaction T_2	Transaction T_3
<code>read_item(X);</code> <code>write_item(X);</code> <code>read_item(Y);</code> <code>write_item(Y);</code>	 <code>read_item(Z);</code> <code>read_item(Y);</code> <code>write_item(Y);</code> <code>read_item(X);</code> <code>write_item(X);</code>	<code>read_item(Y);</code> <code>read_item(Z);</code> <code>write_item(Y);</code> <code>write_item(Z);</code>



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

For a schedule to be conflict serializable, there should be no cycle present in its precedence graph.

EXAMPLE 1

T1	T2	T3
R(X)		
		R(Z)
		W(Z)
	R(Y)	
R(Y)		
	W(Y)	
		W(X)
	W(Z)	
W(X)		

EXAMPLE 2

T1	T2	T3
R(X)		
	R(Y)	
		R(Y)
	W(Y)	
W(X)		
		W(X)
	R(X)	
	W(X)	

EXAMPLE 3

T1	T2	T3
R(A)		
	R(B)	
		R(C)
		W(C)
	W(B)	
W(A)		

EXAMPLE 4

T1	T2	T3	T4
	R(X)		
		W(X) COMMIT	
W(X) COMMIT			
	W(Y)		
	R(Z) COMMIT		
			R(X)
			R(Y) COMMIT

Example 5

Consider the following schedule for transactions T1, T2 and T3:

T1	T2	T3
Read(X)		
	Read (Y)	
		Read (Y)
	Write (Y)	
Write (X)		
		Write (X)
	Read (X)	
	Write (X)	

Which one of the schedules below is the correct serialization of the above?

1. $T1 \rightarrow T3 \rightarrow T2$

2. $T2 \rightarrow T1 \rightarrow T3$

3. $T2 \rightarrow T3 \rightarrow T1$

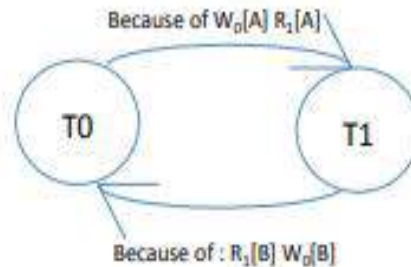
4. $T3 \rightarrow T1 \rightarrow T2$

- (a) Consider the following two transactions and schedule (time goes from top to bottom). Is this schedule conflict-serializable? Explain why or why not.

Transaction T_0	Transaction T_1
$r_0[A]$	
$w_0[A]$	
	$r_1[A]$
	$r_1[B]$
	c_1
$r_0[B]$	
$w_0[B]$	
c_0	

Solution:

The schedule is not conflict serializable because the precedence graph contains a cycle. The graph has an edge $T_0 \rightarrow T_1$ because the schedule contains $w_0[A] \rightarrow r_1[A]$. The graph has an edge $T_1 \rightarrow T_0$ because the schedule contains $r_1[B] \rightarrow w_0[B]$.

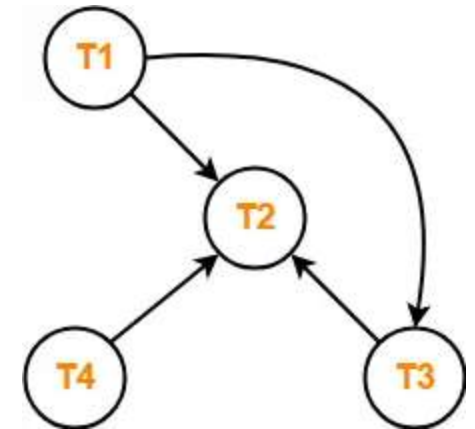


- **Problem-01:**

- Check whether the given schedule S is conflict serializable or not-

- **S : $R_1(A)$, $R_2(A)$, $R_1(B)$, $R_2(B)$, $R_3(B)$, $W_1(A)$, $W_2(B)$**

T1	T2	T3	T4
			R(A)
	R(A)		
		R(A)	
W(B)			
	W(A)		
		R(B)	
	W(B)		



Check whether the given schedule S is conflict serializable or not.

- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.

The given schedule S can be rewritten
as-

T1	T2
R(A)	
A = A-10	
	R(A)
	Temp = 0.2 x A
	W(A)
	R(B)
W(A)	
R(B)	
B = B+10	
W(B)	
	B = B+Temp
	W(B)

T1	T2
R(A)	
	R(A)
	W(A)
	R(B)
W(A)	
R(B)	
W(B)	
	W(B)

- This is because we are only concerned about the read and write operations taking place on the database.
-

View Serializability

- If a schedule is conflict serializable, then it will be view serializable.
- The view serializable which does not conflict serializable contains blind writes.

- Two schedules S and S' are said to be **view equivalent** if the following three conditions hold:

1. The same set of transactions participates in S and S' , and S and S' include the same operations of those transactions.
2. For any operation $r_i(X)$ of T_i in S , if the value of X read by the operation has been written by an operation $w_j(X)$ of T_j (or if it is the original value of X before the schedule started), the same condition must hold for the value of X read by operation $r_i(X)$ of T_i in S' .
3. If the operation $w_k(Y)$ of T_k is the last operation to write item Y in S , then $w_k(Y)$ of T_k must also be the last operation to write item Y in S' .

- **SIMPLY:**
 - **Initial Read:** An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	Write(A)

Schedule S1

T1	T2
Read(A)	Write(A)

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

- **Updated Read:** In schedule S1, if T_i is reading A which is updated by T_j then in S2 also, T_i should read A which is updated by T_j .

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S1

T1	T2	T3
Write(A)	Write(A)	<u>Read(A)</u>

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

- **Final Write:** A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

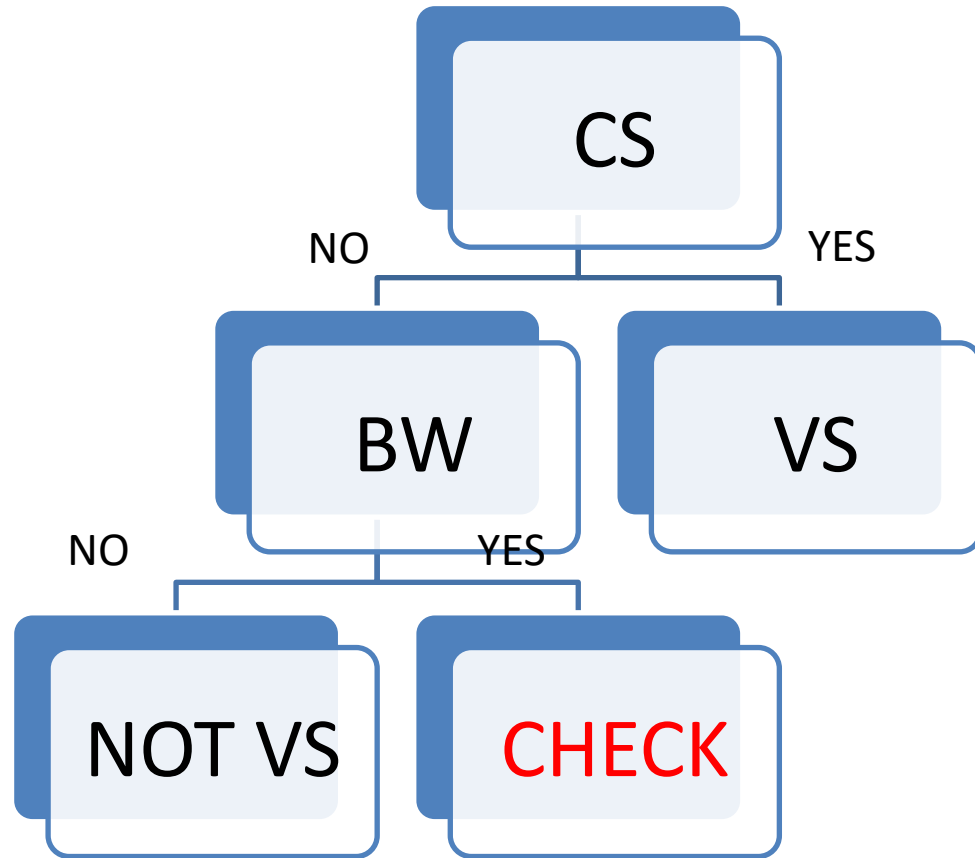
T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.



EXAMPLE 1

T1	T2	T3
Read(A)	Write(A)	
Write(A)		Write(A)

With 3 transactions, the total number of possible schedule

$$= 3! = 6$$

S1 = <T1 T2 T3>

S2 = <T1 T3 T2>

S3 = <T2 T3 T1>

S4 = <T2 T1 T3>

S5 = <T3 T1 T2>

S6 = <T3 T2 T1>

- Taking first schedule S1:

T1	T2	T3
Read(A) Write(A)	Write(A)	Write(A)

Schedule S1

- **Step 1: final updation on data items:** In both schedules S and S1, there is no read except the initial read that's why we don't need to check that condition.
- **Step 2: Initial Read:** The initial read operation in S is done by T1 and in S1, it is also done by T1.
- **Step 3: Final Write:** The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are view Equivalent.

The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.

Hence, view equivalent serial schedule is: T1 → T2 → T3

- NON-SERIAL SCHEDULE IS VIEW EQUIVALENT TO SERIAL SCHEDULE, SO IT IS VIEW SERIALIZABLE.
- What is view equivalency?

EXAMPLE 2

R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)

EXAMPLE 2

S		S'	
T1	T2	T1	T2
R(A)		R(A)	
W(A)		W(A)	
	R(A)	R(B)	
	W(A)	W(B)	
R(B)			R(A)
W(B)			W(A)
	R(B)		R(B)
	W(B)		W(B)

- **CONDITIONS CHECK:**
 - **Initial read:** In the example above, the Initial read is made by the T1 transaction for both the data items A and B in Schedule S and in S' as well initial read is made by transaction T1 for both. Thus, the initial read condition is satisfied in S and S' for data items A and B.
 - **Final Write:** The final write for data items A and B are made by Transaction T2 in Schedule S. Also, the final write is made by T2 in Schedule S' in Schedule S as well as serial schedule S'. Thus, the condition for the final write is also satisfied.
 - **Update Read:** Value of A and B is updated by transaction T1 in schedule S, which is read by transaction T2. In Schedule S' the A and B data items are updated by T1 and are read by T2 as well. Thus, in the above two schedules, the update read condition is also satisfied.

If the above mentioned conditioned are satisfied then the schedule is view serializable or else not view serializable.

Concurrency Control Techniques

- These protocols guarantee to generate schedules which satisfy the properties of serializability (CS).
- Avoid conflicting instructions (accessing same data item at the same time (one of the instructions is write operation))
- **concurrency control protocols** (sets of rules) that guarantee serializability (ISOLATION or consistency).
- One important set of protocols—known as *two-phase locking protocols*—employ the technique of **locking** data items to prevent multiple transactions from accessing the items concurrently.
- Another set of concurrency control protocols use **timestamps (ORDERING, the moment they enter the system)**. A timestamp is a unique identifier for each transaction, generated by the system.

- A **lock** is a variable associated with a data item that describes the status of the item with respect to possible operations that can be applied to it.
- Generally, there is one lock for each data item in the database. Locks are used as a means of synchronizing the access by concurrent transactions to the database items.
- *shared/exclusive* locks—also known as *read/write* locks—which provide more general locking capabilities and are used in practical database locking schemes.

- **Binary Locks.**

- A **binary lock** can have two **states** or **values**: locked and unlocked (or 1 and 0, for simplicity). A distinct lock is associated with each database item X . If the value of the lock on X is 1, item X *cannot be accessed* by a database operation that requests the item.
- If the value of the lock on X is 0, the item can be accessed when requested, and the lock value is changed to 1.
- We refer to the current value (or state) of the lock associated with item X as **lock(X)**.

```

lock_item(X):
  B:  if LOCK(X) = 0          (* item is unlocked *)
      then LOCK(X) ← 1      (* lock the item *)
      else
        begin
          wait (until LOCK(X) = 0
                and the lock manager wakes up the transaction);
        go to B
        end;
unlock_item(X):
  LOCK(X) ← 0;                (* unlock the item *)
  if any transactions are waiting
    then wakeup one of the waiting transactions;

```

- Two operations, **lock_item** and **unlock_item**, are used with binary locking.
- A transaction requests access to an item X by first issuing a **lock_item(X)** operation. If $\text{LOCK}(X) = 1$, the transaction is forced to wait.
- If $\text{LOCK}(X) = 0$, it is set to 1 (the transaction **locks** the item) and the transaction is allowed to access item X .
- When the transaction is through using the item, it issues an **unlock_item(X)** operation, which sets $\text{LOCK}(X)$ back to 0 (**unlocks** the item) so that X may be accessed by other transactions.
- Hence, a binary lock enforces **mutual exclusion** on the data item.

- The preceding binary locking scheme is too restrictive for database items because at most, one transaction can hold a lock on a given item. We should allow several transactions to access the same item X if they all access X for *reading purposes only*.
- This is because read operations on the same item by different transactions are not conflicting.
- However, if a transaction is to write an item X , it must have exclusive access to X .
- For this purpose, a different type of lock called a **multiple-mode lock** is used. In this scheme—called **shared/exclusive** or **read/write** locks—there are three locking operations: `read_lock( $X$ )`, `write_lock( $X$ )`, and `unlock( $X$ )`.

- A **read-locked item** is also called **share-locked** because other transactions are allowed to read the item, whereas a **write-locked item** is called **exclusive-locked** because a single transaction exclusively holds the lock on the item.

- Modes of locks: (The DBMS has a **lock manager subsystem** to keep track of and control access to locks.)
 - **SHARED:** Another transaction that tries to read the same data is permitted to read, but a transaction that tries to update the data will be prevented from doing so until the shared lock is released. Shared lock is also called read lock, used for reading data items only.
 - Lock $-S(a)$
 - **EXCLUSIVE:** When a statement modifies data, its transaction holds an exclusive lock on data that prevents other transactions from accessing the data. This lock remains in place until the transaction holding the lock issues a commit or rollback.
 - Lock $-X(a)$

	S	X
S	✓	X
X	X	X

read_lock(X):

B: if LOCK(X) = "unlocked"
 then **begin** LOCK(X) ← "read-locked";
 no_of_reads(X) ← 1
 end
else if LOCK(X) = "read-locked"
 then no_of_reads(X) ← no_of_reads(X) + 1
else **begin**
 wait (until LOCK(X) = "unlocked"
 and the lock manager wakes up the transaction);
 go to **B**
 end;

write_lock(X):

B: if LOCK(X) = "unlocked"
 then LOCK(X) ← "write-locked"
else **begin**
 wait (until LOCK(X) = "unlocked"
 and the lock manager wakes up the transaction);
 go to **B**
 end;

unlock (X):

if LOCK(X) = "write-locked"
 then **begin** LOCK(X) ← "unlocked";
 wakeup one of the waiting transactions, if any
 end
else if LOCK(X) = "read-locked"
 then **begin**
 no_of_reads(X) ← no_of_reads(X) - 1;
 if no_of_reads(X) = 0
 then **begin** LOCK(X) = "unlocked";
 wakeup one of the waiting transactions, if any
 end
 end;

T1	T2
Lock – X(A)	
R(A)	
W(A)	
Unlock – X(A)	
	Lock –S(B)
	R(B)
	unlock –S(B)
Lock – X(B)	
R(B)	
W(B)	
Unlock – X(B)	
	Lock –S(A)
	R(A)
	unlock –S(A)

By unlocking inconsistency will arise, if not then concurrency will be poor.

Guaranteeing Serializability by Two-Phase Locking

- A transaction is said to follow the **two-phase locking protocol** if *all* locking operations (read_lock, write_lock) precede the *first* unlock operation in the transaction.
- Such a transaction can be divided into two phases: an **expanding** or **growing (first) phase**, during which new locks on items can be acquired but none can be released; and a **shrinking (second) phase**, during which existing locks can be released but no new locks can be acquired.

- Growing phase: granting locks and not releasing locks.
- Shrinking Phase: Releasing locks and not granting locks.

T_1	T_2
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>unlock(Y);</code> <code>write_lock(X);</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>unlock(X);</code> <code>write_lock(Y);</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>

- Transactions T_1 and T_2 in Figure **above** do not follow the two-phase locking protocol because the `write_lock(X)` operation follows the `unlock(Y)` operation in T_1 , and similarly the `write_lock(Y)` operation follows the `unlock(X)` operation in T_2 .
- If we enforce two-phase locking, the transactions can be rewritten as T_1 and T_2 , as shown in **below**.

T_1'	T_2'
<code>read_lock(Y);</code> <code>read_item(Y);</code> <code>write_lock(X);</code> <code>unlock(Y)</code> <code>read_item(X);</code> <code>X := X + Y;</code> <code>write_item(X);</code> <code>unlock(X);</code>	<code>read_lock(X);</code> <code>read_item(X);</code> <code>write_lock(Y);</code> <code>unlock(X)</code> <code>read_item(Y);</code> <code>Y := X + Y;</code> <code>write_item(Y);</code> <code>unlock(Y);</code>

- NOTE: if *every* transaction in a schedule follows the two-phase locking protocol, the schedule is *guaranteed to be serializable*,

Static or Conservative 2 PL

- There are a number of variations of two-phase locking (2PL). The technique just described is known as **basic 2PL**.
- A variation known as **conservative 2PL** (or **static 2PL**) requires a transaction to lock all the items it accesses *before the transaction begins execution*, by **pre-declaring** its *read-set* and *write-set*.

- the **read-set** of a transaction is the set of all items that the transaction reads, and the **write-set** is the set of all items that it writes.
- If any of the pre-declared items needed cannot be locked, the transaction does not lock any item; instead, it waits until all the items are available for locking.
- Conservative 2PL is a deadlock-free protocol.
- Violating Hold and wait.
- Prior knowledge of what data items will be required during execution.

Strict 2PL

- In this variation, a transaction T does not release any of its exclusive (write) locks until *after* it commits or aborts. Hence, no other transaction can read or write an item that is written by T unless T has committed, leading to a strict schedule for recoverability.

Rigorous 2 PL

- A more restrictive variation of strict 2PL is **rigorous 2PL**, which also guarantees strict schedules.
- In this variation, a transaction T does not release any of its locks (exclusive or shared) until after it commits or aborts, and so it is easier to implement than strict 2PL.
- Notice the difference between conservative and rigorous 2PL: the former must lock all its items *before it starts*, so once the transaction starts it is in its shrinking phase; the latter does not unlock any of its items until *after it terminates* (by committing or aborting), so the transaction is in its expanding phase until it ends.

Recoverable schedule

- Recoverable Schedule: A schedule is recoverable if it allows for the recovery of the database to a consistent state after a transaction failure.
- In a recoverable schedule, a transaction that has updated the database must commit before any other transaction reads or writes the same data.
- If a transaction fails before committing, its updates must be rolled back, and any transactions that have read its uncommitted data must also be rolled back.
- Committed transaction can be rollback.

S	
T1	T2
R(A)	
A=A+10	
W(A)	
	R(A)
	A=A-5
	W(A)
	COMMIT
COMMIT	→ failure

IRRECOVERABLE SCHEDULE

S	
T1	T2
R(A)	
A=A+10	
W(A)	
	R(A)
	A=A-5
	W(A)
COMMIT	→ failure
	→ failure
	COMMIT

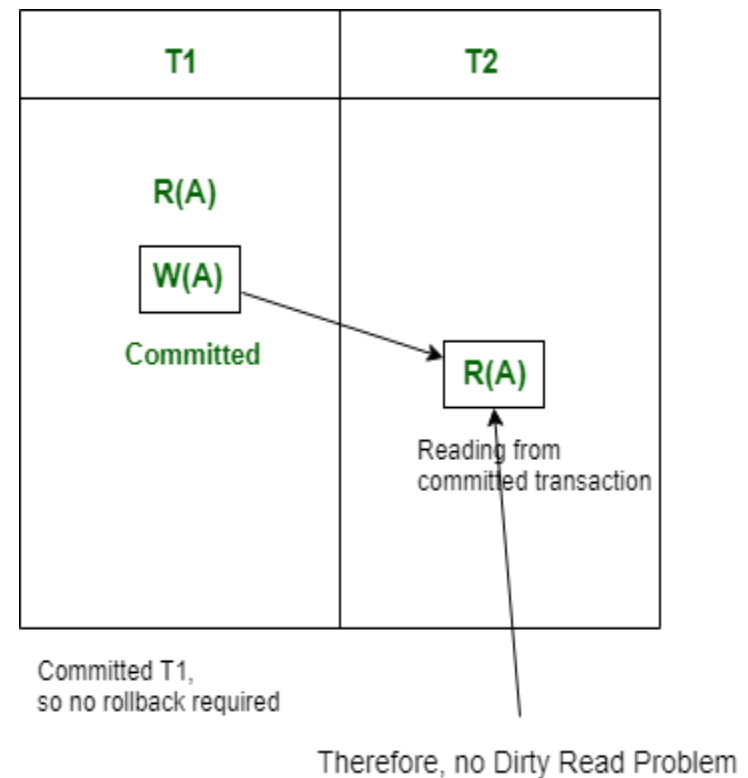
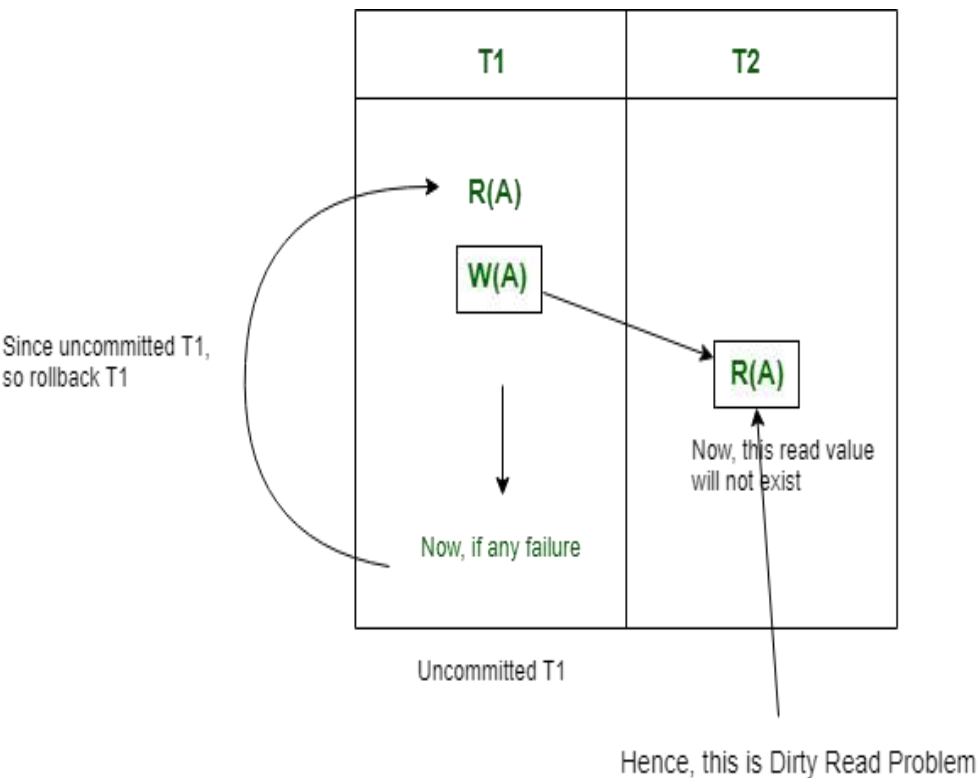
IRRECOVERABLE SCHEDULE

- Note: The transaction which is doing ***dirty read*** should commit after the main transaction from which dirty read is performed.
 - No dirty read= Recoverable
 - Dirty read= commit after the main transaction to be recoverable.

Cascade less Schedule

- A schedule is cascaded less if it does not result in a cascading rollback of transactions after a failure.
- In a cascade-less schedule, a transaction that has read uncommitted data (Dirty Read) from another transaction cannot commit before that transaction commits.
- What is cascading rollback?

- Dirty Read not allowed, means reading the data written by an uncommitted transaction is not allowed.
- uncommitted transaction might make other transactions read a value that does not even exist. This leads to inconsistency of the database.



Note : there is no Dirty Read problem, if a transaction is reading from another committed transaction. So, no rollback required.

T1	T2	T3
R(A)		
W(A)		
	R(A)	
	W(A)	
		R(A)
	C	
C		
		C

→ failure

- T2 can't rollback (irrecoverable state)

- SOLUTION: to make it recoverable (commit in same order of dirty read. (dirty read is present but recoverable))

T1	T2	T3
R(A)		
W(A)		
	R(A)	
	W(A)	
		R(A)
C		
	C	
		C

- **Cascading roll back:** after failure, T1 rollback which makes T2 rollback and finally t3 rollback because it is doing dirty read from T2.
- **DEFINITION:** The phenomenon in which a single transaction failure leads to a series of transaction rollbacks is called **Cascading rollback**.

T1	T2	T3
R(A)		
W(A)		
	R(A)	
	W(A)	
		R(A)
C		
	C	
		C

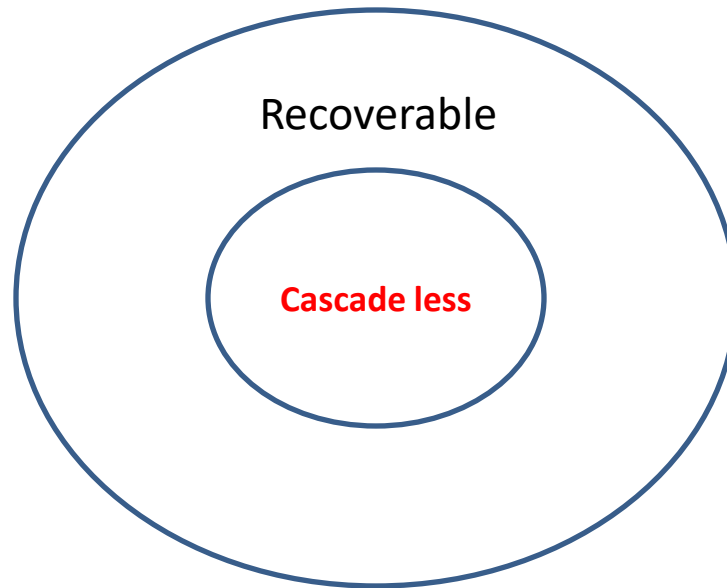
→ failure

- Consistent but not efficient.

- Cascade less Schedule avoids cascading aborts/rollbacks (avoid dirty read).
- Schedules in which transactions read values only after all transactions whose changes they are going to read commit are called cascade less schedules

T1	T2	T3
R(A)		
W(A)		
C		
	R(A)	
	W(A)	
	C	
		R(A)
		C

- Cascade less Schedules are always recoverable, but all recoverable transactions may not be Cascade less Schedule.



- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

- Here,
 - Transaction T2 depends on transaction T1.
 - Transaction T3 depends on transaction T2.
 - Transaction T4 depends on transaction T3.
- In this schedule,
 - The failure of transaction T1 causes the transaction T2 to rollback.
 - The rollback of transaction T2 causes the transaction T3 to rollback.
 - The rollback of transaction T3 causes the transaction T4 to rollback.

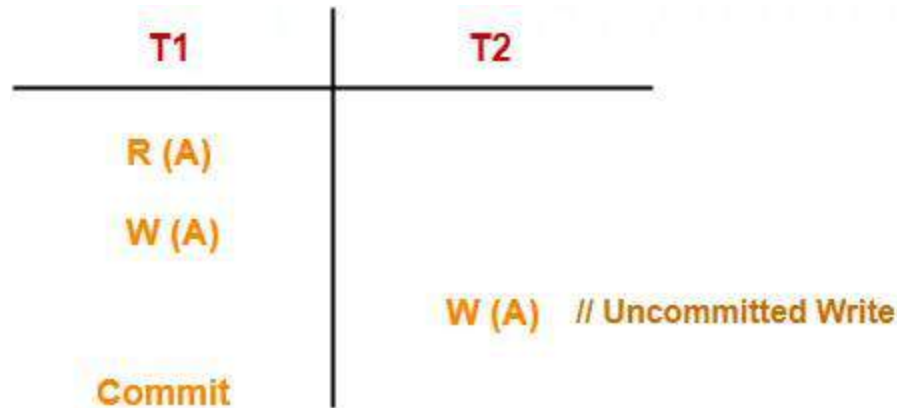
Such a rollback is called as a **Cascading Rollback**.

If the transactions T2, T3 and T4 would have committed before the failure of transaction T1, then the schedule would have been irrecoverable.

T1	T2	T3
R (A)		
W (A)		
Commit		
	R (A)	
	W (A)	
	Commit	
		R (A)
		W (A)
		Commit

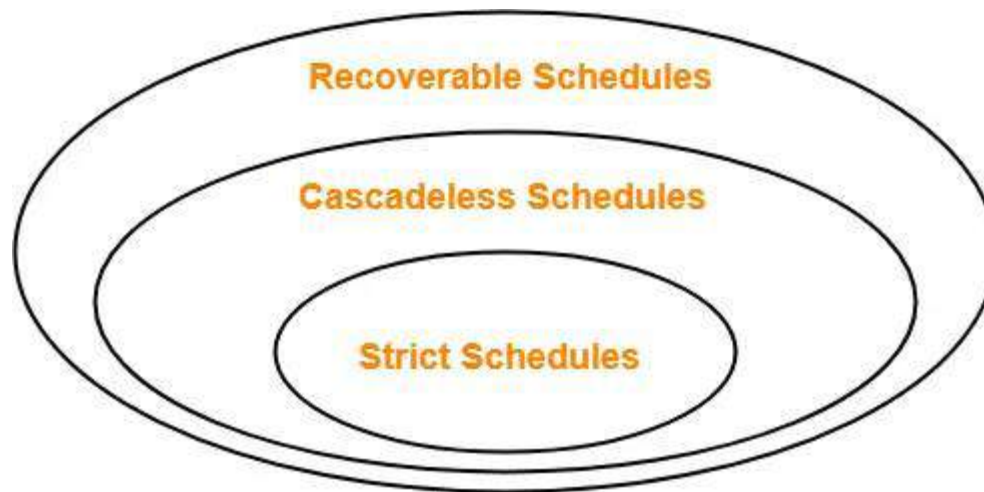
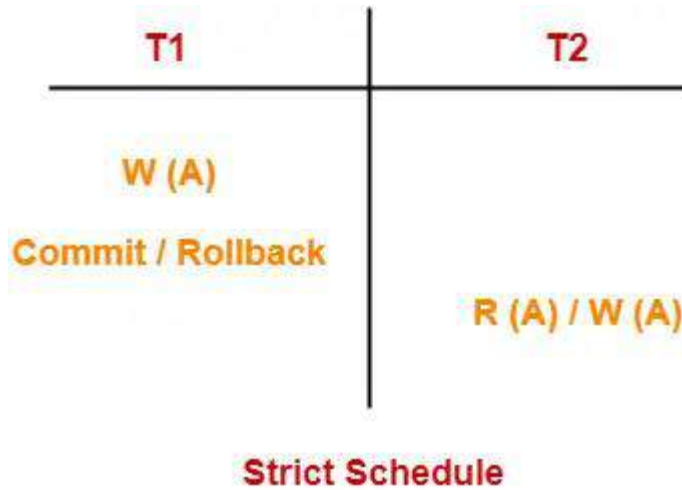
Cascadeless Schedule

Strict Schedule



Cascadeless Schedule

- Cascade less schedule allows only committed read operations.
- However, it allows uncommitted write operations.
- If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.



- Strict schedules are more strict than cascade less schedules.
- All strict schedules are cascade less schedules

EXAMPLE OF STRICT SCHEDULES

S	
T1	T2
R(A)	
W(A)	
COMMIT	
	W(A)
	R(A)
	COMMIT

S	
T1	T2
R(A)	
	R(B)
W(A)	
	W(B)
COMMIT	
	R(A)
	COMMIT

- Strict schedules are all recoverable and cascade schedules. All cascade-less schedules are recoverable.

VIEW Serializable

Conflict serializable



Figure - Venn diagram of these schedules

Time Stamping Protocol

- The use of locks, combined with the 2PL protocol, guarantees serializability of schedules.
- The serializable schedules produced by 2PL have their equivalent serial schedules based on the order in which executing transactions lock the items they acquire.
- A different approach that guarantees serializability involves using transaction timestamps to order transaction execution for an equivalent serial schedule.

- **Timestamps**

- a **timestamp** is a unique identifier created by the DBMS to identify a transaction.
- Typically, timestamp values are assigned in the order in which the transactions are submitted to the system, so a timestamp can be thought of as the *transaction start time*.
- We will refer to the timestamp of transaction T_i as **TS(T_i)**. Concurrency control techniques based on timestamp ordering do not use locks.

- Timestamps can be generated in several ways:
 - One possibility is to use a counter that is incremented each time its value is assigned to a transaction.
 - Another way to implement timestamps is to use the current date/time value of the system clock and ensure that no two timestamp values are generated during the same tick of the clock.

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses **system time or logical counter**.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.

Timestamp of data items

- **W_TS(X)** is the largest timestamp of any transaction that executed **write(X)** successfully.
- **R_TS(X)** is the largest timestamp of any transaction that executed **read(X)** successfully.
- The Basic TO algorithm compares the timestamp of T with **R_TS(X)** & **W_TS(X)** to ensure that the Timestamp order is not violated.

- INITIAL
 - $W_TS(X) = \text{NIL}$
 - $R_TS(X) = \text{NIL}$
- When T_i performs write operation on X , The timestamp of T_i will be assigned to X by $W_TS(X)$.
- EXAMPLE: T_i executes at 03:00 the value of $W_TS(X) = 03:00$.
- **NOTE: the above time is the timestamp of T_i and not the actual time at when T_i executes write operation. (T_i enters at that time in a system (time stamp))**

- **Basic Timestamp ordering protocol works as follows:**

1. Check the following condition whenever a transaction T_i issues a **Read (X)** operation:
 - If $W_TS(X) > TS(T_i)$ then the operation is rejected.
 - If $W_TS(X) \leq TS(T_i)$ then the operation is executed. (Set $R_TS = \max\{R_TS(A), TS(T_i)\}$).

Timestamps of all the data items are updated.

2. Check the following condition whenever a transaction T_i issues a **Write(X)** operation:
 - If $TS(T_i) < R_TS(X)$ then the operation is rejected.
 - If $TS(T_i) < W_TS(X)$ then the operation is rejected and T_i is rolled back otherwise the operation is executed.

Where,

$TS(T_i)$ denotes the timestamp of the transaction T_i .

$R_TS(X)$ denotes the Read time-stamp of data-item X .

$W_TS(X)$ denotes the Write time-stamp of data-item X .

Thomas Write Rule

- **Thomas Write Rule** allows such operations and is a modification of the Basic Timestamp Ordering protocol.
 - If $\text{read_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation.
 - If $\text{write_TS}(X) > \text{TS}(T)$, then do not execute the write operation but continue processing.
 - If neither the condition in part (1) nor the condition in part (2) occurs, then execute the $\text{write_item}(X)$ operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.

Recovery

- Recovery from transaction failures usually means that the database is restored to the most recent consistent state just before the time of failure.
- To do this, the system must keep information about the changes that were applied to data items by the various transactions.

- If there is extensive damage to a wide portion of the database due to catastrophic failure, such as a disk crash, the recovery method restores a past copy of the database that was backed up to archival storage (typically tape or other large capacity offline storage media) and reconstructs a more current state by reapplying or redoing the operations of committed transactions from the backed up log, up to the time of failure.

- When the database on disk is not physically damaged, and a non-catastrophic failure, the recovery strategy is to identify any changes that may cause an inconsistency in the database.
- Conceptually, we can distinguish two main techniques for recovery from non-catastrophic transaction failures: **deferred update and immediate update.**

- The deferred update techniques do not physically update the database on disk until after a transaction reaches its commit point; then the updates are recorded in the database.
 - Before reaching commit, all transaction updates are recorded in the local transaction workspace or in the main memory buffers that the DBMS maintains (the DBMS main memory cache). Before commit, the updates are recorded persistently in the log, and then after commit, the updates are written to the database on disk.

- If a transaction fails before reaching its commit point, it will not have changed the database in any way, so UNDO is not needed.
- Deferred update is also known as the NO-UNDO/REDO algorithm.
- In the immediate update techniques, the database may be updated by some operations of a transaction before the transaction reaches its commit point.

- these operations must also be recorded in the log on disk by force-writing before they are applied to the database on disk, making recovery still possible.
- In the general case of immediate update, both *undo* and *redo* may be required during recovery. This technique, known as the **UNDO/REDO algorithm**, requires both operations during recovery, and is used most often in practice.

DEFERRED (Log-Based Recovery)

- The log is a sequence of records.
- Log of each transaction is maintained in some stable storage so that if any failure occurs, then it can be recovered from there.
- If any operation is performed on the database, then it will be recorded in the log.
- But the process of storing the logs should be done before the actual transaction is applied in the database (COMMIT).

- Let T_0 be a transaction that transfers 50 from account A to account B:
 - T_0 : read(A);
 - $A := A - 50$;
 - write(A);
 - read(B);
 - $B := B + 50$;
 - write(B).
- Let T_1 be a transaction that withdraws \$100 from account C:
 - T_1 : read(C);
 - $C := C - 100$;
 - write(C).

```

<T0 start>
<T0, A, 950>
<T0, B, 2050>
<T0 commit>
<T1 start>
<T1, C, 600>
<T1 commit>

```

Suppose that these transactions are executed serially, in the order T_0 followed by T_1 , and that the values of accounts A, B, and C before the execution took place were 1000, 2000, and 700, respectively. The portion of the log containing the relevant information on these two transactions appears to be (LOG FILE)

- (REDO after failure for recovery (by recovery manager))

IMMEDIATE (Log-Based Recovery)

- Let T_0 be a transaction that transfers 50 from account A to account B:
 - T_0 : read(A);
 - $A := A - 50$;
 - write(A);
 - read(B);
 - $B := B + 50$;
 - write(B).
- Let T_1 be a transaction that withdraws \$100 from account C:
 - T_1 : read(C);
 - $C := C - 100$;
 - write(C).

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$
 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

- **Caching (Buffering) of Disk Blocks**

- In general, it is convenient to consider recovery in terms of the database disk pages (blocks).

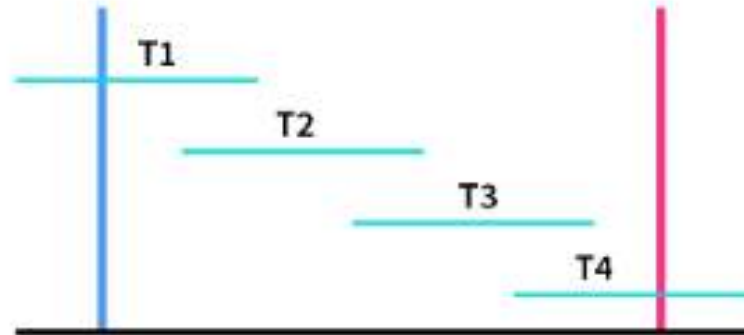
- Typically a collection of in-memory buffers, called the DBMS cache, is kept under the control of the DBMS for the purpose of holding these buffers. A directory for the cache is used to keep track of which database items are in the buffers.

CHECKPOINTS

- The Checkpoint is used to declare a point before which the DBMS was in a consistent state, and all transactions were committed.
- Upon reaching the savepoint/checkpoint, the log file is destroyed by saving its update to the database.
- Then a new log is created with upcoming execution operations of the transaction and it will be updated until the next checkpoint and the process continues.

Checkpoint

Failure



T1	T2	T3	T4
START			
	START		
	COMMIT		
		START	
		COMMIT	
			START
			FAILURE

- **Table of transactions operations and the lists they are placed in**

Operations	List
$\langle T_n, \text{start} \rangle$ and $\langle T_n, \text{commit} \rangle$	Redo list
$\langle T_n, \text{commit} \rangle$	Redo list
$\langle T_n, \text{start} \rangle$	Undo list

Types of Checkpoints: There are basically two main types of Checkpoints:

- Automatic Checkpoint
- Manual Checkpoint

1. Automatic Checkpoint: These checkpoints occur very frequently like every hour or every day. These intervals are set by the [database administrator](#). They are generally used by heavy databases as they are frequently updated, and we can recover the data easily in case of failure.

2. Manual Checkpoint: These are the checkpoints that are manually set by the database administrator. Manual checkpoints are generally used for smaller databases. They are updated very less frequently only when they are set by the database administrator.

Advantages of Checkpoints

- Checkpoints help us in recovering the transaction of the database in case of a random shutdown of the database.
- It enhancing the consistency of the database in case when multiple transactions are executing in the database simultaneously.
- Checkpoint records in the log file are used to prevent unnecessary redo operations.